BỘ LAO ĐỘNG – THƯƠNG BINH VÀ XÃ HỘI

TRƯỜNG CAO ĐẲNG CÔNG NGHỆ THÔNG TIN TP.HCM

KHOA CÔNG NGHỆ THÔNG TIN – ĐIỆN TỬ

BÁO CÁO ĐỒ ÁN MÔN HỌC

CHUYÊN ĐỀ BACK-END

ĐỀ TÀI:

XÂY DỰNG HỆ THỐNG BACK-END CHO WEBSITE THƯƠNG MẠI ĐIỆN TỬ BÁN LAPTOP LAPTOPNEW

Express.js · MongoDB Atlas · JWT · PayOS · Telegram Bot · Gemini AI

Giảng viên hướng dẫn: **ThS. Nguyễn Minh Hải**

Sinh viên thực hiện: **Lê Minh Trí – 501240045**

Âu Lê Thành Tài – 501250054

Lớp / Khóa: .....

TP. Hồ Chí Minh, tháng 9 năm 2026

LỜI MỞ ĐẦU

Thương mại điện tử đang thay đổi cách người Việt mua sắm thiết bị công nghệ. Với mặt hàng có giá trị cao và nhiều thông số kỹ thuật như máy tính xách tay, khách hàng cần một hệ thống vừa cung cấp thông tin cấu hình chính xác, vừa đặt hàng và thanh toán nhanh, lại vừa được tư vấn đúng nhu cầu. Phía sau mọi trải nghiệm đó là tầng Back-End: nơi lưu trữ dữ liệu, xử lý nghiệp vụ, xác thực người dùng và kết nối với các dịch vụ bên ngoài.

Xuất phát từ yêu cầu của môn **Chuyên đề Back-End**, nhóm chúng em thực hiện đề tài **"Xây dựng hệ thống Back-End cho website thương mại điện tử bán laptop LaptopNew bằng Express.js và MongoDB"**. Hệ thống được viết bằng Node.js và Express 5 theo mô hình phân tầng Route – Middleware – Controller – Service – Model, lưu trữ trên MongoDB Atlas thông qua Mongoose, xác thực bằng JSON Web Token và mã hóa mật khẩu bằng bcrypt.

Ngoài các chức năng quản lý cơ bản (sản phẩm, danh mục, đơn hàng, người dùng, mã giảm giá, đánh giá, tin tức), hệ thống còn tích hợp nhiều dịch vụ thực tế: gửi mã OTP và hóa đơn qua email (Brevo API và Nodemailer), thanh toán chuyển khoản tự động qua PayOS và VietQR, thông báo và duyệt đơn qua Telegram Bot, và Trợ lý AI tư vấn laptop dựa trên Google Gemini.

Báo cáo này trình bày toàn bộ quá trình phân tích, thiết kế và hiện thực hệ thống: từ kiến trúc tổng thể, thiết kế cơ sở dữ liệu, đặc tả API, luồng xử lý của từng chức năng, cơ chế xử lý lỗi và bảo mật, cho đến kết quả kiểm thử thực tế và hướng phát triển. Đặc biệt, **Chương 5** được dành riêng cho bảo mật: nhóm tự đóng vai kẻ tấn công để rà soát mã nguồn theo chuẩn **OWASP API Security Top 10**, khắc phục **21 lỗ hổng** và chứng minh bằng **40 ca kiểm thử tự động** cùng **15 kịch bản tấn công thử**. Nhóm hy vọng đây vừa là sản phẩm hoàn thành yêu cầu học phần, vừa là bước chuẩn bị cho công việc phát triển phần mềm sau này.

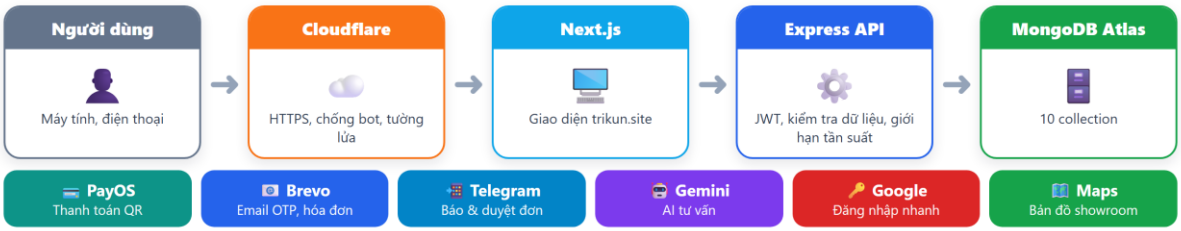
Hệ thống đã được triển khai thực tế trên nền tảng đám mây Railway, chạy dưới tên miền riêng **https://trikun.site** (website) và **https://api.trikun.site** (API), đặt sau mạng Cloudflare để có HTTPS bắt buộc, chống bot và giới hạn tần suất ngay tại lớp biên. Toàn bộ API được công bố kèm tài liệu tương tác chuẩn OpenAPI 3.0 tại **https://api.trikun.site/api-docs**. Các hình giao diện và kết quả kiểm thử trong báo cáo

đều được thực hiện trên server đang chạy thật; riêng hình minh họa bước cài đặt (mục 6.3) và kết quả chạy bộ kiểm thử Jest (mục 6.8) chụp ở máy phát triển và trên GitHub Actions; các hình giỏ hàng nhiều thiết bị và thông báo đổi giá (mục 4.15.3) chụp ở môi trường kiểm thử cục bộ với dữ liệu mẫu.

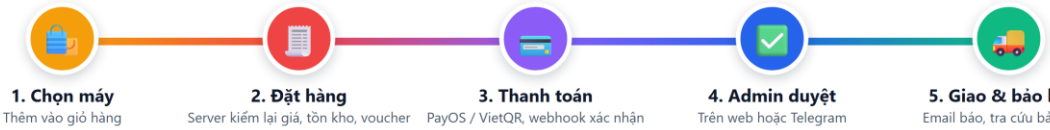
TÓM TẮT DỰ ÁN BẰNG HÌNH

Hai trang dưới đây tóm tắt toàn bộ dự án để người đọc nắm nhanh trước khi vào chi tiết ở các chương.

Hệ thống hoạt động thế nào?



Một đơn hàng đi qua 5 bước



Bảo mật & chất lượng

Báo cáo gồm 6 chương

LỜI CẢM ƠN

Trước hết, chúng em xin gửi lời cảm ơn chân thành đến Ban Giám hiệu và quý Thầy Cô Khoa Công nghệ Thông tin – Điện tử, Trường Cao đẳng Công nghệ Thông tin TP.HCM đã tạo môi trường học tập và truyền đạt nền tảng kiến thức trong suốt thời gian qua.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn đến **Thầy Nguyễn Minh Hải**, giảng viên phụ trách môn Chuyên đề Back-End. Những bài giảng về Node.js, Express, MongoDB, cách tổ chức mã nguồn và bảo mật API của Thầy là nền tảng để nhóm hoàn thành đồ án này.

Chúng em cũng cảm ơn các thành viên trong nhóm đã hỗ trợ nhau trong quá trình thực hiện, cùng gia đình và bạn bè đã luôn động viên.

Do thời gian và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi thiếu sót. Nhóm rất mong nhận được nhận xét và góp ý của quý Thầy Cô để hoàn thiện hệ thống tốt hơn.

Chúng em xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 9 năm 2026

Nhóm sinh viên thực hiện

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm đánh giá: .....

TP. Hồ Chí Minh, ngày ..... tháng ..... năm 2026

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

MỤC LỤC

LỜI MỞ ĐẦU ..... **i**

TÓM TẮT DỰ ÁN BẰNG HÌNH ..... **iii**

LỜI CẢM ƠN ..... **v**

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN ..... **vi**

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ ..... **xii**

DANH MỤC HÌNH ẢNH ..... **xiv**

DANH MỤC BẢNG ..... **xxi**

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI **1**

1.1. Lý do chọn đề tài 1

1.2. Mục tiêu dự án 1

1.2.1. Các chức năng cơ bản 1

1.2.2. Mục đích phát triển ứng dụng Back-End 2

1.3. Phạm vi và đối tượng sử dụng 2

1.4. Công nghệ sử dụng 3

1.4.1. Node.js 5

1.4.2. Express.js 6

1.4.3. MongoDB Atlas và Mongoose 6

1.4.4. JSON Web Token (JWT) 7

1.4.5. Bcrypt 7

1.4.6. Nodemailer và Brevo Email API 7

1.4.7. Dotenv và CORS 8

1.4.8. PayOS và VietQR 8

1.4.9. Telegram Bot API 8

1.4.10. Google Gemini và 9Router Gateway 9

1.4.11. Công cụ phát triển 9

1.4.12. Swagger UI và OpenAPI 3.0 10

1.4.13. Railway, Cloudflare và tên miền riêng 10

1.4.14. Jest, Supertest, ESLint và GitHub Actions 10

1.4.15. Next.js, React, Tailwind CSS và Recharts (Front-End) 11

1.4.16. Xuất Excel, in hóa đơn và Google Maps 11

1.5. Lý do chọn công nghệ 12

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 13
2.1. Kiến trúc ứng dụng 13
2.1.1. Client (Front-End) 13
2.1.2. Server (Back-End) 13
2.1.3. Database (MongoDB Atlas) 14
2.1.4. Middleware 14
2.1.5. Vòng đời một request 15
2.2. Cấu trúc thư mục 15
2.2.1. config/ 15
2.2.2. routes/ 16
2.2.3. middleware/ 16
2.2.4. controllers/ 16
2.2.5. services/ 16
2.2.6. models/, utils/, scripts-local/ 16
2.3. Sơ đồ Use Case 17
2.4. Thiết kế cơ sở dữ liệu 17
2.4.1. Tổng quan các collection 17
2.4.2. Sơ đồ quan hệ dữ liệu 18
2.4.3. Collection san_pham 19
2.4.4. Collection don_hang 20
2.4.5. Collection nguoi_dung và dang_ky_tam 21
2.4.6. Collection ma_giam_gia 22
2.4.7. Collection danh_gia 23
2.4.8. Collection danh-muc, tin_tuc, lien_he, cai_dat 23
2.4.9. Chỉ mục (Index) và ràng buộc dữ liệu 24
CHƯƠNG 3. ĐẶC TẢ HỆ THỐNG API 25
3.1. Quy ước chung 25
3.1.1. Các mức quyền truy cập 25
3.2. Bảng tổng hợp API 26
3.3. Đặc tả chi tiết từng nhóm API 27
3.3.1. Nhóm Xác thực và Tài khoản — /api/auth 27
3.3.2. Nhóm Sản phẩm — /api/san-pham 28
3.3.3. Nhóm Danh mục — /api/danh-muc 29
3.3.4. Nhóm Đơn hàng — /api/don-hang 29
3.3.5. Nhóm Thanh toán — /api/thanh-toan 30

3.3.6. Nhóm Mã giảm giá — /api/ma-giam-gia 31
3.3.7. Nhóm Đánh giá — /api/danh-gia 31
3.3.8. Nhóm Người dùng — /api/nguoi-dung 31
3.3.9. Nhóm Tin tức — /api/tin-tuc 32
3.3.10. Nhóm Bảo hành — /api/bao-hanh 32
3.3.11. Nhóm Liên hệ — /api/lien-he 32
3.3.12. Nhóm Cài đặt showroom — /api/cai-dat 33
3.3.13. Nhóm Trợ lý AI — /api/tro-ly-ai 33
3.3.14. Nhóm Giám sát máy chủ 33
3.4. Tài liệu API trực tuyến với Swagger UI (OpenAPI 3.0) 34
CHƯƠNG 4. HIỆN THỰC CÁC CHỨC NĂNG CHÍNH 40
4.1. Chức năng đăng ký và kích hoạt tài khoản bằng OTP 40
4.2. Chức năng đăng nhập, xác thực JWT và phân quyền 43
4.3. Chức năng đăng nhập bằng Google 46
4.4. Chức năng quên mật khẩu và đổi mật khẩu 48
4.5. Chức năng quản lý sản phẩm 51
4.5.1. Tìm kiếm, lọc và sắp xếp 51
4.5.2. Thêm, sửa, xóa sản phẩm (Admin) 53
4.6. Chức năng giỏ hàng và mã giảm giá 54
4.7. Chức năng đặt hàng và quản lý trạng thái đơn 57
4.7.1. Tạo đơn hàng 57
4.7.2. Phân quyền xem và cập nhật đơn 58
4.7.3. Trạng thái đơn và đồng bộ tồn kho 60
4.7.4. Địa chỉ giao hàng theo địa giới hành chính mới (Provinces Open API v2) 62
4.8. Chức năng thanh toán PayOS và VietQR 63
4.9. Chức năng gửi email tự động 66
4.10. Chức năng quản trị đơn hàng qua Telegram Bot 67
4.11. Chức năng Trợ lý AI tư vấn laptop 69
4.12. Chức năng tra cứu bảo hành điện tử 71
4.13. Các chức năng bổ trợ 73
4.14. Một số giao diện kết quả 73
4.15. Tự động đồng bộ giao diện khi Admin thay đổi dữ liệu 79
4.15.1. Cơ chế tự đồng bộ 80
4.15.2. Sản phẩm ẩn, hết hàng và giá 81

4.15.3. Giỏ hàng theo tài khoản 82
4.15.4. Kết quả kiểm thử 84
4.16. Giao diện quản trị Admin: thêm, xem, sửa, xóa (CRUD) 85
4.16.1. Quản lý sản phẩm 86
4.16.2. Quản lý hãng và danh mục 87
4.16.3. Quản lý đơn hàng 88
4.16.4. Quản lý tin tức và bài viết 89
4.16.5. Cấu hình banner và khuyến mãi 90
4.16.6. Quản lý showroom và bản đồ 91
4.16.7. Quản lý yêu cầu tư vấn (CSKH) 91
4.16.8. Quản lý tài khoản khách hàng 92
4.16.9. Quản lý mã giảm giá (voucher) 93
4.16.10. Xuất báo cáo kế toán ra Excel 94
CHƯƠNG 5. XỬ LÝ LỖI VÀ BẢO MẬT 96
5.1. Chuẩn hóa mã trạng thái HTTP 96
5.2. Xử lý từng loại lỗi 97
5.2.1. Lỗi 400 – Bad Request 97
5.2.2. Lỗi 401 – Unauthorized 97
5.2.3. Lỗi 403 – Forbidden 98
5.2.4. Lỗi 404 – Not Found 99
5.2.5. Lỗi 500 – Internal Server Error 100
5.2.6. Lỗi 429 – Too Many Requests 100
5.3. Mô hình bảo mật nhiều lớp (Defense in Depth) 101
5.4. Đối chiếu với OWASP API Security Top 10 (2023) 102
5.5. Rà soát và khắc phục lỗ hổng bảo mật 103
5.6. Hiện thực các cơ chế bảo mật tiêu biểu 105
5.6.1. Chống NoSQL Injection bằng middleware 105
5.6.2. Khóa đăng nhập khi bị dò mật khẩu 105
5.6.3. OTP an toàn: sinh ngẫu nhiên mật mã, lưu dạng băm, giới hạn số lần nhập 106
5.6.4. Chống IDOR khi xem và cập nhật đơn hàng 106
5.6.5. Thu hồi phiên đăng nhập phía server (tokenVersion) 106
5.6.6. Giới hạn tần suất (Rate Limiting) 106
5.7. Kiểm thử bảo mật bằng Postman 107
5.8. Hạn chế còn lại và hướng hoàn thiện 118

CHƯƠNG 6. TRIỂN KHAI VÀ KIỂM THỬ 119
6.1. Yêu cầu môi trường 119
6.2. Cấu hình biến môi trường 119
6.3. Cài đặt và khởi chạy 120
6.4. Kiểm thử API 121
6.5. Minh chứng kiểm thử API bằng Postman 124
6.5.1. Nhóm Xác thực và Tài khoản 125
6.5.2. Nhóm Sản phẩm và Danh mục 126
6.5.3. Nhóm Đơn hàng và Thanh toán 128
6.5.4. Nhóm Mã giảm giá, Đánh giá, Bảo hành và Liên hệ 132
6.5.5. Nhóm Trợ lý AI, Cài đặt và Giám sát máy chủ 134
6.6. Minh chứng dữ liệu thay đổi đúng trong MongoDB 135
6.7. Triển khai thực tế trên Railway, tên miền riêng và Cloudflare 144
6.7.1. Mô hình triển khai 144
6.7.2. Tên miền và cấu hình DNS 145
6.7.3. Bảo vệ lớp biên với Cloudflare 146
6.7.4. Cấu hình môi trường production 148
6.7.5. Kiểm chứng trên môi trường production 148
6.8. Kiểm thử tự động, chất lượng mã nguồn và CI 149
6.8.1. Tổ chức bộ kiểm thử 149
6.8.2. Tích hợp liên tục với GitHub Actions 152
6.8.3. Chất lượng mã nguồn và an toàn thư viện 154
CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 156
7.1. Kết luận 156
7.2. Hướng phát triển 157
TÀI LIỆU THAM KHẢO 159

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Viết tắt / Thuật ngữ	Tên đầy đủ	Ý nghĩa trong đồ án
API	Application Programming Interface	Giao diện lập trình để Front-End gọi tới Back-End
REST / RESTful	Representational State Transfer	Phong cách thiết kế API dựa trên URL và phương thức HTTP
CRUD	Create – Read – Update – Delete	Bốn thao tác cơ bản trên dữ liệu
JSON	JavaScript Object Notation	Định dạng dữ liệu trao đổi giữa Client và Server
JWT	JSON Web Token (RFC 7519)	Chuỗi token ký số dùng để xác thực người dùng
RBAC	Role-Based Access Control	Phân quyền theo vai trò: khach_hang / admin
OTP	One-Time Password	Mã 6 số dùng một lần, hết hạn sau 10 phút
ODM	Object Document Mapping	Thư viện ánh xạ document ↔ đối tượng (Mongoose)
NoSQL	Not only SQL	Cơ sở dữ liệu phi quan hệ, lưu dạng document
BSON	Binary JSON	Định dạng nhị phân MongoDB dùng để lưu document
TTL Index	Time-To-Live Index	Chỉ mục tự xóa document khi hết hạn
CORS	Cross-Origin Resource Sharing	Cơ chế cho phép domain Front-End gọi API
MVC	Model – View – Controller	Mô hình tách lớp dữ liệu, xử lý và hiển thị
HMAC-SHA256	Hash-based Message Authentication Code	Thuật toán tạo/kiểm tra chữ ký PayOS
Webhook	HTTP Callback	Dịch vụ ngoài chủ động gọi vào API khi có sự kiện
NDJSON	Newline Delimited JSON	Mỗi dòng một JSON — dùng để stream câu trả lời AI
SKU	Stock Keeping Unit	Mã quản lý hàng hóa (ma_san_pham)
COD	Cash On Delivery	Thanh toán khi nhận hàng
CSKH	Chăm sóc khách hàng	Bộ phận xử lý yêu cầu liên hệ
OWASP	Open Worldwide Application Security Project	Tổ chức công bố danh sách rủi ro bảo mật phổ biến (API Security Top 10)

Viết tắt / Thuật ngữ	Tên đầy đủ	Ý nghĩa trong đồ án
IDOR / BOLA	Insecure Direct Object Reference	Lỗi xem/sửa được dữ liệu của người khác do thiếu kiểm tra chủ sở hữu
NoSQL Injection	—	Chèn toán tử MongoDB ($ne, $gt...) vào dữ liệu gửi lên để thay đổi truy vấn
ReDoS	Regular expression Denial of Service	Biểu thức chính quy độc hại làm server xử lý rất lâu
Rate Limiting	—	Giới hạn số request trong một khoảng thời gian, vượt quá trả 429
Race Condition	—	Hai request đồng thời cùng sửa một dữ liệu gây sai lệch (vd: trừ kho hai lần)
Mass Assignment	—	Gửi thêm trường không được phép (vd: vaiTro) để server ghi vào DB
XSS / HTML Injection	Cross-Site Scripting	Chèn mã HTML/JS độc hại vào nội dung hiển thị cho người khác
HSTS	HTTP Strict Transport Security	Header buộc trình duyệt chỉ kết nối qua HTTPS
CDN	Content Delivery Network	Mạng phân phối nội dung; Cloudflare phục vụ website qua các máy chủ biên
DNS	Domain Name System	Hệ thống phân giải tên miền; bản ghi CNAME trỏ trikun.site về Railway
SSL / TLS	Secure Sockets Layer / Transport Layer Security	Giao thức mã hóa HTTPS giữa trình duyệt, Cloudflare và server
WAF	Web Application Firewall	Tường lửa ứng dụng web, chặn request độc hại trước khi tới server
OpenAPI / Swagger	OpenAPI Specification 3.0	Chuẩn mô tả REST API; Swagger UI hiển thị tài liệu tương tác tại /api-docs
PaaS	Platform as a Service	Nền tảng chạy ứng dụng không cần tự quản lý máy chủ (Railway)
CI / CD	Continuous Integration / Continuous Delivery	Tự động build và triển khai mỗi khi có mã nguồn mới
Mock	Mock object	Đối tượng giả thay cho dịch vụ thật (email, Telegram) khi chạy kiểm thử

DANH MỤC HÌNH ẢNH

Hình 1.1. Bộ công nghệ sử dụng trong hệ thống LaptopNew, chia theo từng lớp 5
Hình 1.2. Node.js: môi trường chạy JavaScript phía server 5
Hình 1.3. Express.js: framework web để xây dựng REST API.....6
Hình 1.4. MongoDB Atlas (cơ sở dữ liệu) và Mongoose (ODM).....6
Hình 1.5. JSON Web Token: xác thực không cần lưu phiên trên server.....7
Hình 1.6. bcrypt: băm mật khẩu một chiều có salt 7
Hình 1.7. Nodemailer và Brevo Email API: gửi email OTP và xác nhận đơn hàng.....8
Hình 1.8. dotenv (biến môi trường) và CORS (giới hạn origin được gọi API).....8
Hình 1.9. PayOS và VietQR: thanh toán chuyển khoản bằng mã QR 8
Hình 1.10. Telegram Bot API: thông báo và duyệt đơn hàng 9
Hình 1.11. Google Gemini và 9Router: trợ lý AI tư vấn laptop.....9
Hình 1.12. Công cụ phát triển: Visual Studio Code, Postman, Git và GitHub 9
Hình 1.13. Swagger UI và OpenAPI 3.0: tài liệu API tương tác.....10
Hình 1.14. Railway (chạy ứng dụng) và Cloudflare (DNS, HTTPS, chống bot).....10
Hình 1.15. Jest, ESLint, Prettier và GitHub Actions: kiểm thử, chất lượng mã và CI.11
Hình 1.16. Next.js, React, Tailwind CSS và Recharts: xây dựng giao diện web.....11
Hình 1.17. Xuất Excel, in hóa đơn và Google Maps: tiện ích báo cáo và bản đồ.....11
Hình 2.1. Sơ đồ kiến trúc tổng thể hệ thống LaptopNew 13
Hình 2.2. Vòng đời xử lý một request qua các tầng của server.....15
Hình 2.3. Cấu trúc thư mục mã nguồn Back-End.....15
Hình 2.4. Sơ đồ use case của hệ thống Back-End LaptopNew 17
Hình 2.5. Sơ đồ quan hệ giữa các collection (ERD logic).....19
Hình 3.1. Kiểm thử GET /api/san-pham bằng Postman: trả về danh sách sản phẩm (200 OK).....34
Hình 3.2. Trang tài liệu Swagger UI của LaptopNew tại https://api.trikun.site/api-docs 36
Hình 3.3. Chi tiết API đăng nhập trên Swagger: mô tả nghiệp vụ, giới hạn tần suất, body mẫu và phản hồi mẫu.....37
Hình 3.4. Nhóm API Sản phẩm và Danh mục trên Swagger: API công khai (GET) và API cần JWT của Admin (biểu tượng ổ khóa).....37
Hình 3.5. Gọi thử GET /api/san-pham?limit=2 bằng Try it out: server production trả 200 kèm dữ liệu và các header bảo mật 38
Hình 3.6. Gọi thử GET /api/auth/toi khi chưa đăng nhập: server từ chối 401 (thiếu token xác thực) 39

Hình 4.1. Sơ đồ tuần tự đăng ký và kích hoạt tài khoản bằng OTP 41
Hình 4.2. Giao diện form đăng ký tài khoản 42
Hình 4.3. Email chứa mã OTP kích hoạt tài khoản 42
Hình 4.4. Kiểm thử POST /api/auth/kich-hoat bằng Postman: kích hoạt thành công, nhận JWT 43
Hình 4.5. Sơ đồ tuần tự đăng nhập và gọi API được bảo vệ 44
Hình 4.6. Giao diện đăng nhập (email/mật khẩu và đăng nhập nhanh với Google) 45
Hình 4.7. Kiểm thử POST /api/auth/dang-nhap bằng Postman: đăng nhập thành công, nhận token JWT 46
Hình 4.8. Chọn tài khoản Google khi đăng nhập 47
Hình 4.9. Google xác nhận chia sẻ tên, ảnh và email với website 47
Hình 4.10. Đăng nhập Google thành công, thanh điều hướng hiển thị tên người dùng 48
Hình 4.11. Sơ đồ tuần tự quên mật khẩu bằng OTP 49
Hình 4.12. Trang quên mật khẩu: nhập email để nhận mã OTP 49
Hình 4.13. Bước nhập mã OTP xác thực 50
Hình 4.14. Email chứa mã OTP đặt lại mật khẩu 50
Hình 4.15. Kiểm thử POST /api/auth/xac-nhan-otp bằng Postman: OTP hợp lệ 51
Hình 4.16. Trang danh sách sản phẩm với bộ lọc thương hiệu, khoảng giá, CPU, card đồ họa 52
Hình 4.17. Trang chi tiết sản phẩm: thư viện ảnh, phiên bản cấu hình, giá khuyến mãi 53
Hình 4.18. Trang quản trị: danh sách và form thêm/sửa sản phẩm 54
Hình 4.19. Lưu đồ kiểm tra và tính tiền mã giảm giá 55
Hình 4.20. Giỏ hàng áp dụng mã giảm giá TNTP3TR thành công 56
Hình 4.21. Kiểm thử POST /api/ma-giam-gia/kiem-tra bằng Postman: mã hợp lệ, giảm 3.000.000 đ 56
Hình 4.22. Lưu đồ xử lý tạo đơn hàng 58
Hình 4.23. Trang tra cứu tiến độ đơn hàng: khách nhập mã đơn và số điện thoại 59
Hình 4.24. Kiểm thử GET /api/don-hang/:id?sdt=… khi chưa đăng nhập: thông tin cá nhân đã được che 59
Hình 4.25. Kiểm thử GET /api/don-hang/:id với token chủ đơn: trả đầy đủ thông tin đơn hàng 60
Hình 4.26. Kiểm thử tra cứu đơn thiếu hoặc sai số điện thoại bị từ chối (403) 60
Hình 4.27. Vòng đời trạng thái đơn hàng và quy tắc trừ/hoàn tồn kho 60

Hình 4.28. Trang thanh toán: thông tin người nhận, địa chỉ hành chính, phương thức thanh toán 61
Hình 4.29. Email xác nhận đặt hàng thành công gửi tới khách hàng 61
Hình 4.30. Trang quản trị đơn hàng (Admin): lọc đơn và đổi trạng thái 62
Hình 4.31. Sơ đồ tuần tự thanh toán PayOS / VietQR với Webhook và polling 64
Hình 4.32. Màn hình mã QR thanh toán PayOS / VietQR 65
Hình 4.33. Thanh toán thành công — đơn tự chuyển trạng thái 66
Hình 4.34. Email xác nhận đơn hàng (hóa đơn HTML) 67
Hình 4.35. Sơ đồ tuần tự thông báo và duyệt đơn qua Telegram Bot 68
Hình 4.36. Tin nhắn đơn hàng mới kèm nút duyệt trên Telegram 69
Hình 4.37. Kết quả lệnh /thongke trên Telegram 69
Hình 4.38. Luồng xử lý của Trợ lý AI với cơ chế dự phòng nhiều tầng 70
Hình 4.39. Hộp chat Trợ lý AI tư vấn và gợi ý sản phẩm trên website 71
Hình 4.40. Kiểm thử POST /api/tro-ly-ai/chat bằng Postman: câu trả lời kèm sản phẩm gợi ý 71
Hình 4.41. Lưu đồ tra cứu bảo hành điện tử 72
Hình 4.42. Giao diện tra cứu bảo hành điện tử 72
Hình 4.43. Trang chủ website 73
Hình 4.44. Trang tin tức công nghệ 74
Hình 4.45. Kiểm thử GET /api/tin-tuc bằng Postman: danh sách bài viết 74
Hình 4.46. Trang liên hệ và hệ thống showroom có bản đồ 74
Hình 4.47. Trang so sánh: danh sách laptop đã chọn để so sánh 75
Hình 4.48. Bảng đối chiếu nhanh thương hiệu, giá và đánh giá 75
Hình 4.49. So sánh chi tiết bộ xử lý (CPU) giữa các máy 75
Hình 4.50. Trang tổng quan quản trị: thẻ thống kê doanh thu, đơn hàng, khách hàng, kho 76
Hình 4.51. Biểu đồ xu hướng doanh thu và cơ cấu ngành hàng 76
Hình 4.52. Kiểm thử GET / bằng Postman: thông tin server và danh sách endpoint chính 77
Hình 4.53. Dữ liệu JSON thực tế trả về từ GET /api/san-pham 77
Hình 4.54. Các collection trên MongoDB Atlas (cơ sở dữ liệu laptop_store, 04/10/2026): đủ 10 collection đang dùng, có gio_hang; thuong_hieu ở cuối danh sách là collection cũ không còn dùng 77
Hình 4.55. Trang chủ chạy trên tên miền chính thức https://trikun.site (qua Cloudflare) 78

Hình 4.56. Trang thanh toán trên trikun.site: chọn Tỉnh/Thành và Phường/Xã theo địa giới sau sáp nhập (Provinces Open API v2) 78
Hình 4.57. Trang quản trị trên trikun.site: danh sách tài khoản khách hàng kèm thống kê, bộ lọc và các thao tác khóa / sửa / xóa 79
Hình 4.58. Cơ chế tự đồng bộ giao diện: Admin thay đổi → chữ ký phiên bản → trình duyệt của khách nạp lại đúng bảng 80
Hình 4.59. Gọi thử GET /api/dong-bo trên Swagger của server thật: trả chữ ký các bảng, không cache 81
Hình 4.60. Luồng xử lý sản phẩm ẩn, hết hàng, đổi giá: từ lúc khách xem, bỏ vào giỏ đến lúc thanh toán 82
Hình 4.61. Luồng giỏ hàng theo tài khoản: hợp nhất khi đăng nhập, dùng nhiều thiết bị, đăng xuất 83
Hình 4.62. Giỏ hàng cùng một tài khoản hiển thị giống nhau trên hai trình duyệt riêng biệt (môi trường kiểm thử cục bộ, dữ liệu mẫu) 83
Hình 4.63. Thông báo giá thay đổi "cũ → mới" hiện trong giỏ sau khi Admin đổi giá (môi trường kiểm thử cục bộ, dữ liệu mẫu) 84
Hình 4.64. Gọi thử GET /api/gio-hang khi chưa đăng nhập trên Swagger của server thật: bị từ chối 401 84
Hình 4.65. Trang quản trị: danh sách Kho Sản Phẩm với bộ lọc và các nút xem / sửa / xóa 86
Hình 4.66. Trang quản trị: biểu mẫu chỉnh sửa sản phẩm (thông tin, giá bán và tồn kho) 87
Hình 4.67. Trang quản trị: danh sách thương hiệu và danh mục kèm số sản phẩm, trạng thái 87
Hình 4.68. Trang quản trị: biểu mẫu chỉnh sửa thương hiệu 88
Hình 4.69. Trang quản trị: danh sách đơn hàng với bộ lọc, nút tạo đơn, xuất Excel và đổi trạng thái 88
Hình 4.70. Trang quản trị: chi tiết đơn hàng, lộ trình giao hàng và in phiếu hóa đơn .89
Hình 4.71. Trang quản trị: danh sách bài viết kèm thống kê, bộ lọc và các nút xem / sửa / xóa 89
Hình 4.72. Trang quản trị: biểu mẫu chỉnh sửa bài viết tin tức 90
Hình 4.73. Trang quản trị: cấu hình banner Flash Sale và khuyến mãi kèm bản xem trước giao diện thật 90
Hình 4.74. Trang quản trị: cấu hình showroom, tọa độ GPS và xem trước Google Maps 91
Hình 4.75. Trang quản trị: danh sách phiếu yêu cầu tư vấn với trạng thái và các thao tác nhanh 92

Hình 4.76. Trang quản trị: chi tiết phiếu tư vấn, trạng thái xử lý và ghi chú nội bộ 92
Hình 4.77. Trang quản trị: danh sách tài khoản khách hàng kèm thống kê, bộ lọc và các thao tác khóa / sửa / xóa 93
Hình 4.78. Trang quản trị: biểu mẫu cập nhật tài khoản khách hàng 93
Hình 4.79. Trang quản trị: danh sách mã giảm giá với số lượng đã dùng, hạn dùng và công tắc bật / tắt 94
Hình 4.80. Trang quản trị: biểu mẫu sửa mã khuyến mãi 94
Hình 4.81. Tệp Excel báo cáo kế toán do chức năng Xuất Excel tạo ra: chỉ tiêu tổng hợp và chi tiết từng đơn 95
Hình 5.1. Kiểm thử lỗi 400 bằng Postman: đăng ký thiếu thông tin 97
Hình 5.2. Kiểm thử lỗi 401 bằng Postman: gọi /api/auth/toi không gửi token....98
Hình 5.3. Kiểm thử lỗi 403 bằng Postman: tài khoản khách gọi API thêm sản phẩm .99
Hình 5.4. Kiểm thử lỗi 404 bằng Postman: sản phẩm không tồn tại 100
Hình 5.5. Kiểm thử lỗi 429 bằng Postman: tra cứu đơn vượt giới hạn 101
Hình 5.6. BM01 — Xem đơn của tài khoản khác bị chặn (403, chống IDOR) 108
Hình 5.7. BM02 — Khách tự xác nhận "đã thanh toán" bị từ chối (403) 109
Hình 5.8. BM03 — Gửi vaiTro: admin khi cập nhật hồ sơ nhưng không được nâng quyền 109
Hình 5.9. BM04 — Hồ sơ người dùng không còn trả mật khẩu băm và OTP 110
Hình 5.10. BM05 — Nhập sai mật khẩu 5 lần, tài khoản bị tạm khóa đăng nhập (429) 111
Hình 5.11. BM06 — Nhập sai OTP: đếm ngược số lần còn lại và hủy mã 113
Hình 5.12. BM08 – BM10 — Admin khóa tài khoản, token cũ và đăng nhập lại đều bị từ chối 115
Hình 5.13. BM15 — Token cũ bị thu hồi sau khi đăng xuất (401) 116
Hình 5.14. TC31 — Tấn công NoSQL Injection vào API đăng nhập bị chặn 117
Hình 5.15. TC35 — HTTP Security Headers trong phản hồi của server 117
Hình 5.16. Mật khẩu (bcrypt) và OTP (HMAC-SHA256) chỉ được lưu dạng băm trong MongoDB 118
Hình 6.1. Terminal khởi chạy Back-End ở máy phát triển (npm run dev)....121
Hình 6.2. Postman Runner chạy 40 ca kiểm thử tự động (58 bài kiểm tra): 58/58 đạt, 0 lỗi 124
Hình 6.3. Kiểm thử POST /api/auth/dang-ky bằng Postman: đăng ký hợp lệ, gửi OTP 125
Hình 6.4. Kiểm thử GET /api/auth/toi với token hợp lệ: hồ sơ không chứa mật khẩu băm và OTP 125

Hình 6.5. Kiểm thử PUT /api/auth/doi-mat-khau bằng Postman 126
Hình 6.6. Kiểm thử lọc sản phẩm theo nhiều hãng và danh mục 126
Hình 6.7. Kiểm thử POST /api/san-pham bằng tài khoản Admin (201 Created).....127
Hình 6.8. Kiểm thử GET /api/danh-muc?loai=thuong_hieu 127
Hình 6.9. Kiểm thử POST /api/don-hang: tạo đơn thành công (201 Created) 128
Hình 6.10. Kiểm thử PUT /api/don-hang/:id: Admin cập nhật trạng thái đơn 129
Hình 6.11. Kiểm thử POST /api/thanh-toan/tao-link bằng token chủ đơn: tạo mã QR thanh toán 130
Hình 6.12. Kiểm thử GET /api/thanh-toan/kiem-tra/:orderCode 130
Hình 6.13. Kiểm thử Webhook PayOS với chữ ký giả mạo bị từ chối (400) 131
Hình 6.14. Kiểm thử lưu voucher và xem ví voucher của tôi.....132
Hình 6.15. Kiểm thử POST /api/danh-gia: gửi đánh giá sản phẩm 133
Hình 6.16. Kiểm thử GET /api/bao-hanh/tra-cuu: tra cứu bảo hành điện tử.....133
Hình 6.17. Kiểm thử POST /api/lien-he: gửi phiếu yêu cầu tư vấn 134
Hình 6.18. Kiểm thử GET /api/tro-ly-ai/trang-thai.....134
Hình 6.19. Kiểm thử GET /api/cai-dat/showroom 135
Hình 6.20. Kiểm thử GET /api/health: kiểm tra sức khỏe máy chủ 135
Hình 6.21. Collection dang_ky_tam sau khi đăng ký: mật khẩu và OTP đều đã băm 136
Hình 6.22. Sau khi kích hoạt OTP: tài khoản xuất hiện trong nguoi_dung, bản ghi tạm đã bị xóa.....136
Hình 6.23. Đơn hàng mới được ghi vào collection don_hang.....137
Hình 6.24. Tồn kho sản phẩm TRƯỚC khi Admin chuyển đơn sang "đã giao" 137
Hình 6.25. Tồn kho SAU khi đơn "đã giao": giảm đúng số lượng, số đã bán tăng ...138
Hình 6.26. Tồn kho được hoàn lại sau khi Admin hủy đơn 139
Hình 6.27. Lưu voucher: số lượt còn lại giảm và mã được thêm vào ví người dùng.140
Hình 6.28. Đánh giá mới và điểm trung bình sản phẩm được cập nhật 141
Hình 6.29. Tài khoản bị Admin khóa: biKhoa = true và tokenVersion tăng.....143
Hình 6.30. Phiếu yêu cầu tư vấn mới trong collection lien_he 143
Hình 6.31. Mô hình triển khai production: Cloudflare – Railway – MongoDB Atlas và các dịch vụ bên thứ ba 144
Hình 6.32. Service BE trên Railway: bản deploy mới nhất từ GitHub ở trạng thái ACTIVE – Deployment successful 145
Hình 6.33. Bản ghi DNS trên Cloudflare: hai CNAME ở chế độ Proxied và hai TXT xác minh 146

Hình 6.34. Railway gắn tên miền api.trikun.site cho service BE và nhận diện Cloudflare proxy.....146
Hình 6.35. SSL/TLS ở chế độ Full: mã hóa cả chặng Browser → Cloudflare và Cloudflare → Origin Server.....147
Hình 6.36. Luật Rate limiting chong-do-mat-khau đang hoạt động cho /api/auth/*: ghi nhận 19 request bị chặn trong lần kiểm thử.....148
Hình 6.37. Chạy bộ kiểm thử Jest: 11 tệp, 103/103 ca đạt, kèm tóm tắt độ phủ mã nguồn.....151
Hình 6.38. Lịch sử các lần chạy CI trên nhánh main: lần đầu bị Cloudflare chặn ở bước smoke test (đỏ), sau khi sửa các lần tiếp theo đều đạt, kể cả lần thêm ESLint, Prettier và npm audit.....153
Hình 6.39. Các lần chạy CI gần nhất trên nhánh main (tổng 20 lần chạy) đều đạt, gồm các commit giỏ hàng theo tài khoản và đồng bộ giá, tồn kho.....153
Hình 6.40. Chi tiết một lần chạy CI: cả 3 job Back-End, Front-End và Production smoke test đều đạt.....154
Hình 6.41. Chạy ESLint, kiểm tra định dạng Prettier và npm audit: không còn lỗi, không còn lỗ hổng.....155

DANH MỤC BẢNG

Bảng 1.1. Các nhóm đối tượng sử dụng hệ thống....3
Bảng 1.2. Công nghệ và thư viện sử dụng ở Back-End....3
Bảng 1.3. Lý do lựa chọn các công nghệ chính....12
Bảng 2.1. Vai trò các tầng trong server Express....14
Bảng 2.2. Danh sách collection trong cơ sở dữ liệu....18
Bảng 2.3. Cấu trúc collection san_pham....19
Bảng 2.4. Cấu trúc collection don_hang....20
Bảng 2.5. Cấu trúc collection nguoi_dung....21
Bảng 2.6. Cấu trúc collection ma_giam_gia....22
Bảng 2.7. Cấu trúc collection danh_gia....23
Bảng 2.8. Cấu trúc các collection danh-muc, tin_tuc, lien_he, cai_dat....23
Bảng 2.9. Các chỉ mục chính trong cơ sở dữ liệu....24
Bảng 3.1. Quy ước chung của API....25
Bảng 3.2. Ký hiệu mức quyền dùng trong các bảng đặc tả....25
Bảng 3.3. Thống kê số route theo nhóm chức năng....26
Bảng 3.4. Đặc tả API nhóm /api/auth....27
Bảng 3.5. Đặc tả API nhóm /api/san-pham....28
Bảng 3.6. Giá trị của tham số sap_xep....28
Bảng 3.7. Đặc tả API nhóm /api/danh-muc....29
Bảng 3.8. Đặc tả API nhóm /api/don-hang....29
Bảng 3.9. Đặc tả API nhóm /api/thanh-toan....30
Bảng 3.10. Đặc tả API nhóm /api/ma-giam-gia....31
Bảng 3.11. Đặc tả API nhóm /api/danh-gia....31
Bảng 3.12. Đặc tả API nhóm /api/nguoi-dung....31
Bảng 3.13. Đặc tả API nhóm /api/tin-tuc....32
Bảng 3.14. Đặc tả API nhóm /api/bao-hanh....32
Bảng 3.15. Đặc tả API nhóm /api/lien-he....32
Bảng 3.16. Đặc tả API nhóm /api/cai-dat....33
Bảng 3.17. Đặc tả API nhóm /api/tro-ly-ai....33
Bảng 3.18. Đặc tả API giám sát máy chủ....33
Bảng 3.19. Nội dung tài liệu API trên Swagger UI....34
Bảng 4.1. Các chức năng bổ trợ khác....73
Bảng 4.2. Các lỗi đồng bộ dữ liệu giữa Admin và khách, và cách khắc phục....79

Bảng 4.3. Thời gian phía khách tự cập nhật sau khi Admin thay đổi.....85
Bảng 5.1. Các mã trạng thái HTTP được sử dụng.....96
Bảng 5.2. Các lớp bảo vệ một request theo thứ tự xử lý.....102
Bảng 5.3. Đối chiếu hệ thống LaptopNew với OWASP API Security Top 10.....102
Bảng 5.4. Các lỗ hổng phát hiện khi rà soát và cách đã khắc phục.....103
Bảng 5.5. Hạn mức giới hạn tần suất đang áp dụng.....107
Bảng 5.6. Kịch bản kiểm thử bảo mật (thư mục 16 của collection Postman).....107
Bảng 6.1. Danh sách biến môi trường của Back-End.....119
Bảng 6.2. Kết quả kiểm thử API thực tế.....121
Bảng 6.3. Tổng hợp minh chứng kiểm thử Postman theo nhóm API.....124
Bảng 6.4. Các minh chứng dữ liệu MongoDB trước / sau thao tác.....135
Bảng 6.5. Bản ghi DNS của tên miền trikun.site trên Cloudflare.....145
Bảng 6.6. Cấu hình bảo vệ đã bật trên Cloudflare.....146
Bảng 6.7. Kết quả kiểm chứng trên môi trường production.....148
Bảng 6.8. Bộ kiểm thử tự động Back-End (Jest + Supertest).....150
Bảng 6.9. Các job trong quy trình CI.....152
Bảng 6.10. Kết quả quét chất lượng mã nguồn và lỗ hổng thư viện.....154

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Laptop là mặt hàng có nhiều biến thể cấu hình (CPU, RAM, ổ cứng, card đồ họa, màn hình), giá trị cao và thay đổi nhanh theo từng thế hệ. Một website bán laptop vì vậy phải giải quyết nhiều bài toán Back-End điển hình:

- **Dữ liệu sản phẩm phức tạp:** mỗi máy có hàng chục thông số, nhiều phiên bản, quà tặng, tồn kho theo từng chi nhánh — rất phù hợp để khai thác mô hình document của MongoDB.
- **Xác thực và phân quyền:** khách hàng và quản trị viên có quyền khác nhau; tài khoản mới cần xác minh email để tránh đăng ký rác.
- **Đơn hàng và tồn kho:** trạng thái đơn thay đổi nhiều bước, tồn kho phải trừ và hoàn trả chính xác.
- **Thanh toán trực tuyến:** chuyển khoản thủ công dễ nhầm lẫn; cần đối soát tự động qua cổng thanh toán.
- **Tích hợp dịch vụ ngoài:** email, bot thông báo, trí tuệ nhân tạo — đòi hỏi kỹ năng gọi API bên thứ ba và xử lý lỗi, dự phòng.

Đề tài vừa bám sát nội dung môn học (Node.js, Express, MongoDB, JWT, Nodemailer), vừa đủ độ phức tạp để nhóm rèn luyện kỹ năng thiết kế một hệ thống Back-End hoàn chỉnh như trong thực tế.

1.2. Mục tiêu dự án

1.2.1. Các chức năng cơ bản

- **Đăng ký tài khoản** có xác thực mã OTP gửi qua email; **đăng nhập** bằng email/mật khẩu hoặc **Google**; **đăng xuất**.
- **Quên mật khẩu, đổi mật khẩu** bằng mã OTP; xem và cập nhật hồ sơ cá nhân, sổ địa chỉ.
- **Quản lý sản phẩm:** xem danh sách, tìm kiếm, lọc theo hãng/danh mục/khoảng giá, sắp xếp; xem chi tiết; admin thêm, sửa, xóa.
- **Quản lý danh mục, thương hiệu, tin tức, showroom** phục vụ giao diện.

- **Mã giảm giá:** kiểm tra và tính số tiền giảm; lưu voucher vào ví cá nhân; admin quản lý voucher.
- **Đặt hàng:** tạo đơn, xem lịch sử, tra cứu đơn, hủy đơn; admin cập nhật trạng thái, hệ thống tự trừ/hoàn tồn kho.
- **Thanh toán:** tạo mã QR PayOS/VietQR, kiểm tra trạng thái, nhận Webhook xác nhận tiền về.
- **Đánh giá sản phẩm**, **tra cứu bảo hành**, **gửi liên hệ tư vấn**, **chat với Trợ lý AI**.
- **Quản trị người dùng** và **nhận thông báo, duyệt đơn qua Telegram Bot**.

1.2.2. Mục đích phát triển ứng dụng Back-End

- **Cung cấp API RESTful:** 15 nhóm tài nguyên với 74 route để Front-End Next.js gọi tới.
- **Xử lý nghiệp vụ:** kiểm tra dữ liệu, tính tiền giảm giá, quản lý trạng thái đơn, tồn kho, thời hạn bảo hành.
- **Lưu trữ dữ liệu:** thiết kế 10 collection chính trên MongoDB Atlas và 1 collection tạm cho đăng ký.
- Bảo mật nhiều lớp (Defense in Depth): băm mật khẩu bcrypt, JWT có thu hồi phiên, phân quyền RBAC, chống NoSQL Injection, giới hạn tần suất, khóa đăng nhập khi bị dò mật khẩu, OTP được băm và giới hạn số lần nhập, chữ ký Webhook, HTTP security headers — đối chiếu đầy đủ theo OWASP API Security Top 10.
- **Tích hợp:** kết nối Brevo/Gmail, PayOS, VietQR, Telegram, Google OAuth, Google Gemini, Provinces Open API v2 (địa giới hành chính sau sáp nhập).
- **Triển khai thực tế:** chạy trên Railway với tên miền riêng trikun.site, đặt sau Cloudflare (HTTPS, chống bot, rate limit lớp biên); công bố tài liệu API bằng Swagger/OpenAPI 3.0.

1.3. Phạm vi và đối tượng sử dụng

Phạm vi đồ án tập trung vào **phần Back-End** (thư mục `BE/`). Phần Front-End Next.js (thư mục `FE/`) được dùng để minh họa kết quả và kiểm thử luồng nghiệp vụ. Hệ thống phục vụ bốn nhóm đối tượng:

***Bảng 1.1.** Các nhóm đối tượng sử dụng hệ thống*

Đối tượng	Cách nhận diện ở Back-End	Quyền chính
Khách vãng lai	Không gửi token	Xem sản phẩm, tin tức, đánh giá; đăng ký, đăng nhập; tra cứu bảo hành; gửi liên hệ; chat AI
Thành viên	Token JWT hợp lệ, `vaiTro = khach_hang`	Mọi quyền của khách + đặt hàng, thanh toán, xem đơn của mình, ví voucher, hồ sơ
Quản trị viên	Token JWT hợp lệ, `vaiTro = admin`	Mọi quyền của thành viên + CRUD sản phẩm, danh mục, voucher, người dùng; xem và xử lý toàn bộ đơn
Hệ thống ngoài	Webhook có chữ ký / Bot token	PayOS báo thanh toán; Telegram gửi thao tác duyệt đơn của admin

1.4. Công nghệ sử dụng

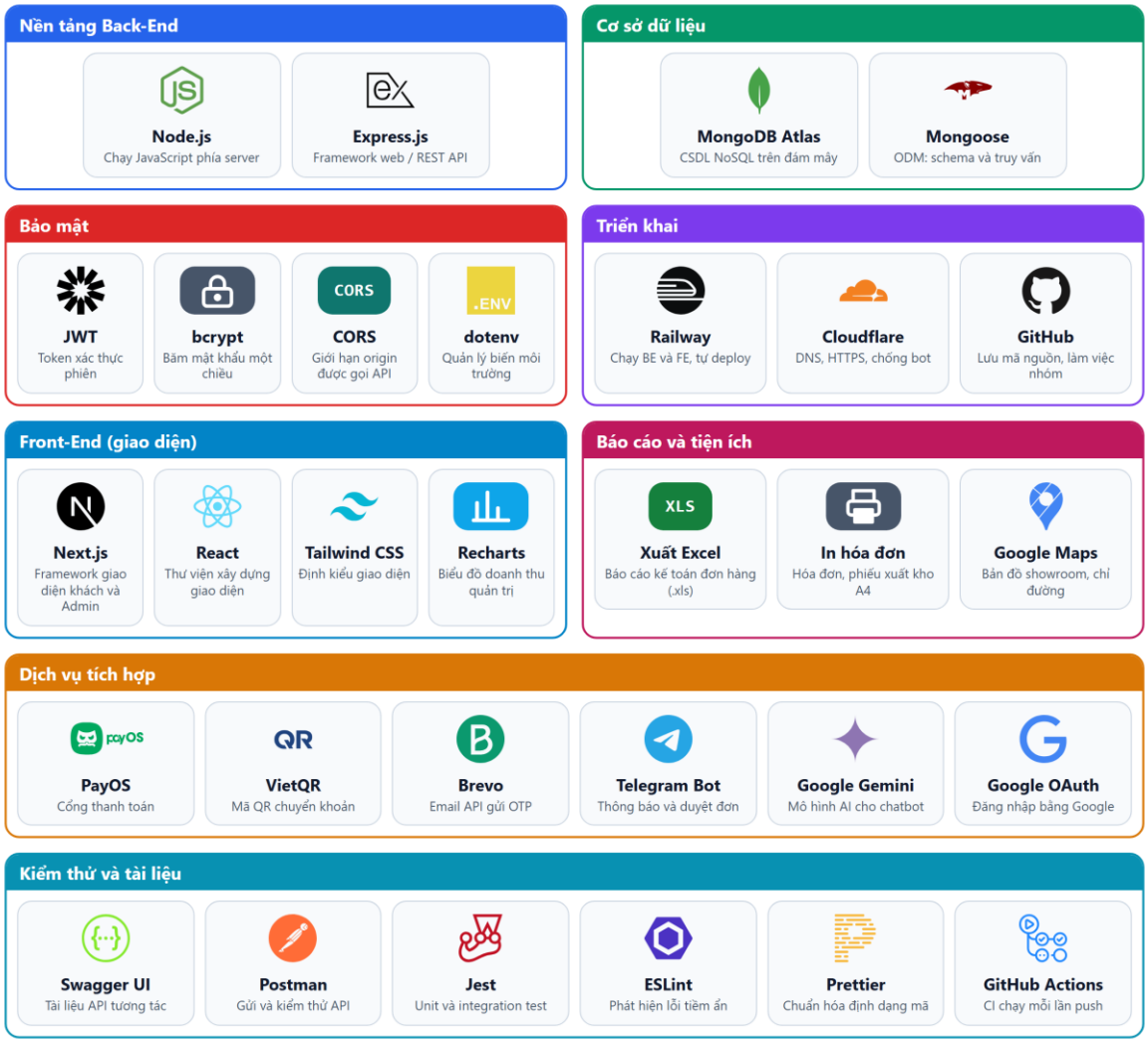
Bảng dưới đây tổng hợp các thư viện chính khai báo trong `BE/package.json` và các dịch vụ được tích hợp.

***Bảng 1.2.** Công nghệ và thư viện sử dụng ở Back-End*

Thành phần	Phiên bản	Vai trò trong hệ thống
Node.js	v24	Môi trường chạy JavaScript phía server
express	^5.2.1	Framework xây dựng REST API, router, middleware
mongoose	^9.9.4	ODM: định nghĩa Schema, kiểm tra dữ liệu, truy vấn MongoDB
MongoDB Atlas	Cloud	Cơ sở dữ liệu NoSQL lưu toàn bộ dữ liệu (database `laptop_store`)
jsonwebtoken	^9.0.3	Tạo và xác minh JWT (hạn 7 ngày)
bcryptjs	^3.0.3	Băm mật khẩu với salt 10 vòng
nodemailer	^10.0.9	Gửi email qua Gmail SMTP khi chạy local
Brevo REST API v3	—	Gửi email khi triển khai cloud (SMTP bị chặn)
google-auth-library	^11.0.2	Xác minh Google ID Token khi đăng nhập Google
cors	^2.8.6	Chỉ cho phép domain Front-End khai báo trong `CLIENT_URL`
dotenv	^17.4.2	Nạp biến môi trường từ tệp `.env`
nodemon	^3.1.14 (dev)	Tự khởi động lại server khi sửa mã nguồn
PayOS + VietQR	REST API	Tạo link/QR thanh toán, Webhook đối soát
Telegram Bot API	REST API	Thông báo đơn mới, nút duyệt đơn, lệnh thống kê

Thành phần	Phiên bản	Vai trò trong hệ thống
Google Gemini / 9Router	REST API	Mô hình ngôn ngữ cho Trợ lý AI tư vấn laptop
swagger-ui-express	5.x · OAS 3.0.3	Phục vụ tài liệu API tương tác tại /api-docs
Railway	PaaS	Chạy 2 service BE và FE; tự build và deploy mỗi lần push GitHub
Cloudflare	Gói Free	DNS, HTTPS, CDN, Bot Fight Mode, rate limit lớp biên
Provinces Open API	v2 · 34 tỉnh, 2 cấp	Địa giới sau sáp nhập 2025: Tỉnh → Phường/Xã cho địa chỉ giao hàng
Jest + Supertest	29.x · 7.x	Kiểm thử tự động bằng mã: unit test và integration test gọi API
mongodb-memory-server	10.x	MongoDB chạy trong RAM cho integration test, không đụng dữ liệu thật
GitHub Actions	CI	Tự chạy kiểm thử, lint, build mỗi lần push / Pull Request
ESLint + Prettier	9.x · 3.x	Phân tích tĩnh phát hiện lỗi tiềm ẩn và chuẩn hóa định dạng mã nguồn Back-End
Next.js + React	16.3 · 19.2	Giao diện website và trang quản trị (App Router)
Tailwind CSS	4.3	Định kiểu giao diện
Recharts	3.10	Biểu đồ doanh thu và cơ cấu ngành hàng trong trang quản trị
Xuất Excel · In hóa đơn	Không dùng thư viện	Xuất báo cáo `.xls` cho đơn hàng; in hóa đơn, phiếu xuất kho khổ A4
Google Maps (nhúng)	iframe	Bản đồ showroom và chỉ đường ở trang Liên hệ

Hình 1.1 tóm tắt toàn bộ công nghệ của hệ thống theo từng lớp (nền tảng, dữ liệu, bảo mật, giao diện, báo cáo và tiện ích, dịch vụ tích hợp, triển khai, kiểm thử); các mục từ 1.4.1 đến 1.4.16 giới thiệu cơ bản từng công nghệ, mỗi mục kèm logo để dễ nhận biết. Các logo thuộc quyền sở hữu của chủ sở hữu thương hiệu tương ứng và chỉ được dùng với mục đích minh họa học tập; những công nghệ không có logo chính thức (bcrypt, CORS, 9Router, Recharts, Xuất Excel, In hóa đơn) được thể hiện bằng huy hiệu chữ.



Hình 1.1. *Bộ công nghệ sử dụng trong hệ thống LaptopNew, chia theo từng lớp*

1.4.1. Node.js

Node.js là môi trường thực thi JavaScript mã nguồn mở chạy trên V8 Engine của Google Chrome. Node.js hoạt động theo mô hình **hướng sự kiện, vào/ra không chặn (non-blocking I/O)**: khi chờ truy vấn cơ sở dữ liệu hoặc gọi API bên ngoài, luồng chính vẫn tiếp tục phục vụ yêu cầu khác. Điều này rất phù hợp với hệ thống của nhóm, vốn phải gọi nhiều dịch vụ ngoài (PayOS, Telegram, Gemini) cùng lúc.

Hình 1.2. *Node.js: môi trường chạy JavaScript phía server*

Công cụ hỗ trợ — Nodemon: tự động khởi động lại server mỗi khi mã nguồn thay đổi (lệnh `npm run dev`), giúp tăng tốc quá trình phát triển.

1.4.2. Express.js

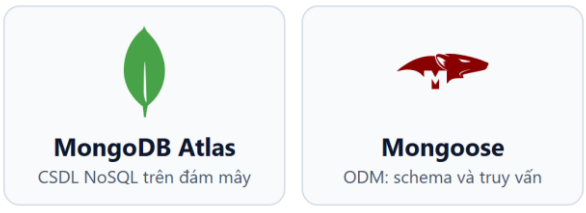
Express là framework web tối giản và phổ biến nhất của Node.js. Dự án dùng **Express 5**, phiên bản tự động chuyển lỗi của hàm `async` sang middleware xử lý lỗi. Các tính năng được khai thác:

Hình 1.3. *Express.js: framework web để xây dựng REST API*

- `express.Router()` để tách 15 nhóm route thành 15 tệp riêng.
- Chuỗi **middleware** (CORS → parse JSON → log → xác thực → phân quyền) trước khi vào controller.
- Middleware xử lý lỗi 404 và 500 tập trung ở cuối `app.js`.

1.4.3. MongoDB Atlas và Mongoose

MongoDB là cơ sở dữ liệu NoSQL hướng tài liệu: dữ liệu được lưu thành các **document** dạng BSON (tương tự JSON) trong các **collection**. Nhờ đó, thông số kỹ thuật, danh sách phiên bản, quà tặng, tồn kho chi nhánh của một laptop có thể nằm gọn trong một document mà không cần nhiều bảng và phép JOIN như SQL. Dự án dùng **MongoDB Atlas** — dịch vụ MongoDB trên đám mây, có sẵn Replica Set sao lưu dữ liệu.



Hình 1.4. *MongoDB Atlas (cơ sở dữ liệu) và Mongoose (ODM)*

Mongoose là thư viện ODM giúp định nghĩa Schema (kiểu dữ liệu, bắt buộc, duy nhất, giá trị mặc định, enum), tạo index, viết hook `pre('validate')` để tự sinh slug và thêm phương thức cho model (ví dụ `soSanhMatKhau`).

1.4.4. JSON Web Token (JWT)

JWT là tiêu chuẩn mở (RFC 7519) để truyền thông tin xác thực dưới dạng chuỗi đã ký số gồm ba phần *Header.Payload.Signature*. Sau khi đăng nhập, server ký token chứa `userId`, `email`, `vaiTro` bằng khóa `JWT_SECRET`, thời hạn 7 ngày. Client gửi token ở header `Authorization: Bearer <token>`; server chỉ cần xác minh chữ ký mà không phải lưu phiên (stateless).

Hình 1.5. *JSON Web Token: xác thực không cần lưu phiên trên server*

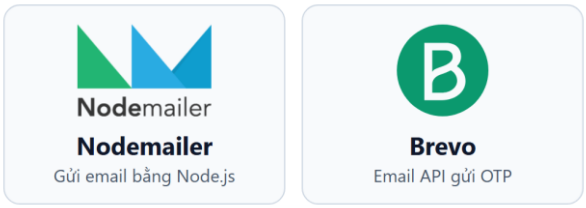
1.4.5. Bcrypt

Bcrypt là hàm băm mật khẩu được thiết kế **chậm có chủ đích** và có **salt ngẫu nhiên**, khiến tấn công dò mật khẩu (brute-force) và bảng tra sẵn (rainbow table) trở nên rất tốn kém. Hệ thống dùng `bcrypt.genSalt(10)` và `bcrypt.hash()` khi lưu, `bcrypt.compare()` khi đăng nhập; mật khẩu gốc không bao giờ được lưu.

Hình 1.6. *bcrypt: băm mật khẩu một chiều có salt*

1.4.6. Nodemailer và Brevo Email API

Nodemailer là module gửi email phổ biến của Node.js, hỗ trợ nội dung HTML. Tuy nhiên, nhiều nền tảng cloud (như Railway) chặn cổng SMTP, nên dịch vụ email của nhóm được thiết kế **hai tầng**: ưu tiên gọi **Brevo REST API v3** qua HTTPS; nếu chưa cấu hình Brevo và đang chạy local thì dùng Nodemailer với Gmail. Hệ thống gửi bốn loại email: OTP kích hoạt, OTP quên mật khẩu, OTP đổi mật khẩu và hóa đơn xác nhận đơn hàng.



Hình 1.7. *Nodemailer và Brevo Email API: gửi email OTP và xác nhận đơn hàng*

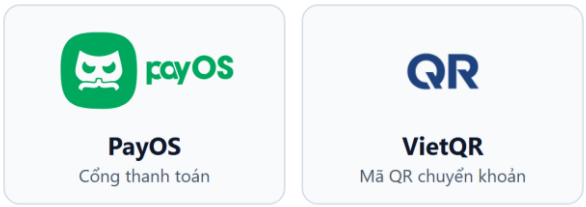
1.4.7. Dotenv và CORS

Dotenv nạp cấu hình nhạy cảm (chuỗi kết nối MongoDB, khóa JWT, khóa PayOS, token Telegram, khóa Gemini) từ tệp `.env` vào `process.env`, giúp tách bí mật khỏi mã nguồn và cấu hình khác nhau cho từng môi trường. **CORS** được cấu hình chỉ chấp nhận các origin liệt kê trong `CLIENT_URL`.

Hình 1.8. *dotenv (biến môi trường) và CORS (giới hạn origin được gọi API)*

1.4.8. PayOS và VietQR

PayOS là cổng thanh toán cho phép tạo yêu cầu thanh toán, sinh mã QR chuyển khoản có sẵn số tiền và nội dung, sau đó gọi **Webhook** về server khi tiền vào tài khoản. Mọi yêu cầu và Webhook đều được ký bằng **HMAC-SHA256** với `checksumKey`. Song song, hệ thống dựng sẵn ảnh **VietQR** (chuẩn Napas) để khách luôn có QR, kể cả khi PayOS phản hồi chậm.



Hình 1.9. *PayOS và VietQR: thanh toán chuyển khoản bằng mã QR*

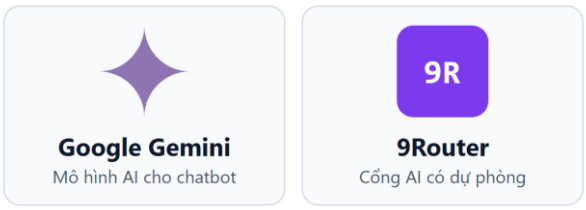
1.4.9. Telegram Bot API

Telegram Bot API cho phép server gửi tin nhắn có **bàn phím nút bấm (Inline Keyboard)** tới quản trị viên và nhận lại thao tác bằng cơ chế **Long Polling** (`getUpdates`). Nhờ đó admin có thể xác nhận, giao hàng hoặc hủy đơn ngay trên điện thoại, và dùng các lệnh `/thongke`, `/donmoi`, `/tatcadon`, `/timdon`.

Hình 1.10. *Telegram Bot API: thông báo và duyệt đơn hàng*

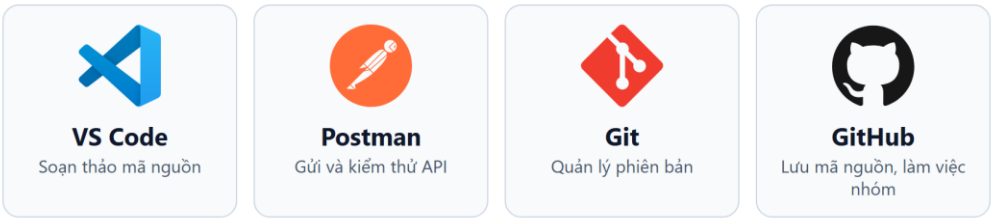
1.4.10. Google Gemini và 9Router Gateway

Trợ lý AI dùng mô hình ngôn ngữ lớn để hiểu câu hỏi tiếng Việt và tư vấn laptop dựa trên **dữ liệu sản phẩm thật** trong MongoDB. Yêu cầu được gửi trước tới cổng trung gian **9Router** (chuẩn OpenAI `/chat/completions`, hỗ trợ stream); nếu lỗi sẽ chuyển sang gọi trực tiếp **Google Gemini** với cơ chế **xoay vòng nhiều API key**, và cuối cùng là câu trả lời dự phòng.



Hình 1.11. *Google Gemini và 9Router: trợ lý AI tư vấn laptop*

1.4.11. Công cụ phát triển



Hình 1.12. *Công cụ phát triển: Visual Studio Code, Postman, Git và GitHub*

- **Visual Studio Code:** soạn thảo mã nguồn.
- **Postman:** gửi request và kiểm thử API.
- **MongoDB Compass / Atlas Web UI:** xem và kiểm tra dữ liệu.
- **Git & GitHub:** quản lý phiên bản mã nguồn và làm việc nhóm.
- **Swagger UI, Newman, Railway CLI:** tài liệu API tương tác, chạy collection Postman bằng dòng lệnh và vận hành production.

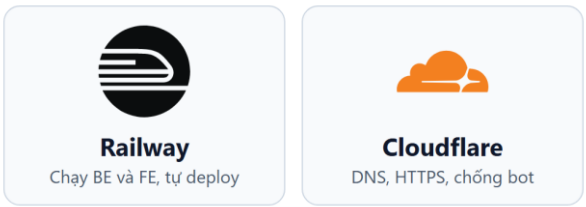
1.4.12. Swagger UI và OpenAPI 3.0

OpenAPI là chuẩn mô tả REST API ở dạng máy đọc được. Swagger UI biến bản đặc tả đó thành trang tài liệu có thể gọi thử trực tiếp (Try it out), kèm schema dữ liệu và ví dụ phản hồi. Nhóm dùng thư viện `swagger-ui-express` để phục vụ tài liệu ngay trên server tại `/api-docs` (mục 3.4).

Hình 1.13. *Swagger UI và OpenAPI 3.0: tài liệu API tương tác*

1.4.13. Railway, Cloudflare và tên miền riêng

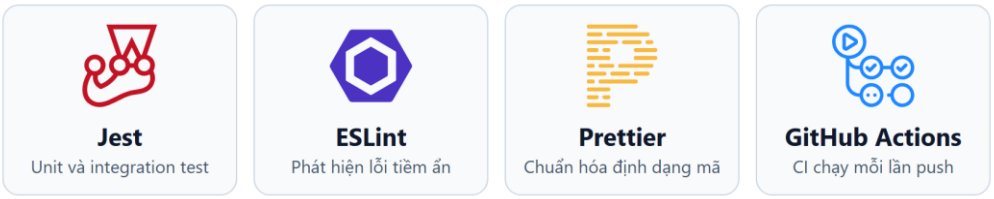
Railway là nền tảng PaaS chạy hai service BE và FE, tự động build và deploy mỗi lần mã nguồn được push lên GitHub. Cloudflare quản lý DNS cho tên miền `trikun.site` và làm proxy đứng trước Railway: cung cấp HTTPS, CDN, Bot Fight Mode và luật giới hạn tần suất ở lớp biên (mục 6.7).



Hình 1.14. *Railway (chạy ứng dụng) và Cloudflare (DNS, HTTPS, chống bot)*

1.4.14. Jest, Supertest, ESLint và GitHub Actions

Jest là framework kiểm thử JavaScript: chạy các tệp `*.test.js` và so sánh kết quả thực tế với kết quả mong đợi. Supertest gửi request HTTP thẳng vào ứng dụng Express trong bộ nhớ, không cần mở cổng mạng. GitHub Actions là dịch vụ CI của GitHub: mỗi lần mã nguồn được push, GitHub cấp một máy ảo để chạy toàn bộ kiểm thử và báo kết quả đạt / không đạt ngay trên commit. ESLint phân tích tĩnh mã nguồn để phát hiện lỗi tiềm ẩn, còn Prettier tự định dạng mã theo một chuẩn chung cho cả nhóm (mục 6.8).



Hình 1.15. *Jest, ESLint, Prettier và GitHub Actions: kiểm thử, chất lượng mã và CI*

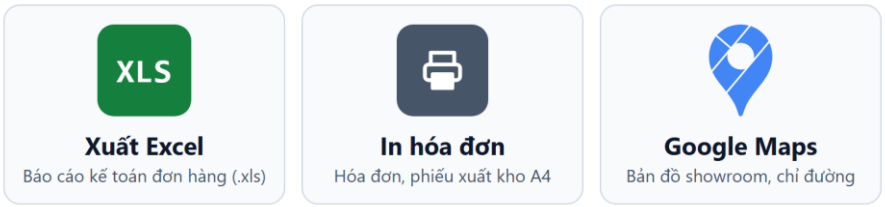
1.4.15. Next.js, React, Tailwind CSS và Recharts (Front-End)

Giao diện web viết bằng **Next.js 16** (App Router) trên **React 19**, định kiểu bằng **Tailwind CSS 4**. Trang quản trị dùng **Recharts 3** để vẽ biểu đồ xu hướng doanh thu và cơ cấu ngành hàng (Hình 4.51). Front-End gọi API qua một lớp tiện ích chung `apiFetch` tự gắn JWT và xử lý lỗi, nên mọi trang dùng cùng một cách xác thực với Back-End.

Hình 1.16. *Next.js, React, Tailwind CSS và Recharts: xây dựng giao diện web*

1.4.16. Xuất Excel, in hóa đơn và Google Maps

Ba tiện ích phục vụ vận hành cửa hàng, đều làm trực tiếp ở Front-End mà không cần thư viện ngoài. **Xuất Excel:** trang quản trị có nút xuất báo cáo kế toán cho danh sách đơn hàng đang lọc; Front-End dựng bảng định dạng Excel, lưu thành tệp `.xls` mã hóa UTF-8 có BOM để giữ đúng tiếng Việt rồi tải về bằng `Blob` (tệp `Bao_Cao_Ke_Toan_Don_Hang_<ngày>.xls`). **In hóa đơn:** tệp `utils/inHoaDon.js` tạo hóa đơn bán hàng và phiếu xuất kho khổ A4 trên một trang, đọc số tiền thành chữ tiếng Việt và in qua iframe ẩn. **Google Maps:** trang Liên hệ nhúng bản đồ để hiển thị showroom (tọa độ lấy từ collection cài đặt) và chỉ đường từ vị trí của khách.



Hình 1.17. *Xuất Excel, in hóa đơn và Google Maps: tiện ích báo cáo và bản đồ*

1.5. Lý do chọn công nghệ

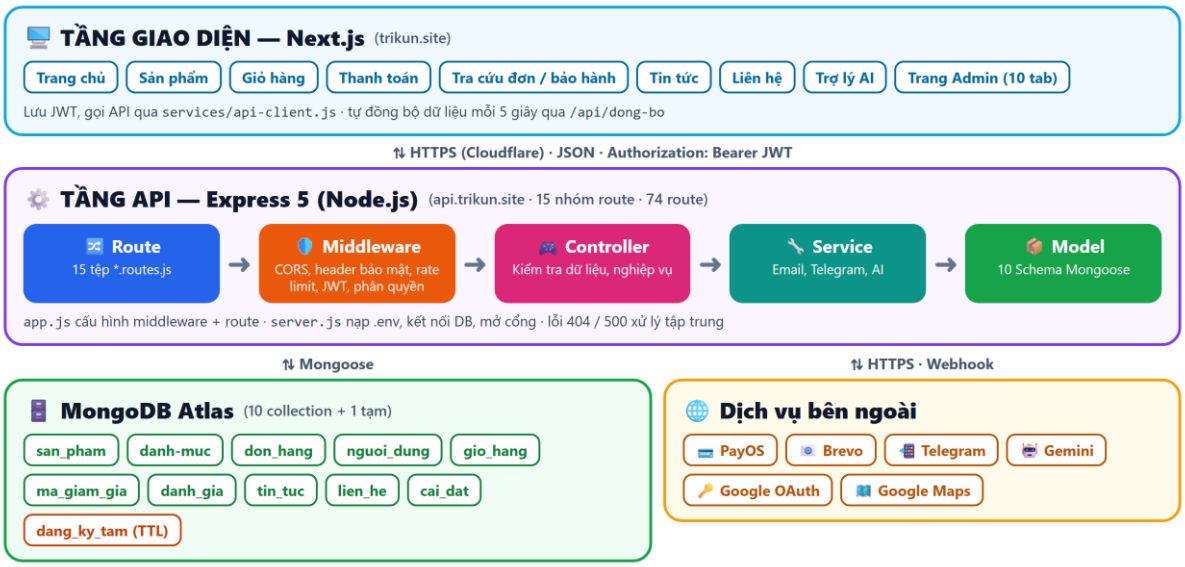
Bảng 1.3. *Lý do lựa chọn các công nghệ chính*

Công nghệ	Lý do lựa chọn
Node.js	Dùng chung JavaScript cho cả Front-End và Back-End; xử lý bất đồng bộ tốt cho ứng dụng nhiều I/O; hệ sinh thái npm rất lớn.
Express	Đơn giản, linh hoạt, cộng đồng lớn; cơ chế middleware giúp tách xác thực, phân quyền, xử lý lỗi rõ ràng.
MongoDB	Schema linh hoạt, phù hợp dữ liệu laptop nhiều thông số lồng nhau; dễ mở rộng; Atlas miễn phí cho môi trường học tập.
Mongoose	Ràng buộc dữ liệu ngay ở tầng ứng dụng, hỗ trợ hook, index, phương thức model.
JWT	Xác thực không lưu phiên, dễ dùng cho SPA/Next.js, mang theo vai trò để phân quyền.
Bcrypt	Chuẩn công nghiệp cho lưu mật khẩu, có salt và độ trễ điều chỉnh được.
Nodemailer / Brevo	Gửi email OTP và hóa đơn; Brevo khắc phục việc cloud chặn SMTP.
PayOS	Đối soát chuyển khoản tự động qua Webhook, không cần kiểm tra thủ công.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Kiến trúc ứng dụng

Hệ thống được tổ chức theo mô hình **Client – Server** ba tầng: tầng giao diện Next.js, tầng API Express và tầng dữ liệu MongoDB Atlas, cùng với các dịch vụ bên thứ ba. Bên trong server, mã nguồn được chia theo **kiến trúc phân tầng** (biến thể của MVC cho Web API, trong đó "View" chính là dữ liệu JSON trả về).



Hình 2.1. *Sơ đồ kiến trúc tổng thể hệ thống LaptopNew*

Ở môi trường production, cả Client và Server đều chạy trên Railway dưới tên miền riêng và được đặt sau Cloudflare; mô hình triển khai chi tiết được trình bày ở mục 6.7 (Hình 6.31).

2.1.1. Client (Front-End)

- Là nơi người dùng tương tác: xem sản phẩm, đặt hàng, đăng nhập, quản trị.
- Giao tiếp với server qua các endpoint `/api/...`; token JWT được lưu ở client và gắn vào header `Authorization` bởi `services/api-client.js`.
- Dữ liệu trả về ở định dạng JSON để hiển thị.

2.1.2. Server (Back-End)

Server Express nhận request, xử lý nghiệp vụ và trả JSON. Mã nguồn được chia thành các tầng có trách nhiệm riêng:

Bảng 2.1. *Vai trò các tầng trong server Express*

Tầng	Thư mục	Trách nhiệm
Routes	`routes/`	Khai báo URL + phương thức HTTP, gắn middleware và hàm controller tương ứng.
Middleware	`middleware/`	Xử lý trước controller: giải mã JWT, kiểm tra quyền admin, nhận diện người dùng tùy chọn.
Controllers	`controllers/`	Đọc tham số, kiểm tra dữ liệu, thực hiện nghiệp vụ, trả mã HTTP và JSON.
Services	`services/`	Đóng gói việc gọi dịch vụ ngoài: email, Telegram, AI; controller không cần biết chi tiết.
Models	`models/`	Định nghĩa Schema Mongoose, ràng buộc dữ liệu và truy cập MongoDB.
Config / Utils	`config/`, `utils/`	Kết nối CSDL, cấu hình AI; hàm tiện ích như tạo slug tiếng Việt.

2.1.3. Database (MongoDB Atlas)

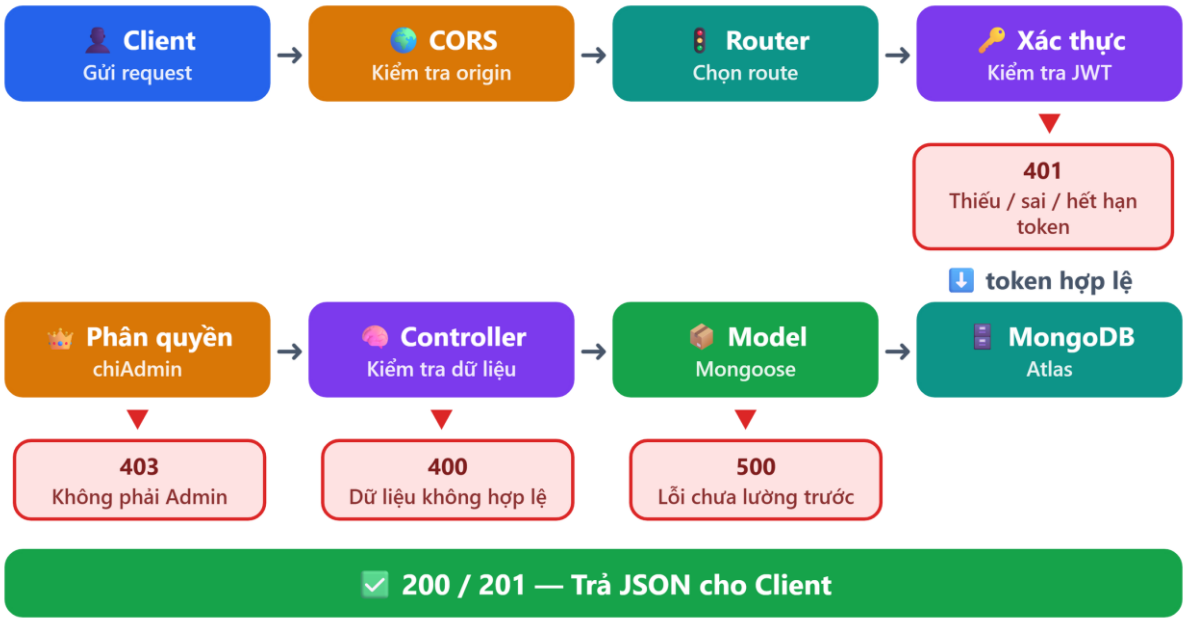
- Lưu toàn bộ dữ liệu: sản phẩm, đơn hàng, người dùng, voucher, đánh giá, danh mục, tin tức, liên hệ, cấu hình showroom.
- Hỗ trợ đầy đủ thao tác CRUD thông qua Mongoose; kết nối bằng biến `MONGODB_URI` với thời gian chờ chọn server 6 giây.

2.1.4. Middleware

- **Toàn cục:** `cors` (kiểm tra origin), `express.json()` giới hạn 1 MB, `datHeaderBaoMat` (HTTP security headers), `chongNoSqlInjection` (lọc khóa `$` và `.`), `gioiHanRequest` (300 request/phút/IP), middleware ghi log `[giờ] METHOD URL`.
- **Xác thực:** `xacThucToken` bắt buộc JWT hợp lệ, còn hiệu lực và chưa bị thu hồi (`tokenVersion`); `nhanDienNguoiDungTuyChon` / `nhanDienNguoiDungMem` gắn `req.user` nếu có token nhưng không chặn khách.
- **Phân quyền:** `chiAdmin` chỉ cho `vaiTro = admin` đi tiếp.
- **Xử lý lỗi:** middleware 404 cho route không tồn tại và middleware 4 tham số `(err, req, res, next)` trả lỗi 500.

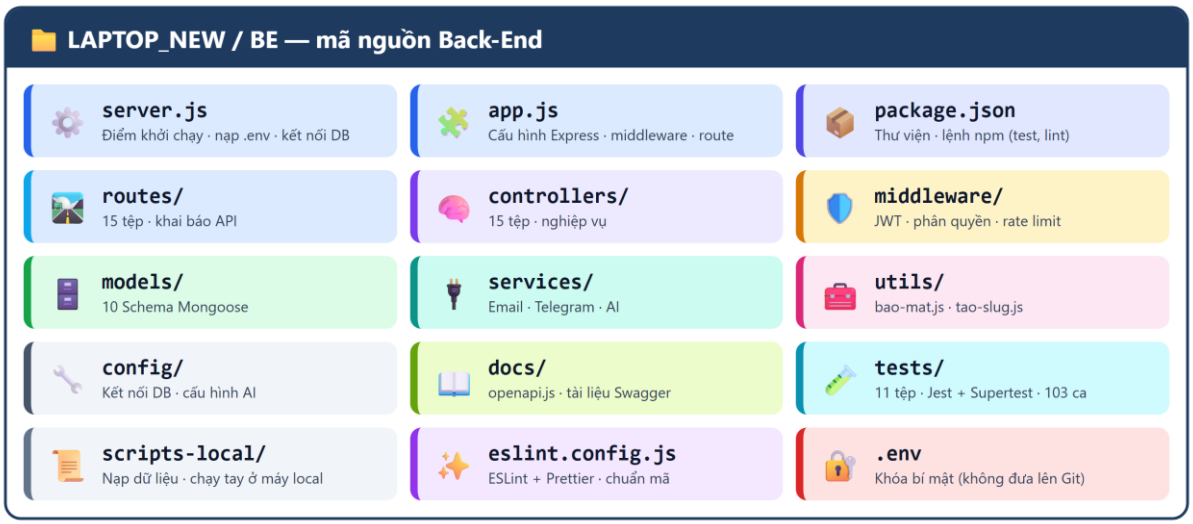
2.1.5. Vòng đời một request

Sơ đồ tuần tự sau mô tả đường đi của một request tới route cần bảo vệ và các nhánh lỗi có thể xảy ra ở từng tầng.



Hình 2.2. *Vòng đời xử lý một request qua các tầng của server*

2.2. Cấu trúc thư mục



Hình 2.3. *Cấu trúc thư mục mã nguồn Back-End*

2.2.1. config/

- `db.js`: hàm `ketNoiCoSoDuLieu()` kết nối MongoDB Atlas qua Mongoose, in hướng dẫn mở Network Access nếu lỗi.
- `gemini.config.js`: đọc danh sách API key từ `GEMINI_API_KEYS`, thứ tự mô hình ưu tiên và tham số sinh nội dung (temperature 0.35, tối đa 2048 token).

2.2.2. routes/

Gồm 15 tệp *.routes.js, mỗi tệp ứng với một nhóm tài nguyên và được gắn vào tiền tố tương ứng trong `app.js`: `/api/auth`, `/api/san-pham`, `/api/don-hang`, `/api/ma-giam-gia`, `/api/danh-gia`, `/api/nguoi-dung`, `/api/danh-muc`, `/api/tin-tuc`, `/api/bao-hanh`, `/api/thanh-toan`, `/api/lien-he`, `/api/cai-dat`, `/api/tro-ly-ai`.

Ví dụ khai báo route có gắn middleware phân quyền

```
// routes/san-pham.routes.js
router.get('/', layTatCaSanPham);                          // Công khai
router.get('/:id', laySanPhamTheoId);                      // Công khai
router.post('/', xacThucToken, chiAdmin, themSanPham);     // Chỉ Admin
router.put('/:id', xacThucToken, chiAdmin, capNhatSanPham); // Chỉ Admin
router.delete('/:id', xacThucToken, chiAdmin, xoaSanPham);  // Chỉ Admin
```

2.2.3. middleware/

Gồm 3 tệp: `xac-thuc.middleware.js` (`xacThucToken`, `chiAdmin`, `nhanDienNguoiDungTuyChon`, `nhanDienNguoiDungMem`); `bao-mat.middleware.js` (security headers, chống NoSQL Injection); `gioi-han-request.middleware.js` (giới hạn tần suất theo IP). Chi tiết ở mục 4.2 và Chương 5.

2.2.4. controllers/

15 tệp controller chứa logic nghiệp vụ, ví dụ `xac-thuc.controller.js` (đăng ký, OTP, đăng nhập, Google, hồ sơ), `don-hang.controller.js` (tạo đơn, phân quyền xem đơn, trừ/hoàn kho), `thanh-toan.controller.js` (PayOS, VietQR, Webhook).

2.2.5. services/

- `email.service.js`: gửi email qua Brevo API hoặc Gmail SMTP, dựng nội dung HTML cho OTP và hóa đơn.
- `telegram-bot.service.js`: gửi thông báo đơn, xử lý nút bấm, lệnh quản trị, vòng Long Polling.
- `tro-ly-ai.service.js`: điều phối Trợ lý AI; `nine-router.service.js` và `gemini-xoay-key.service.js`: gọi mô hình; `tim-kiem-san-pham-ai.service.js`: phân tích ý định và tìm sản phẩm phù hợp.

2.2.6. models/, utils/, scripts-local/

- `models/: 10 tệp Schema Mongoose`(mục 2.4).
- `utils/tao-slug.js`: chuyển tiêu đề tiếng Việt có dấu thành slug URL, ví dụ “ASUS Zenbook 14” → `asus-zenbook-14`.

- `utils/bao-mat.js`: các hàm bảo mật dùng chung — `escapeRegex` (chống ReDoS), `taoMaOtp` / `bamOtp` / `soSanhOtp` (OTP an toàn), `kiemTraMatKhauManh`, `laUrlAnhHopLe`.
- `scripts-local/`: các tập lệnh nạp dữ liệu mẫu, migrate và đồng bộ dữ liệu (chạy tay ở máy local, không thuộc ứng dụng; chạy bằng `node scripts-local/<tên>.js`).

2.3. Sơ đồ Use Case

Sơ đồ use case thể hiện các chức năng chính theo ba nhóm quyền và nhóm hệ thống tích hợp bên ngoài. Dưới mỗi use case là endpoint hiện thực chức năng đó.

Hình 2.4. *Sơ đồ use case của hệ thống Back-End LaptopNew*

2.4. Thiết kế cơ sở dữ liệu

2.4.1. Tổng quan các collection

Cơ sở dữ liệu `laptop_store` trên MongoDB Atlas gồm 10 collection được quản lý bằng Mongoose và 1 collection tạm `dang_ky_tam` được thao tác trực tiếp qua driver. Số lượng bản ghi dưới đây được lấy từ API thực tế ngày 29/09/2026.

***Bảng 2.2.** Danh sách collection trong cơ sở dữ liệu*

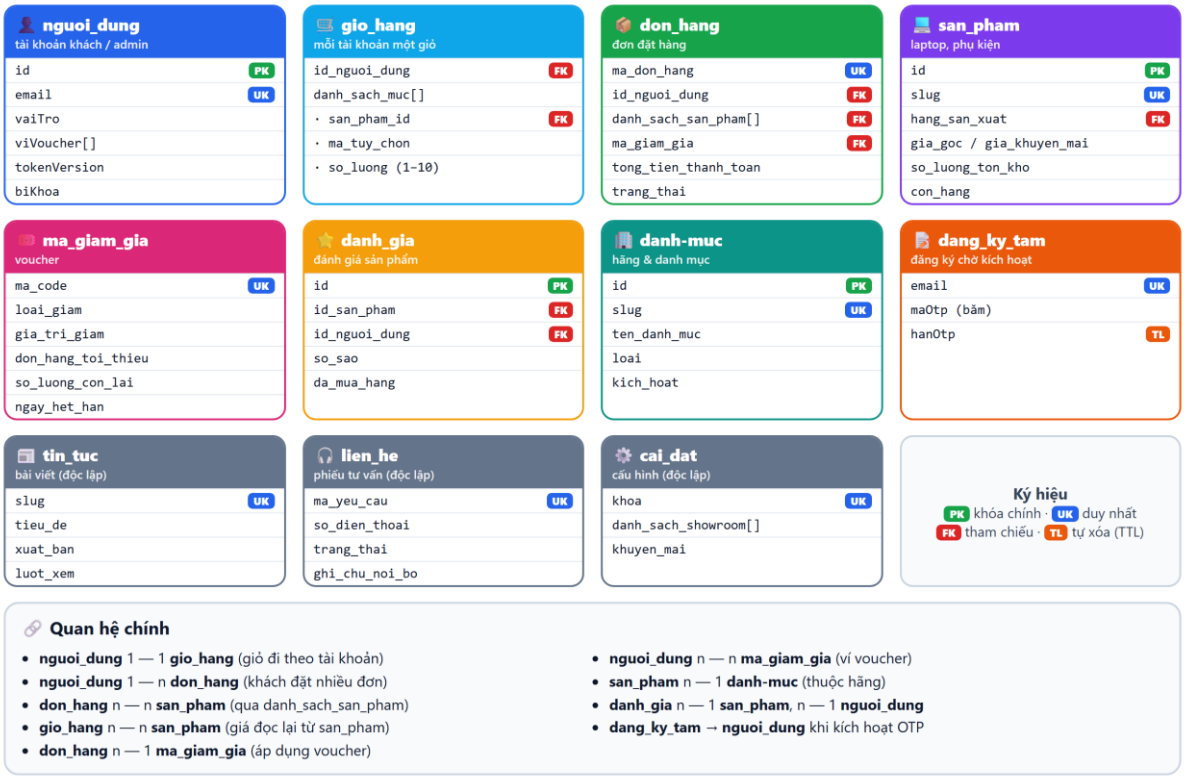
Collection	Model / Tệp	Nội dung lưu trữ	Số bản ghi
`san_pham`	SanPham · san-pham.model.js	Laptop, phụ kiện: giá, thông số, phiên bản, tồn kho	61 (35 hãng)
`don_hang`	DonHang · don-hang.model.js	Đơn hàng, giao hàng, thanh toán, lịch trình	—
`nguoi_dung`	NguoiDung · nguoi-dung.model.js	Tài khoản, vai trò, OTP, sổ địa chỉ, ví voucher	—
`dang_ky_tam`	(driver trực tiếp)	Thông tin đăng ký chờ xác thực OTP, tự xóa khi hết hạn	tạm thời
`ma_giam_gia`	MaGiamGia · ma-giam-gia.model.js	Voucher giảm tiền mặt hoặc phần trăm	14
`danh_gia`	DanhGia · danh-gia.model.js	Đánh giá sao và bình luận sản phẩm	30
`danh-muc`	DanhMuc · danh-muc.model.js	Thương hiệu, nhu cầu, phân khúc	21
`tin_tuc`	TinTuc · tin-tuc.model.js	Bài viết tin tức công nghệ	10
`lien_he`	LienHe · lien-he.model.js	Phiếu yêu cầu tư vấn của khách	—
`cai_dat`	CaiDat · cai-dat.model.js	Cấu hình hệ thống: danh sách showroom, tọa độ	1 (3 showroom)
`gio_hang`	GioHang · gio-hang.model.js	Giỏ hàng theo tài khoản: mã sản phẩm, tùy chọn, số lượng (không lưu giá)	—

> **Ghi chú:** "—" là các collection chứa dữ liệu cá nhân hoặc chỉ admin mới xem được qua API, nên nhóm không thống kê trong báo cáo.

2.4.2. Sơ đồ quan hệ dữ liệu

MongoDB không có khóa ngoại như SQL; quan hệ được thể hiện bằng **tham chiếu theo mã** (lưu `id` của document khác) hoặc **nhúng (embedding)** dữ liệu vào trong document. Nhóm lựa chọn như sau:

- **Nhúng** thông số kỹ thuật, phiên bản cấu hình, quà tặng, tồn kho chi nhánh vào `san_pham` — vì luôn được đọc cùng sản phẩm.
- **Nhúng bản chụp (snapshot)** sản phẩm vào `danh_sach_san_pham` của đơn hàng — để giá và tên trong đơn không đổi khi sản phẩm được sửa sau này.
- **Tham chiếu** `id_nguoi_dung` trong đơn hàng và đánh giá; `id_san_pham` trong đánh giá; mảng mã `viVoucher` trong người dùng.



Hình 2.5. *Sơ đồ quan hệ giữa các collection (ERD logic)*

2.4.3. Collection san_pham

Bảng 2.3. *Cấu trúc collection san_pham*

Tên trường	Kiểu	Ràng buộc	Mô tả
`id`	String	Bắt buộc, duy nhất	Mã định danh, ví dụ `asus-zenbook-14-oled-ai`
`slug`	String	Duy nhất, index, tự sinh	Đường dẫn SEO sinh từ tên bằng hook `pre('validate')`
`ma_san_pham`	String	Bắt buộc	Mã part-number / SKU, ví dụ `UX3405MA-PP152W`
`ten_san_pham`	String	Bắt buộc	Tên thương mại kèm cấu hình nổi bật
`hang_san_xuat`	String	Bắt buộc, lowercase	Hãng: asus, dell, lenovo, apple, msi...
`danh_muc`	[String]	Mặc định []	Nhu cầu: gaming, van-phong-mong-nhe, ai-pc...
`gia_goc`	Number	Mặc định 0	Giá niêm yết (VNĐ)
`gia_khuyen_mai`	Number	Bắt buộc	Giá bán thực tế — dùng để lọc và sắp xếp
`phan_tram_giam_gia`	Number	Mặc định 0	Phần trăm giảm hiển thị
`hinh_anh_chinh`	String	Bắt buộc	Ảnh đại diện
`thu_vien_hinh_anh`	[String]	Mặc định []	Các ảnh chi tiết
`video_demo`	String	Mặc định ""	Video giới thiệu

Tên trường	Kiểu	Ràng buộc	Mô tả
`thong_so`	Mixed	Mặc định {}	Object thông số tự do: cpu, card_do_hoa, ram, o_cung, man_hinh, dung_luong_pin, chip_ai_npu...
`tuy_chon_phien_ban`	[Object]	Nhúng	`ma_tuy_chon`, `ten_tuy_chon`, `ram`, `o_cung`, `chenh_lech_gia`, `hinh_anh`
`qua_tang`	[Object]	Nhúng	`ten_qua_tang`, `gia_tri`
`ton_kho`	[Object]	Nhúng	Theo chi nhánh: `ten_chi_nhanh`, `dia_chi`, `so_dien_thoai`, `so_luong_con`
`danh_gia`	[Object]	Nhúng	Đánh giá rút gọn hiển thị nhanh trên trang chi tiết
`dac_diem_noi_bat`	[String]	Mặc định []	Các điểm nổi bật
`mo_ta_chi_tiet`	String	Mặc định ""	Bài mô tả dài
`so_luong_ton_kho`	Number	Mặc định 10	Tổng tồn kho — bị trừ khi đơn giao thành công
`so_luong_da_ban`	Number	Mặc định 0	Số đã bán — dùng sắp xếp "bán chạy"
`con_hang`	Boolean	Mặc định true	Tự chuyển `false` khi tồn kho về 0
`diem_danh_gia_tb`, `so_luong_danh_gia`	Number	Mặc định 5 / 0	Điểm trung bình và số lượt đánh giá
`la_flash_sale`, `la_ban_chay`, `la_san_pham_moi`	Boolean	Mặc định false	Cờ phục vụ các khối trên trang chủ
`ho_tro_tra_gop_0_phan_tram`	Boolean	Mặc định true	Hỗ trợ trả góp 0%
`createdAt, updatedAt`	Date	Tự động	Do tùy chọn `timestamps: true`

Hook tự sinh slug trước khi kiểm tra dữ liệu

```
// models/san-pham.model.js (trích)
SanPhamSchema.pre('validate', function () {
    if (!this.slug || this.isModified('ten_san_pham')) {
        this.slug = taoSlug(this.ten_san_pham) || this.id;
    }
});
const SanPham = mongoose.model('SanPham', SanPhamSchema, 'san_pham');
```

2.4.4. Collection don_hang

Bảng 2.4. *Cấu trúc collection don_hang*

Tên trường	Kiểu	Ràng buộc	Mô tả
`id`	String	Bắt buộc, duy nhất	Tự sinh `dh-<timestamp>` nếu client không gửi
`ma_don_hang`	String	Bắt buộc, duy nhất, index	Mã hiển thị, dạng `LPN-2026-xxxx`
`id_nguoi_dung`	String	Index	Chủ đơn — lấy từ token khi có

Tên trường	Kiểu	Ràng buộc	Mô tả
`ngay_tao`	String	Bắt buộc	Thời điểm đặt theo định dạng vi-VN
`trang_thai`	String	Enum	`cho_xac_nhan` · `da_xac_nhan` · `dang_giao` · `da_giao` · `da_huy`
`thong_tin_giao_hang`	Object	SĐT bắt buộc	`ho_ten`, `so_dien_thoai`, `email`, `dia_chi_chi_tiet`, `tinh_thanh`, `quan_huyen`, `phuong_xa`, `ghi_chu`
`danh_sach_san_pham`	[Object]	Nhúng	`san_pham_id`, `ten_san_pham`, `hinh_anh`, `so_luong`, `gia`, `tong_tien_muc`, `tuy_chon_chon`, snapshot `san_pham`
`tam_tinh`	Number	Bắt buộc	Tổng tiền hàng trước giảm giá
`tien_giam_gia / ma_giam_gia`	Number / String	Mặc định 0 / ""	Số tiền giảm và mã voucher đã áp dụng
`phi_van_chuyen`	Number	Mặc định 0	Phí giao hàng
`tong_tien_thanh_toan`	Number	Bắt buộc	Số tiền khách phải trả
`hinh_thuc_thanh_toan`	String	Mặc định `tien_mat_cod`	COD, chuyển khoản, PayOS
`da_thanh_toan`	Boolean	Mặc định false, index	Đã nhận tiền hay chưa
`trang_thai_thanh_toan`	String	Enum	`chua_thanh_toan` · `cho_thanh_toan` · `da_thanh_toan` · `that_bai` · `thanh_toan_khi_nhan_hang`
`payos_order_code`	Number	Index	Mã giao dịch PayOS để đối soát Webhook
`noi_dung_chuyen_khoan`	String		Nội dung CK, dạng `TRIKUN xxxxxx`
`da_tru_ton_kho`	Boolean	Mặc định false	Cờ chống trừ kho hai lần
`lich_trinh_giao_hang`	[Object]	Nhúng	`thoi_gian`, `tieu_de`, `mo_ta_chi_tiet`, `hoan_thanh`

2.4.5. Collection nguoi_dung và dang_ky_tam

Bảng 2.5. *Cấu trúc collection nguoi_dung*

Tên trường	Kiểu	Ràng buộc	Mô tả
`id`	String	Bắt buộc, duy nhất	Mã người dùng
`hoTen`	String	Bắt buộc	Họ và tên
`email`	String	Bắt buộc, duy nhất, lowercase	Dùng làm tên đăng nhập
`matKhau`	String	Bắt buộc	Chuỗi băm bcrypt; bị loại khỏi JSON trả về nhờ `toJSON()`
`soDienThoai, avatar`	String	Mặc định ""	Liên lạc, ảnh đại diện

Tên trường	Kiểu	Ràng buộc	Mô tả
`vaiTro`	String	Enum	`khach_hang` (mặc định) hoặc `admin` — nền tảng RBAC
`hangThanhVien`	String	Mặc định	Hạng thành viên, ví dụ "Thành Viên Mới"
`viVoucher`	[String]	Mặc định []	Mã voucher đã lưu
`gioiTinh, ngaySinh`	String	Enum / ""	`nam` · `nu` · `khac`
`danhSachDiaChi`	[Object]	Nhúng	Sổ địa chỉ: người nhận, SĐT, tỉnh/quận/phường, `macDinh`, `loaiDiaChi`
`googleId, authProvider`	String	Enum	`local` hoặc `google`
`maOtp, hanOtp, loaiOtp`	String, Date, String	Mặc định null	**Bản băm HMAC-SHA256** của OTP (không lưu mã gốc), hạn 10 phút, loại: quen_mat_khau · kich_hoat · doi_mat_khau
tokenVersion	Number	Mặc định 0	Phiên bản phiên đăng nhập; tăng khi đăng xuất / đổi mật khẩu / bị khóa → mọi JWT cũ mất hiệu lực
soLanNhapSaiOtp	Number	Mặc định 0	Đếm số lần nhập sai OTP; đủ 5 lần thì mã bị hủy
soLanDangNhapSai, khoaDangNhapDen	Number, Date	Mặc định 0 / null	Sai mật khẩu 5 lần liên tiếp → khóa đăng nhập 15 phút (chống brute-force)
`daKichHoat`	Boolean	Mặc định true	Tài khoản đã xác thực email
`trangThai, biKhoa, lyDoKhoa`	String, Boolean, String		Khóa tài khoản — đăng nhập bị từ chối với mã 403

Collection `dang_ky_tam` lưu thông tin đăng ký chưa xác thực gồm `hoTen`, `email`, `soDienThoai`, `matKhau` (đã băm bcrypt), `maOtp` (đã băm HMAC), `hanOtp`, `loaiOtp`, `soLanNhapSaiOtp`. Collection có **TTL index** trên `hanOtp` với `expireAfterSeconds: 0`, nên MongoDB tự xóa bản ghi khi OTP hết hạn. Nhờ vậy, bảng `nguoi_dung` chỉ chứa tài khoản đã xác thực, không bị đầy tài khoản rác. Toàn bộ trường nhạy cảm (`matKhau`, `maOtp`, `hanOtp`, `tokenVersion`, các bộ đếm) bị loại khỏi JSON trả về nhờ phương thức `toJSON()` của Model.

2.4.6. Collection ma_giam_gia

Bảng 2.6. *Cấu trúc collection ma_giam_gia*

Tên trường	Kiểu	Ràng buộc	Mô tả
`ma_code`	String	Bắt buộc, duy nhất, uppercase	Mã khách nhập, ví dụ `TNTP3TR`
`tieu_de, mo_ta`	String	tieu_de bắt buộc	Nội dung hiển thị

Tên trường	Kiểu	Ràng buộc	Mô tả
`loai_giam`	String	Enum	`tien_mat` (mặc định) hoặc `phan_tram`
`gia_tri_giam`	Number	Bắt buộc	Số tiền hoặc số phần trăm giảm
`giam_toi_da`	Number	Mặc định 0	Trần giảm cho loại phần trăm
`don_hang_toi_thieu`	Number	Mặc định 0	Tạm tính tối thiểu để áp dụng
`ngay_het_han`	String	Mặc định ""	Thông tin hạn dùng hiển thị
`kich_hoat`	Boolean	Mặc định true	Chỉ mã đang bật mới kiểm tra hợp lệ
`chuyen_muc, badge, hot_deal, mau_sac`	Mixed		Thông tin trình bày trên trang khuyến mãi
`so_luong_phat_hanh, so_luong_con_lai, phan_tram_da_dung, khung_gio_san`	Number / String		Thông tin "săn mã" theo khung giờ

2.4.7. Collection danh_gia

Bảng 2.7. *Cấu trúc collection danh_gia*

Tên trường	Kiểu	Ràng buộc	Mô tả
`id_san_pham`	String	Bắt buộc	Sản phẩm được đánh giá
`id_nguoi_dung`	String		Người viết (nếu đăng nhập)
`ho_ten`	String	Bắt buộc	Tên hiển thị
`so_sao`	Number	Bắt buộc, 1–5	Số sao
`tieu_de, noi_dung`	String	noi_dung bắt buộc	Nội dung nhận xét
`hinh_anh_dinh_kem`	[String]	Mặc định []	Ảnh khách gửi kèm
`luot_thich`	Number	Mặc định 0	Tăng qua `PATCH /:id/thich`
`da_mua_hang`	Boolean	Mặc định true	Nhãn "Đã mua hàng"
`ngheNghiep, avatar, diaDiemMua, thoi_gian_dung, ngay_danh_gia`	String		Thông tin bổ sung khi hiển thị

2.4.8. Collection danh-muc, tin_tuc, lien_he, cai_dat

Bảng 2.8. *Cấu trúc các collection danh-muc, tin_tuc, lien_he, cai_dat*

Collection	Trường chính	Ràng buộc nổi bật
`danh-muc`	`id`, `slug`, `ma_danh_muc`, `ten_danh_muc`, `loai`, `logo`, `mo_ta`, `xuat_xu`, `kich_hoat`, `thu_tu_hien_thi`	`ma_danh_muc` duy nhất, lowercase; `loai` ∈ {`thuong_hieu`, `nhu_cau`, `phan_khuc`}; slug tự sinh
`tin_tuc`	`id`, `slug`, `tieu_de`, `tom_tat`, `noi_dung`, `hinh_anh`, `ngay_dang`, `tac_gia`, `chuyen_muc`, `thoi_gian_doc`, `luot_xem`, `la_tieu_diem`, `xuat_ban`, `tags`	`tieu_de`, `tom_tat` bắt buộc; slug duy nhất tự sinh

Collection	Trường chính	Ràng buộc nổi bật
`lien_he`	`ma_yeu_cau`, `ho_ten`, `so_dien_thoai`, `email`, `chu_de`, `noi_dung`, `trang_thai`, `ghi_chu_noi_bo`, `nguon_tiep_nhan`	`ma_yeu_cau` duy nhất; thông báo lỗi tiếng Việt cho trường bắt buộc; `trang_thai` ∈ {`chua_xu_ly`, `dang_xu_ly`, `da_hoan_thanh`, `da_huy`}; index {SĐT, ngày tạo}
`cai_dat`	`khoa` (mặc định `he_thong`), `danh_sach_showroom[]`: `id`, `ten`, `dia_chi`, `hotline`, `gio_mo_cua`, `vi_do`, `kinh_do`, `la_mac_dinh`	`khoa` duy nhất (một bản ghi cấu hình); tọa độ bắt buộc để vẽ bản đồ

2.4.9. Chỉ mục (Index) và ràng buộc dữ liệu

Bảng 2.9. *Các chỉ mục chính trong cơ sở dữ liệu*

Collection	Chỉ mục	Mục đích
san_pham	`id` (unique), `slug` (unique, sparse)	Tra chi tiết theo id hoặc slug SEO
don_hang	`id`, `ma_don_hang` (unique); `payos_order_code`, `id_nguoi_dung`, `da_thanh_toan`	Tra đơn nhanh, đối soát Webhook, lọc đơn theo người dùng
nguoi_dung	`id`, `email` (unique)	Không cho trùng email khi đăng ký
dang_ky_tam	`hanOtp` (TTL, expireAfterSeconds = 0)	Tự xóa đăng ký chưa xác thực khi hết hạn
ma_giam_gia	`ma_code` (unique)	Mỗi mã chỉ tồn tại một lần
lien_he	`ma_yeu_cau` (unique), `trang_thai`, {`so_dien_thoai`, `ngay_tao`}	Lọc phiếu theo trạng thái, tra theo SĐT

CHƯƠNG 3. ĐẶC TẢ HỆ THỐNG API

3.1. Quy ước chung

Bảng 3.1. *Quy ước chung của API*

Hạng mục	Quy ước
Địa chỉ gốc (Base URL)	Production: `https://api.trikun.site` (qua Cloudflare, bắt buộc HTTPS) · phát triển: `http://localhost:5000` · Front-End cấu hình qua biến `NEXT_PUBLIC_API_URL`
Tiền tố	Mọi tài nguyên nằm dưới `/api/<tên-nhóm>`; tên nhóm viết thường, không dấu, nối bằng gạch ngang
Định dạng dữ liệu	Request và response dùng JSON (`Content-Type: application/json`); riêng stream AI dùng NDJSON
Xác thực	Header `Authorization: Bearer <JWT>` nhận được sau khi đăng nhập / kích hoạt
Phương thức	`GET` đọc · `POST` tạo mới / hành động · `PUT` cập nhật toàn phần · `PATCH` cập nhật một phần · `DELETE` xóa
Thông báo lỗi	Trả kèm mã HTTP phù hợp và trường `thong_diep` tiếng Việt (một số nhóm dùng `message` / `thongBao`). Ở môi trường production, chi tiết lỗi kỹ thuật (`chi_tiet`) bị ẩn
Định danh linh hoạt	Các API `/:id` chấp nhận `id` chuỗi, `slug` hoặc `_id` ObjectId của MongoDB
Giới hạn tần suất	Mọi API tối đa 300 request/phút/IP; các API nhạy cảm có hạn mức riêng (Bảng 5.5). Vượt quá trả `429` kèm header `Retry-After`
Tài liệu trực tuyến	Swagger UI: https://api.trikun.site/api-docs · đặc tả JSON: https://api.trikun.site/api-docs.json

3.1.1. Các mức quyền truy cập

Bảng 3.2. *Ký hiệu mức quyền dùng trong các bảng đặc tả*

Ký hiệu	Middleware	Ý nghĩa
Công khai	(không có)	Ai cũng gọi được
Tùy chọn	`nhanDienNguoiDungTuyChon`	Không bắt buộc token; nếu có token hợp lệ thì hệ thống biết người gọi là ai để lọc dữ liệu
JWT	`xacThucToken`	Bắt buộc token hợp lệ, nếu không trả 401
JWT + Admin	`xacThucToken` + `chiAdmin`	Bắt buộc token của tài khoản `vaiTro = admin`, nếu không trả 401 / 403

3.2. Bảng tổng hợp API

Hệ thống khai báo 74 route trong 15 tệp route và `app.js`. Một số route là **bí danh (alias)** trỏ cùng một hàm để tương thích Front-End cũ (ví dụ `/tao-link` và `/tao_link`), nên số endpoint chức năng khác nhau là 68.

Bảng 3.3. *Thống kê số route theo nhóm chức năng*

STT	Tiền tố	Nhóm chức năng	Số route	Tệp route
1	`/api/auth`	Xác thực & tài khoản	13	xac-thuc.routes.js
2	`/api/san-pham`	Sản phẩm	5	san-pham.routes.js
3	`/api/danh-muc`	Danh mục & thương hiệu	4	danh-muc.routes.js
4	`/api/don-hang`	Đơn hàng	5	don-hang.routes.js
5	`/api/thanh-toan`	Thanh toán PayOS / VietQR	6 (3 alias)	thanh-toan.routes.js
6	`/api/ma-giam-gia`	Mã giảm giá & ví voucher	7	ma-giam-gia.routes.js
7	`/api/danh-gia`	Đánh giá sản phẩm	4	danh-gia.routes.js
8	`/api/nguoi-dung`	Quản trị người dùng	4	nguoi-dung.routes.js
9	`/api/tin-tuc`	Tin tức	5	tin-tuc.routes.js
10	`/api/bao-hanh`	Tra cứu bảo hành	2 (1 alias)	bao-hanh.routes.js
11	`/api/lien-he`	Liên hệ / CSKH	5 (1 alias)	lien-he.routes.js
12	`/api/cai-dat`	Cấu hình showroom & khuyến mãi	5	cai-dat.routes.js
13	`/api/tro-ly-ai`	Trợ lý AI	4 (1 alias)	tro-ly-ai.routes.js
14	`/api/gio-hang`	Giỏ hàng theo tài khoản	2	gio-hang.routes.js
15	`/api/dong-bo`	Phiên bản dữ liệu (tự làm mới giao diện)	1	dong-bo.routes.js
16	`/`, `/api/health`	Giám sát máy chủ	2	app.js
		Tổng cộng	**74**	

3.3. Đặc tả chi tiết từng nhóm API

3.3.1. Nhóm Xác thực và Tài khoản — /api/auth

Bảng 3.4. *Đặc tả API nhóm /api/auth*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
POST	/dang-ky	Công khai	hoTen, email, soDienThoai, matKhau (≥ 8 ký tự, có chữ và số)	200 / 400 / 429 / 502	Kiểm tra dữ liệu, chính sách mật khẩu mạnh, băm mật khẩu, lưu dang_ky_tam, gửi OTP (lưu bản băm)
POST	/kich-hoat	Công khai	`email`, `otp`	200 / 400 / 404 / 429	Xác thực OTP (sai 5 lần → hủy đăng ký tạm), tạo tài khoản chính thức, trả JWT
POST	/dang-nhap	Công khai	`email`, `matKhau`	200 / 400 / 401 / 403 / 429	So khớp bcrypt; sai 5 lần liên tiếp → khóa đăng nhập 15 phút (429); kiểm tra khóa / kích hoạt; trả token + nguoiDung
POST	/dang-xuat	JWT	Header Bearer	200 / 401	Tăng tokenVersion → thu hồi phía server mọi token đang dùng của tài khoản
POST	/google	Công khai	credential (Google ID Token)	200 / 400 / 401 / 403	Bắt buộc verifyIdToken (đúng audience, email_verified); tạo hoặc liên kết tài khoản, trả JWT
POST	/quen-mat-khau	Công khai	`email`	200 / 429 / 502	Luôn trả 200 cùng một thông điệp dù email có tồn tại hay không (chống dò email); sinh OTP loại quen_mat_khau, gửi email
POST	/gui-otp	Công khai	`email`	200 / 429	Gửi lại OTP mới (tối đa 6 lần / 15 phút / IP)
POST	/xac-nhan-otp	Công khai	`email`, `otp`	200 / 400 / 429	Chỉ chấp nhận OTP đúng loại quen_mat_khau, còn hạn; sai 5 lần → mã bị hủy
POST	/dat-lai-mat-khau	Công khai	email, otp, matKhauMoi (≥ 8 ký tự, có chữ và số)	200 / 400 / 429	Băm và lưu mật khẩu mới, xóa OTP, thu hồi mọi phiên cũ
GET	/toi	JWT	Header Bearer	200 / 401	Lấy hồ sơ người đang đăng nhập
PUT	/cap-nhat-ho-so	JWT	`hoTen`, `soDienThoai`, `avatar`, `gioiTinh`, `ngaySinh`, `danhSachDiaChi`	200 / 400 / 404	Chỉ nhận các trường hồ sơ cho phép (chống Mass Assignment); kiểm tra SĐT, URL ảnh
POST	/doi-mat-khau/gui-otp	JWT	`matKhauCu` (nếu tài khoản có mật khẩu)	200 / 400 / 502	Kiểm tra mật khẩu cũ, gửi OTP loại `doi_mat_khau`
PUT	/doi-mat-khau	JWT	matKhauCu, matKhauMoi (≥ 8 ký tự), otp	200 / 400	Đổi mật khẩu khi OTP đúng loại doi_mat_khau; thu hồi phiên cũ, cấp token mới

```
Ví dụ request / response đăng nhập
POST /api/auth/dang-nhap
Content-Type: application/json

{ "email": "khach@example.com", "matKhau": "matkhau123" }

HTTP/1.1 200 OK
{
  "thong_diep": "Đăng nhập thành công! Xin chào Nguyễn Văn A.",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOi...",
  "nguoiDung": { "id": "...", "hoTen": "Nguyễn Văn A", "email": "khach@example.com",
"vaiTro": "khach_hang" }
}
```

3.3.2. Nhóm Sản phẩm — /api/san-pham

Bảng 3.5. *Đặc tả API nhóm /api/san-pham*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	Công khai	Query: `tu_khoa`/`q`, `hang_san_xuat`, `danh_muc` (nhiều giá trị cách bằng dấu phẩy), `gia_min`, `gia_max`, `flash_sale`, `ban_chay`, `moi_nhat`, `sap_xep`, `limit`	200	Tìm kiếm, lọc, sắp xếp; có header cache 60 giây
GET	/:id	Công khai	Param: `id` / `slug` / `ma_san_pham` / `_id`	200 / 404	Chi tiết một sản phẩm
POST	/	JWT + Admin	Document sản phẩm	201 / 409	Thêm sản phẩm; trùng id trả 409
PUT	/:id	JWT + Admin	Các trường cần sửa	200 / 404	Cập nhật sản phẩm
DELETE	/:id	JWT + Admin	Param: `id`	200 / 404	Xóa sản phẩm

Bảng 3.6. *Giá trị của tham số sap_xep*

Giá trị	Sắp xếp MongoDB	Ý nghĩa
`gia_tang_dan`	`{ gia_khuyen_mai: 1 }`	Giá thấp đến cao
`gia_giam_dan`	`{ gia_khuyen_mai: -1 }`	Giá cao đến thấp
`ban_chay_nhat`	`{ so_luong_da_ban: -1 }`	Bán chạy nhất
`danh_gia_cao`	`{ diem_danh_gia_tb: -1 }`	Đánh giá cao nhất
`moi_nhat`	`{ la_san_pham_moi: -1, createdAt: -1 }`	Hàng mới về

Kết quả thực tế khi lọc laptop ASUS dưới 30 triệu

```
GET /api/san-pham?hang_san_xuat=asus&gia_max=30000000&sap_xep=gia_tang_dan&limit=1

HTTP/1.1 200 OK
[{
  "id": "asus-zenbook-14-oled-ai",
  "ma_san_pham": "UX3405MA-PP152W",
  "ten_san_pham": "ASUS Zenbook 14 OLED (2026) Intel Core Ultra 7 155H | 32GB | 1TB ...",
  "hang_san_xuat": "asus",
  "gia_goc": 34990000, "gia_khuyen_mai": 29990000, "phan_tram_giam_gia": 14,
  "thong_so": { "cpu": "Intel® Core™ Ultra 7 155H (16 Nhân 22 Luồng...)", "ram": "32GB
LPDDR5X 7467MHz", ... },
  ...
}]
```

3.3.3. Nhóm Danh mục — /api/danh-muc

Bảng 3.7. *Đặc tả API nhóm /api/danh-muc*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	Công khai	Query: `loai` (thuong_hieu / nhu_cau / phan_khuc)	200	Danh sách danh mục, sắp theo thứ tự hiển thị
POST	/	JWT + Admin	`ma_danh_muc`, `ten_danh_muc`, ...	201 / 400 / 409	Tạo danh mục; thiếu mã/tên trả 400, trùng mã trả 409
PUT	/:id	JWT + Admin	Các trường cần sửa	200 / 404	Cập nhật danh mục
DELETE	/:id	JWT + Admin	Param: `id`	200 / 404	Xóa danh mục

3.3.4. Nhóm Đơn hàng — /api/don-hang

Bảng 3.8. *Đặc tả API nhóm /api/don-hang*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
POST	/	JWT	thong_tin_giao_hang, danh_sach_san_pham (id, số lượng, tùy chọn), ma_giam_gia, hinh_thuc_thanh_toan	201 / 400 / 401	Server tự tính lại đơn giá, voucher, tổng tiền từ DB (bỏ qua giá client gửi), kiểm tra tồn kho; gửi email + Telegram
GET	/	JWT	Query: trang_thai (Admin thêm id_nguoi_dung, email, sdt)	200 / 401	Admin: tất cả đơn · Thành viên: chỉ đơn gắn id_nguoi_dung của chính mình
GET	/:id	Tùy chọn + giới hạn	Param: `id` / `ma_don_hang` / orderCode / `_id`; Query: `sdt` (bắt buộc với khách)	200 / 403 / 404 / 429	Admin, chủ đơn: xem đầy đủ. Khách: phải nhập đúng mã đơn + SĐT nhận hàng, dữ liệu cá nhân được che; tối đa 30 lần / 10 phút / IP

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
PUT	/:id	JWT	Admin: các trường nghiệp vụ (trừ trường hệ thống) · Chủ đơn: chỉ trang_thai = da_huy khi đơn chờ xác nhận	200 / 403 / 404	Cập nhật trạng thái; trừ/hoàn kho bằng thao tác nguyên tử; đồng bộ trạng thái thanh toán
DELETE	/:id	JWT + Admin	Param: `id`	200 / 404	Xóa đơn; hoàn kho (nguyên tử) nếu đơn đã trừ kho

3.3.5. Nhóm Thanh toán — /api/thanh-toan

Bảng 3.9. *Đặc tả API nhóm /api/thanh-toan*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
POST	/tao-link (alias /tao_link)	JWT (chủ đơn)	ma_don_hang hoặc id_don_hang	200 / 401 / 403 / 404 / 409	Số tiền lấy từ đơn trong DB; chặn đơn đã thanh toán / đã hủy; tạo PayOS + ảnh VietQR
GET	/kiem-tra/:orderCode (alias /kiem_tra/:orderCode)	Công khai	Param: `orderCode`	200 / 400	Chỉ trả trạng thái thanh toán (DB → cache 3 giây → PayOS); giới hạn 60 lần/phút/IP
POST	/webhook (alias /payos-webhook)	PayOS	`{ data, signature }`	200 / 400	Xác minh chữ ký HMAC (so sánh thời gian hằng), kiểm tra số tiền khớp đơn, xử lý idempotent; đánh dấu đã thanh toán

Ví dụ tạo yêu cầu thanh toán

```
POST /api/thanh-toan/tao-link
Authorization: Bearer <token của chủ đơn>
{ "ma_don_hang": "LPN-2026-4821" }        // Không gửi số tiền: server lấy từ đơn trong
DB


HTTP/1.1 200 OK
{
  "success": true,
  "orderCode": 20264821,
  "amount": 29990000,
  "description": "TRIKUN 6-4821",
  "qrImageUrl": "https://img.vietqr.io/image/<BIN>-<STK>-
compact2.png?amount=29990000&addInfo=...",
  "checkoutUrl": "https://pay.payos.vn/web/...",
  "thongTinNganHang": { "nganHang": "KienlongBank", "soTien": 29990000, "noiDung":
"TRIKUN 6-4821", ... }
}
```

3.3.6. Nhóm Mã giảm giá — /api/ma-giam-gia

Bảng 3.10. *Đặc tả API nhóm /api/ma-giam-gia*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	Công khai	Query: `kich_hoat`	200	Danh sách voucher
POST	/kiem-tra	Công khai	`ma_code`, `tong_tien_tam_tinh`	200 / 400 / 404 / 429	Kiểm tra mã và tính `so_tien_giam`
POST	/luu-ma	JWT	`ma_code`	200 / 400 / 404 / 409	Trừ lượt bằng thao tác nguyên tử ($inc có điều kiện); $addToSet chống lưu trùng
GET	/vi-cua-toi	JWT	—	200 / 404	Danh sách voucher trong ví
POST	/	JWT + Admin	Document voucher	201 / 400	Thêm voucher; trùng mã trả 400
PUT	/:id	JWT + Admin	Các trường cần sửa	200 / 404	Cập nhật, bật/tắt voucher
DELETE	/:id	JWT + Admin	Param: `id`	200 / 404	Xóa voucher

3.3.7. Nhóm Đánh giá — /api/danh-gia

Bảng 3.11. *Đặc tả API nhóm /api/danh-gia*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/ và /:productId	Công khai	Param `productId` hoặc query `id_san_pham`	200	Danh sách đánh giá của sản phẩm (hoặc tất cả)
POST	/	JWT	id_san_pham, so_sao (1–5), tieu_de, noi_dung	201 / 400 / 401 / 403 / 409	Chỉ tài khoản đã mua và nhận hàng mới được đánh giá, mỗi sản phẩm 1 lần; tính lại điểm trung bình
PATCH	/:id/thich	Công khai	Param: `id`	200 / 404 / 429	Tăng lượt thích

3.3.8. Nhóm Người dùng — /api/nguoi-dung

Bảng 3.12. *Đặc tả API nhóm /api/nguoi-dung*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	JWT + Admin	Query: `email`	200 / 401 / 403	Danh sách / tìm người dùng theo email
POST	/	JWT + Admin	hoTen, email, matKhau, vaiTro	201 / 400 / 401 / 403	Chỉ nhận trường cho phép, kiểm tra mật khẩu mạnh, băm bcrypt

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
PUT	/:id	JWT + Admin	Các trường cần sửa	200 / 404	Khóa/mở khóa, đổi vai trò → thu hồi phiên ngay; không cho Admin tự khóa / tự hạ quyền
DELETE	/:id	JWT + Admin	Param: `id`	200 / 404	Xóa người dùng; không cho Admin tự xóa chính mình

3.3.9. Nhóm Tin tức — /api/tin-tuc

Bảng 3.13. *Đặc tả API nhóm /api/tin-tuc*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	Công khai	Query: `chuyen_muc`, `q`, `la_tieu_diem`, `tat_ca`	200	Khách chỉ thấy bài đã xuất bản; Admin (có token) xem cả bản nháp; từ khóa được escape
GET	/:id	Công khai	Param: `id` / `slug`	200 / 404	Chi tiết bài viết, tự tăng `luot_xem`
POST	/	JWT + Admin	Nội dung bài viết	201 / 401 / 403	Tạo bài viết
PUT	/:id	JWT + Admin	Các trường cần sửa	200 / 401 / 403 / 404	Cập nhật bài viết
DELETE	/:id	JWT + Admin	Param: `id`	200 / 401 / 403 / 404	Xóa bài viết

3.3.10. Nhóm Bảo hành — /api/bao-hanh

Bảng 3.14. *Đặc tả API nhóm /api/bao-hanh*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/ và /tra-cuu	Công khai	Query: `tu_khoa` (hoặc `q`) — mã đơn, SĐT, serial, mã sản phẩm	200 / 429	Tìm đơn khớp chính xác, tính hạn bảo hành 24 tháng, che SĐT; tối đa 30 lần / 15 phút / IP

3.3.11. Nhóm Liên hệ — /api/lien-he

Bảng 3.15. *Đặc tả API nhóm /api/lien-he*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
POST	/	Công khai	`ho_ten`, `so_dien_thoai`, `email`, `chu_de`, `noi_dung`	201 / 400 / 429	Kiểm tra dữ liệu (SĐT Việt Nam), sinh ma_yeu_cau; tối đa 5 phiếu / giờ / IP

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	JWT + Admin	Query: `trang`, `gioi_han`, `trang_thai`, `chu_de`, `tu_khoa`	200	Danh sách phiếu có phân trang và thống kê
PATCH / PUT	/:id	JWT + Admin	`trang_thai`, `ghi_chu_noi_bo`	200 / 404	Cập nhật tiến độ xử lý
DELETE	/:id	JWT + Admin	Param: `id`	200 / 404	Xóa phiếu

3.3.12. Nhóm Cài đặt showroom — /api/cai-dat

Bảng 3.16. *Đặc tả API nhóm /api/cai-dat*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/showroom	Công khai	—	200	Danh sách showroom và tọa độ GPS
PUT	/showroom	JWT + Admin	`danh_sach_showroom[]`	200 / 400 / 401 / 403	Lưu danh sách showroom (upsert bản ghi `he_thong`)
POST	/showroom/khoi-phuc	JWT + Admin	—	410	Chức năng khôi phục mặc định đã ngừng hỗ trợ (410 Gone)

3.3.13. Nhóm Trợ lý AI — /api/tro-ly-ai

Bảng 3.17. *Đặc tả API nhóm /api/tro-ly-ai*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
POST	/chat/stream	Công khai	`tin_nhan`, `lich_su_chat[]`	200 (NDJSON) / 400 / 429	Trả từng đoạn văn bản, cuối cùng là kết quả đầy đủ kèm sản phẩm gợi ý
POST	/chat (alias /)	Công khai	`tin_nhan` (hoặc `message`), `lich_su_chat`	200 / 400 / 429 / 500	Trả toàn bộ câu trả lời một lần dạng JSON
GET	/trang-thai	Công khai	—	200	Trạng thái 9Router, số key Gemini, danh sách model, tình trạng từng key (đã che)

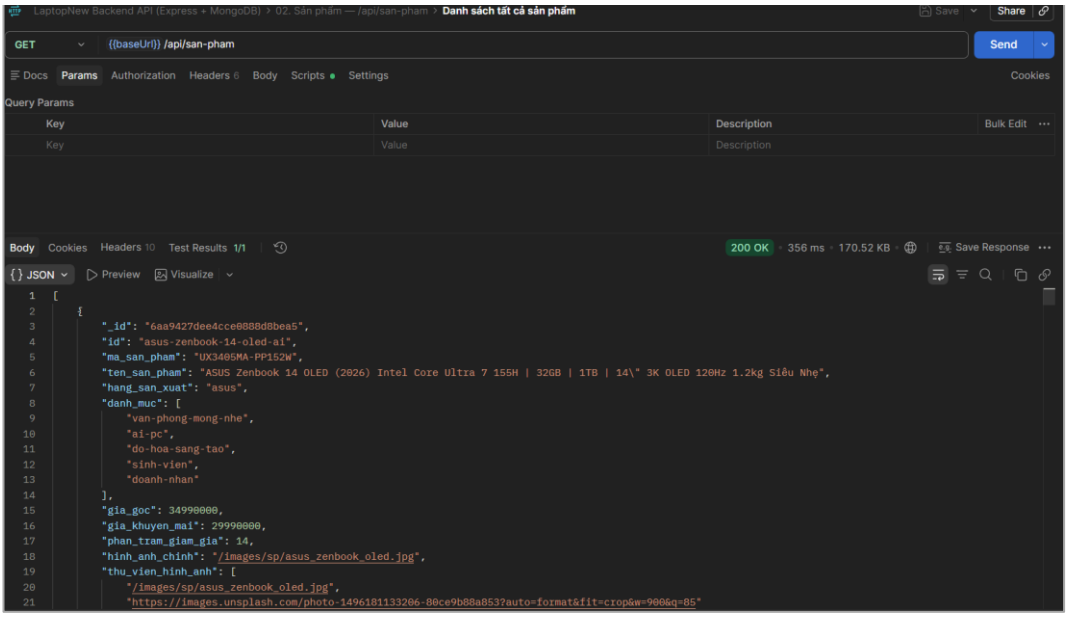
3.3.14. Nhóm Giám sát máy chủ

Bảng 3.18. *Đặc tả API giám sát máy chủ*

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/	Công khai	—	200	Thông điệp server, phiên bản 1.1.0, mô hình, danh sách endpoint chính

Method	Endpoint	Quyền	Tham số / Body	Phản hồi	Chức năng
GET	/api/health	Công khai	—	200	`{ trang_thai: "OK", thoi_gian }` dùng cho health check khi triển khai

> **Ghi chú bảo mật:** toàn bộ thao tác ghi dữ liệu quản trị (sản phẩm, danh mục, voucher, người dùng, tin tức, liên hệ, showroom, xóa đơn) đều đi qua hai lớp `xacThucToken` → `chiAdmin`. Ngoài ra mọi API chịu giới hạn tần suất chung 300 request/phút/IP; các API nhạy cảm có hạn mức riêng (Bảng 5.5).



Hình 3.1. *Kiểm thử GET /api/san-pham bằng Postman: trả về danh sách sản phẩm (200 OK)*

3.4. Tài liệu API trực tuyến với Swagger UI (OpenAPI 3.0)

Bên cạnh các bảng đặc tả ở mục 3.3, nhóm công bố toàn bộ API dưới dạng tài liệu tương tác theo chuẩn OpenAPI 3.0.3, hiển thị bằng Swagger UI ngay trên server production. Đặc tả được viết tay trong tệp `BE/docs/openapi.js`, bám sát route, controller và model thực tế: mỗi endpoint có mô tả nghiệp vụ, quyền truy cập, tham số, body mẫu, schema dữ liệu và đầy đủ mã lỗi.

> **Tài liệu API:** https://api.trikun.site/api-docs · **Đặc tả OpenAPI (JSON):** https://api.trikun.site/api-docs.json

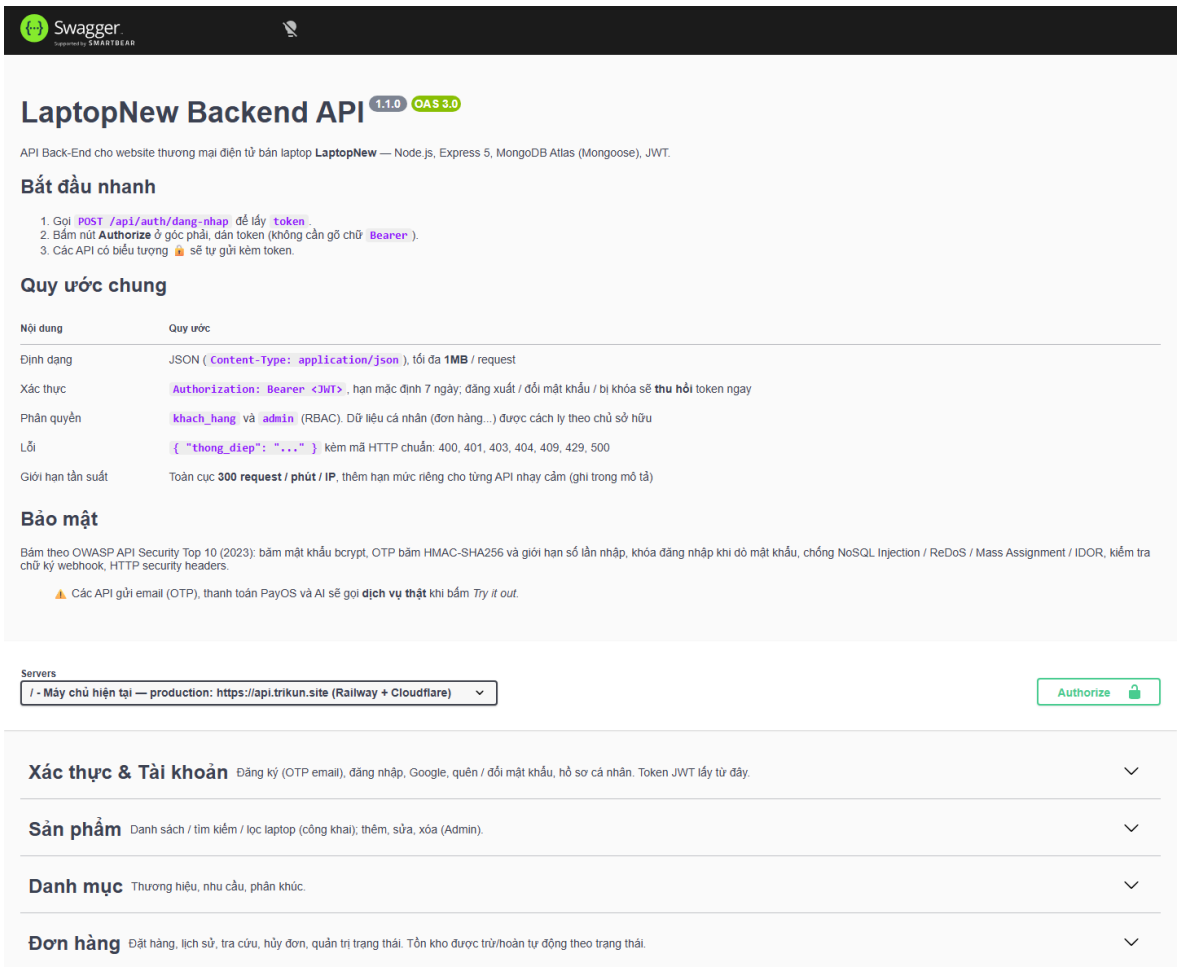
Bảng 3.19. *Nội dung tài liệu API trên Swagger UI*

Hạng mục	Nội dung
Chuẩn đặc tả	OpenAPI 3.0.3 — đã kiểm tra hợp lệ bằng swagger-cli

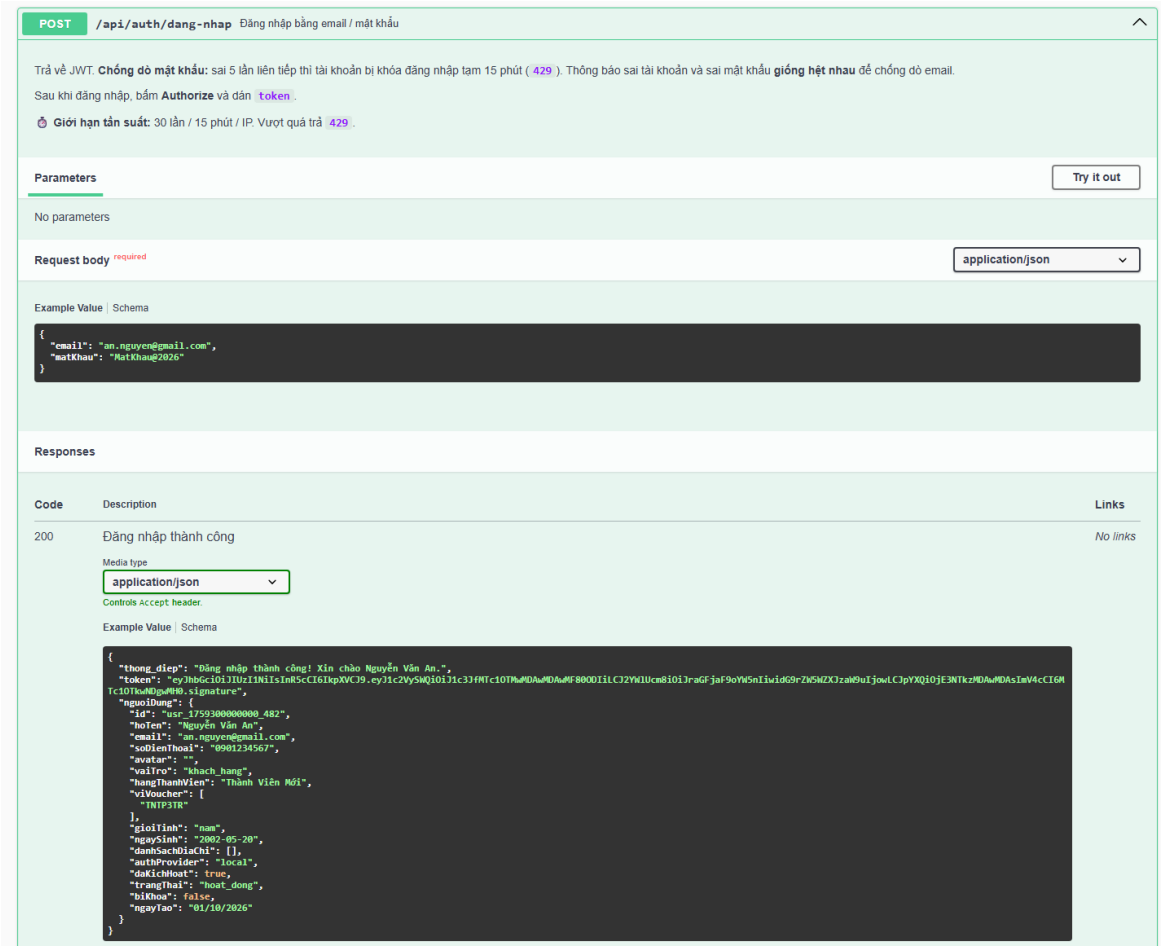
Hạng mục	Nội dung
Phạm vi	47 đường dẫn, 68 thao tác; 6 route alias ghi trong mô tả → phủ đủ 74 route
Nhóm chức năng	15 nhóm theo nghiệp vụ: Xác thực & Tài khoản, Sản phẩm, Danh mục, Đơn hàng, Thanh toán, Mã giảm giá, …
Schema dữ liệu	37 schema: SanPham, DonHang, NguoiDung, MaGiamGia, DanhGia, TinTuc, LienHe, Showroom, PayosWebhook, …
Mã lỗi dùng chung	400, 401, 403, 404, 409, 429, 500 — mỗi mã có ví dụ phản hồi tiếng Việt
Thông tin nghiệp vụ	Quyền truy cập, hạn mức rate limit, quy tắc tồn kho, chống IDOR … ghi trong mô tả từng API
Xác thực khi thử API	Nút **Authorize** nhận JWT Bearer; token được ghi nhớ khi tải lại trang
An toàn trang tài liệu	CSP riêng cho `/api-docs`; mọi API khác giữ CSP chặt `default-src 'none'`; không cho CDN cache để tài liệu luôn mới

Cách dùng nhanh để kiểm thử trực tiếp trên trang tài liệu:

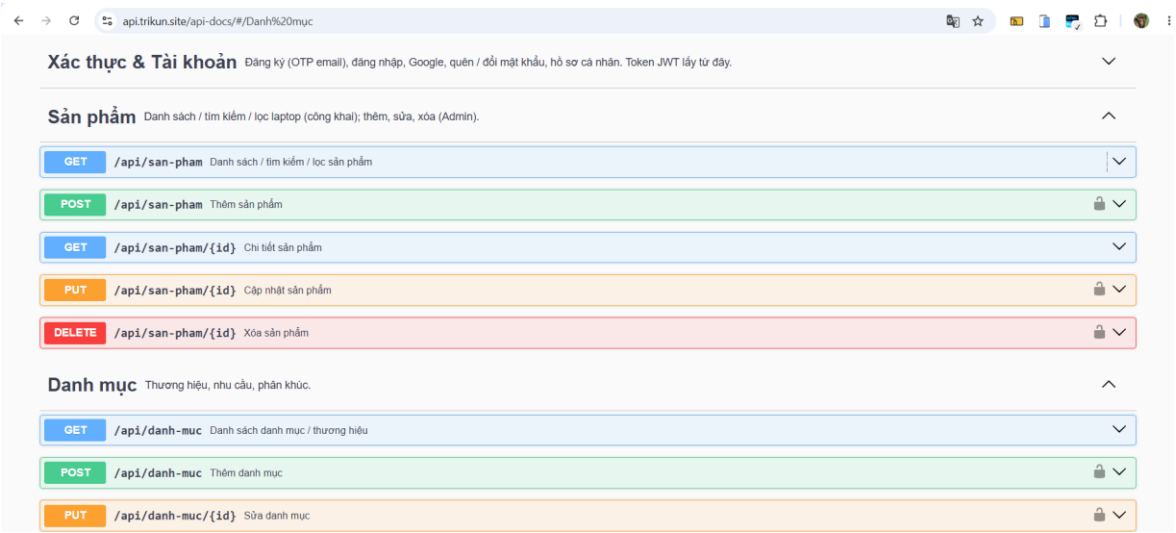
- Mở `POST /api/auth/dang-nhap`, bấm **Try it out** và gửi email/mật khẩu để nhận `token`.
- Bấm **Authorize**, dán token (không cần gõ chữ Bearer); các API có biểu tượng ổ khóa sẽ tự gửi kèm token.
- Mỗi API có đường dẫn riêng (deep link) nên có thể gửi thẳng link một endpoint cho người khác xem.



Hình 3.2. *Trang tài liệu Swagger UI của LaptopNew tại https://api.trikun.site/api-docs*



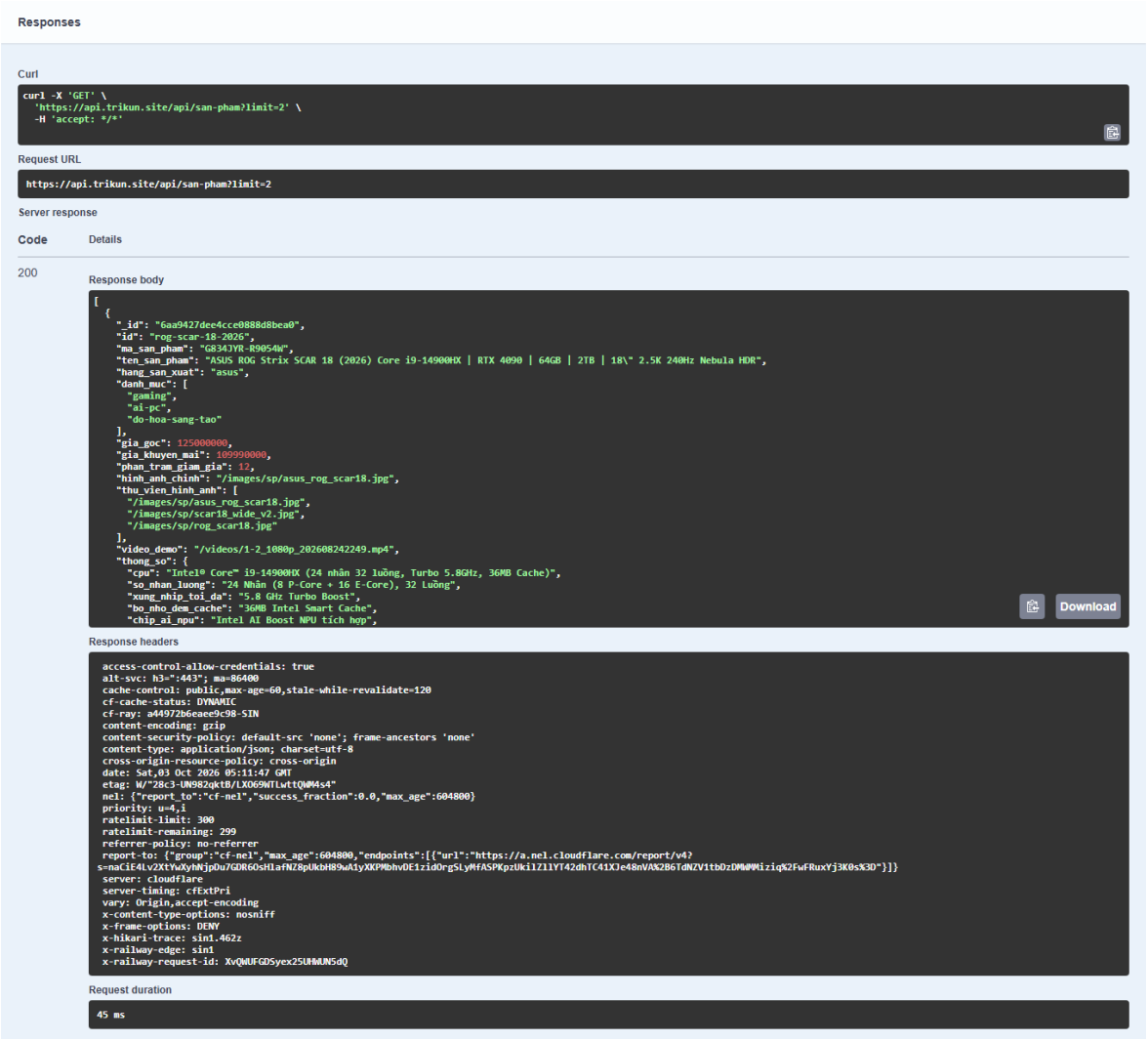
Hình 3.3. *Chi tiết API đăng nhập trên Swagger: mô tả nghiệp vụ, giới hạn tần suất, body mẫu và phản hồi mẫu*



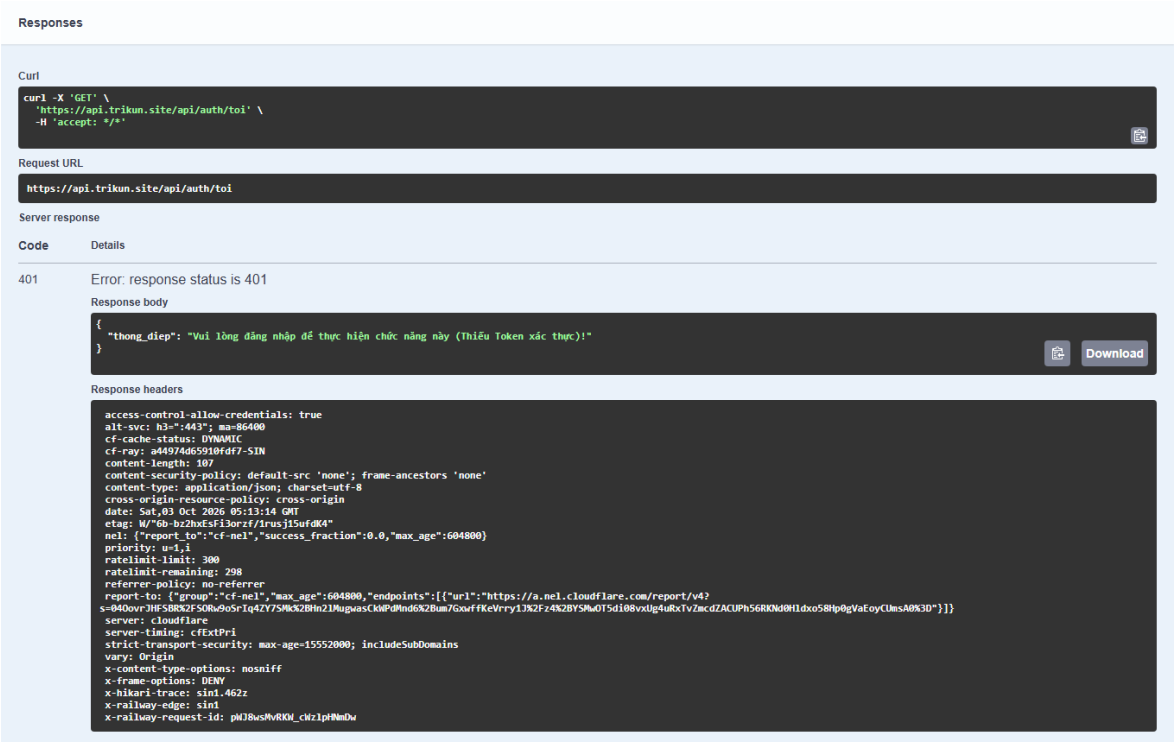
Hình 3.4. *Nhóm API Sản phẩm và Danh mục trên Swagger: API công khai (GET) và API cần JWT của Admin (biểu tượng ổ khóa)*

Nút **Try it out** cho phép gọi thử API ngay trên trang tài liệu và nhận phản hồi thật từ server production. Hình 3.5 gọi `GET /api/san-pham?limit=2` (API công khai): server trả mã **200** kèm dữ liệu sản phẩm, header cho thấy yêu cầu đi qua Cloudflare (`server: cloudflare`), bắt buộc HTTPS (`strict-transport-security`), các header bảo mật và

bộ đếm giới hạn tần suất (`ratelimit-limit: 300`). Hình 3.6 gọi `GET /api/auth/toi` khi chưa đăng nhập: API được bảo vệ nên server từ chối với mã **401**.



Hình 3.5. *Gọi thử GET /api/san-pham?limit=2 bằng Try it out: server production trả 200 kèm dữ liệu và các header bảo mật*



Hình 3.6. *Gọi thử GET /api/auth/toi khi chưa đăng nhập: server từ chối 401 (thiếu token xác thực)*

Song song với Swagger, bộ collection Postman (17 thư mục, gồm 40 ca kiểm thử tự động) cùng hai environment Railway và Cloudflare được lưu tại `_tai_lieu_bao_cao/postman/` để chạy lại toàn bộ kiểm thử (mục 6.4, 6.7.5).

CHƯƠNG 4. HIỆN THỰC CÁC CHỨC NĂNG CHÍNH

Chương này trình bày cách hiện thực từng chức năng quan trọng theo cấu trúc thống nhất: **mục tiêu**, **luồng hoạt động**, **sơ đồ**, **đoạn mã tiêu biểu** và **hình ảnh kết quả**. Các đoạn mã được trích trực tiếp từ mã nguồn thư mục `BE/`, có lược bớt phần log để dễ đọc.

4.1. Chức năng đăng ký và kích hoạt tài khoản bằng OTP

❖ **Mục tiêu:**

- Thu thập họ tên, email, số điện thoại, mật khẩu và kiểm tra hợp lệ.
- Mã hóa mật khẩu trước khi lưu; chỉ tạo tài khoản chính thức sau khi người dùng chứng minh sở hữu email.
- Không để lại tài khoản rác trong `nguoi_dung` khi người dùng bỏ dở.

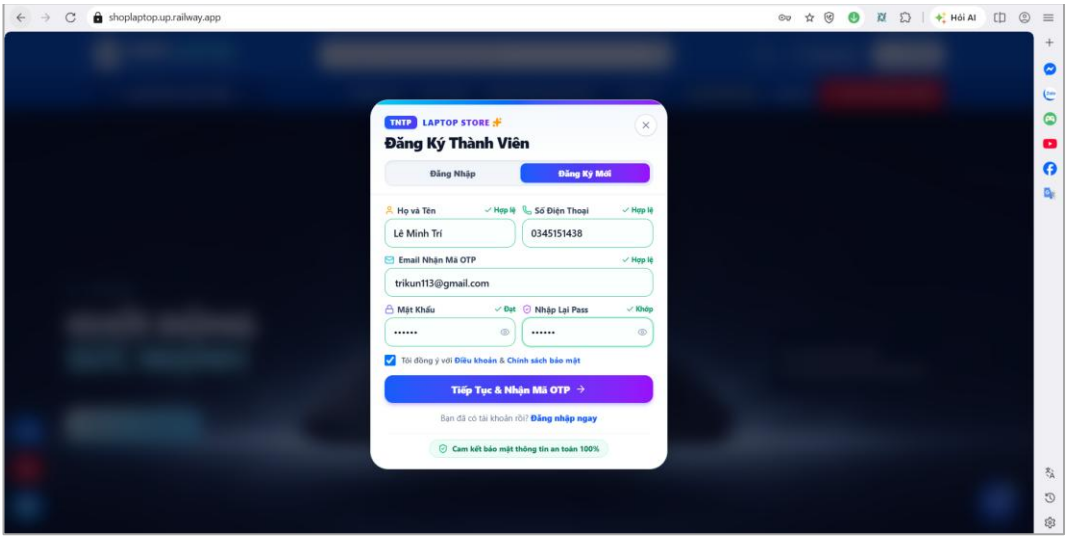
❖ **Luồng hoạt động:**

- **Bước 1:** Client gửi `POST /api/auth/dang-ky`. Server ép kiểu chuỗi, làm sạch dữ liệu (trim, lowercase email, bỏ khoảng trắng SĐT) và kiểm tra: đủ trường, họ tên ≥ 2 ký tự, email đúng định dạng, SĐT Việt Nam 10 số, **mật khẩu mạnh ≥ 8 ký tự có cả chữ và số**.
- **Bước 2:** Nếu email đã có tài khoản kích hoạt → trả **400**.
- **Bước 3:** Băm mật khẩu bằng bcrypt (salt 10); sinh OTP 6 số bằng `crypto.randomInt` (không đoán được như `Math.random`), **chỉ lưu bản băm HMAC-SHA256 của OTP**, hạn 10 phút; upsert vào collection tạm `dang_ky_tam` (có TTL index tự xóa).
- **Bước 4:** Gửi email kích hoạt chứa OTP. Nếu gửi thất bại → trả **502**, cho phép gửi lại qua `/gui-otp`.
- **Bước 5:** Người dùng nhập OTP → `POST /api/auth/kich-hoat`. Server so sánh bản băm bằng `timingSafeEqual`; sai → 400 kèm số lần thử còn lại, **sai 5 lần thì bản ghi tạm bị xóa**; hợp lệ → tạo document trong `nguoi_dung` với `daKichHoat = true`, xóa bản ghi tạm và trả JWT để đăng nhập luôn.

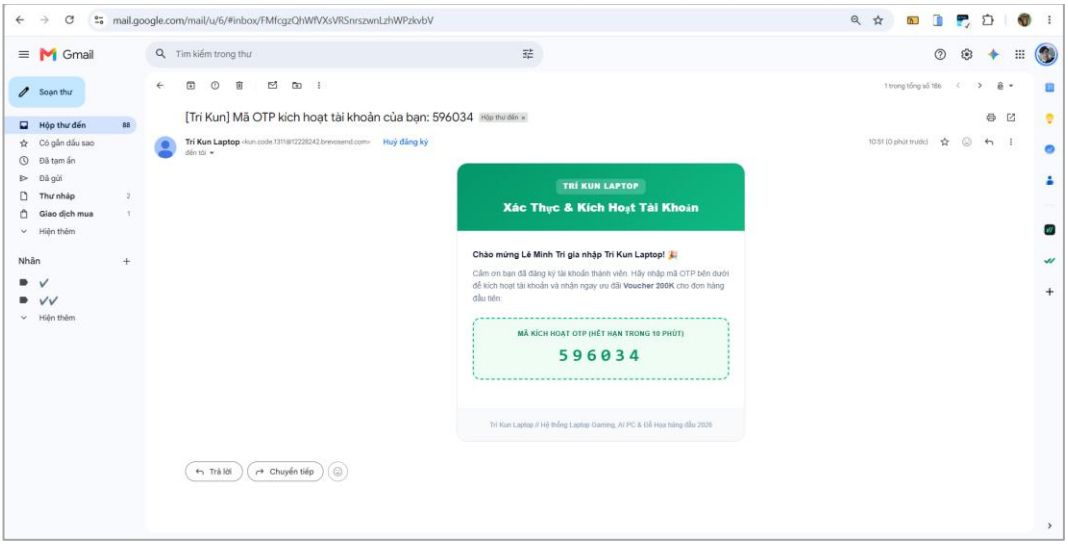
Hình 4.1. *Sơ đồ tuần tự đăng ký và kích hoạt tài khoản bằng OTP*

Lưu đăng ký tạm và gửi OTP

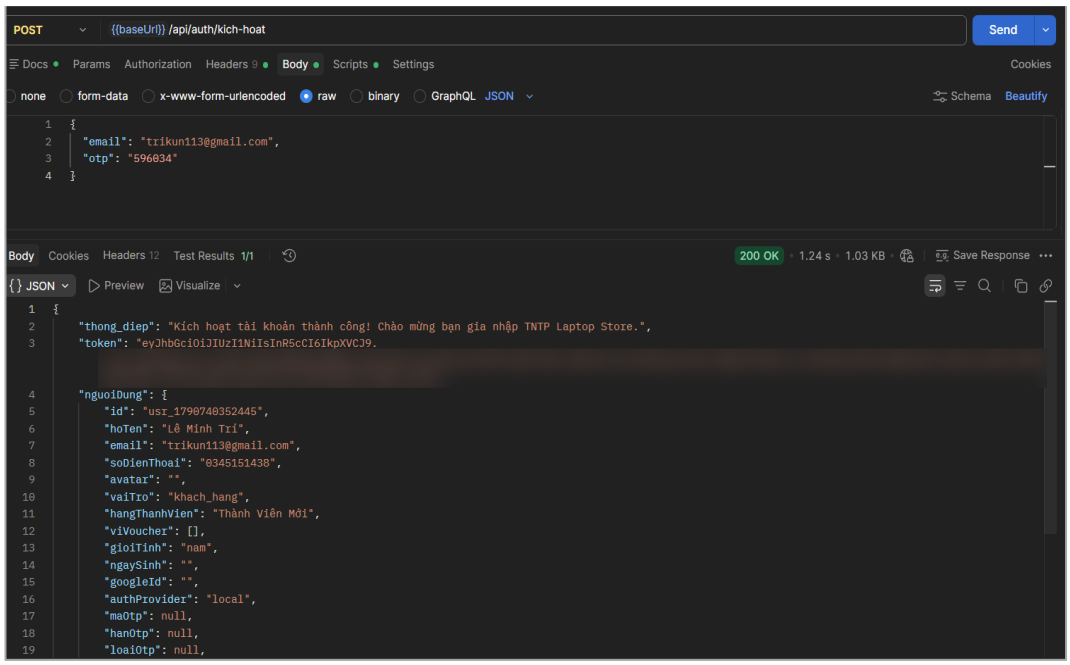
```
// controllers/xac-thuc.controller.js — dangKy (trích)
const salt = await bcrypt.genSalt(10);
const matKhauHash = await bcrypt.hash(mkClean, salt);
const maOtp = taoMaOtp();                          // crypto.randomInt — an toàn mật mã
const hanOtp = new Date(Date.now() + 10 * 60 * 1000);            // 10 phút

const bangTam = mongoose.connection.collection('dang_ky_tam');
bangTam.createIndex({ hanOtp: 1 }, { expireAfterSeconds: 0 });   // TTL: tự xóa khi hết
hạn
await bangTam.updateOne(
    { email: emailClean },
    { $set: { hoTen: hoTenClean, email: emailClean, soDienThoai: sdtClean,
              matKhau: matKhauHash, maOtp: bamOtp(maOtp), hanOtp, loaiOtp:
'kich_hoat' } },  // chỉ lưu bản băm
    { upsert: true }
);
const ketQuaGuiMail = await guiMailKichHoatTaiKhoan(emailClean, hoTenClean, maOtp);
if (!ketQuaGuiMail.thanhCong) return res.status(502).json({ thong_diep: 'Không thể gửi
email OTP...' });
```



Hình 4.2. *Giao diện form đăng ký tài khoản*



Hình 4.3. *Email chứa mã OTP kích hoạt tài khoản*



Hình 4.4. *Kiểm thử POST /api/auth/kich-hoat bằng Postman: kích hoạt thành công, nhận JWT*

4.2. Chức năng đăng nhập, xác thực JWT và phân quyền

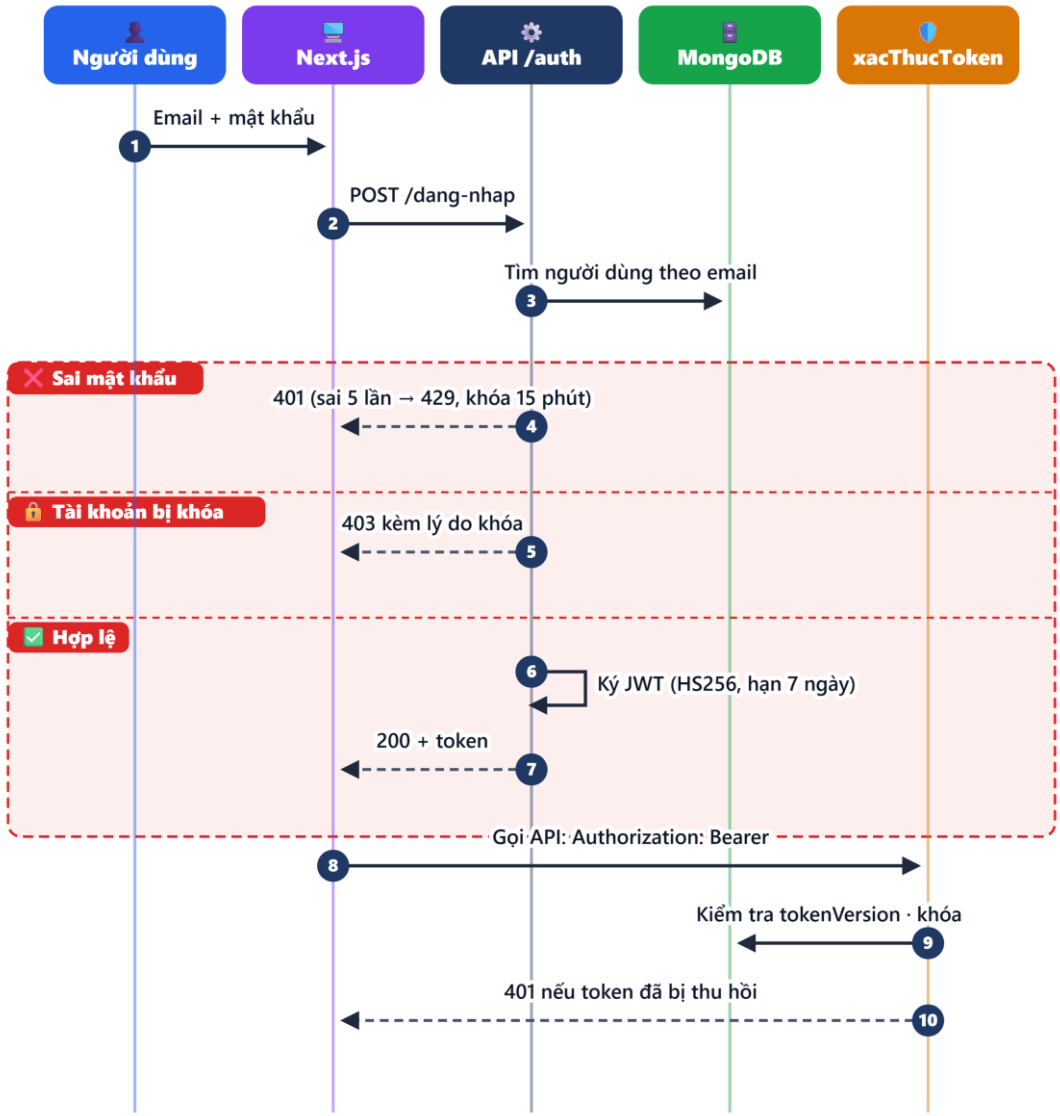
❖ Mục tiêu:

- Cho phép người dùng đăng nhập bằng email và mật khẩu, nhận JWT để gọi các API cần bảo vệ.
- Chặn tài khoản bị khóa; phân biệt quyền khách hàng và quản trị viên ở mọi route nhạy cảm.

❖ Luồng hoạt động:

- **Bước 1:** `POST /api/auth/dang-nhap` với `email`, `matKhau`. Thiếu trường → **400**.
- **Bước 2:** Tìm người dùng theo email; không có hoặc `bcrypt.compare` sai → 401 "Tài khoản hoặc Mật khẩu không chính xác!" (không tiết lộ trường nào sai). Khi email không tồn tại, server vẫn chạy bcrypt với một hash giả để **thời gian phản hồi như nhau**, kẻ tấn công không dò được email nào có thật. **Sai 5 lần liên tiếp → khóa đăng nhập 15 phút (429).**
- **Bước 3:** Tài khoản có `biKhoa` hoặc `trangThai = bi_khoa` → **403** kèm lý do khóa.
- **Bước 4:** Ký JWT chứa `userId`, `email`, `vaiTro`, hạn 7 ngày → trả **200** cùng thông tin người dùng (đã loại `matKhau`).

- **Bước 5:** Ở các lần gọi sau, `xacThucToken` giải mã token (chỉ chấp nhận thuật toán HS256), tìm lại người dùng trong DB, kiểm tra tài khoản chưa bị khóa và `tokenVersion` trong token khớp DB (token chưa bị thu hồi) rồi gắn vào `req.user`; `chiAdmin` kiểm tra `req.user.vaiTro === 'admin'`.

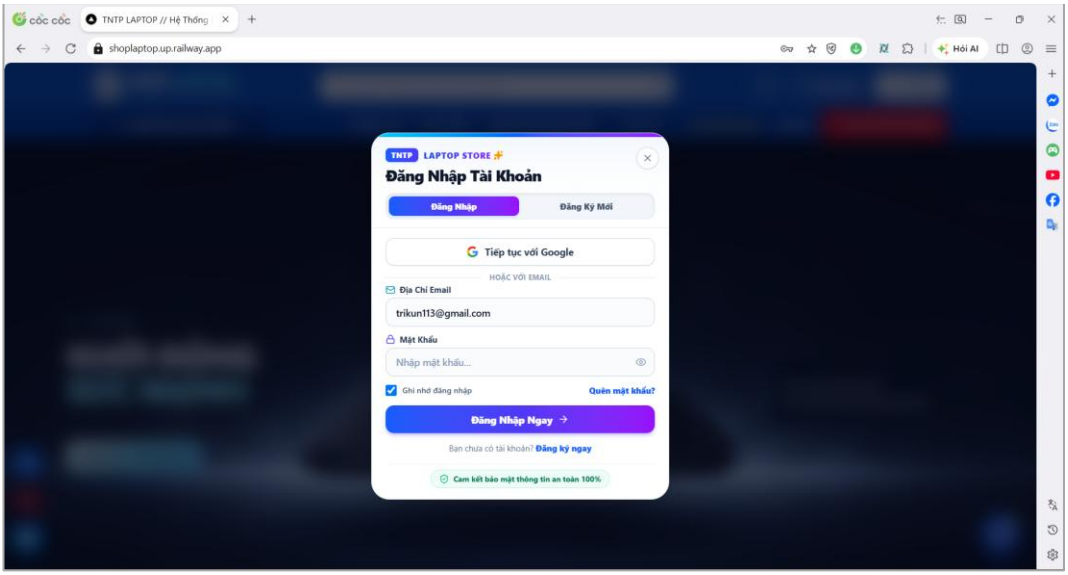


Hình 4.5. *Sơ đồ tuần tự đăng nhập và gọi API được bảo vệ*

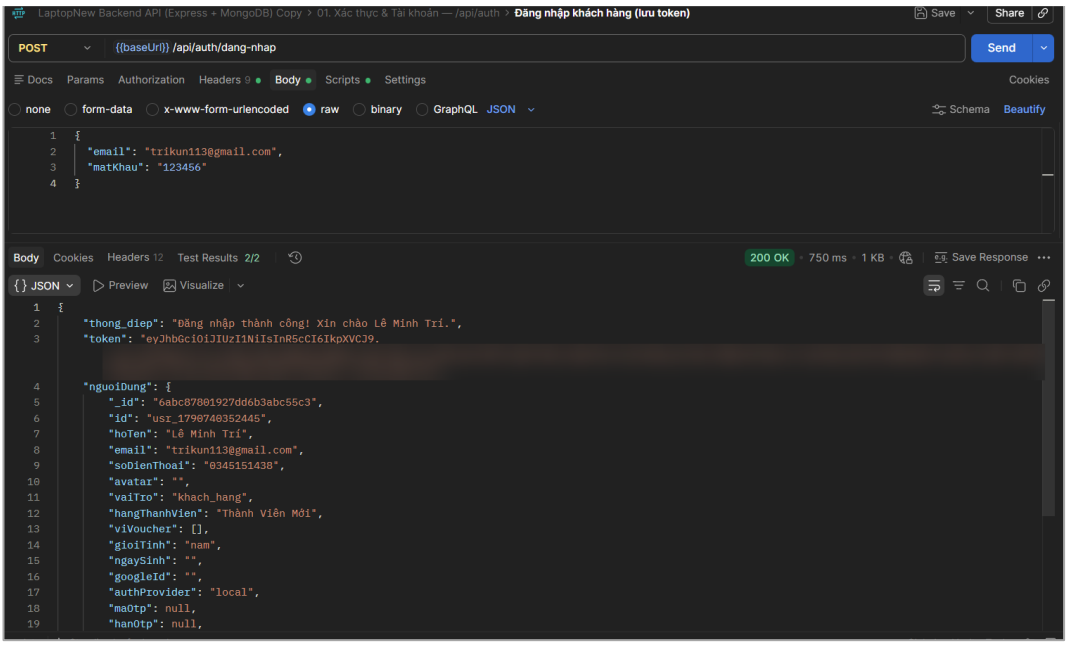
Tạo JWT và hai middleware xác thực, phân quyền

```
// Tạo token – controllers/xac-thuc.controller.js
const taoToken = (nguoiDung) => jwt.sign(
    { userId: nguoiDung.id, email: nguoiDung.email, vaiTro: nguoiDung.vaiTro,
tokenVersion: nguoiDung.tokenVersion },
    JWT_SECRET, { expiresIn: JWT_EXPIRES_IN }            // mặc định '7d'
);

// middleware/xac-thuc.middleware.js
const xacThucToken = async (req, res, next) => {
    try {
        const header = req.headers.authorization;
        const token = header && header.startsWith('Bearer ') ? header.split(' ')[1] :
null;
```

```
        if (!token) return res.status(401).json({ thong_diep: 'Vui lòng đăng nhập ...
(Thiếu Token xác thực)!' });
        const decoded = jwt.verify(token, process.env.JWT_SECRET, { algorithms:
['HS256'] });
        const nguoiDung = await NguoiDung.findOne({ $or: [{ id: decoded.userId },
{ email: decoded.email }] });
        if (!nguoiDung) return res.status(401).json({ thong_diep: 'Tài khoản không tồn
tại hoặc đã bị khóa!' });
        if (decoded.tokenVersion !== nguoiDung.tokenVersion)      // Token đã bị thu hồi
(đăng xuất / đổi MK / bị khóa)
            return res.status(401).json({ thong_diep: 'Phiên đăng nhập đã bị thu hồi. Vui
lòng đăng nhập lại.' });
        req.user = nguoiDung;
        next();
    } catch (loi) {
        if (loi.name === 'TokenExpiredError')
            return res.status(401).json({ thong_diep: 'Phiên đăng nhập đã hết hạn, vui
lòng đăng nhập lại!' });
        return res.status(401).json({ thong_diep: 'Token xác thực không hợp lệ hoặc đã bị
thay đổi!' });
    }
};

const chiAdmin = (req, res, next) => {
    if (!req.user || req.user.vaiTro !== 'admin')
        return res.status(403).json({ thong_diep: 'Truy cập bị từ chối: Bạn không có
quyền Quản trị viên...' });
    next();
};
```



Hình 4.6. *Giao diện đăng nhập (email/mật khẩu và đăng nhập nhanh với Google)*



Hình 4.7. *Kiểm thử POST /api/auth/dang-nhap bằng Postman: đăng nhập thành công, nhận token JWT*

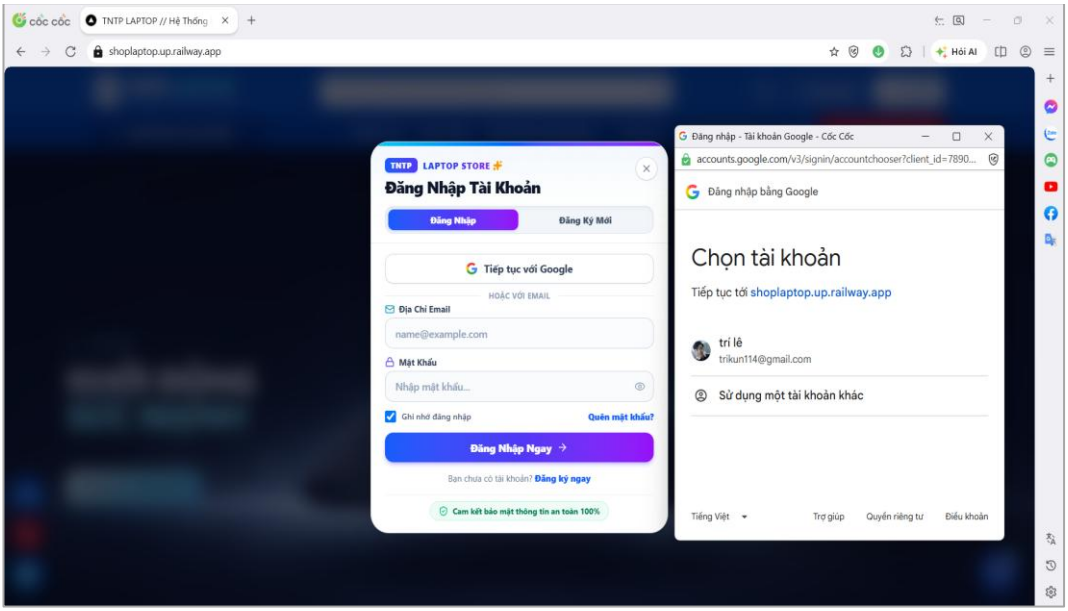
4.3. Chức năng đăng nhập bằng Google

❖ Mục tiêu:

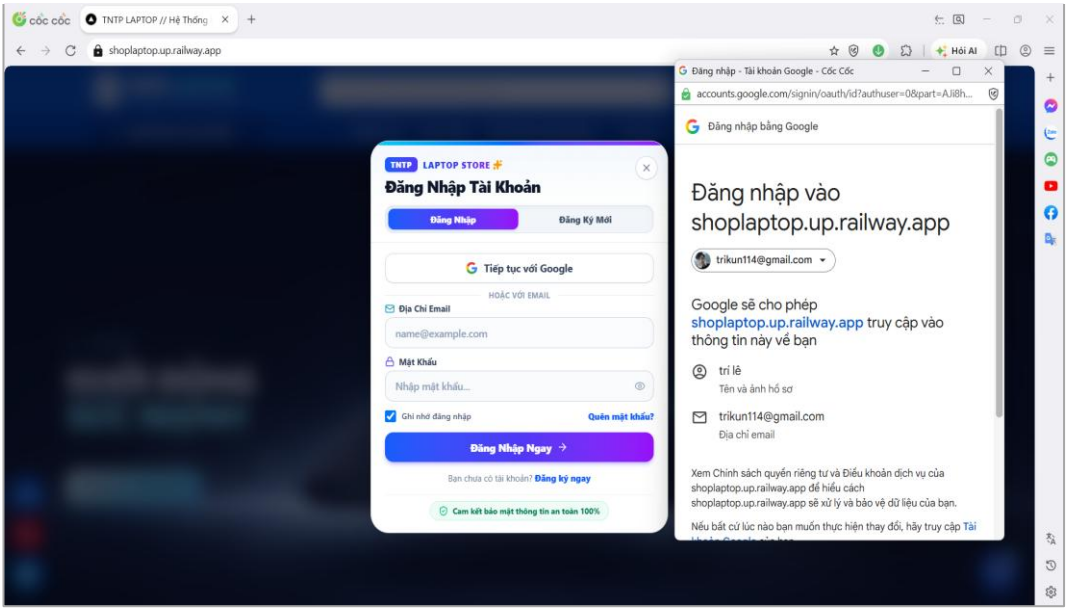
- Cho phép đăng nhập một chạm bằng tài khoản Google, tự tạo tài khoản nếu email chưa có.

❖ Luồng hoạt động:

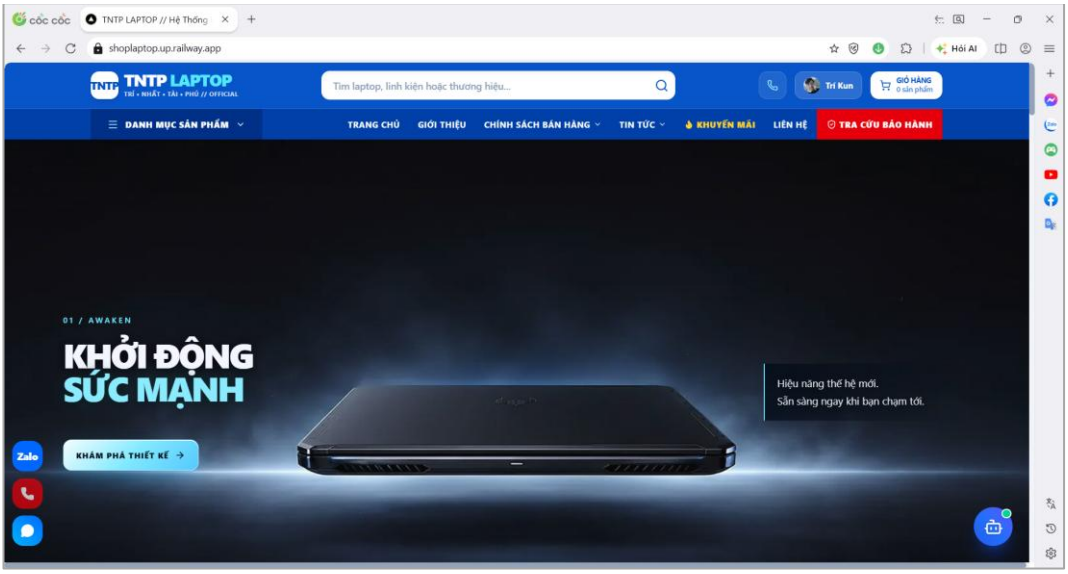
- Front-End dùng Google Sign-In, nhận **ID Token** (`credential`) và gửi tới `POST /api/auth/google`.
- Server **không bao giờ tin email do trình duyệt gửi**, chỉ tin bằng chứng do Google cấp: nếu là **ID Token** thì gọi `verifyIdToken({ idToken, audience: GOOGLE_CLIENT_ID })`; nếu là **Access Token** (cửa sổ OAuth2) thì gọi `getTokenInfo` để kiểm tra token được cấp cho đúng ứng dụng (`aud`), rồi hỏi `userinfo` của Google để lấy email, name, picture, sub thật. Email phải có `email_verified`.
- Tìm người dùng theo `email` hoặc `googleId`. Chưa có → tạo mới với `authProvider = google`, mật khẩu ngẫu nhiên đã băm. Đã có → liên kết `googleId`.
- Kiểm tra khóa tài khoản (**403**), sau đó trả JWT như đăng nhập thường.



Hình 4.8. *Chọn tài khoản Google khi đăng nhập*



Hình 4.9. *Google xác nhận chia sẻ tên, ảnh và email với website*



Hình 4.10. *Đăng nhập Google thành công, thanh điều hướng hiển thị tên người dùng*

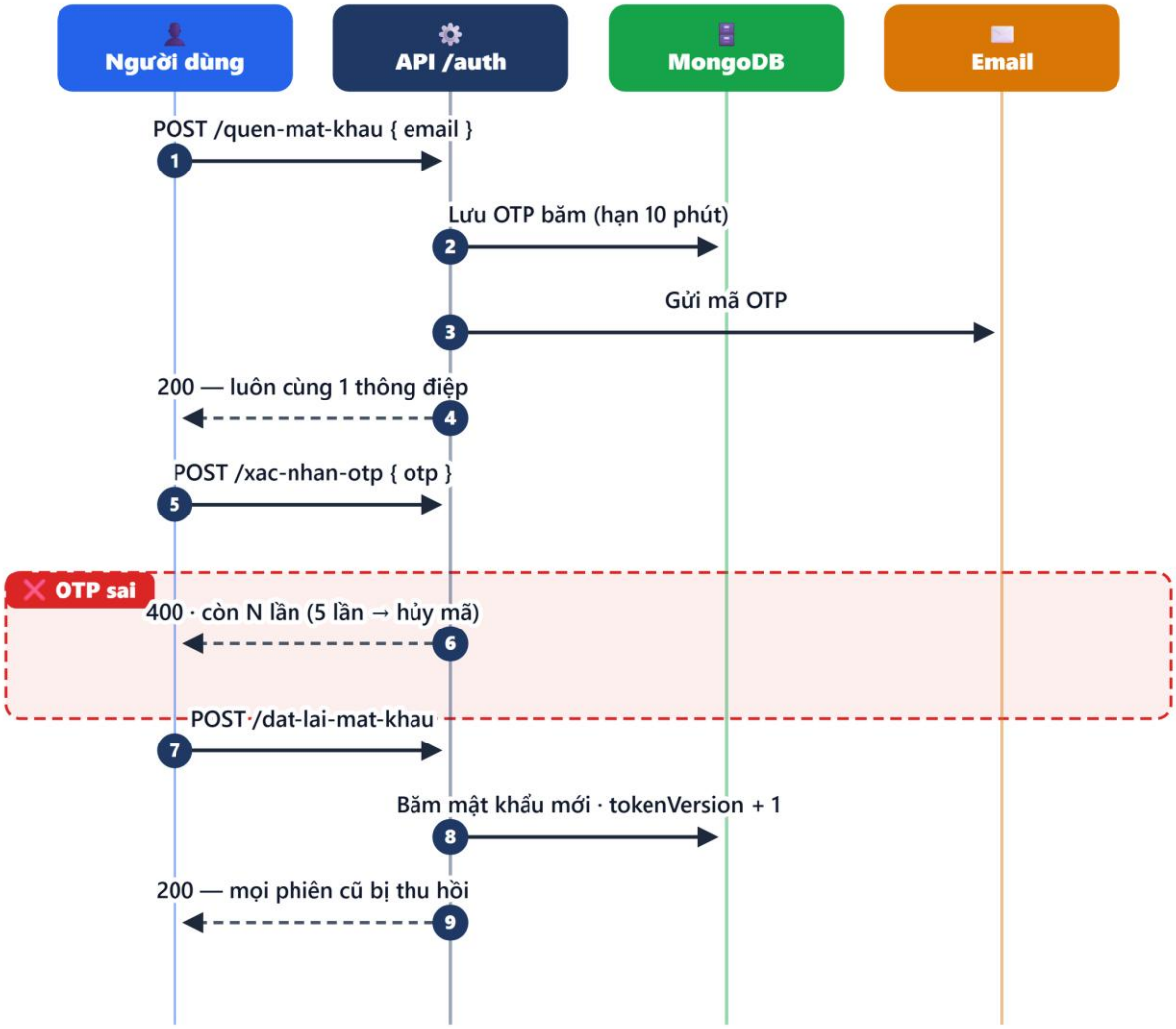
4.4. Chức năng quên mật khẩu và đổi mật khẩu

❖ Mục tiêu:

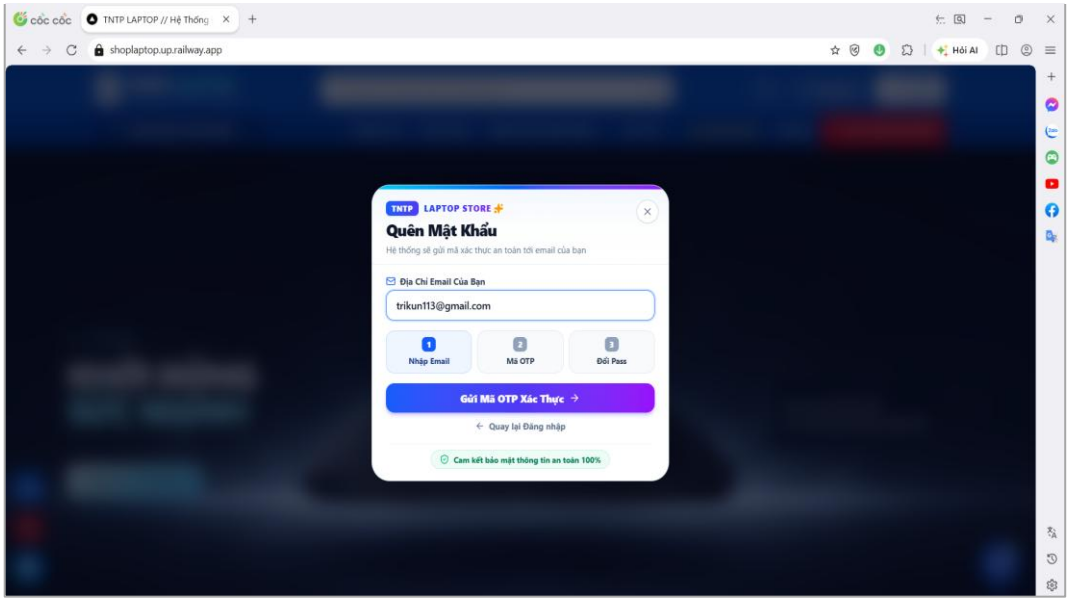
- Hỗ trợ người dùng đặt lại mật khẩu khi quên, bằng mã OTP gửi về email.
- Khi đổi mật khẩu lúc đã đăng nhập, yêu cầu cả mật khẩu cũ và OTP để chống chiếm quyền.

❖ Luồng hoạt động:

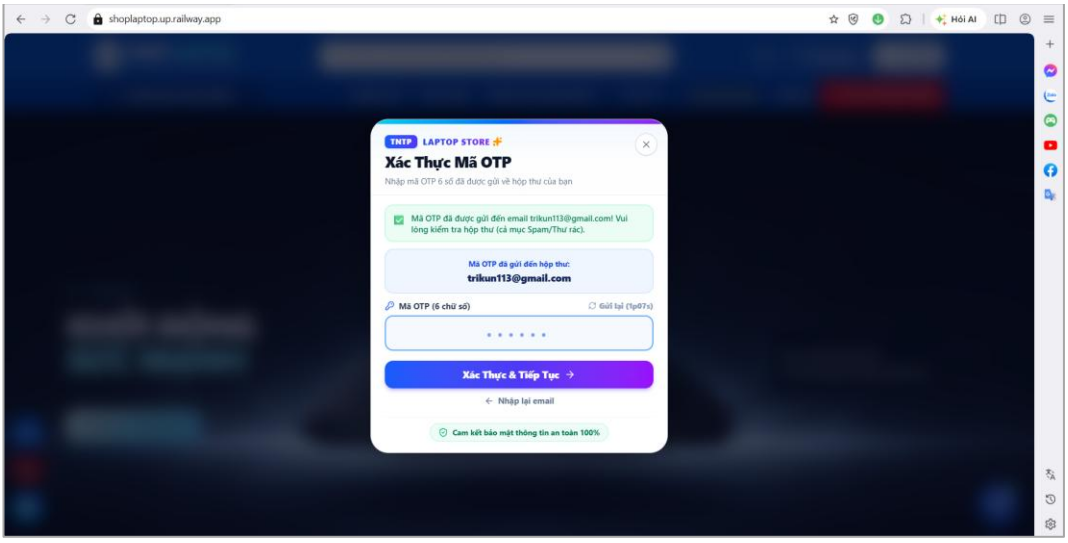
- **Quên mật khẩu:** `POST /quen-mat-khau` **luôn trả 200 với cùng một thông điệp** dù email có tồn tại hay không (chống dò email); nếu tồn tại thì lưu bản băm `maOtp`, `hanOtp` (+10 phút), `loaiOtp = quen_mat_khau` và gửi email.
- `POST /xac-nhan-otp` kiểm tra mã, hạn và **đúng loại OTP** (`quen_mat_khau`) — OTP của luồng khác không dùng được; nhập sai 5 lần thì mã bị hủy. `POST /dat-lai-mat-khau` kiểm tra mật khẩu mới đạt chính sách (≥ 8 ký tự, có chữ và số), băm và lưu, xóa OTP và **thu hồi mọi phiên đăng nhập cũ**.
- **Đổi mật khẩu:** `POST /doi-mat-khau/gui-otp` (JWT) kiểm tra `matKhauCu` rồi gửi OTP loại `doi_mat_khau`; `PUT /doi-mat-khau` chỉ chấp nhận khi đúng loại OTP, đúng mã và còn hạn.



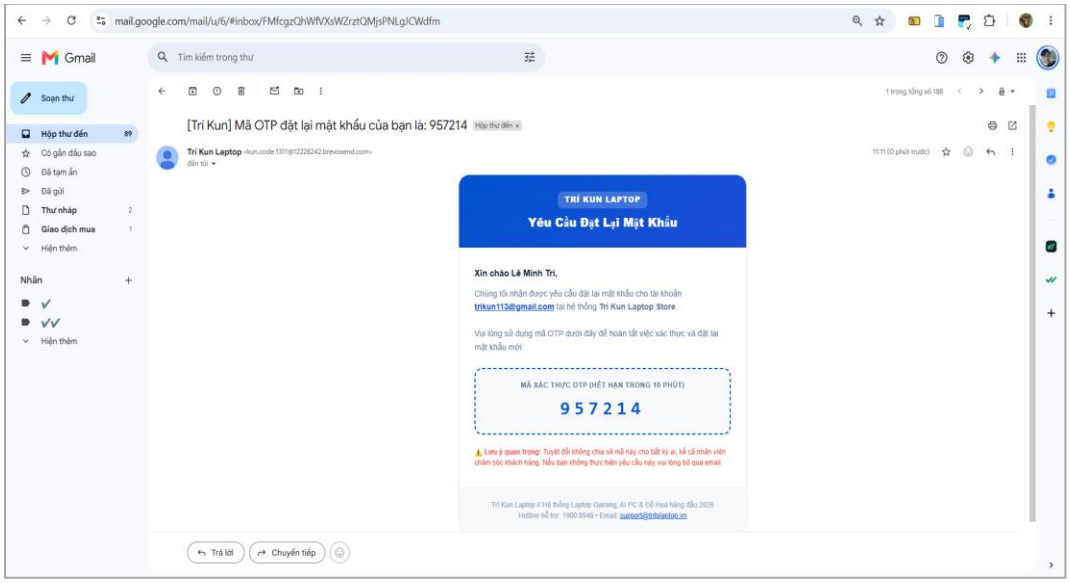
Hình 4.11. *Sơ đồ tuần tự quên mật khẩu bằng OTP*



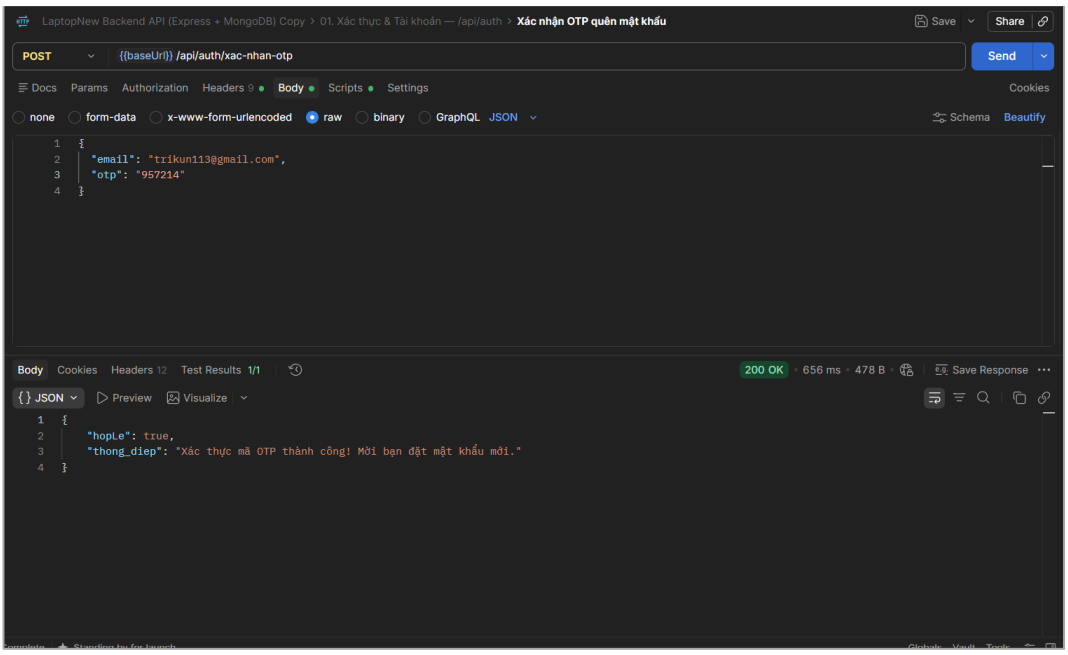
Hình 4.12. *Trang quên mật khẩu: nhập email để nhận mã OTP*



Hình 4.13. *Bước nhập mã OTP xác thực*



Hình 4.14. *Email chứa mã OTP đặt lại mật khẩu*



Hình 4.15. *Kiểm thử POST /api/auth/xac-nhan-otp bằng Postman: OTP hợp lệ*

4.5. Chức năng quản lý sản phẩm

4.5.1. Tìm kiếm, lọc và sắp xếp

❖ **Mục tiêu:**

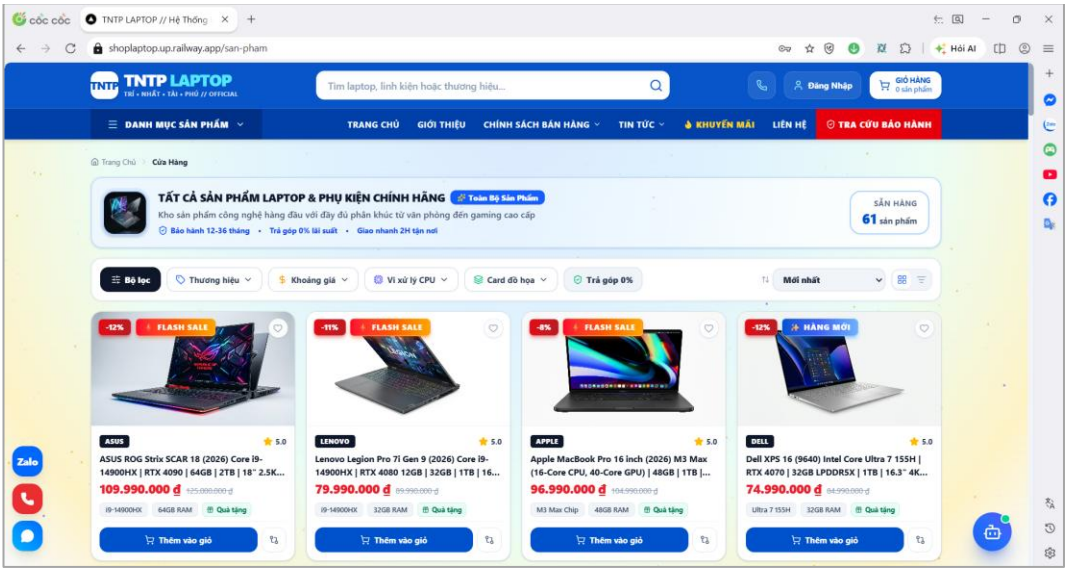
- Cho phép khách tìm laptop theo từ khóa, hãng, nhu cầu, khoảng giá và sắp xếp theo nhiều tiêu chí trong một API duy nhất.

❖ **Luồng hoạt động:**

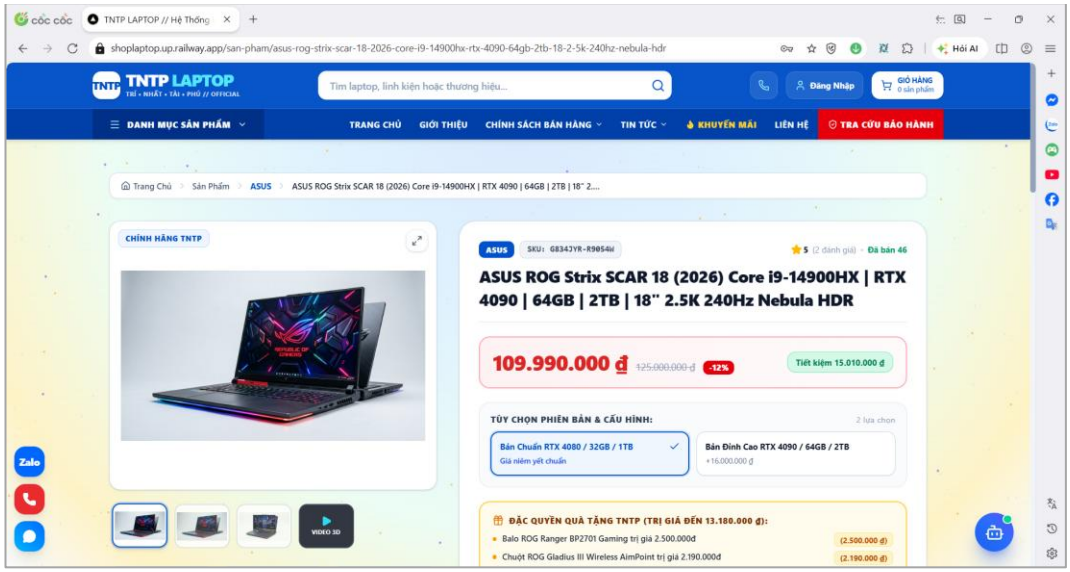
- Controller đọc các tham số query và **xây dựng dần đối tượng** `filter` của MongoDB.
- Từ khóa tìm trên 5 trường (tên, mã, hãng, CPU, card đồ họa) bằng `$regex` không phân biệt hoa thường.
- Hãng và danh mục cho phép nhiều giá trị (`asus,dell`) → toán tử `$in`; khoảng giá → `$gte` / `$lte`.
- Áp dụng `sort` theo `sap_xep`, `limit` nếu có; đặt header `Cache-Control: public, max-age=60` để trình duyệt/CDN lưu tạm 60 giây.

Xây dựng bộ lọc MongoDB từ tham số query

```
// controllers/san-pham.controller.js — layTatCaSanPham (trích)
const filter = {};
const keyword = tu_khoa || q;
if (keyword) filter.$or = [
    { ten_san_pham: { $regex: keyword, $options: 'i' } },
    { ma_san_pham:  { $regex: keyword, $options: 'i' } },
    { hang_san_xuat:{ $regex: keyword, $options: 'i' } },
    { 'thong_so.cpu':         { $regex: keyword, $options: 'i' } },
    { 'thong_so.card_do_hoa': { $regex: keyword, $options: 'i' } }
];
if (hang_san_xuat) filter.hang_san_xuat = { $in: hang_san_xuat.split(',').map(b =>
b.trim().toLowerCase()) };
if (danh_muc)      filter.danh_muc      = { $in: danh_muc.split(',').map(c =>
c.trim().toLowerCase()) };
if (gia_min !== undefined || gia_max !== undefined) {
    filter.gia_khuyen_mai = {};
    if (gia_min !== undefined) filter.gia_khuyen_mai.$gte = Number(gia_min);
    if (gia_max !== undefined) filter.gia_khuyen_mai.$lte = Number(gia_max);
}
let query = SanPham.find(filter).sort(sortOption);
if (limit) query = query.limit(Number(limit));
```



Hình 4.16. *Trang danh sách sản phẩm với bộ lọc thương hiệu, khoảng giá, CPU, card đồ họa*

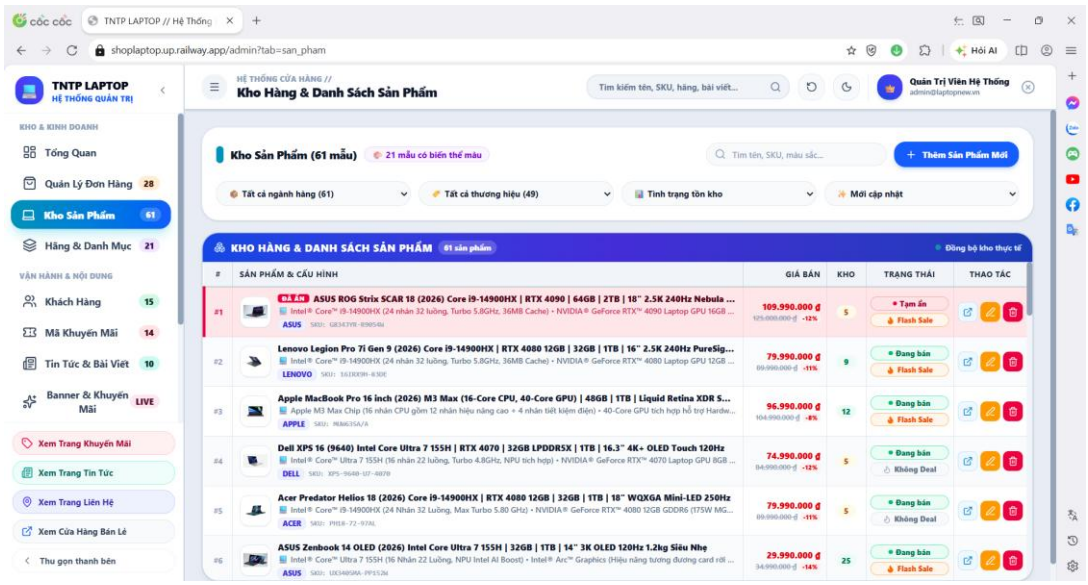


Hình 4.17. *Trang chi tiết sản phẩm: thư viện ảnh, phiên bản cấu hình, giá khuyến mãi*

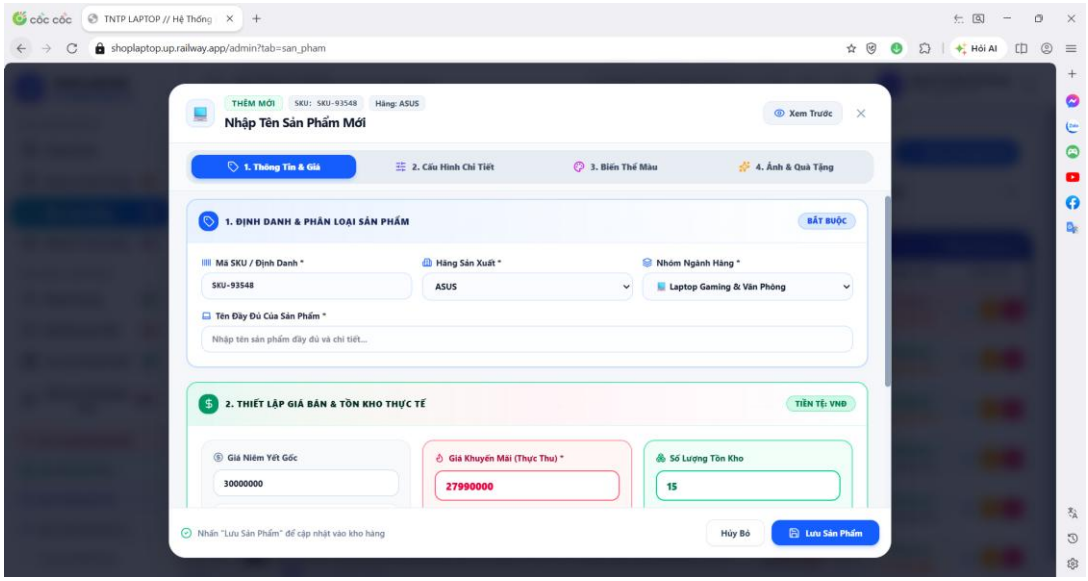
4.5.2. Thêm, sửa, xóa sản phẩm (Admin)

❖ Luồng hoạt động:

- Ba route `POST`, `PUT /:id`, `DELETE /:id` đều qua `xacThucToken` → `chiAdmin`.
- Khi thêm: tự sinh `id` (`sp-<timestamp>`) và `slug` nếu thiếu; kiểm tra trùng `id`/`slug`/`ma_san_pham` → **409 Conflict**.
- Khi sửa / xóa: tìm theo nhiều loại định danh; không thấy → **404**.



(a) *Danh sách sản phẩm trong trang quản trị*



(b) *Form thêm / sửa thông tin sản phẩm*

Hình 4.18. *Trang quản trị: danh sách và form thêm/sửa sản phẩm*

4.6. Chức năng giỏ hàng và mã giảm giá

❖ Mục tiêu:

- Kiểm tra mã voucher khách nhập, tính đúng số tiền được giảm theo loại voucher và điều kiện đơn tối thiểu.

❖ Luồng hoạt động:

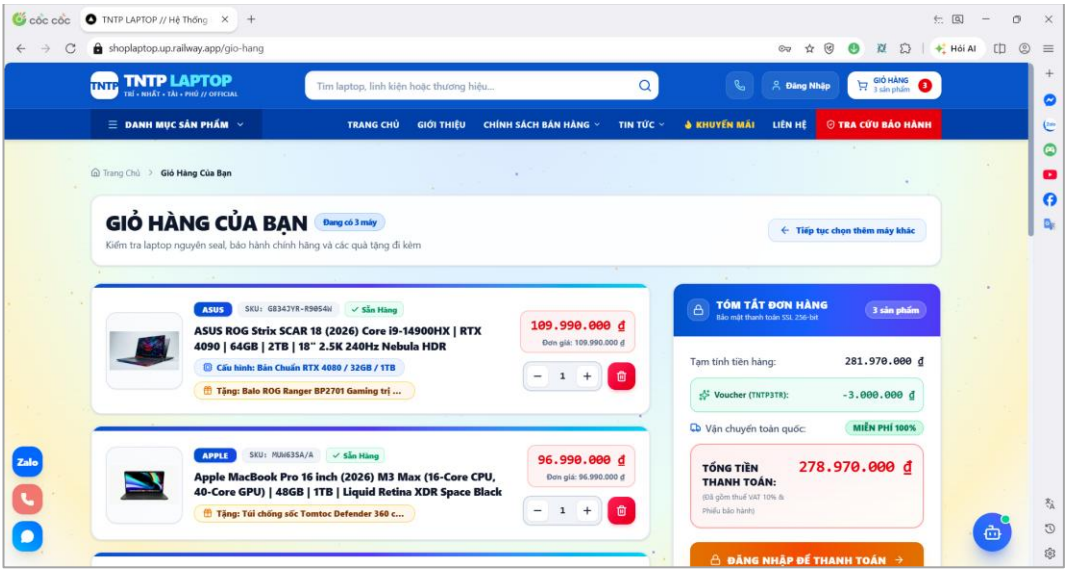
- Giỏ hàng được quản lý ở Front-End (React Context). Khi khách nhập mã, client gọi `POST /api/ma-giam-gia/kiem-tra` với `ma_code` và `tong_tien_tam_tinh`.

- Server chuẩn hóa mã IN HOA, chỉ tìm voucher đang `kich_hoat`; tính tiền giảm: loại `tien_mat` giảm cố định, loại `phan_tram` giảm theo % nhưng không vượt `giam_toi_da`.
- Thành viên có thể "săn mã" vào ví bằng `POST /luu-ma` và xem lại bằng `GET /vi-cua-toi`. Số lượt còn lại được trừ bằng **thao tác nguyên tử** (`findOneAndUpdate` kèm điều kiện `so_luong_con_lai > 0`), nên nhiều người bấm cùng lúc cũng không vượt quá số lượng phát hành.

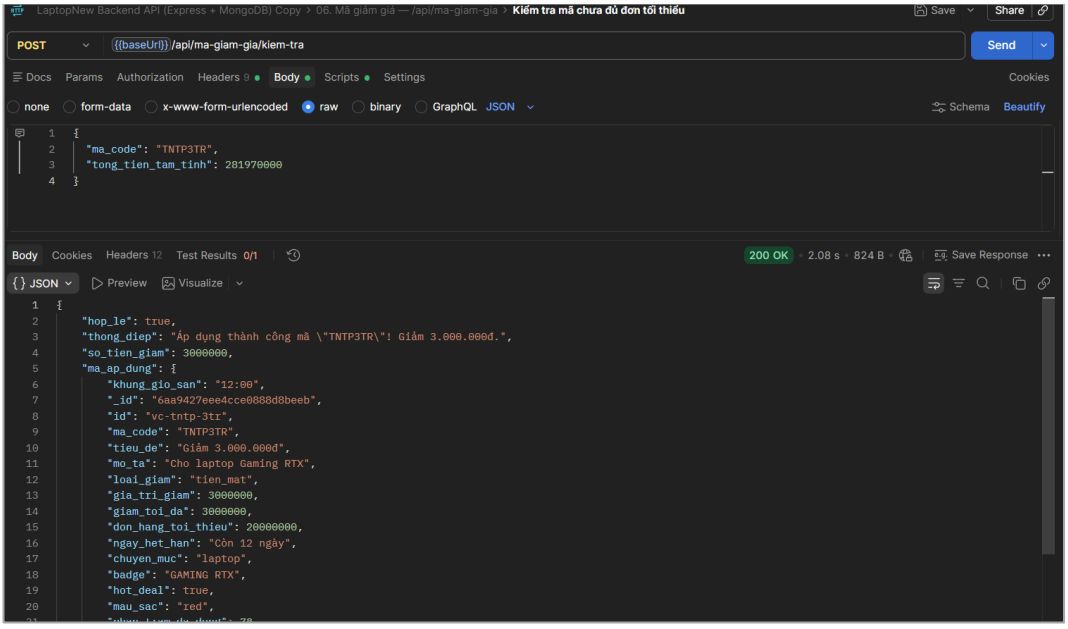
Hình 4.19. *Lưu đồ kiểm tra và tính tiền mã giảm giá*

Tính số tiền giảm của voucher

```
// controllers/ma-giam-gia.controller.js — kiemTraMaGiamGia (trích)
const voucher = await MaGiamGia.findOne({ ma_code: ma_code.trim().toUpperCase(),
kich_hoat: true });
if (!voucher) return res.status(404).json({ hop_le: false, thong_diep: 'Mã giảm giá không
tồn tại...' });
const tamTinh = Number(tong_tien_tam_tinh) || 0;
if (tamTinh < voucher.don_hang_toi_thieu) return res.status(400).json({ hop_le:
false, ... });
let soTienGiam = 0;
if (voucher.loai_giam === 'tien_mat') soTienGiam = voucher.gia_tri_giam;
else {
    soTienGiam = Math.round((tamTinh * voucher.gia_tri_giam) / 100);
    if (voucher.giam_toi_da && soTienGiam > voucher.giam_toi_da) soTienGiam =
voucher.giam_toi_da;
}
return res.status(200).json({ hop_le: true, so_tien_giam: soTienGiam, ma_ap_dung:
voucher });
```



Hình 4.20. *Giỏ hàng áp dụng mã giảm giá TNTP3TR thành công*



Hình 4.21. *Kiểm thử POST /api/ma-giam-gia/kiem-tra bằng Postman: mã hợp lệ, giảm 3.000.000 đ*

4.7. Chức năng đặt hàng và quản lý trạng thái đơn

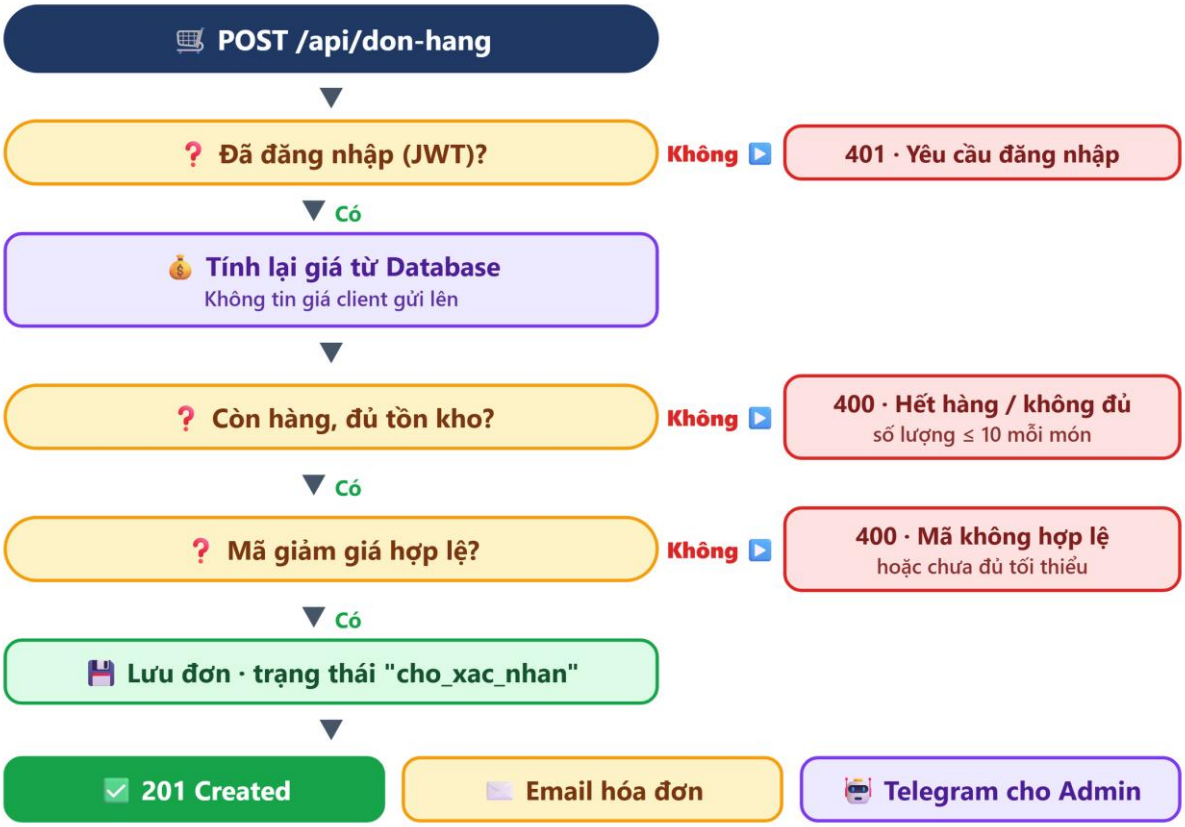
4.7.1. Tạo đơn hàng

❖ **Mục tiêu:**

- Ghi nhận đơn hàng hợp lệ của người dùng đã đăng nhập, chuẩn hóa dữ liệu và thông báo ngay cho khách và quản trị viên.

❖ **Luồng hoạt động:**

- Route gắn `xacThucToken`: chưa đăng nhập → 401 "Quý khách vui lòng đăng nhập...".
- Tự sinh `id`, `ma_don_hang` dạng `LPN-2026-xxxx`, `ngay_tao`; đồng bộ `ho_ten`/`ho_va_ten`.
- **Chống sửa giá:** server bỏ qua giá client gửi lên, tự tra từng sản phẩm trong DB để tính lại đơn giá (kể cả chênh lệch phiên bản), kiểm tra tồn kho, áp lại voucher và tính `tong_tien_thanh_toan`. Thông tin giao hàng được ép kiểu chuỗi, cắt độ dài và kiểm tra SĐT.
- Chuẩn hóa trạng thái thanh toán: COD → `thanh_toan_khi_nhan_hang`; chuyển khoản → `cho_thanh_toan`.
- Lưu đơn, trả **201**, rồi chạy **không chặn** hai tác vụ: gửi email hóa đơn và gửi thông báo Telegram (lỗi ở hai tác vụ này không làm hỏng đơn).



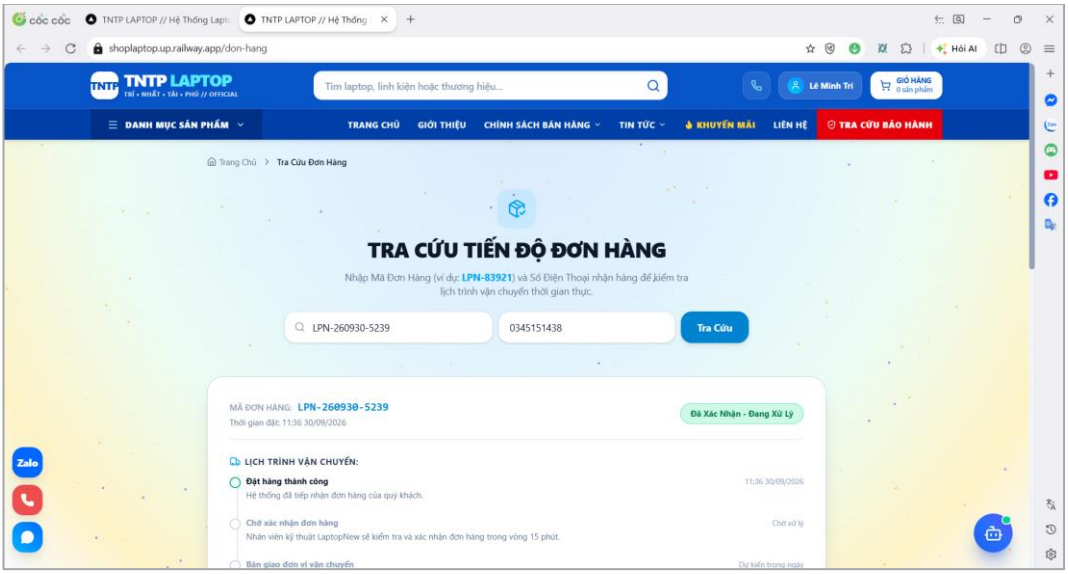
Hình 4.22. *Lưu đồ xử lý tạo đơn hàng*

4.7.2. Phân quyền xem và cập nhật đơn

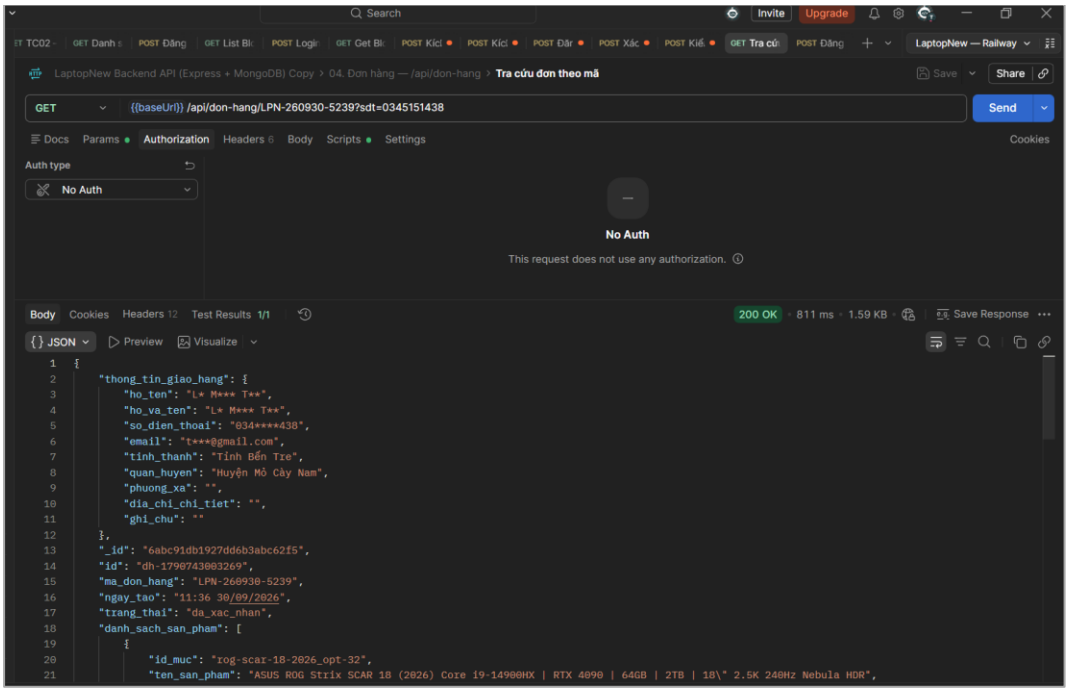
- **Admin** xem được mọi đơn, lọc theo trạng thái/người dùng; cập nhật mọi trường.
- Thành viên chỉ xem đơn gắn `id_nguoi_dung` của chính tài khoản (email chỉ dùng cho đơn cũ chưa gắn id). **Không so khớp theo SĐT** vì SĐT trong hồ sơ do người dùng tự khai — nếu so theo SĐT, kẻ xấu chỉ cần sửa SĐT hồ sơ thành SĐT nạn nhân là xem được đơn của họ (lỗi IDOR). Chủ đơn chỉ được hủy đơn khi còn `cho_xac_nhan`; sửa đơn của người khác → 403.
- `GET /api/don-hang` bắt buộc JWT: không có token → 401, không bao giờ lộ danh sách đơn.
- **Tra cứu công khai** (`GET /api/don-hang/:id`): khách chưa đăng nhập phải nhập đúng **mã đơn** và **số điện thoại nhận hàng** (`?sdt=`); thiếu hoặc sai → **403** kèm `yeu_cau_xac_minh_sdt: true`.
- Dữ liệu trả cho khách được che bởi `anThongTinDonHangCongKhai()`: họ tên `L* M*** T**`, SĐT `034****438`, email `t***@gmail.com`, bỏ địa chỉ chi tiết và ghi chú. Đăng nhập nhưng không phải chủ đơn → **403**.

- Middleware `gioiHanTraCuuDonHang` giới hạn **30 lần / 10 phút** cho mỗi IP; vượt quá → **429**, chống dò mã đơn hàng loạt.

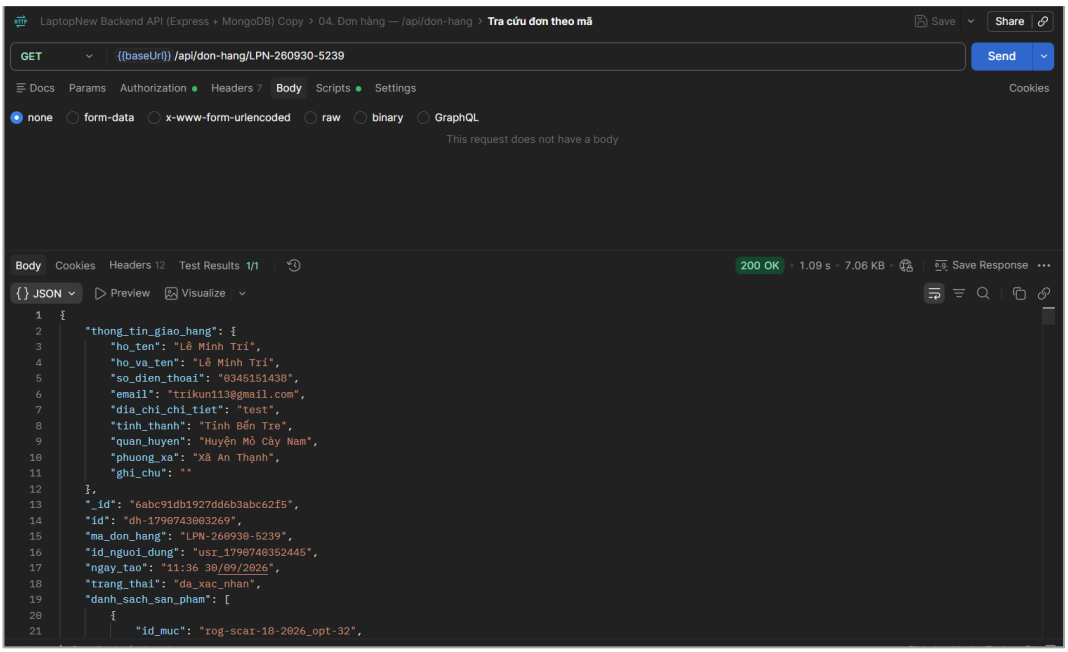
```
Phân quyền xem đơn và che dữ liệu khi tra cứu công khai
// controllers/don-hang.controller.js — layDonHangTheoIdHoacMa (trích)
if (laChuDonHang(req, donHang)) return res.status(200).json(donHang);   // Admin hoặc chủ đơn
if (req.user) return res.status(403).json({ thong_diep: 'Bạn chỉ được xem đơn hàng thuộc tài khoản của mình.' });
if (!maDonDuocNhap || !sdtXacMinh || sdtXacMinh !== sdtKhach)
    return res.status(403).json({ thong_diep: 'Vui lòng nhập đúng mã đơn hàng và số điện thoại...', yeu_cau_xac_minh_sdt: true });
return res.status(200).json(anThongTinDonHangCongKhai(donHang));        // Che họ tên, SĐT, email, địa chỉ
```



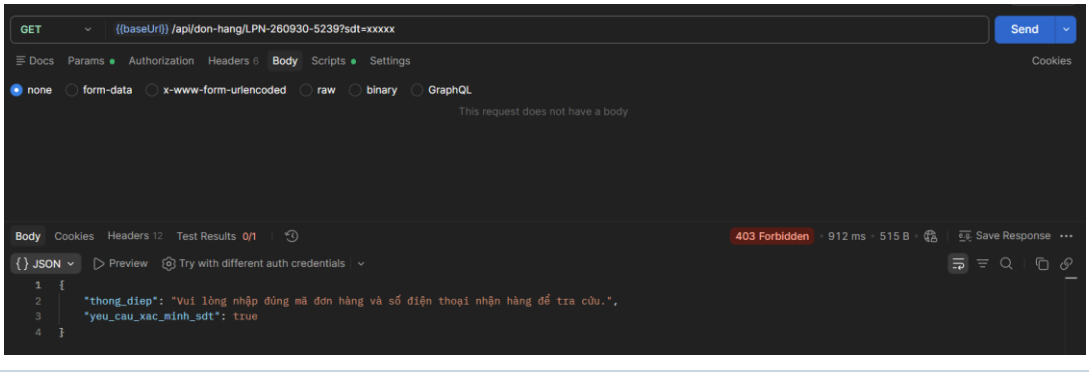
Hình 4.23. *Trang tra cứu tiến độ đơn hàng: khách nhập mã đơn và số điện thoại*



Hình 4.24. *Kiểm thử GET /api/don-hang/:id?sdt=... khi chưa đăng nhập: thông tin cá nhân đã được che*



Hình 4.25. *Kiểm thử GET /api/don-hang/:id với token chủ đơn: trả đầy đủ thông tin đơn hàng*

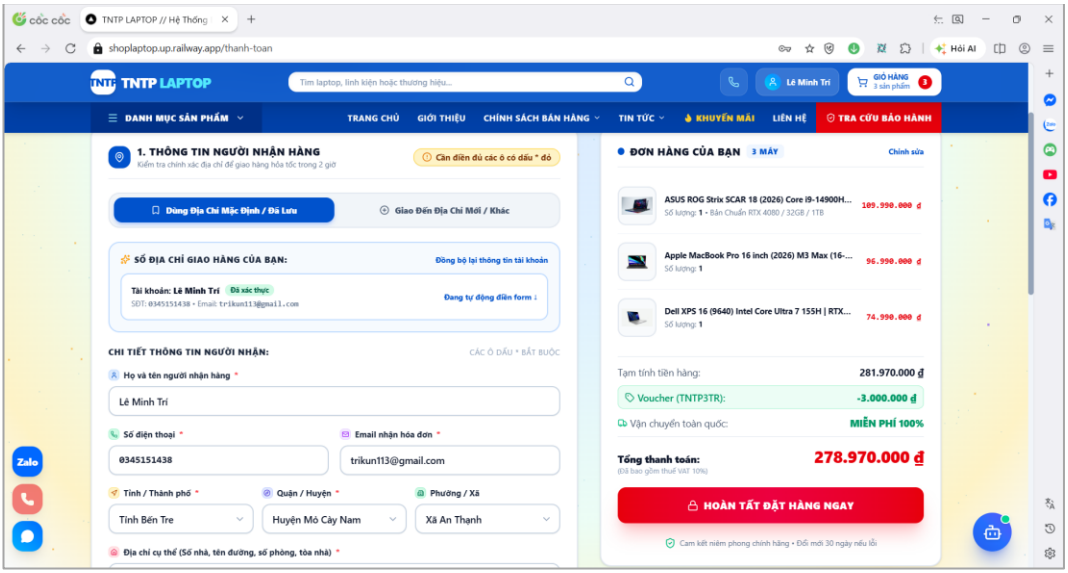


Hình 4.26. *Kiểm thử tra cứu đơn thiếu hoặc sai số điện thoại bị từ chối (403)*

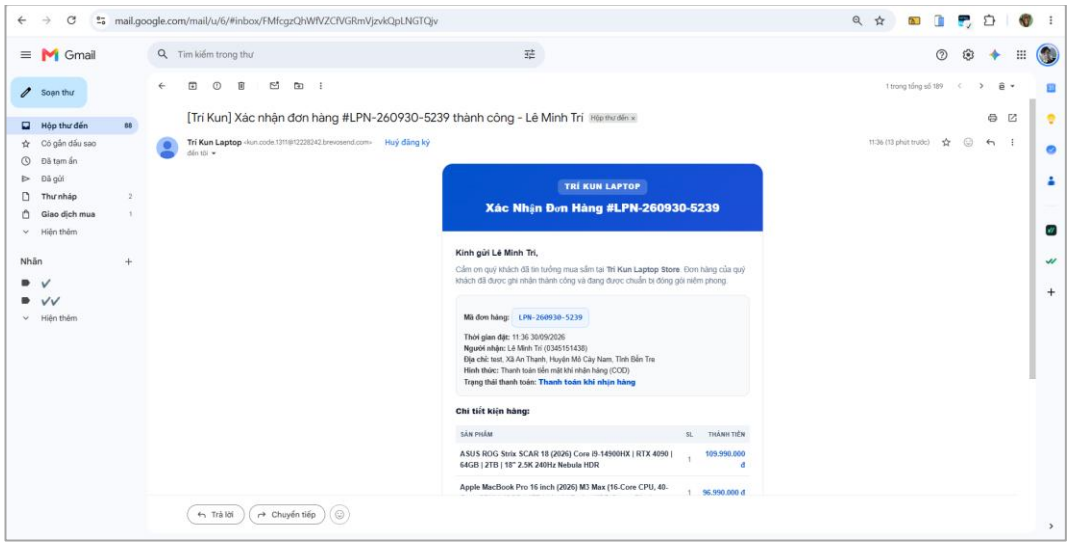
4.7.3. Trạng thái đơn và đồng bộ tồn kho

Hình 4.27. *Vòng đời trạng thái đơn hàng và quy tắc trừ/hoàn tồn kho*

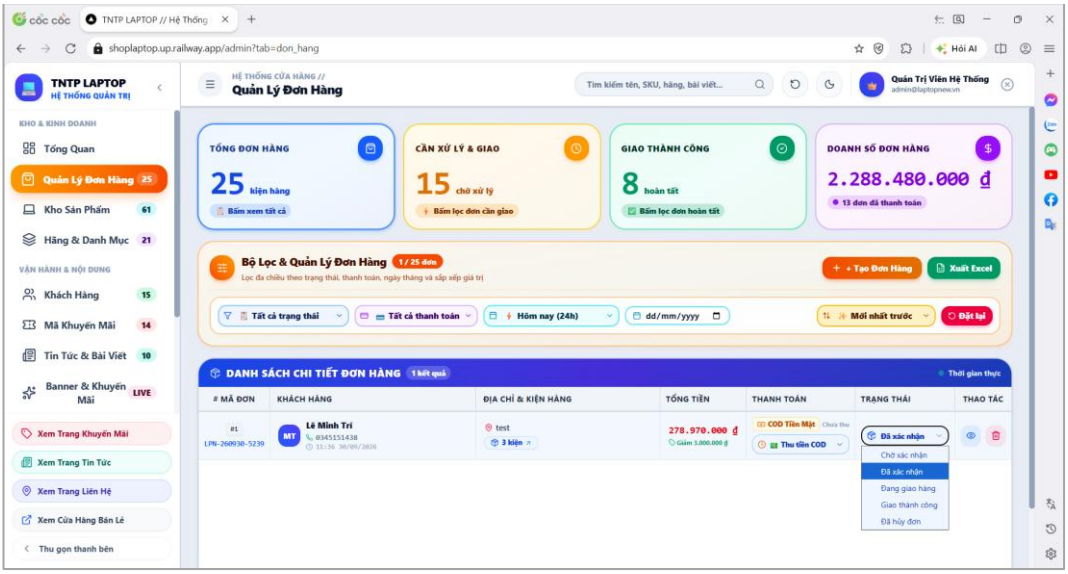
```
Trừ và hoàn tồn kho hai chiều – thao tác nguyên tử chống trừ kho hai lần
// controllers/don-hang.controller.js – capNhatTrangThaiDonHang (nhánh admin, trích)
if (trangThaiMoi === 'da_giao') {
    // "Giành cờ": chỉ 1 request đổi được da_tru_ton_kho từ false → true
    const daGianhCo = await DonHang.findOneAndUpdate(
        { _id: donHang._id, da_tru_ton_kho: { $ne: true } },
        { $set: { da_tru_ton_kho: true } });
    if (daGianhCo) await capNhatTonKhoTheoDonHang(danhSachSp, 'tru'); // giảm tồn, tăng
đã bán
} else {
    const daGianhCo = await DonHang.findOneAndUpdate(
        { _id: donHang._id, da_tru_ton_kho: true },
        { $set: { da_tru_ton_kho: false } });
    if (daGianhCo) await capNhatTonKhoTheoDonHang(danhSachSp, 'hoan'); // hoàn lại kho
}
// Đồng bộ 2 trường thanh toán, tránh "da_thanh_toan = true" mà vẫn "cho_thanh_toan"
if (duLieuCapNhat.da_thanh_toan === true) duLieuCapNhat.trang_thai_thanh_toan =
'da_thanh_toan';
```



Hình 4.28. *Trang thanh toán: thông tin người nhận, địa chỉ hành chính, phương thức thanh toán*



Hình 4.29. *Email xác nhận đặt hàng thành công gửi tới khách hàng*



Hình 4.30. *Trang quản trị đơn hàng (Admin): lọc đơn và đổi trạng thái*

4.7.4. Địa chỉ giao hàng theo địa giới hành chính mới (Provinces Open API v2)

Để địa chỉ giao hàng chính xác và đúng với địa giới hành chính sau sáp nhập năm 2025, trang thanh toán và sổ địa chỉ dùng Provinces Open API v2 (`provinces.open-api.vn/api/v2`): 34 tỉnh/thành, mô hình 2 cấp Tỉnh/Thành → Phường/Xã, khách chọn từ danh sách thay vì gõ tự do (Hình 4.28, Hình 4.56).

- **Tải theo tỉnh:** chọn tỉnh mới nạp toàn bộ phường/xã của tỉnh đó (`/api/v2/p/{mã}?depth=2`); ví dụ Tỉnh Vĩnh Long có 124 phường/xã.
- **Quy đổi địa chỉ cũ:** địa chỉ đã lưu theo 63 tỉnh được tự động quy đổi sang tỉnh mới (Bến Tre → Vĩnh Long, Bình Dương → TP. Hồ Chí Minh…) nhờ bảng tra 29 tỉnh cũ đã sáp nhập; dữ liệu sáp nhập được đối chiếu trực tiếp với API v2.
- **Bộ nhớ đệm, chống gọi trùng:** mỗi danh sách chỉ tải một lần; nhiều thành phần cùng yêu cầu thì dùng chung một promise (chống race condition).
- **Dự phòng khi API ngoài lỗi:** có sẵn danh mục 34 tỉnh/thành nên ô chọn tỉnh không bao giờ trống.
- **Tìm kiếm thân thiện:** khớp không dấu và từ viết tắt phổ biến (hcm, sg, hn, đn, …); tên tỉnh mới có ghi chú các tỉnh cũ đã gộp.
- **Back-End vẫn kiểm tra lại:** khi tạo đơn, `thong_tin_giao_hang` chỉ nhận chuỗi, được cắt độ dài và kiểm tra số điện thoại Việt Nam; đơn cũ có quận/huyện vẫn hiển thị đầy đủ.

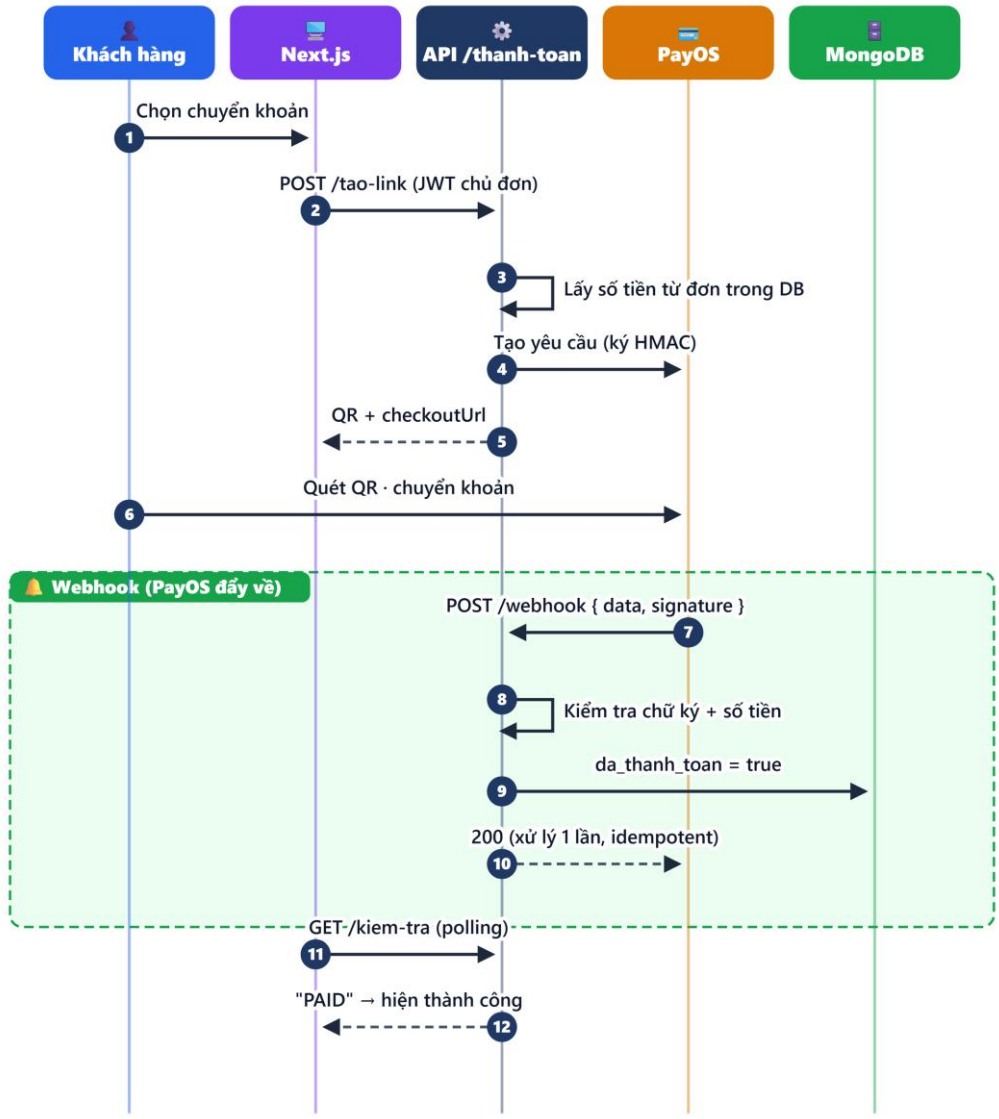
4.8. Chức năng thanh toán PayOS và VietQR

❖ **Mục tiêu:**

- Khách chuyển khoản bằng mã QR có sẵn số tiền và nội dung; hệ thống tự xác nhận khi tiền về, không cần nhân viên đối soát.
- Luôn hiển thị được QR ngay cả khi cổng PayOS phản hồi chậm.

❖ **Luồng hoạt động:**

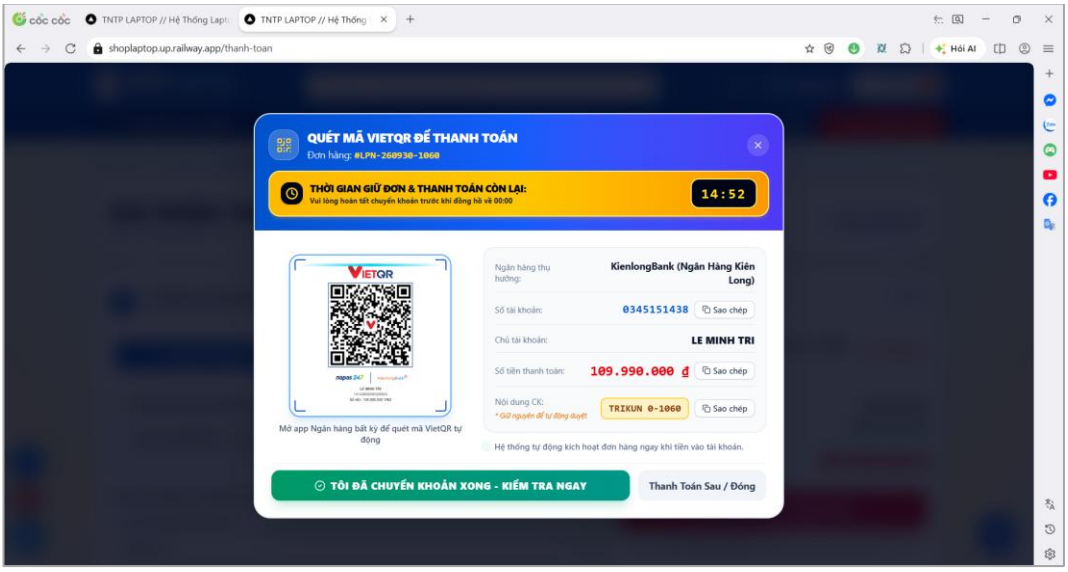
- **Tạo yêu cầu:** bắt buộc JWT của **chủ đơn**; số tiền **luôn lấy từ đơn trong DB** (client không gửi số tiền); chặn đơn đã thanh toán (409) hoặc đã hủy. Sinh `orderCode` là số từ mã đơn, nội dung `TRIKUN xxxxxx` (≤ 25 ký tự theo quy định PayOS); dựng sẵn URL ảnh VietQR; ký payload bằng HMAC-SHA256 rồi gọi PayOS với timeout 2,2 giây.
- **Webhook:** PayOS gọi `POST /api/thanh-toan/webhook`. Server tính lại chữ ký trên `data` và so sánh bằng `crypto.timingSafeEqual` (chống timing attack); sai → 400. Tiếp theo kiểm tra **số tiền khớp đơn** trong DB, đơn đã thanh toán thì bỏ qua (idempotent — PayOS gửi lại nhiều lần không bị xử lý trùng). Hợp lệ → trả 200 ngay cho PayOS, sau đó cập nhật đơn (`da_thanh_toan`, `trang_thai_thanh_toan = da_thanh_toan`, `trang_thai = da_xac_nhan`), gửi email và Telegram.
- **Polling:** trang thanh toán gọi `GET /kiem-tra/:orderCode` định kỳ. Server đọc DB trước (lean query), rồi bộ nhớ đệm 3 giây, cuối cùng mới hỏi PayOS (timeout 1,8 giây) — tránh dồn request lên PayOS.



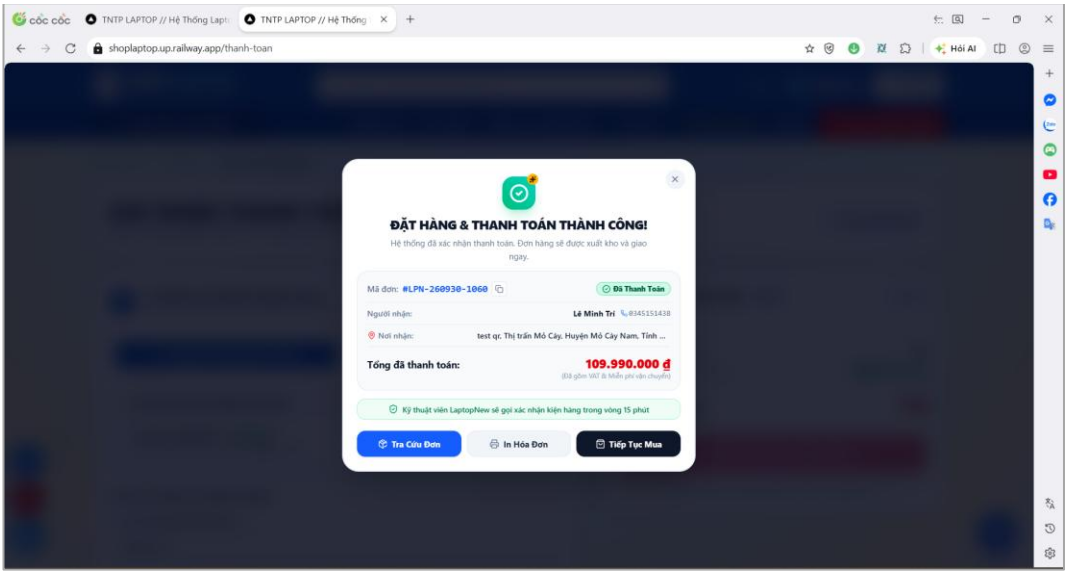
Hình 4.31. *Sơ đồ tuần tự thanh toán PayOS / VietQR với Webhook và polling*

Ký và xác minh chữ ký HMAC-SHA256 của PayOS

```
// controllers/thanh-toan.controller.js (trích)
const taoChuKyPayOS = (data, checksumKey) => {
    const chuoiKy = Object.keys(data).sort()                              // sắp xếp khóa a→z
        .filter(k => data[k] !== undefined && data[k] !== null && typeof data[k] !==
'object')
        .map(k => `${k}=${data[k]}`).join('&');                    //
amount=...&cancelUrl=...
    return crypto.createHmac('sha256', checksumKey).update(chuoiKy).digest('hex');
};

// xuLyWebhookPayOS
const chuKyTinh = taoChuKyPayOS(payload.data, PAYOS_CHECKSUM_KEY);
const hopLe = chuKyNhan.length === chuKyTinh.length
    && crypto.timingSafeEqual(Buffer.from(chuKyTinh), Buffer.from(chuKyNhan)); // so sánh
thời gian hằng
if (!hopLe) return res.status(400).json({ success: false, message: 'Chữ ký không hợp
lệ' });
const donHang = await DonHang.findOne({ payos_order_code: Number(data.orderCode) });
if (Math.round(data.amount) !== Math.round(donHang.tong_tien_thanh_toan))     // số tiền
phải khớp đơn
    return res.status(400).json({ success: false, message: 'Số tiền thanh toán không khớp
đơn hàng' });
if (donHang.da_thanh_toan) return res.status(200).json({ success: true });     //
idempotent
res.status(200).json({ success: true });                                          // phản hồi
PayOS ngay
DonHang.findOneAndUpdate({ _id: donHang._id }, { $set: { da_thanh_toan: true,
trang_thai_thanh_toan: 'da_thanh_toan', trang_thai: 'da_xac_nhan' } })
```



Hình 4.32. *Màn hình mã QR thanh toán PayOS / VietQR*



Hình 4.33. *Thanh toán thành công — đơn tự chuyển trạng thái*

4.9. Chức năng gửi email tự động

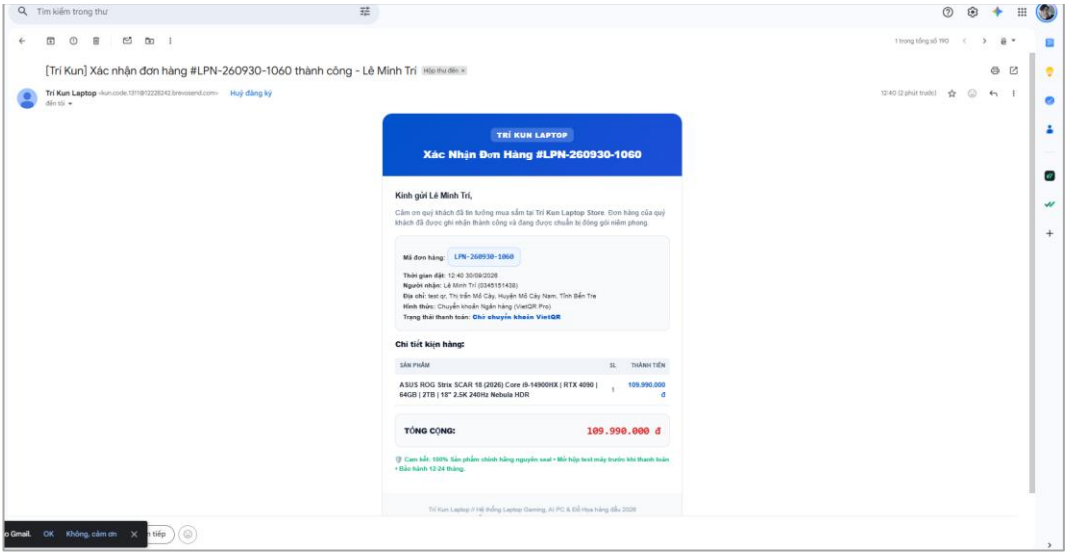
❖ Mục tiêu:

- Gửi email HTML cho OTP kích hoạt, OTP quên mật khẩu, OTP đổi mật khẩu và hóa đơn đơn hàng; hoạt động được cả ở máy cá nhân lẫn trên cloud.

❖ Luồng hoạt động:

- Hàm dùng chung `guiMailBangTransporter()` **ưu tiên Brevo REST API** (`POST https://api.brevo.com/v3/smtp/email` với header `api-key`).
- Nếu không có `BREVO_API_KEY`: khi chạy trên cloud (có biến `RAILWAY_ENVIRONMENT` / `VERCEL`) thì báo lỗi rõ ràng; khi chạy local thì dùng **Nodemailer + Gmail App Password**.
- Các hàm nghiệp vụ trả về `{ thanhCong, loi }` để controller quyết định trả **502** khi không gửi được OTP.

```
Gửi email hai tầng: Brevo API và Nodemailer
// services/email.service.js – guiMailBangTransporter (trích)
if (key) {                                          // Ưu tiên 1: Brevo API qua HTTPS
    const res = await fetch('https://api.brevo.com/v3/smtp/email', {
        method: 'POST',
        headers: { 'api-key': key, 'Content-Type': 'application/json' },
        body: JSON.stringify({ sender: { name: 'Trí Kun Laptop', email: sender },
            to: danhSachTo, subject: mailOptions.subject, htmlContent:
mailOptions.html })
    });
    if (!res.ok) throw new Error(`Brevo error ${res.status}`);
    return { messageId: (await res.json()).messageId };
}
if (isCloud) throw new Error('Railway chặn SMTP. Vui lòng cấu hình BREVO_API_KEY...');
const transporter = nodemailer.createTransport({ service: 'gmail', auth: { user:
emailUser, pass: emailPass } });
return transporter.sendMail(mailOptions);                // Dự phòng: Gmail SMTP khi chạy
local
```



Hình 4.34. *Email xác nhận đơn hàng (hóa đơn HTML)*

4.10. Chức năng quản trị đơn hàng qua Telegram Bot

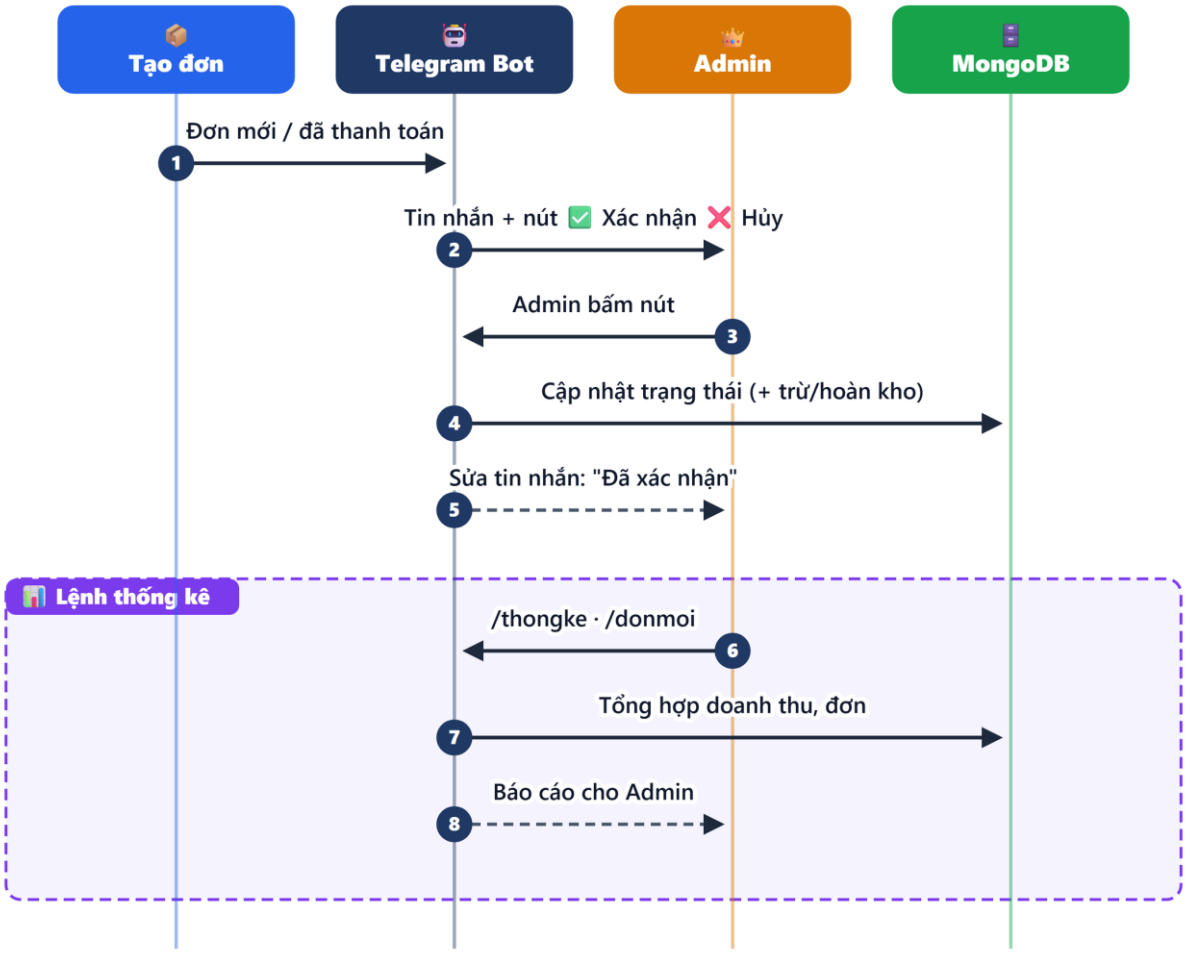
❖ Mục tiêu:

- Quản trị viên nhận thông báo đơn mới tức thì trên điện thoại và xử lý đơn chỉ bằng một lần bấm.

❖ Luồng hoạt động:

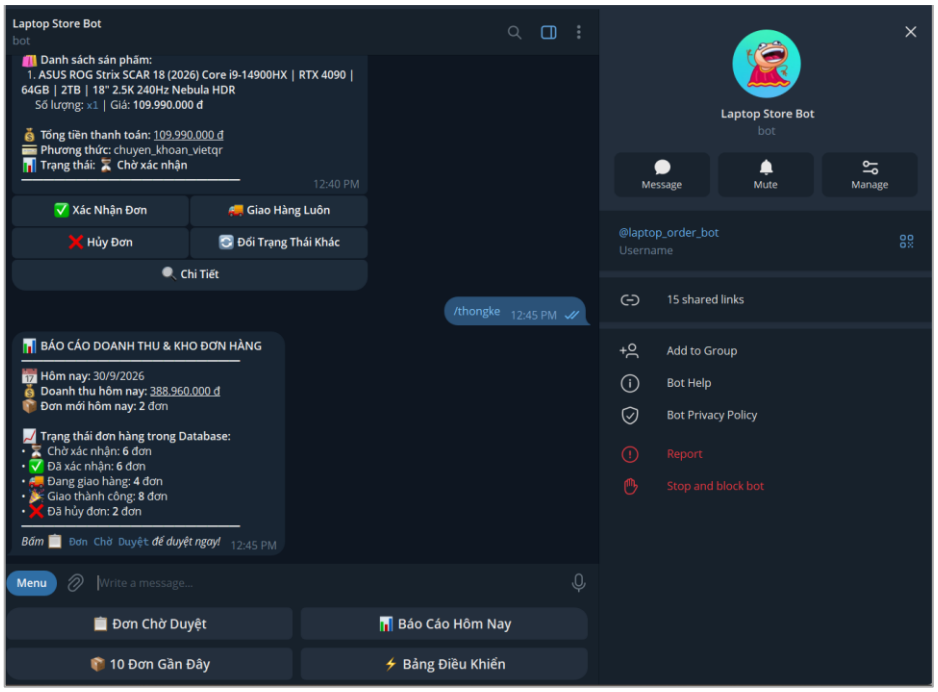
- Khi server khởi động, `khoiTaoTelegramBot()` đăng ký menu lệnh (`setMyCommands`) và chạy vòng **Long Polling** `getUpdates` (timeout 20 giây).
- Có đơn mới hoặc thanh toán thành công → gửi tin nhắn HTML kèm **Inline Keyboard**: Xác nhận, Giao hàng, Hủy, Chi tiết, Đổi trạng thái khác. Bàn phím thay đổi theo trạng thái hiện tại của đơn.

- Admin bấm nút → Telegram trả `callback_query` dạng `doi_tt:<trạng thái>:<mã đơn>` → bot cập nhật MongoDB (kèm trừ/hoàn kho), trả lời `answerCallbackQuery` và sửa lại tin nhắn.
- Lệnh văn bản: `/thongke` (báo cáo hôm nay), `/donmoi` (đơn chờ duyệt), `/tatcadon` (10 đơn gần đây), `/timdon <mã>`.



Hình 4.35. *Sơ đồ tuần tự thông báo và duyệt đơn qua Telegram Bot*

Hình 4.36. *Tin nhắn đơn hàng mới kèm nút duyệt trên Telegram*



Hình 4.37. *Kết quả lệnh /thongke trên Telegram*

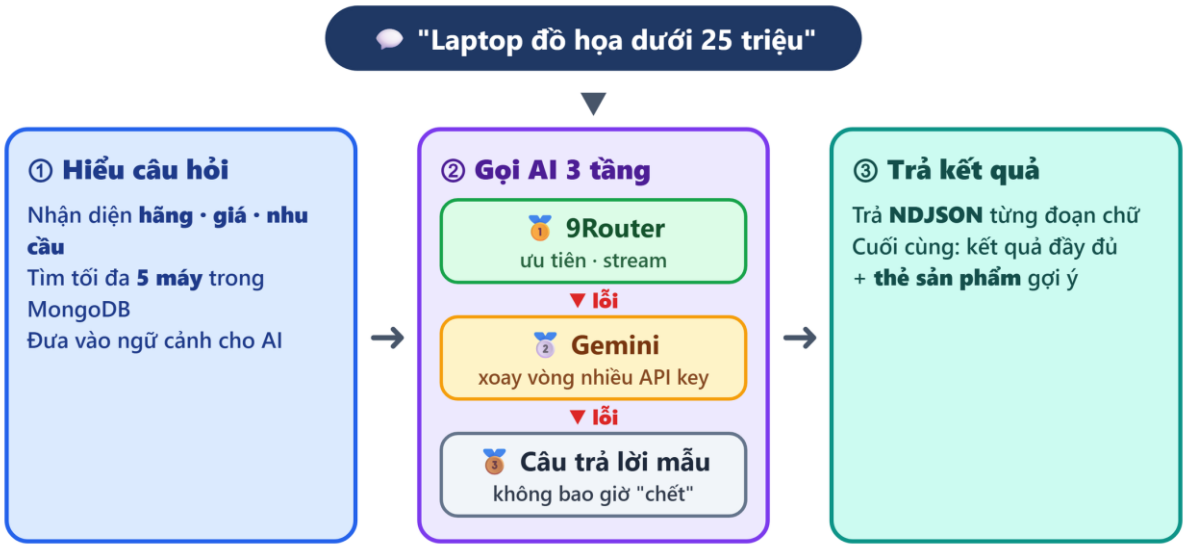
4.11. Chức năng Trợ lý AI tư vấn laptop

❖ Mục tiêu:

- Hiểu câu hỏi tiếng Việt tự nhiên (“laptop đồ họa dưới 25 triệu”), gợi ý **đúng các máy đang bán** thay vì trả lời chung chung.
- Hiển thị câu trả lời dần dần như đang gõ và không bao giờ “chết” khi một nhà cung cấp AI gặp sự cố.

❖ Luồng hoạt động:

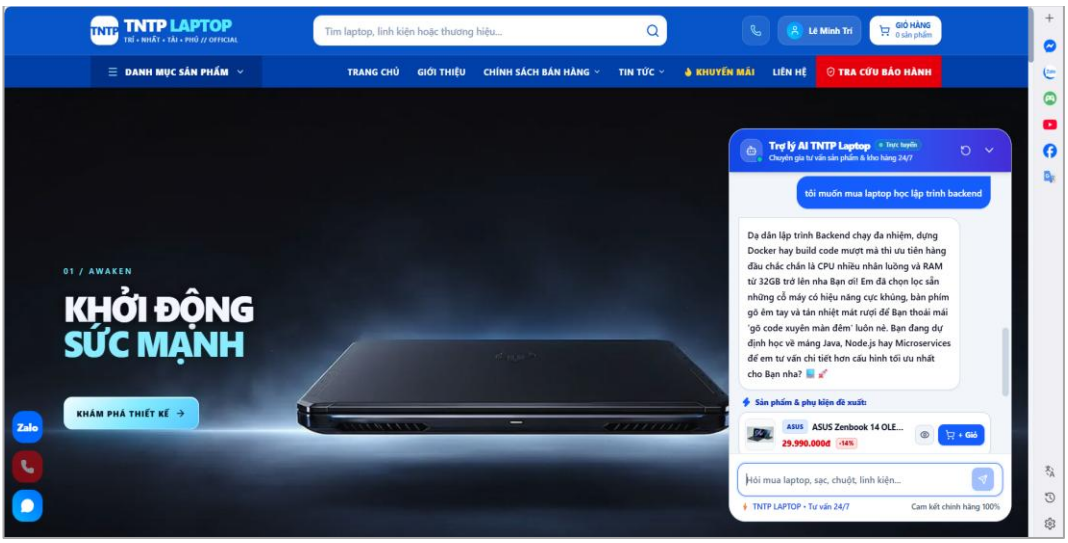
- Phân tích ý định câu hỏi (hãng, khoảng giá, nhu cầu, phụ kiện) → truy vấn tối đa 5 sản phẩm phù hợp trong `san_pham` → đưa vào prompt làm ngữ cảnh.
- Gọi mô hình theo 3 tầng dự phòng: **9Router** (stream) → **Gemini xoay vòng key** (tạm khóa key bị 429) → **câu trả lời mẫu**.
- Trả về dạng **NDJSON**: nhiều dòng `{"type":"text"}` rồi một dòng `{"type":"done","result":...}` chứa câu trả lời, sản phẩm gợi ý và câu hỏi gợi ý tiếp theo.



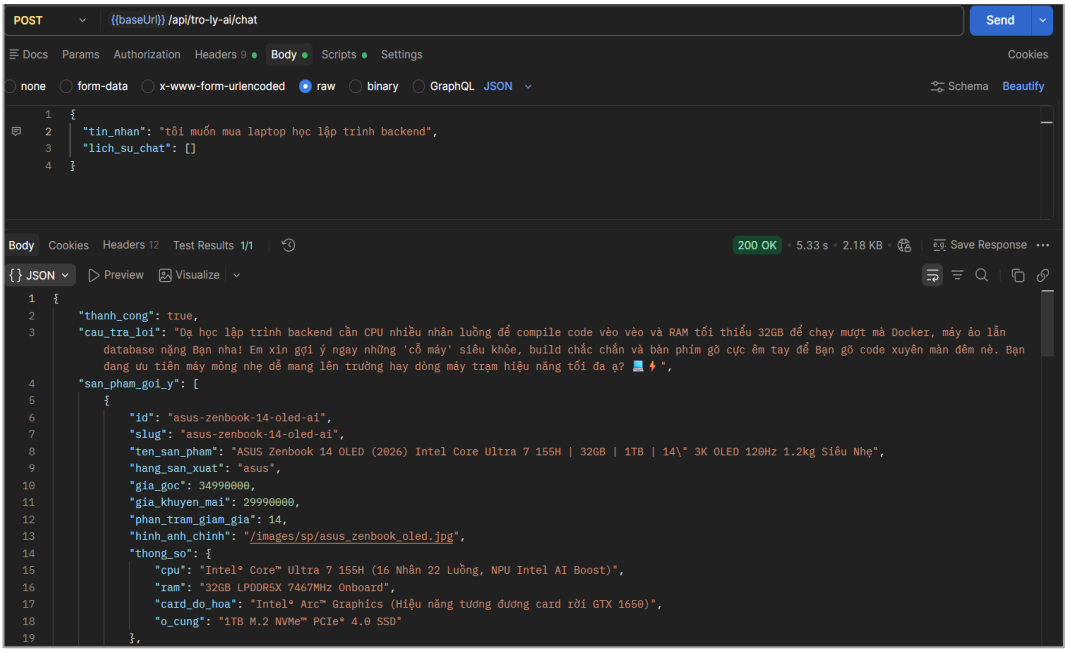
Hình 4.38. *Luồng xử lý của Trợ lý AI với cơ chế dự phòng nhiều tầng*

Truyền câu trả lời AI theo luồng NDJSON

```
// controllers/tro-ly-ai.controller.js – xuLyChatAIStream (trích)
res.setHeader('Content-Type', 'application/x-ndjson; charset=utf-8');
res.setHeader('Cache-Control', 'no-cache, no-transform');
res.flushHeaders();
try {
    const ketQua = await xuLyTroLyChat({
        tinNhan: tin_nhan.trim(), lichSuChat: lichSu,
        onText: text => res.write(JSON.stringify({ type: 'text', text }) + '\n')
    });
    res.write(JSON.stringify({ type: 'done', result: { thanh_cong: true, ...ketQua } }) +
'\n');
} catch (error) {
    res.write(JSON.stringify({ type: 'error', message: 'Không thể kết nối Trợ lý AI.' })
+ '\n');
} finally { res.end(); }
```



Hình 4.39. *Hộp chat Trợ lý AI tư vấn và gợi ý sản phẩm trên website*



Hình 4.40. *Kiểm thử POST /api/tro-ly-ai/chat bằng Postman: câu trả lời kèm sản phẩm gợi ý*

4.12. Chức năng tra cứu bảo hành điện tử

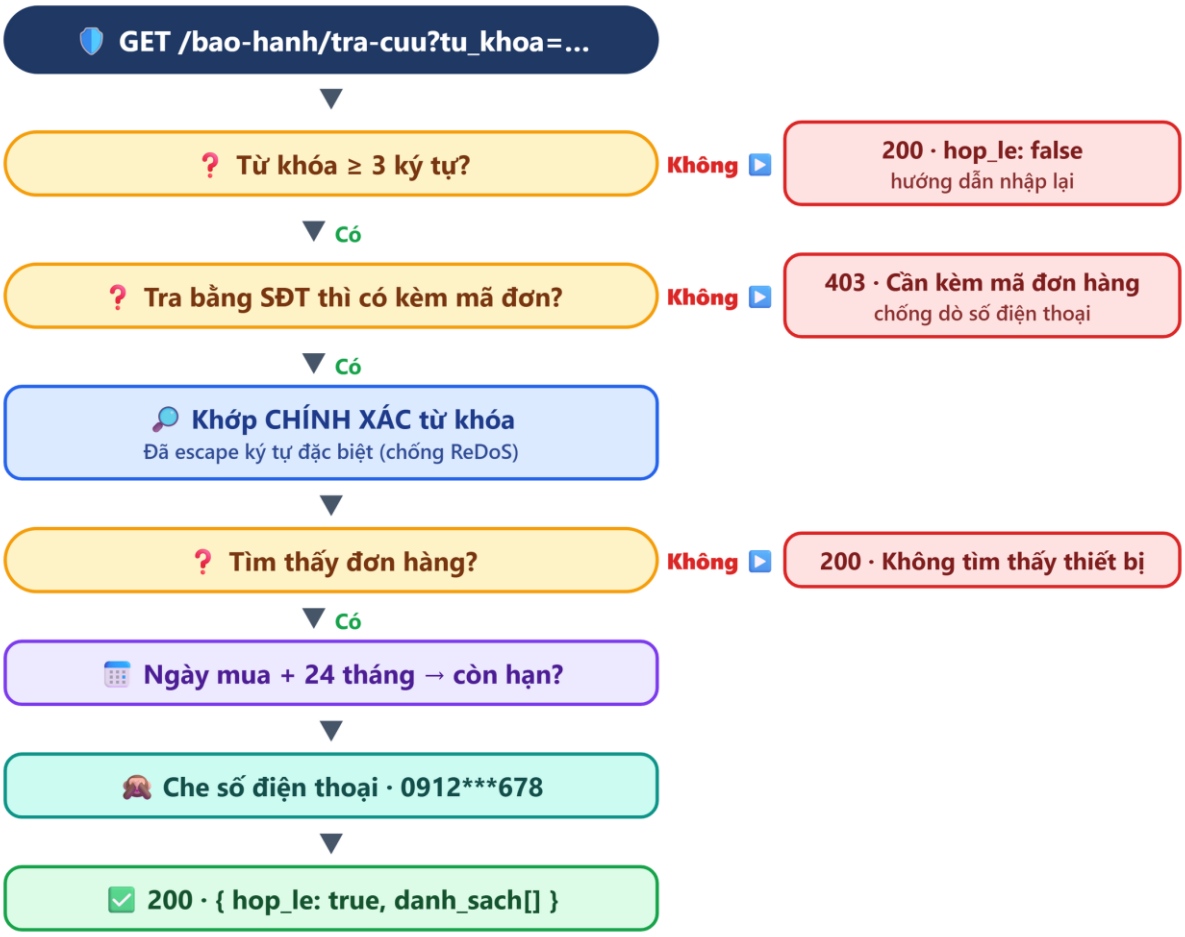
❖ Mục tiêu:

- Khách tra hạn bảo hành bằng mã đơn, số điện thoại, serial hoặc mã sản phẩm mà không cần phiếu giấy.

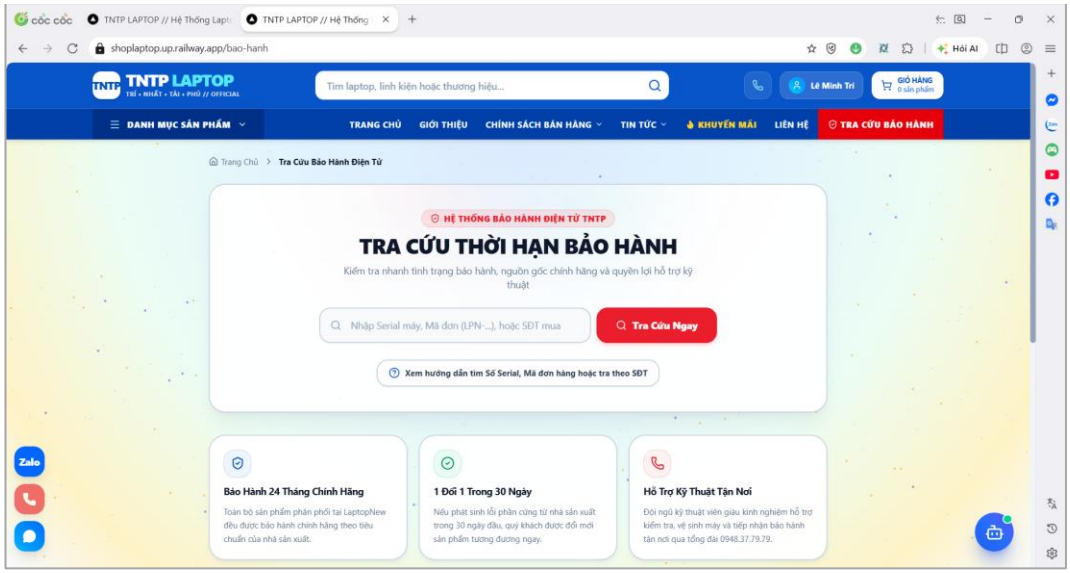
❖ Luồng hoạt động:

- Từ khóa phải ≥ 3 ký tự. Server **escape ký tự đặc biệt** và so khớp **chính xác toàn chuỗi** (`^...$`), tránh việc gõ vài số mà ra đơn của người khác.

- Với mỗi sản phẩm trong đơn khớp: ngày mua + 24 tháng → còn hiệu lực hay không, còn bao nhiêu tháng; số điện thoại được **che** dạng `0912***678`.



Hình 4.41. *Lưu đồ tra cứu bảo hành điện tử*



Hình 4.42. *Giao diện tra cứu bảo hành điện tử*

4.13. Các chức năng bổ trợ

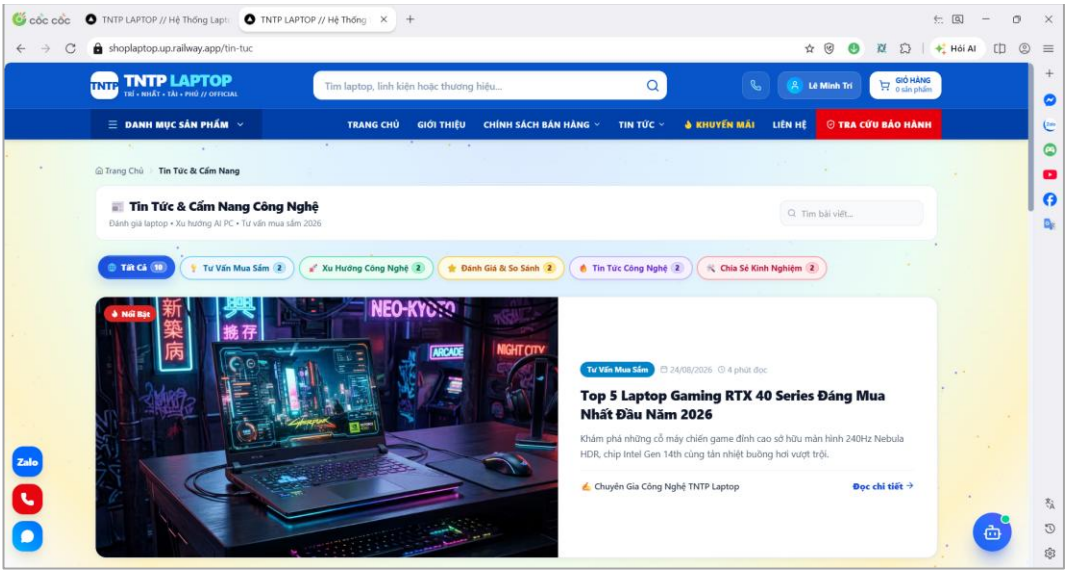
Bảng 4.1. *Các chức năng bổ trợ khác*

Chức năng	API	Điểm kỹ thuật
Đánh giá sản phẩm	`/api/danh-gia`	Sau khi thêm đánh giá, tính lại `diem_danh_gia_tb` và `so_luong_danh_gia` của sản phẩm; tăng lượt thích bằng `PATCH`
Liên hệ tư vấn	`/api/lien-he`	Kiểm tra SĐT Việt Nam, sinh mã phiếu duy nhất; danh sách có phân trang và thống kê theo trạng thái bằng `Promise.all` nhiều `countDocuments`
Tin tức	`/api/tin-tuc`	Tìm kiếm theo tiêu đề/tóm tắt/chuyên mục; xem chi tiết tự tăng `luot_xem` bằng `$inc`
Danh mục	`/api/danh-muc`	Slug tự sinh; sắp xếp theo `thu_tu_hien_thi`
Showroom	`/api/cai-dat/showroom`	Một document cấu hình duy nhất (`khoa = he_thong`), lưu tọa độ để hiển thị bản đồ

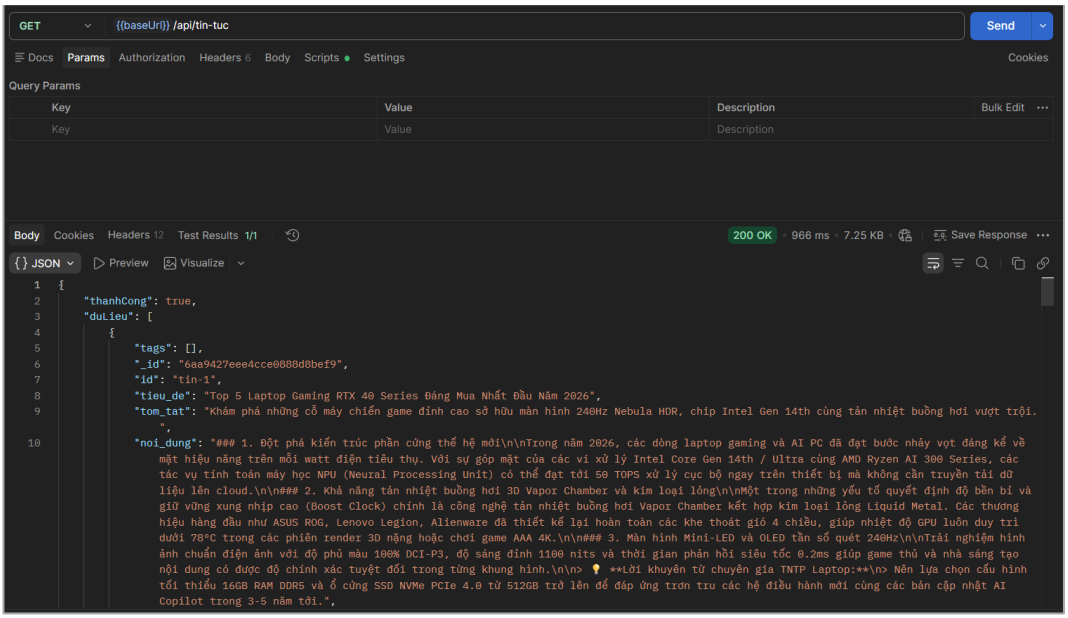
4.14. Một số giao diện kết quả

Các hình dưới đây được chụp trực tiếp trên hệ thống đã triển khai: Front-End Next.js chạy trên Railway (địa chỉ `shoplaptop.up.railway.app`, nay dùng tên miền chính `trikun.site`), lấy dữ liệu thật từ API Express trên Railway và MongoDB Atlas.

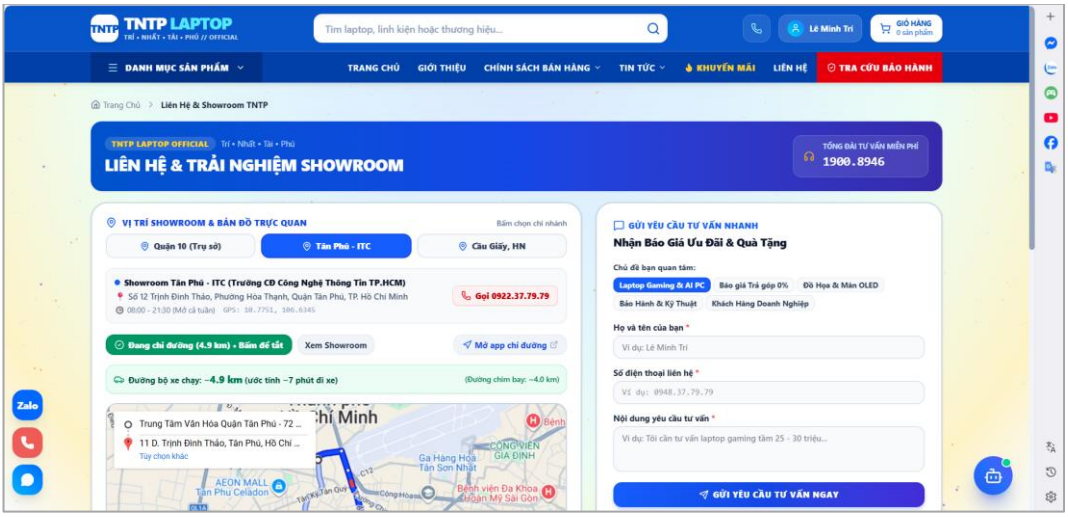
Hình 4.43. *Trang chủ website*



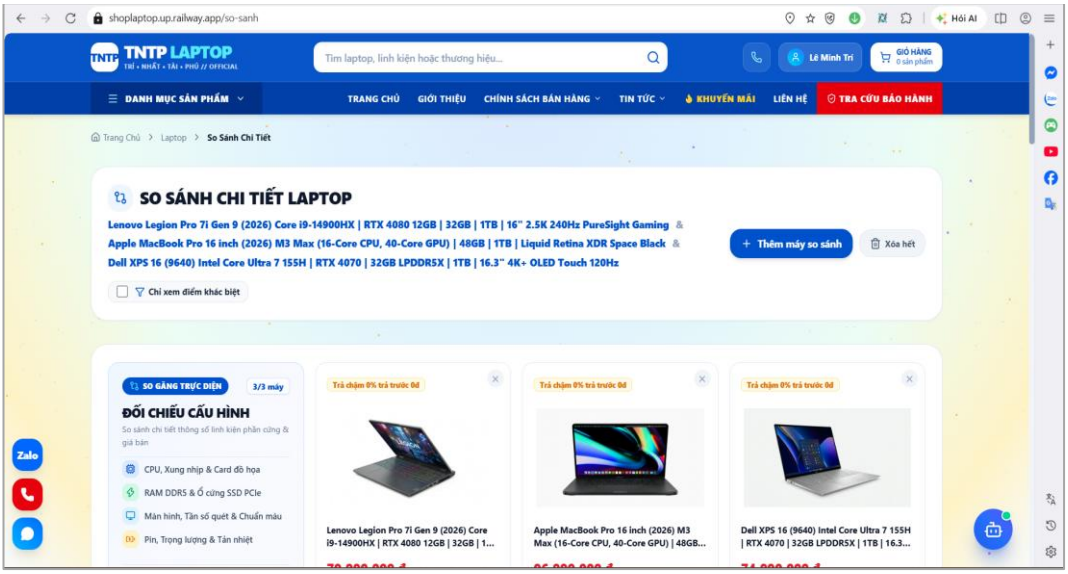
Hình 4.44. *Trang tin tức công nghệ*



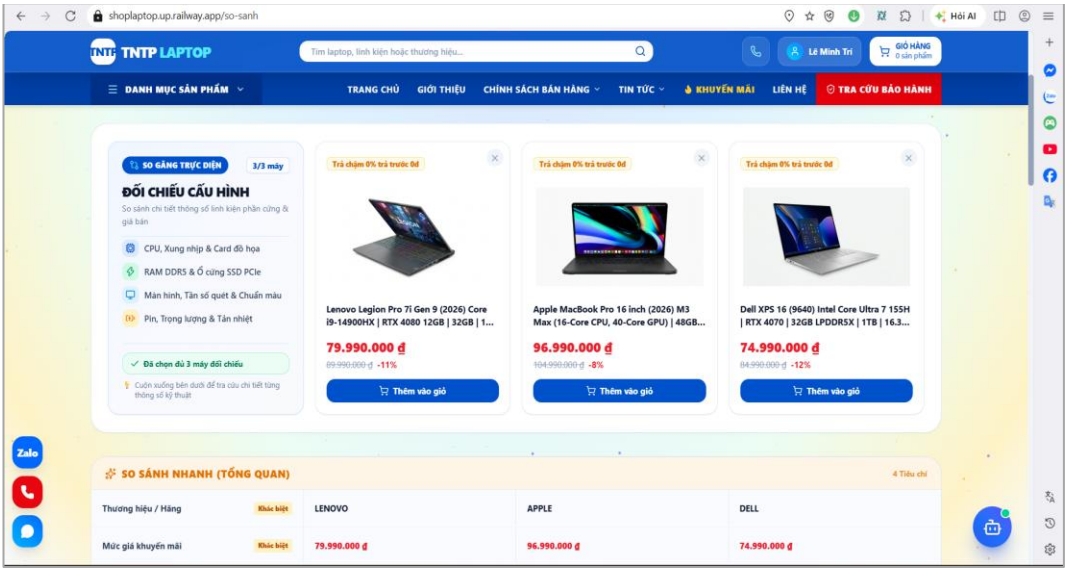
Hình 4.45. *Kiểm thử GET /api/tin-tuc bằng Postman: danh sách bài viết*



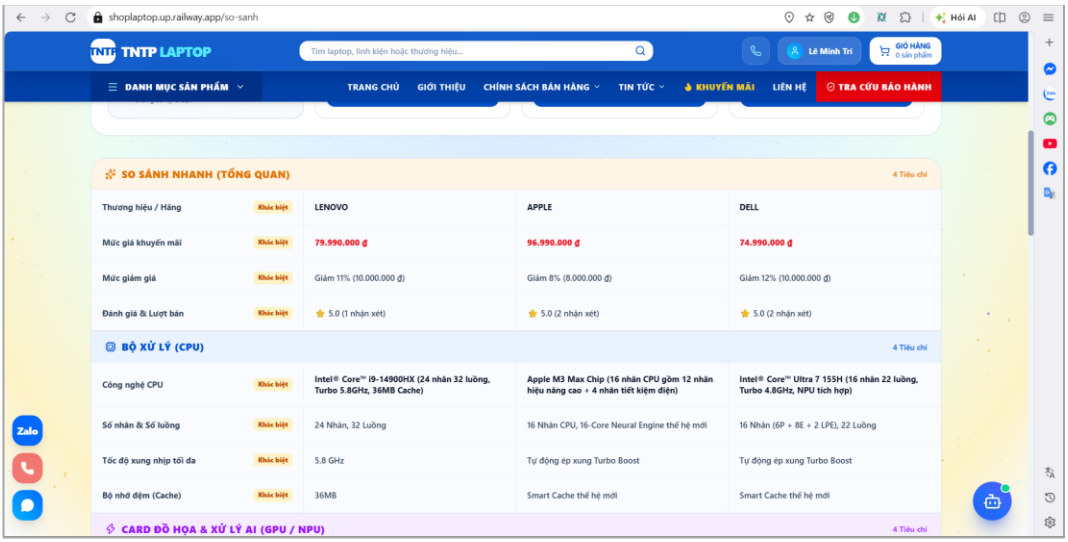
Hình 4.46. *Trang liên hệ và hệ thống showroom có bản đồ*



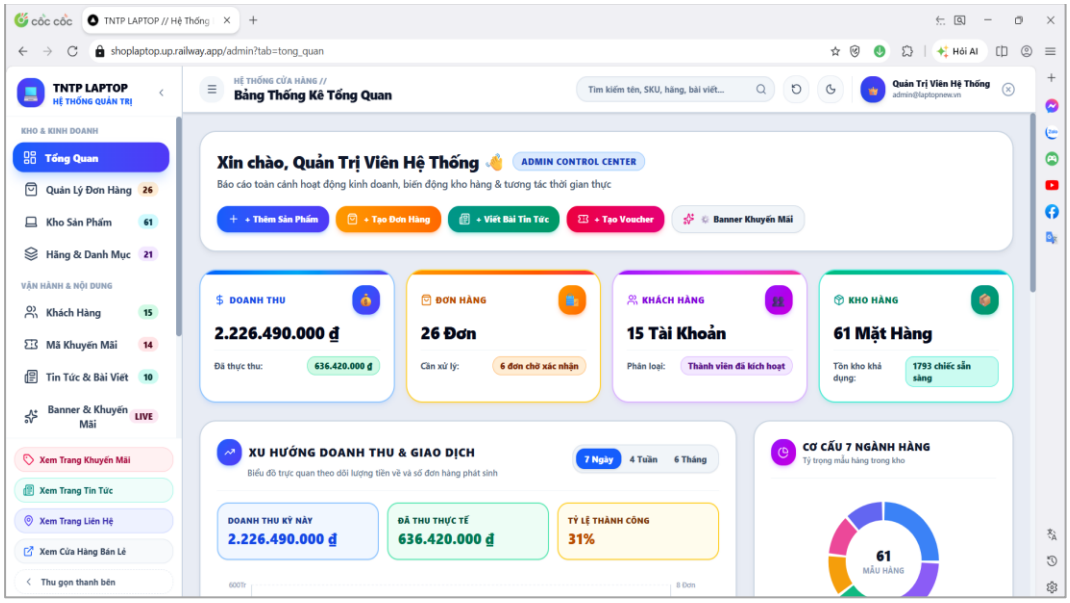
Hình 4.47. *Trang so sánh: danh sách laptop đã chọn để so sánh*



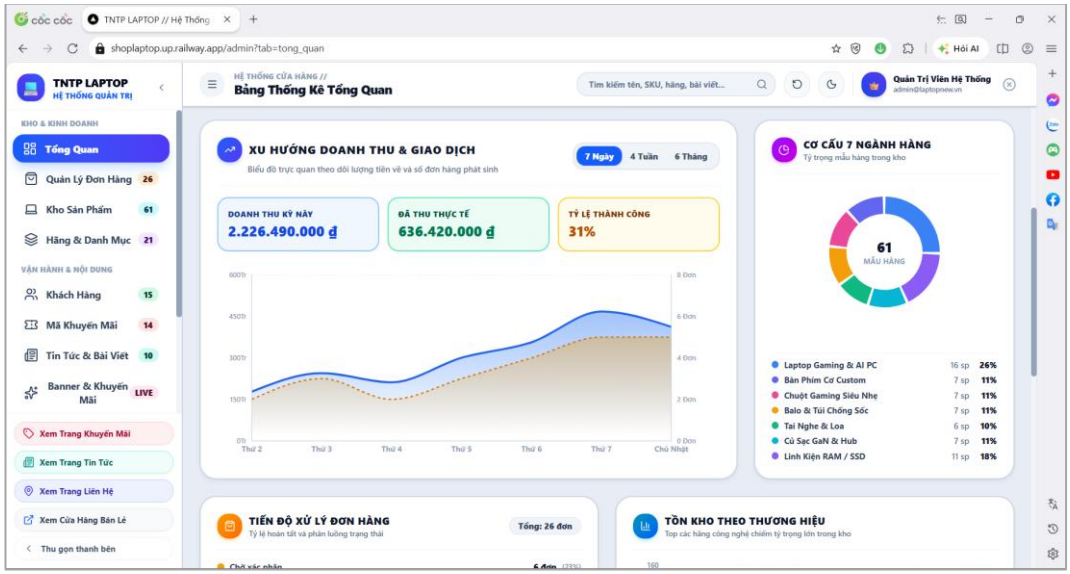
Hình 4.48. *Bảng đối chiếu nhanh thương hiệu, giá và đánh giá*



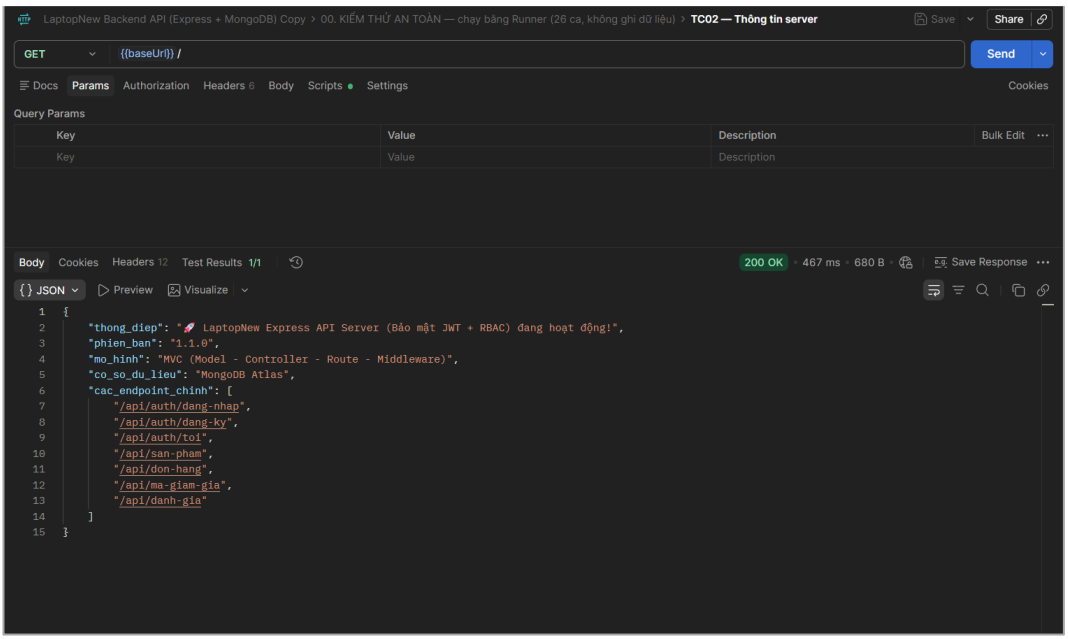
Hình 4.49. *So sánh chi tiết bộ xử lý (CPU) giữa các máy*



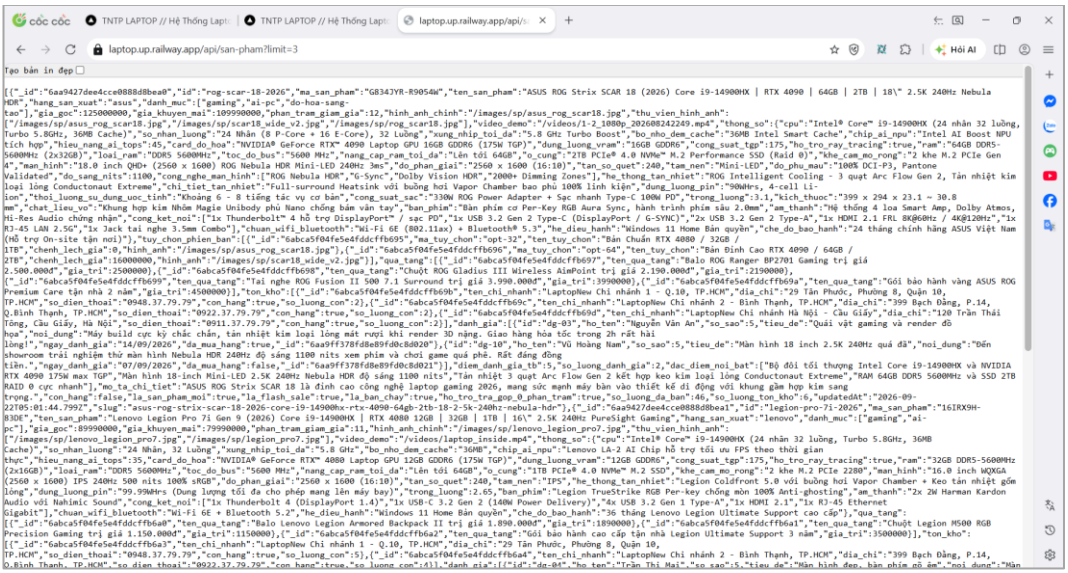
Hình 4.50. *Trang tổng quan quản trị: thẻ thống kê doanh thu, đơn hàng, khách hàng, kho*



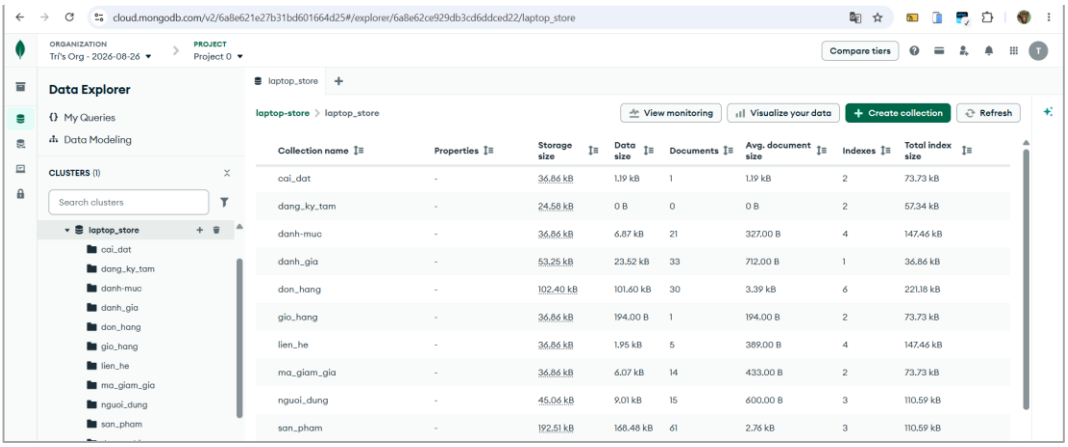
Hình 4.51. *Biểu đồ xu hướng doanh thu và cơ cấu ngành hàng*



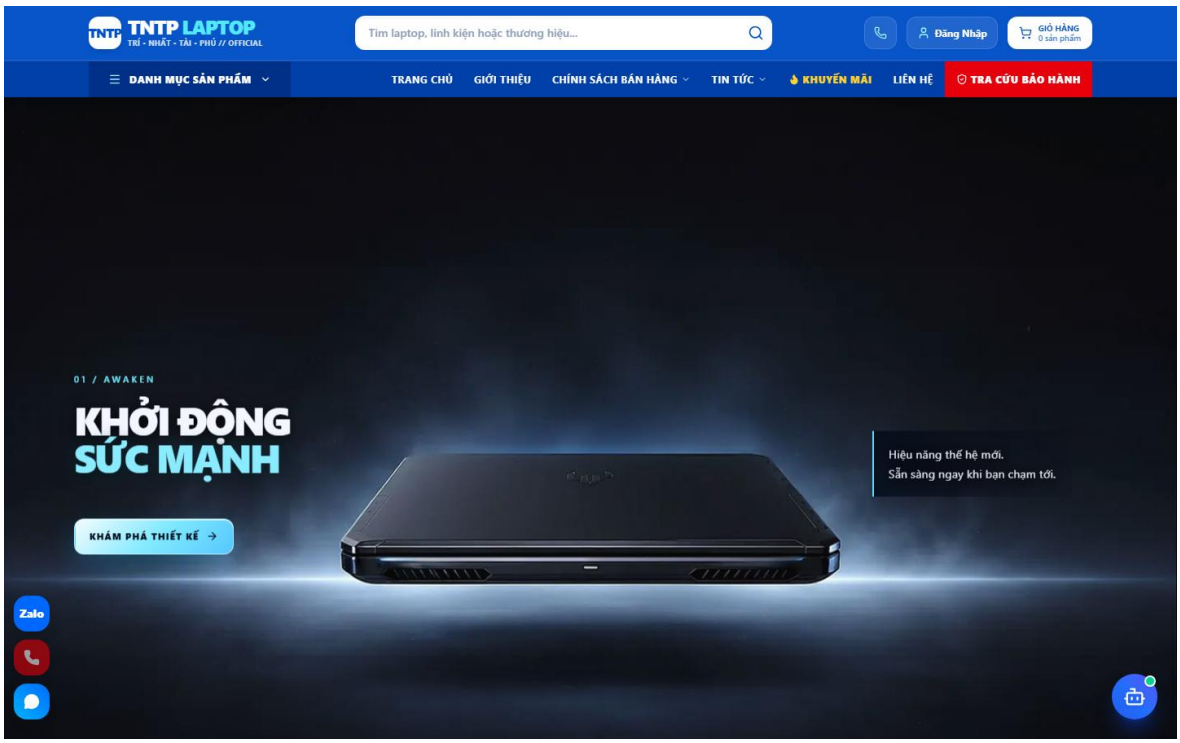
Hình 4.52. *Kiểm thử GET / bằng Postman: thông tin server và danh sách endpoint chính*



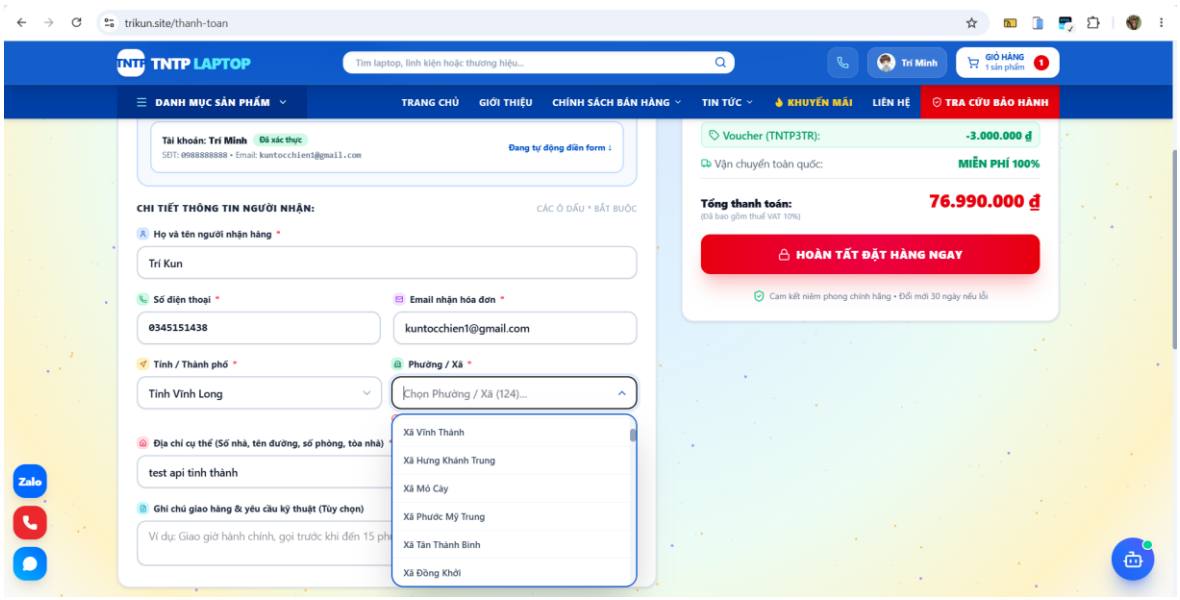
Hình 4.53. *Dữ liệu JSON thực tế trả về từ GET /api/san-pham*



Hình 4.54. *Các collection trên MongoDB Atlas (cơ sở dữ liệu laptop_store, 04/10/2026): đủ 10 collection đang dùng, có gio_hang; thuong_hieu ở cuối danh sách là collection cũ không còn dùng*

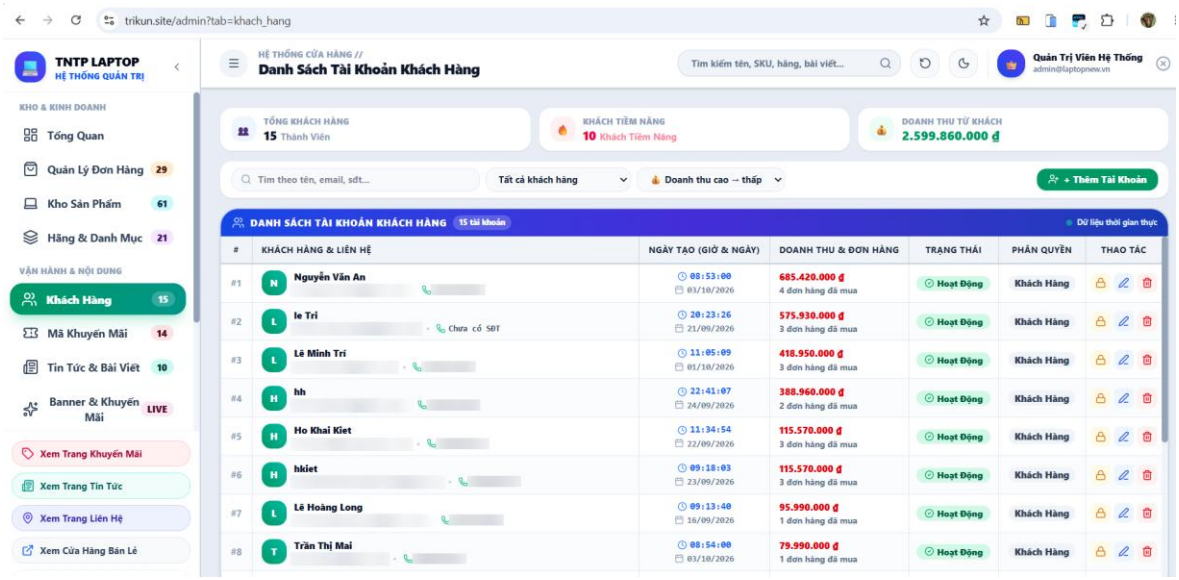


Hình 4.55. *Trang chủ chạy trên tên miền chính thức https://trikun.site (qua Cloudflare)*



Hình 4.56. *Trang thanh toán trên trikun.site: chọn Tỉnh/Thành và Phường/Xã theo địa giới sau sáp nhập (Provinces Open API v2)*

Tab **Khách hàng** của trang quản trị (`/admin?tab=khach_hang`) chỉ mở được bằng tài khoản Admin. Trang tổng hợp số thành viên, số khách tiềm năng và doanh thu từ khách; cho phép tìm theo tên, email, số điện thoại, lọc và sắp xếp theo doanh thu. Mỗi tài khoản có ba thao tác: **khóa / mở khóa** (biểu tượng ổ khóa), **sửa** (đổi thông tin, vai trò) và **xóa**. Phía Back-End, khóa tài khoản hoặc đổi vai trò sẽ thu hồi phiên đăng nhập ngay (tăng `tokenVersion`, Hình 5.12) và Admin không thể tự khóa hay tự xóa chính mình. Email và số điện thoại của khách đã được che trong hình.



Hình 4.57. *Trang quản trị trên trikun.site: danh sách tài khoản khách hàng kèm thống kê, bộ lọc và các thao tác khóa / sửa / xóa*

4.15. Tự động đồng bộ giao diện khi Admin thay đổi dữ liệu

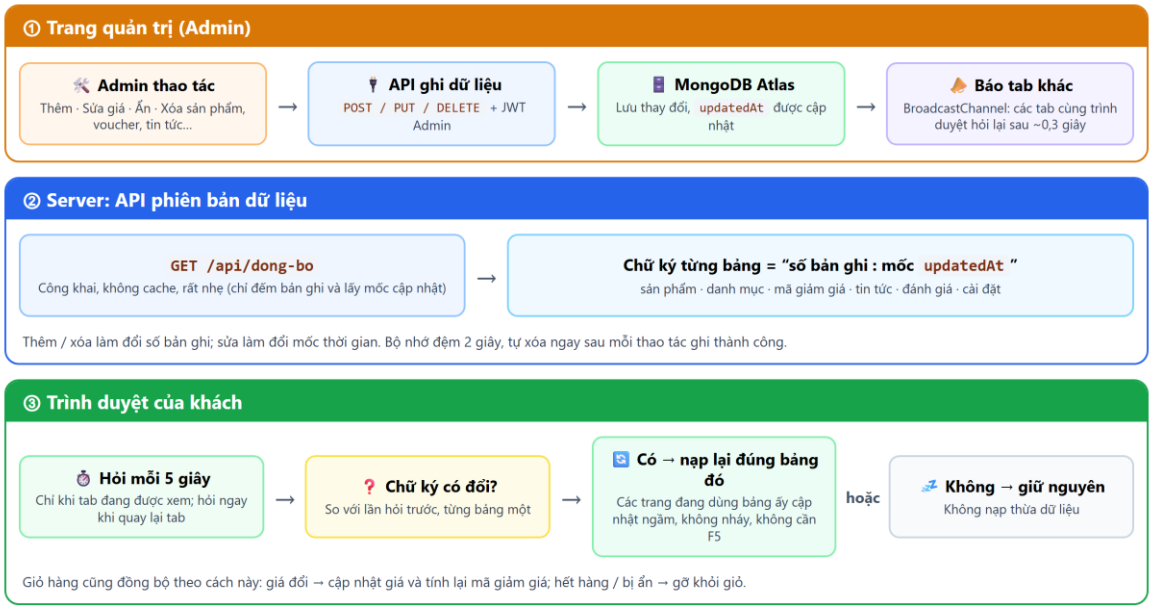
Trong quá trình kiểm thử trên server thật, nhóm phát hiện Admin thêm / sửa / xóa dữ liệu nhưng phía khách không thấy thay đổi ngay. Nguyên nhân không nằm ở bộ nhớ đệm của trình duyệt mà ở năm lỗi thiết kế được tổng hợp ở Bảng 4.2, cùng cách khắc phục.

Bảng 4.2. *Các lỗi đồng bộ dữ liệu giữa Admin và khách, và cách khắc phục*

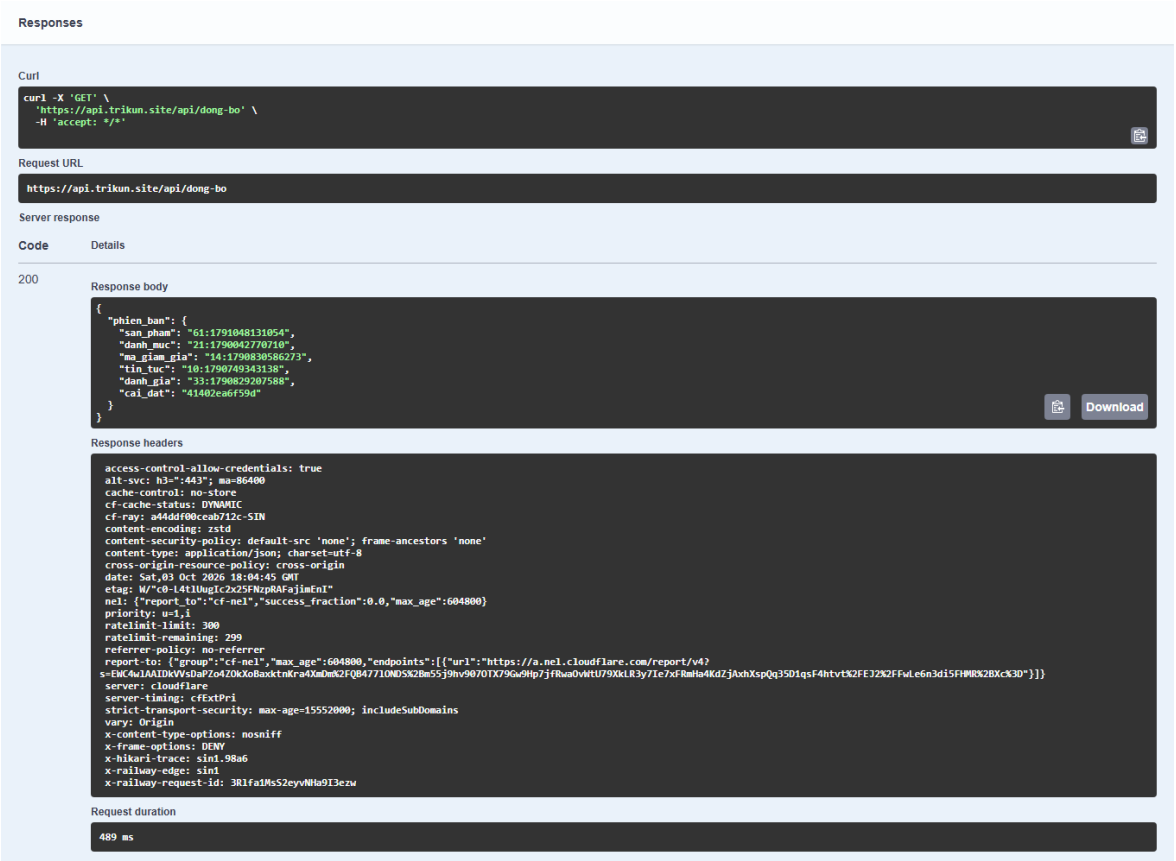
Lỗi trước đây	Triệu chứng	Cách khắc phục
Trang khách chỉ nạp dữ liệu **một lần** lúc mở trang	Admin sửa xong, trang khách đang mở không đổi, phải F5	Tự đồng bộ: hỏi `/api/dong-bo` mỗi 5 giây, bảng nào đổi thì nạp lại (mục 4.15.1)
"Tạm ẩn" chỉ đặt `con_hang = false`, server **không lọc**	Ẩn xong khách vẫn thấy, kể cả mở tab ẩn danh	Server không trả sản phẩm ẩn hoặc hết hàng cho khách (chi tiết trả 404); Admin vẫn thấy
Cấu hình Banner / Flash Sale chỉ lưu trong `localStorage` của Admin	Khách ở máy khác không bao giờ thấy thay đổi	Lưu trong MongoDB: `GET / PUT /api/cai-dat/khuyen-mai`
Giỏ hàng chỉ ở trình duyệt, giữ giá lúc bỏ vào giỏ	Admin đổi giá nhưng giỏ vẫn giá cũ; giỏ không đi theo tài khoản	Giỏ tự đồng bộ giá và tồn kho; giỏ lưu theo tài khoản (`/api/gio-hang`)
Cho nhập giá niêm yết gốc thấp hơn giá bán	Lưu "thành công" nhưng giá bán không đổi, nhãn "-x%" lệch	Server chặn (400, kèm hướng dẫn) và tự tính phần trăm giảm

4.15.1. Cơ chế tự đồng bộ

Back-End bổ sung API công khai `GET /api/dong-bo` trả "chữ ký" của từng bảng (số bản ghi và mốc `updatedAt` mới nhất) mà không trả dữ liệu nghiệp vụ nên rất nhẹ. Trình duyệt của khách hỏi API này mỗi 5 giây (chỉ khi tab đang được xem) và chỉ nạp lại đúng bảng có chữ ký đổi, nên giao diện cập nhật ngầm, không nháy và không tải thừa. Khi Admin vừa lưu xong, trang quản trị còn báo trực tiếp cho các tab cùng trình duyệt qua `BroadcastChannel` nên tab của khách cập nhật gần như tức thì (Hình 4.58). Kết quả gọi thử API trên server thật ở Hình 4.59.



Hình 4.58. *Cơ chế tự đồng bộ giao diện: Admin thay đổi → chữ ký phiên bản → trình duyệt của khách nạp lại đúng bảng*



Hình 4.59. *Gọi thử GET /api/dong-bo trên Swagger của server thật: trả chữ ký các bảng, không cache*

4.15.2. Sản phẩm ẩn, hết hàng và giá

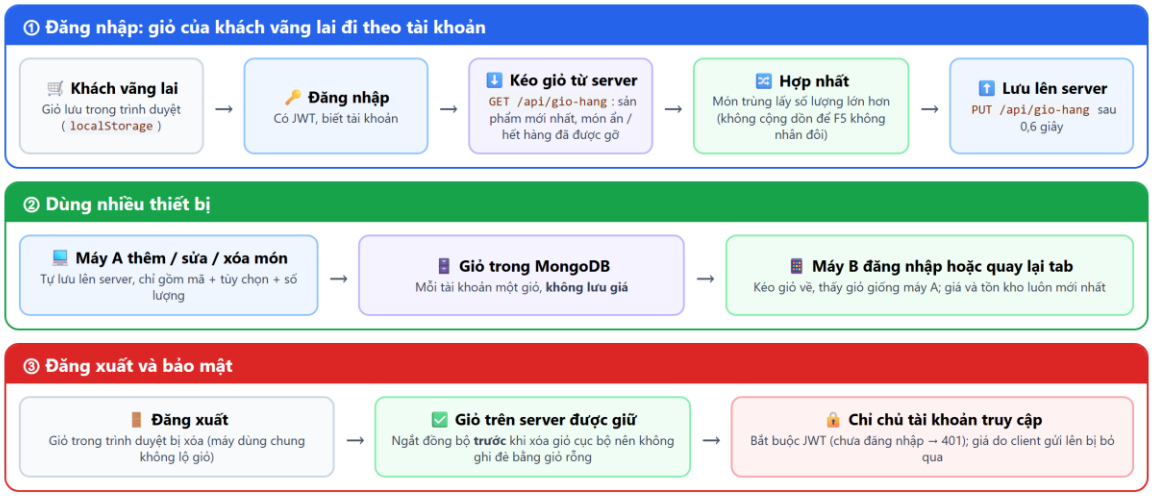
- **Ẩn và hết hàng:** khách không thấy sản phẩm đã bị Admin tạm ẩn hoặc có tồn kho bằng 0, trang chi tiết trả 404; Admin vẫn thấy tất cả để nhập hàng hoặc mở bán lại. Phản hồi dành cho Admin không được cache dùng chung (`Cache-Control: private, no-store`).
- **Giá:** "Giá niêm yết gốc" chỉ để hiện giá gạch ngang nên phải bằng 0 hoặc không thấp hơn "Giá khuyến mãi" (giá bán thật); nếu nhập sai, server trả 400 kèm hướng dẫn. Phần trăm giảm do server tự tính, không tin số client gửi.
- **Cấu hình khuyến mãi:** lưu trong MongoDB thay vì `localStorage` của trình duyệt Admin nên mọi khách đều thấy.

Hình 4.60 tóm tắt vòng đời của một sản phẩm từ lúc khách xem, bỏ vào giỏ đến lúc thanh toán: sản phẩm bị ẩn hoặc hết hàng không hiện với khách, giỏ tự cập nhật khi Admin đổi, còn lúc thanh toán server luôn đọc lại database và lấy giá từ database.

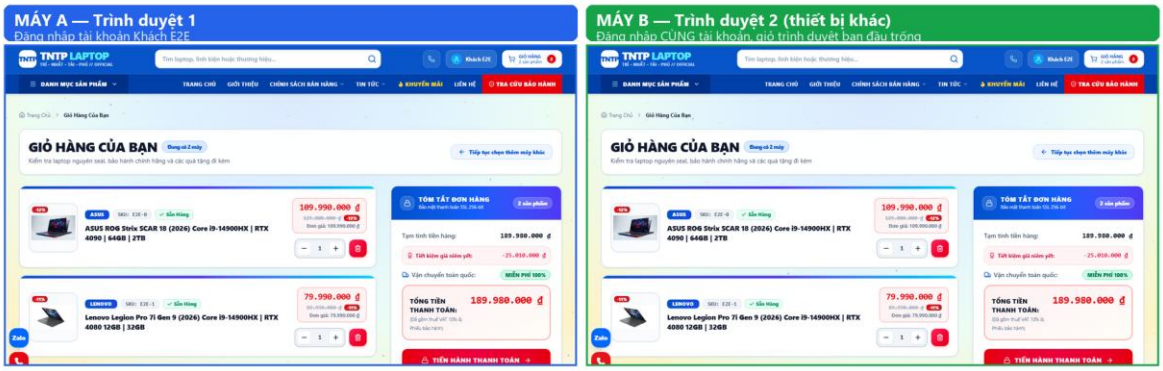
Hình 4.60. *Luồng xử lý sản phẩm ẩn, hết hàng, đổi giá: từ lúc khách xem, bỏ vào giỏ đến lúc thanh toán*

4.15.3. Giỏ hàng theo tài khoản

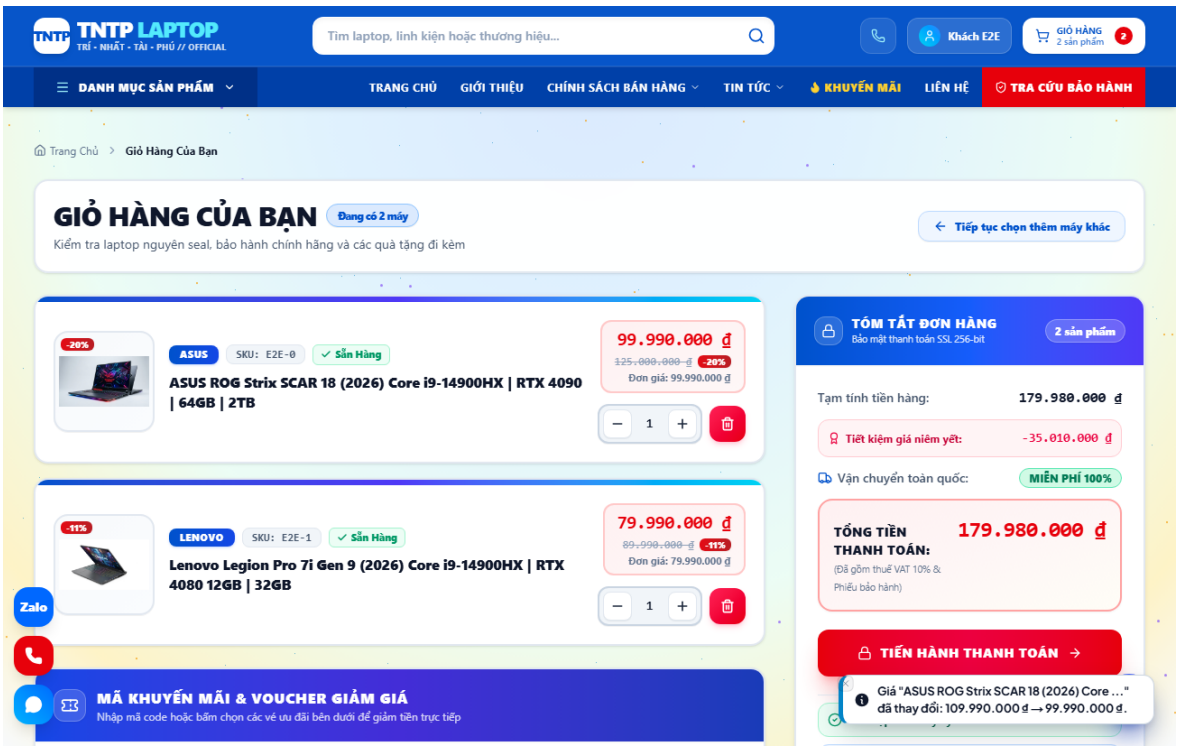
Giỏ hàng của khách vãng lai vẫn lưu trong trình duyệt; khi đăng nhập, giỏ được hợp nhất vào giỏ của tài khoản trong collection `gio_hang` và đi theo tài khoản giữa các thiết bị. Server chỉ lưu mã sản phẩm, tùy chọn và số lượng, **không lưu giá**: mỗi lần đọc giỏ, server lấy giá và tồn kho hiện tại từ bảng sản phẩm, gỡ món đã ẩn / hết hàng và hạ số lượng về đúng tồn kho. API `GET / PUT /api/gio-hang` bắt buộc đăng nhập (chưa đăng nhập trả 401, Hình 4.64; toàn bộ luồng ở Hình 4.61; kết quả thử trên hai trình duyệt ở Hình 4.62 và 4.63), giỏ gắn với tài khoản trong JWT nên người này không đọc hay ghi được giỏ người khác, giá do client gửi lên bị bỏ qua. Phía Front-End, giỏ cũng tự đồng bộ khi Admin đổi giá, ẩn hoặc làm hết hàng và hiện thông báo cho khách; khi đăng xuất, giỏ trong trình duyệt bị xóa nhưng giỏ trên server được giữ lại.



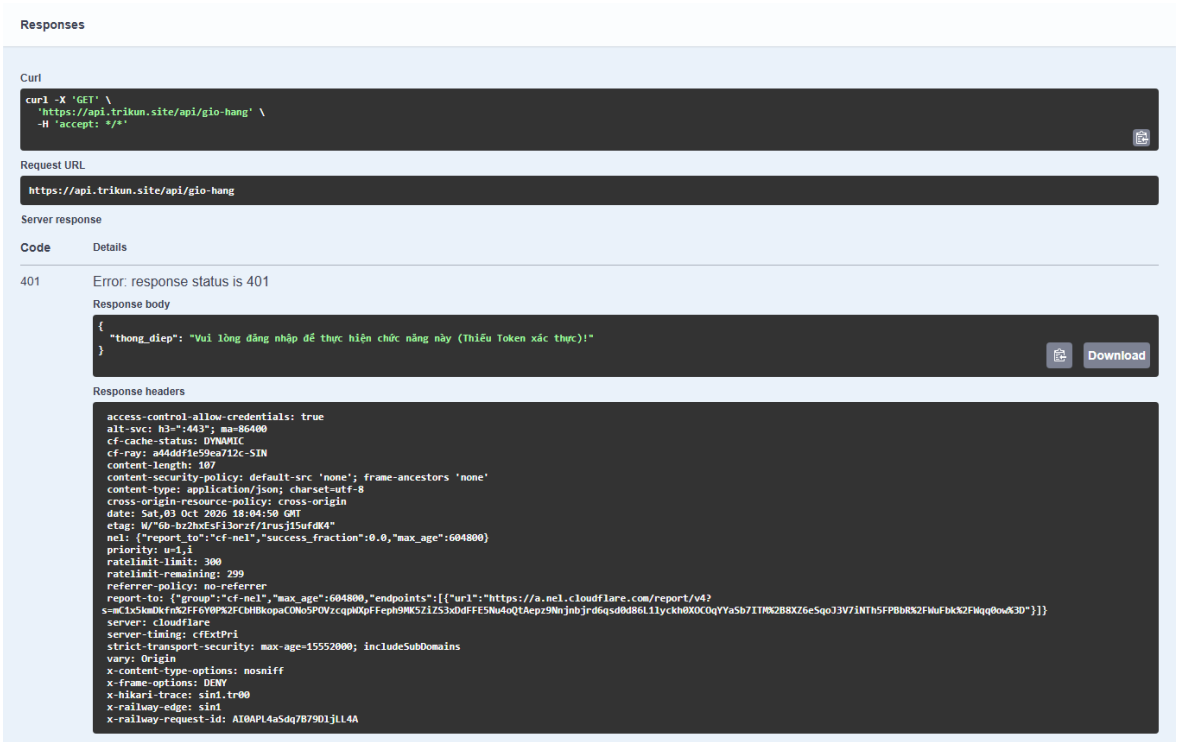
Hình 4.61. *Luồng giỏ hàng theo tài khoản: hợp nhất khi đăng nhập, dùng nhiều thiết bị, đăng xuất*



Hình 4.62. *Giỏ hàng cùng một tài khoản hiển thị giống nhau trên hai trình duyệt riêng biệt (môi trường kiểm thử cục bộ, dữ liệu mẫu)*



Hình 4.63. *Thông báo giá thay đổi "cũ → mới" hiện trong giỏ sau khi Admin đổi giá (môi trường kiểm thử cục bộ, dữ liệu mẫu)*



Hình 4.64. *Gọi thử GET /api/gio-hang khi chưa đăng nhập trên Swagger của server thật: bị từ chối 401*

4.15.4. Kết quả kiểm thử

Nhóm kiểm tra bằng trình duyệt Chrome tự động: mở trang khách, để "Admin" thao tác qua API rồi đo thời gian trang khách tự thay đổi (không reload); giỏ theo tài

khoản được thử bằng hai trình duyệt riêng biệt đăng nhập cùng một tài khoản. Phép đo chạy Back-End và Front-End cục bộ với cơ sở dữ liệu giả trong bộ nhớ (không đụng dữ liệu thật), cùng mã nguồn với bản đã triển khai; riêng trên `trikun.site`, nhóm kiểm tra trang tải đúng, gọi `/api/dong-bo` định kỳ và không có lỗi trong console. Các quy tắc phía server được bảo vệ thêm bằng 36 ca Jest mới (Bảng 6.8).

Bảng 4.3. *Thời gian phía khách tự cập nhật sau khi Admin thay đổi*

Kịch bản (Admin thao tác, khách không F5)	Phía khách cập nhật sau
Admin thêm sản phẩm	3,7 giây
Admin sửa tên / tạm ẩn / mở bán lại	5,0 giây
Admin xóa sản phẩm	5,3 giây
Admin ẩn sản phẩm khi khách đang xem trang chi tiết	4,4 giây (trang chuyển sang "không tìm thấy")
Admin và khách mở hai tab **cùng một trình duyệt**	0,6 giây
Giỏ hàng: Admin đổi giá / làm hết hàng / giảm tồn kho	1,3 – 7 giây (giỏ đổi giá, gỡ món, hạ số lượng và hiện thông báo)
Giỏ theo tài khoản: đăng nhập → giỏ được lưu vào database	0,3 giây
Giỏ theo tài khoản: máy B thêm món → máy A quay lại tab thấy	0,3 giây
Giỏ theo tài khoản: Admin ẩn món → giỏ cả hai máy gỡ món	0,8 giây

Hạn chế còn lại: danh sách yêu thích và so sánh vẫn lưu ở trình duyệt; trạng thái đơn hàng do Admin đổi chưa tự cập nhật cho khách đang mở trang tra cứu (khách cần mở lại trang).

> **Kết quả:** thay đổi của Admin được khách thấy trong khoảng 5 giây (0,6 giây nếu cùng trình duyệt) mà không cần tải lại trang; sản phẩm ẩn hoặc hết hàng không còn hiện với khách; giỏ hàng luôn đúng giá và tồn kho hiện tại và đi theo tài khoản giữa các thiết bị.

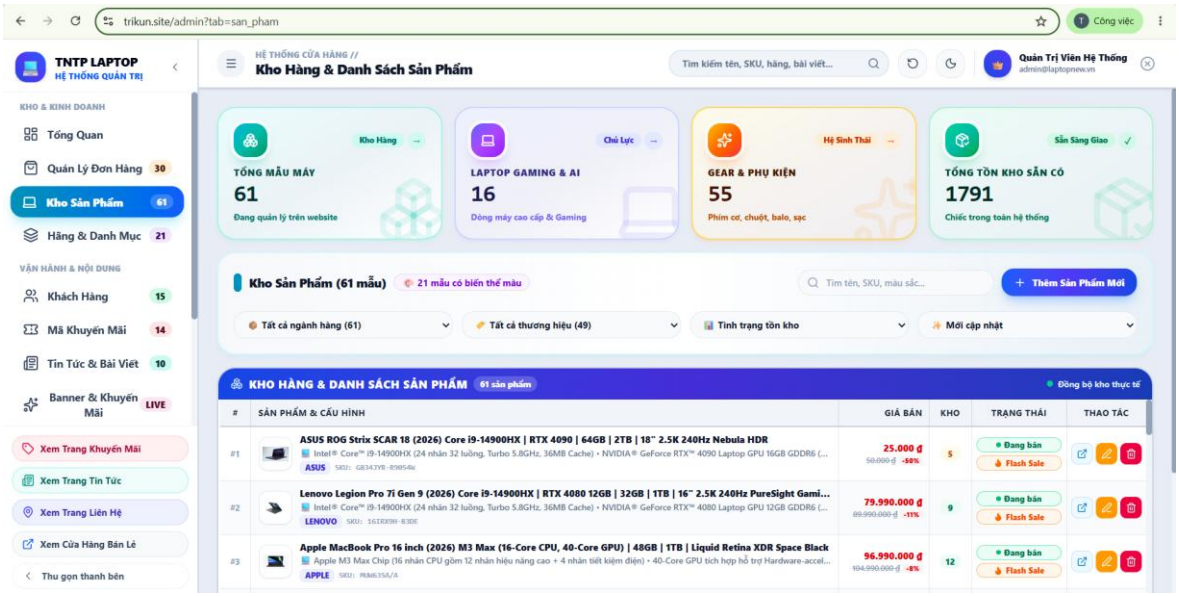
4.16. Giao diện quản trị Admin: thêm, xem, sửa, xóa (CRUD)

Toàn bộ dữ liệu của cửa hàng được quản lý ở trang `/admin`, chỉ mở được bằng tài khoản có vai trò Admin (các API ghi đều qua middleware xác thực và phân quyền). Mỗi nhóm dữ liệu có đủ bốn thao tác CRUD: **thêm** (nút "Thêm …" ở đầu bảng), **xem** (bảng

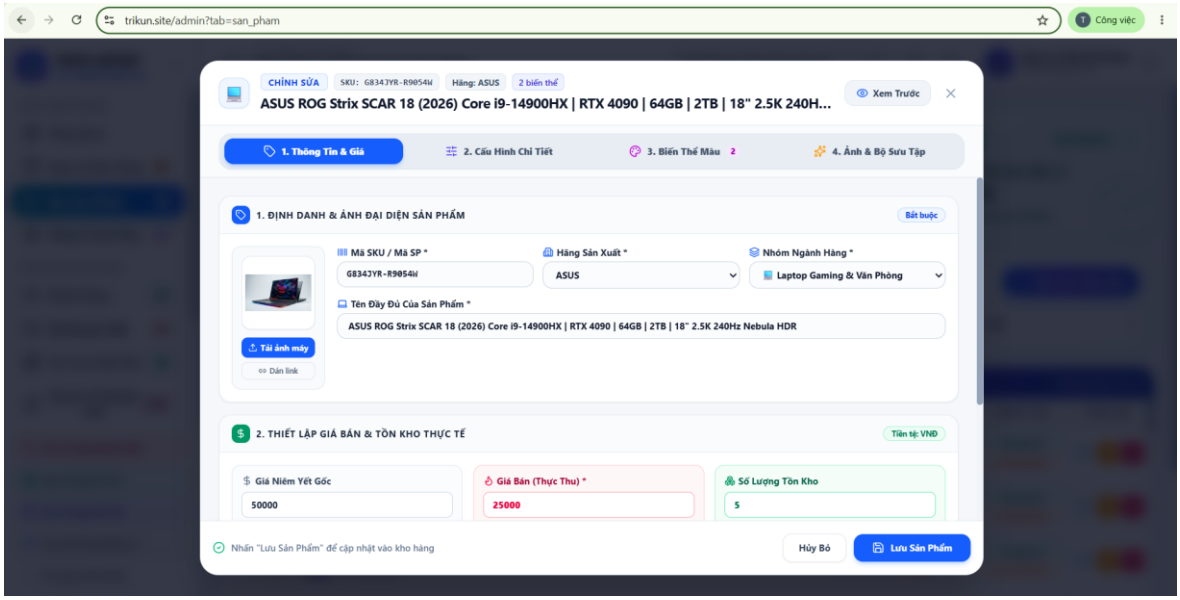
danh sách có tìm kiếm, bộ lọc), **sửa** (biểu tượng cây bút, mở biểu mẫu) và **xóa** (biểu tượng thùng rác, có hộp xác nhận). Các hình dưới đây chụp trực tiếp trên trikun.site; số điện thoại của khách đã được che.

4.16.1. Quản lý sản phẩm

Tab **Kho Sản Phẩm** (Hình 4.65) hiển thị thống kê nhanh (tổng mẫu máy, laptop gaming, phụ kiện, tổng tồn kho), bộ lọc theo ngành hàng, thương hiệu, tình trạng tồn kho, và danh sách gồm ảnh, cấu hình, giá bán, giá niêm yết kèm % giảm, tồn kho, trạng thái (đang bán, Flash Sale). Mỗi dòng có ba nút: xem trang sản phẩm, sửa và xóa. Biểu mẫu thêm / sửa (Hình 4.66) chia thành bốn bước: thông tin và giá, cấu hình chi tiết, biến thể màu, ảnh và bộ sưu tập. Khi lưu, Back-End kiểm tra lại giá (giá niêm yết phải bằng 0 hoặc không nhỏ hơn giá bán) và tự tính phần trăm giảm; thay đổi được khách nhìn thấy sau vài giây nhờ cơ chế đồng bộ ở mục 4.15.



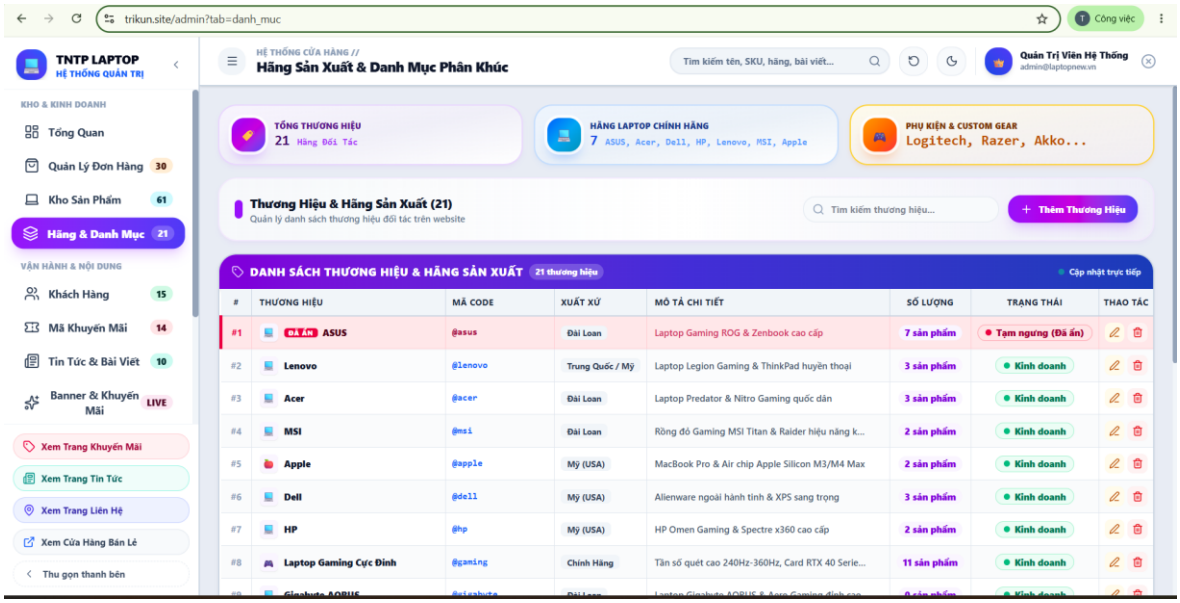
Hình 4.65. *Trang quản trị: danh sách Kho Sản Phẩm với bộ lọc và các nút xem / sửa / xóa*



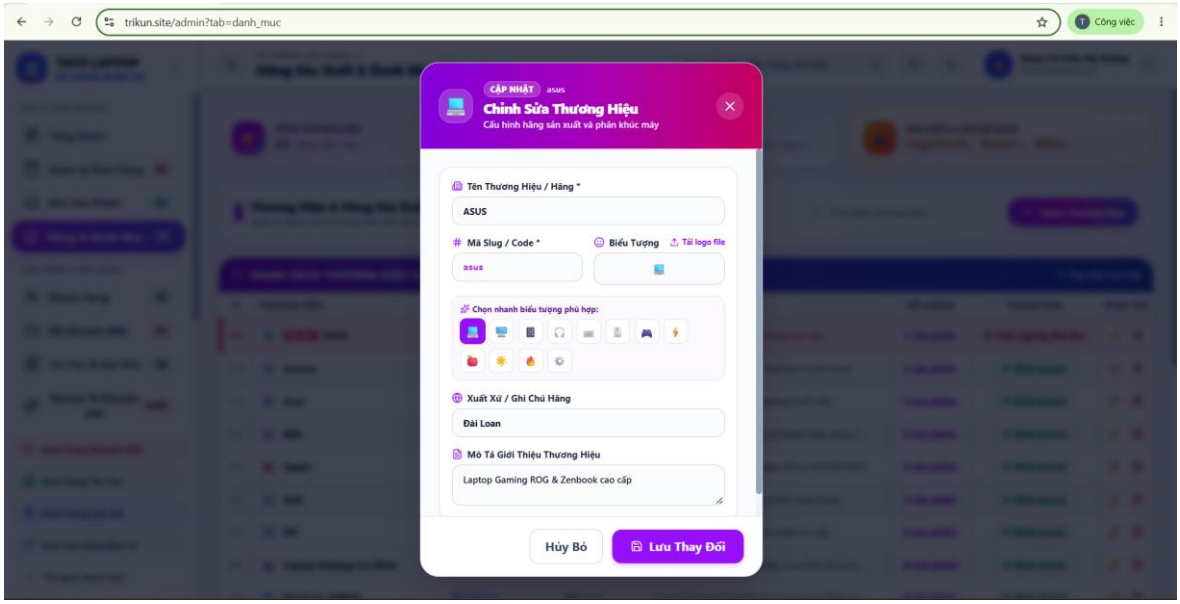
Hình 4.66. *Trang quản trị: biểu mẫu chỉnh sửa sản phẩm (thông tin, giá bán và tồn kho)*

4.16.2. Quản lý hãng và danh mục

Tab **Hãng & Danh Mục** (Hình 4.67) liệt kê 21 thương hiệu cùng mã code, xuất xứ, mô tả, số sản phẩm đang có và trạng thái kinh doanh. Thương hiệu tạm ngưng được gắn nhãn "Đã ẩn" và không hiện với khách. Biểu mẫu thêm / sửa (Hình 4.68) gồm tên, mã slug, biểu tượng, xuất xứ và mô tả.



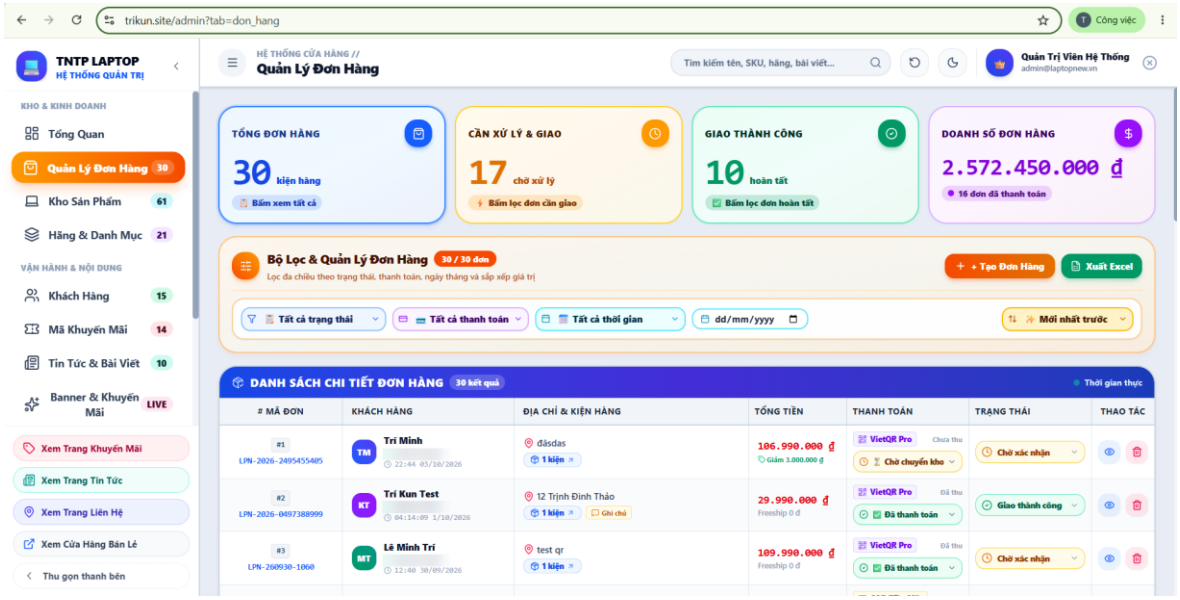
Hình 4.67. *Trang quản trị: danh sách thương hiệu và danh mục kèm số sản phẩm, trạng thái*



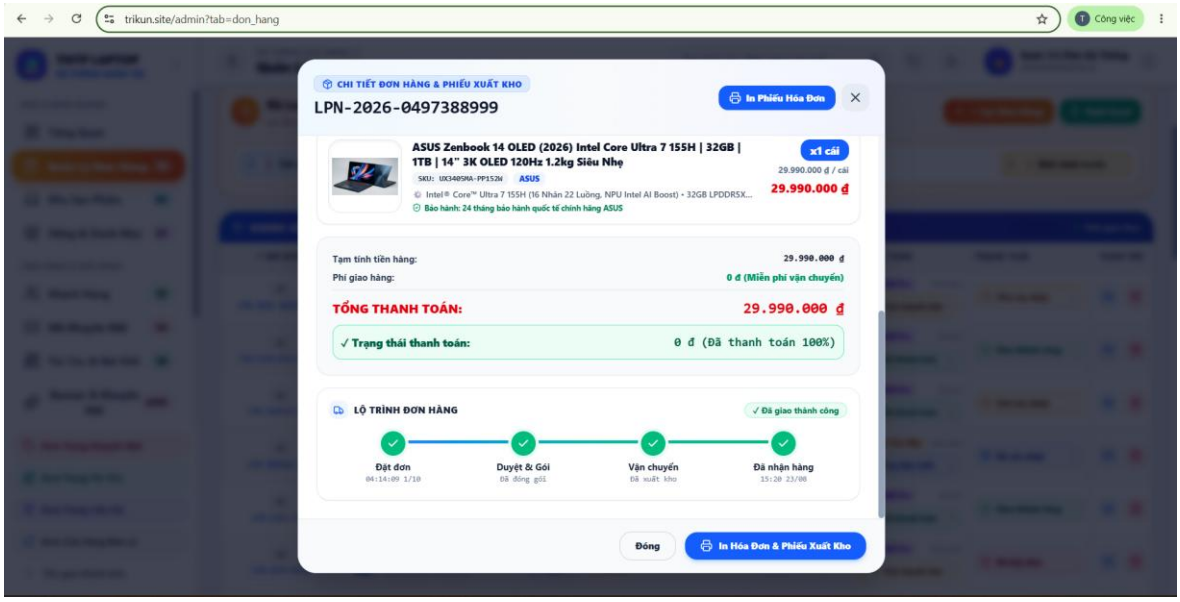
Hình 4.68. *Trang quản trị: biểu mẫu chỉnh sửa thương hiệu*

4.16.3. Quản lý đơn hàng

Tab **Quản Lý Đơn Hàng** (Hình 4.69) có thống kê (tổng đơn, đơn cần xử lý, giao thành công, doanh số), bộ lọc theo trạng thái, thanh toán, thời gian, ngày và cách sắp xếp, nút **Tạo Đơn Hàng** (đặt hộ khách) và nút **Xuất Excel** của danh sách đang lọc. Trên mỗi đơn, Admin đổi trực tiếp trạng thái thanh toán và trạng thái giao hàng bằng danh sách thả xuống, xem chi tiết hoặc xóa. Hộp chi tiết (Hình 4.70) hiển thị từng sản phẩm, tổng tiền, tình trạng thanh toán, lộ trình đơn hàng và nút in phiếu hóa đơn / phiếu xuất kho.



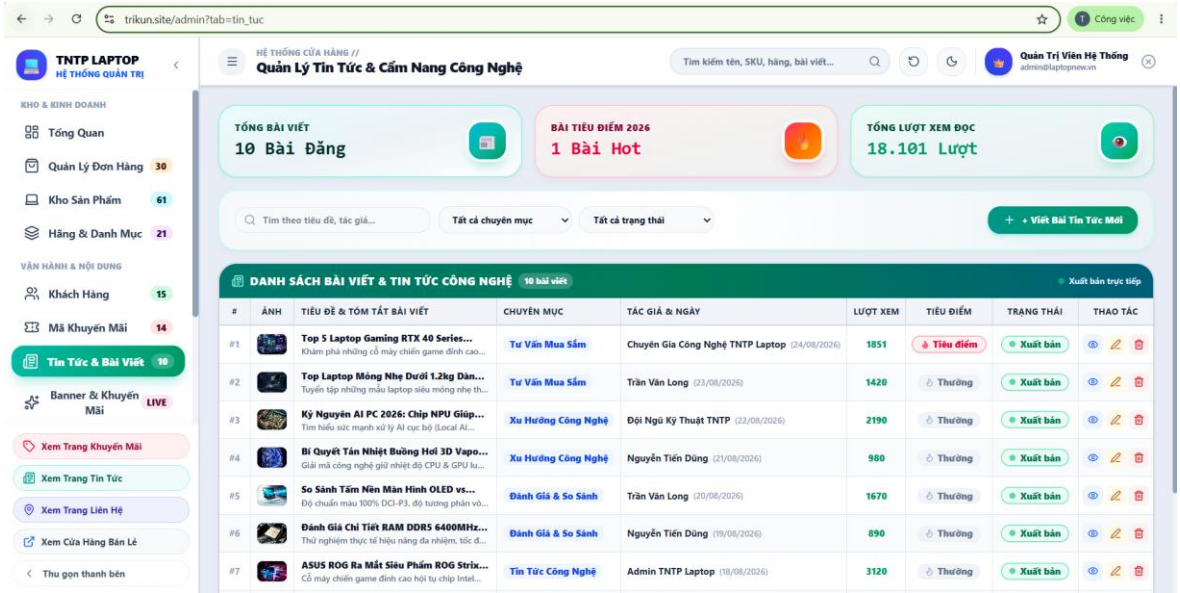
Hình 4.69. *Trang quản trị: danh sách đơn hàng với bộ lọc, nút tạo đơn, xuất Excel và đổi trạng thái*



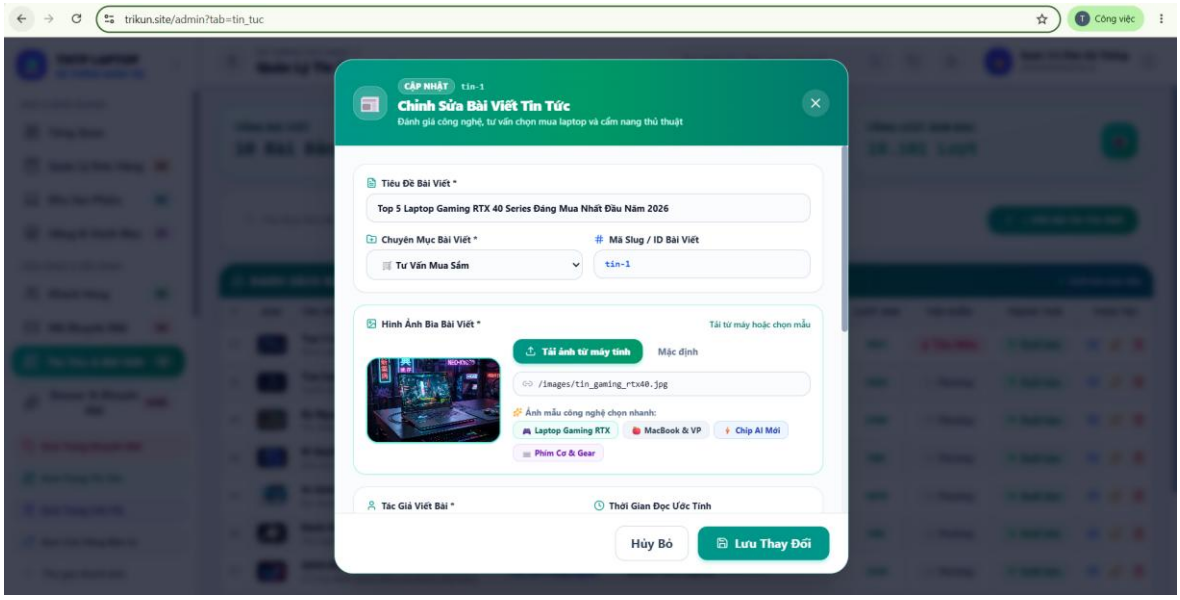
Hình 4.70. *Trang quản trị: chi tiết đơn hàng, lộ trình giao hàng và in phiếu hóa đơn*

4.16.4. Quản lý tin tức và bài viết

Tab **Tin Tức & Bài Viết** (Hình 4.71) thống kê tổng bài đăng, bài tiêu điểm và tổng lượt xem, có bộ lọc theo chuyên mục, trạng thái và ô tìm theo tiêu đề / tác giả. Mỗi bài hiển thị ảnh bìa, tóm tắt, chuyên mục, tác giả, ngày đăng, lượt xem, nhãn tiêu điểm và trạng thái xuất bản, kèm ba nút xem, sửa, xóa. Nút **Viết Bài Tin Tức Mới** và nút sửa mở cùng một biểu mẫu (Hình 4.72): tiêu đề, chuyên mục, mã slug, ảnh bìa (tải từ máy hoặc chọn ảnh mẫu), tác giả và thời gian đọc ước tính.



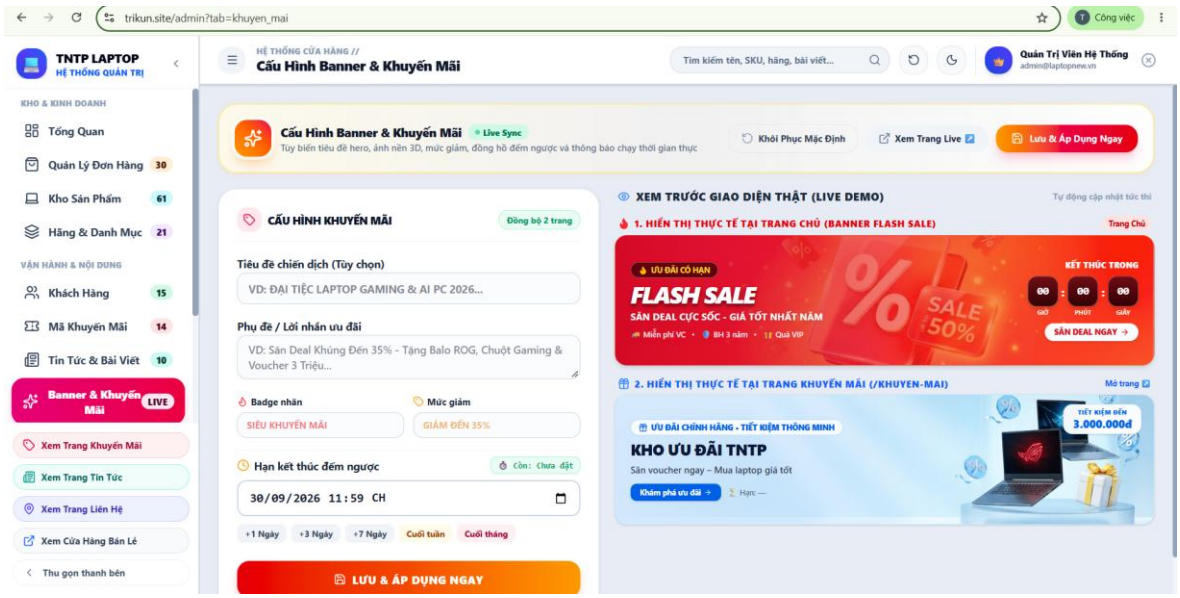
Hình 4.71. *Trang quản trị: danh sách bài viết kèm thống kê, bộ lọc và các nút xem / sửa / xóa*



Hình 4.72. *Trang quản trị: biểu mẫu chỉnh sửa bài viết tin tức*

4.16.5. Cấu hình banner và khuyến mãi

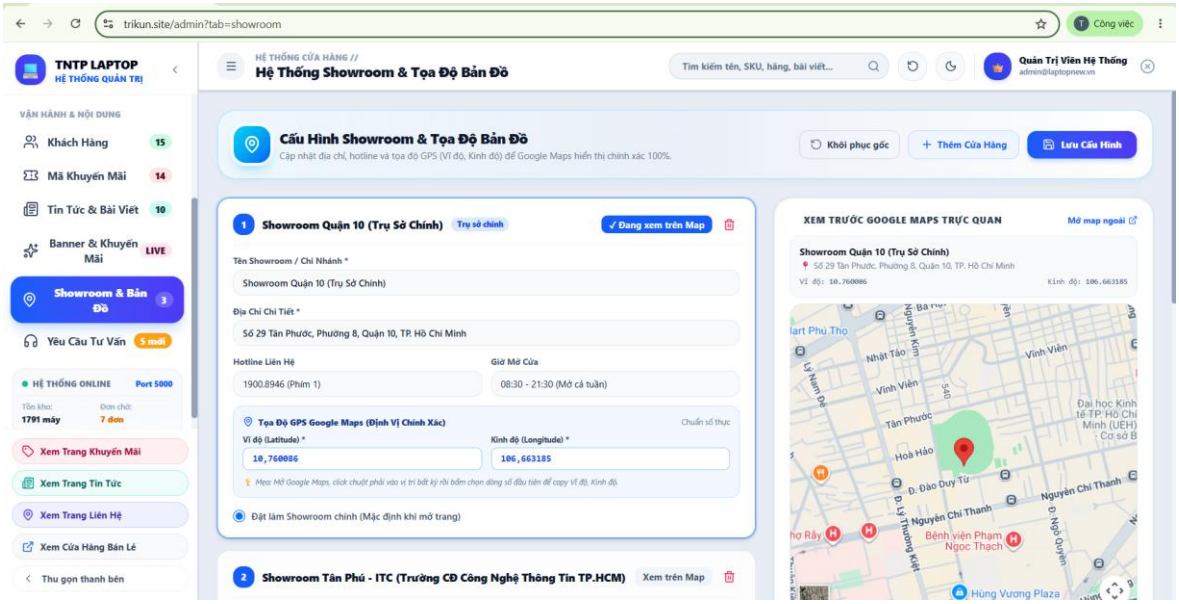
Tab **Banner & Khuyến Mãi** (Hình 4.73) cho Admin sửa tiêu đề chiến dịch, phụ đề, nhãn, mức giảm và hạn kết thúc đếm ngược (có nút chọn nhanh +1 ngày, +3 ngày, cuối tuần, cuối tháng). Bên phải là bản xem trước giao diện thật của banner Flash Sale ở trang chủ và của trang Khuyến mãi. Bấm **Lưu & Áp Dụng Ngay** thì cấu hình được lưu vào MongoDB (collection `cai_dat`) và khách thấy thay đổi sau vài giây nhờ cơ chế đồng bộ ở mục 4.15.



Hình 4.73. *Trang quản trị: cấu hình banner Flash Sale và khuyến mãi kèm bản xem trước giao diện thật*

4.16.6. Quản lý showroom và bản đồ

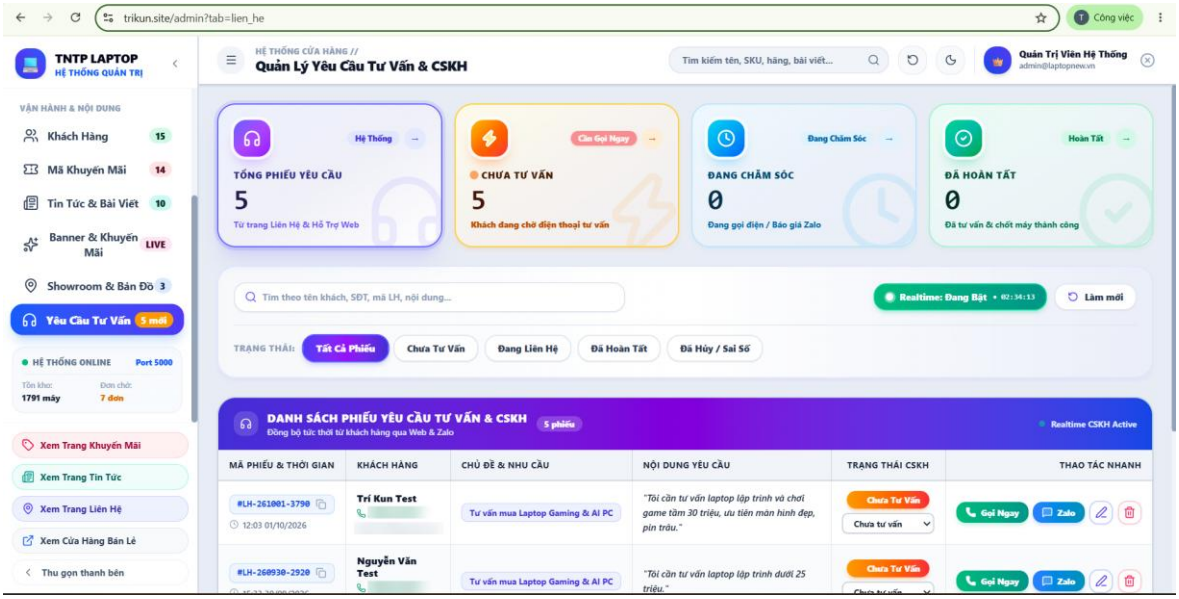
Tab **Showroom & Bản Đồ** (Hình 4.74) quản lý danh sách cửa hàng: tên, địa chỉ chi tiết, hotline, giờ mở cửa, tọa độ GPS (vĩ độ, kinh độ) và đặt một cửa hàng làm showroom chính. Admin có thể thêm hoặc xóa cửa hàng, khôi phục dữ liệu gốc; bản đồ Google Maps bên phải xem trước đúng vị trí vừa nhập trước khi bấm **Lưu Cấu Hình**.



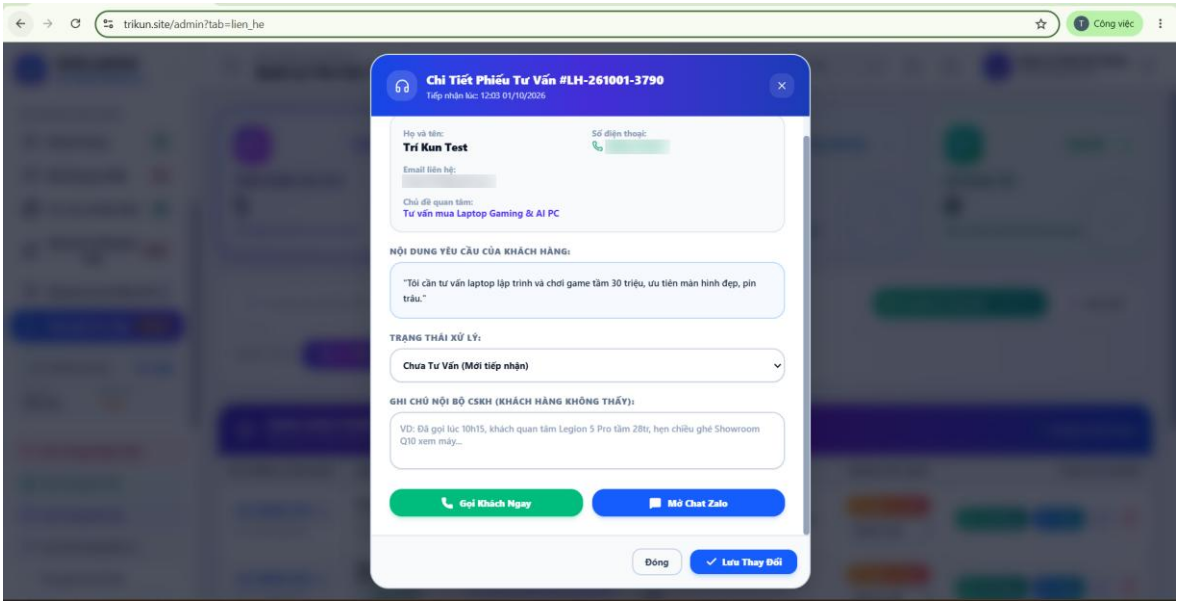
Hình 4.74. *Trang quản trị: cấu hình showroom, tọa độ GPS và xem trước Google Maps*

4.16.7. Quản lý yêu cầu tư vấn (CSKH)

Tab **Yêu Cầu Tư Vấn** (Hình 4.75) nhận các phiếu khách gửi từ trang Liên hệ: thống kê theo trạng thái (chưa tư vấn, đang chăm sóc, đã hoàn tất), tìm theo tên, số điện thoại, mã phiếu, đổi nhanh trạng thái, gọi điện hoặc mở Zalo, sửa và xóa phiếu. Hộp chi tiết (Hình 4.76) hiển thị đầy đủ nội dung yêu cầu, cho chọn trạng thái xử lý và ghi chú nội bộ mà khách không thấy. Số điện thoại và email của khách đã được che trong hình.



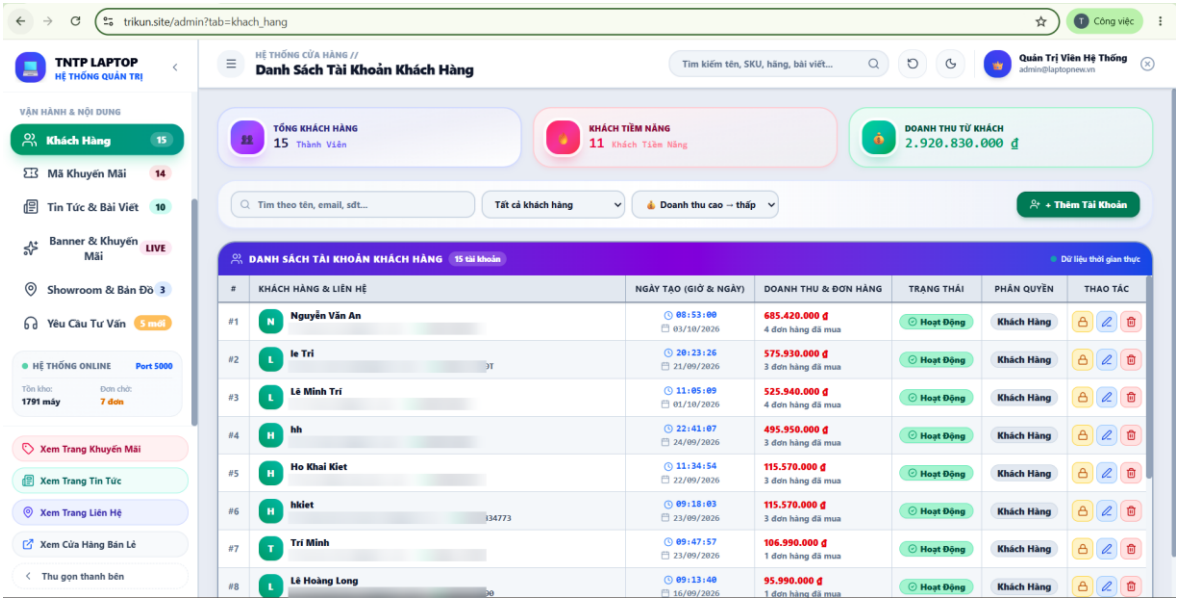
Hình 4.75. *Trang quản trị: danh sách phiếu yêu cầu tư vấn với trạng thái và các thao tác nhanh*



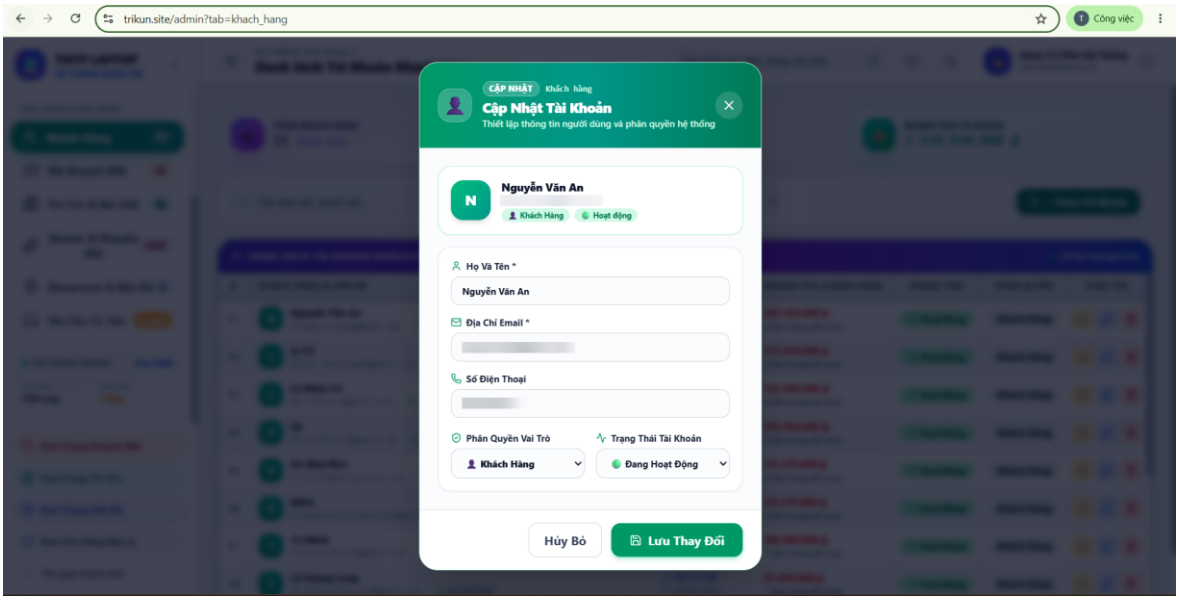
Hình 4.76. *Trang quản trị: chi tiết phiếu tư vấn, trạng thái xử lý và ghi chú nội bộ*

4.16.8. Quản lý tài khoản khách hàng

Tab **Khách Hàng** (Hình 4.77) liệt kê tài khoản kèm thống kê (tổng khách, khách tiềm năng, doanh thu từ khách), tìm kiếm, lọc và sắp xếp theo doanh thu. Nút **Thêm Tài Khoản** tạo tài khoản mới; mỗi dòng có ba nút khóa / mở khóa, sửa và xóa. Biểu mẫu cập nhật (Hình 4.78) cho đổi họ tên, email, số điện thoại, vai trò và trạng thái tài khoản. Khóa tài khoản hoặc đổi vai trò làm phiên đăng nhập cũ hết hiệu lực ngay (mục 4.14). Email và số điện thoại của khách đã được che trong hình.



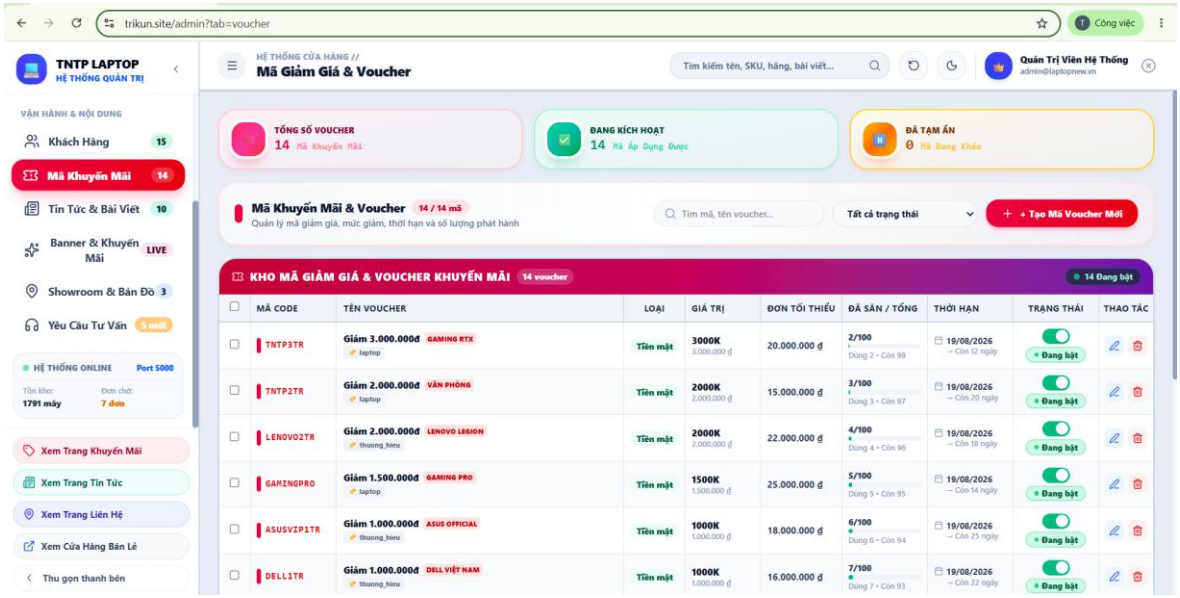
Hình 4.77. *Trang quản trị: danh sách tài khoản khách hàng kèm thống kê, bộ lọc và các thao tác khóa / sửa / xóa*



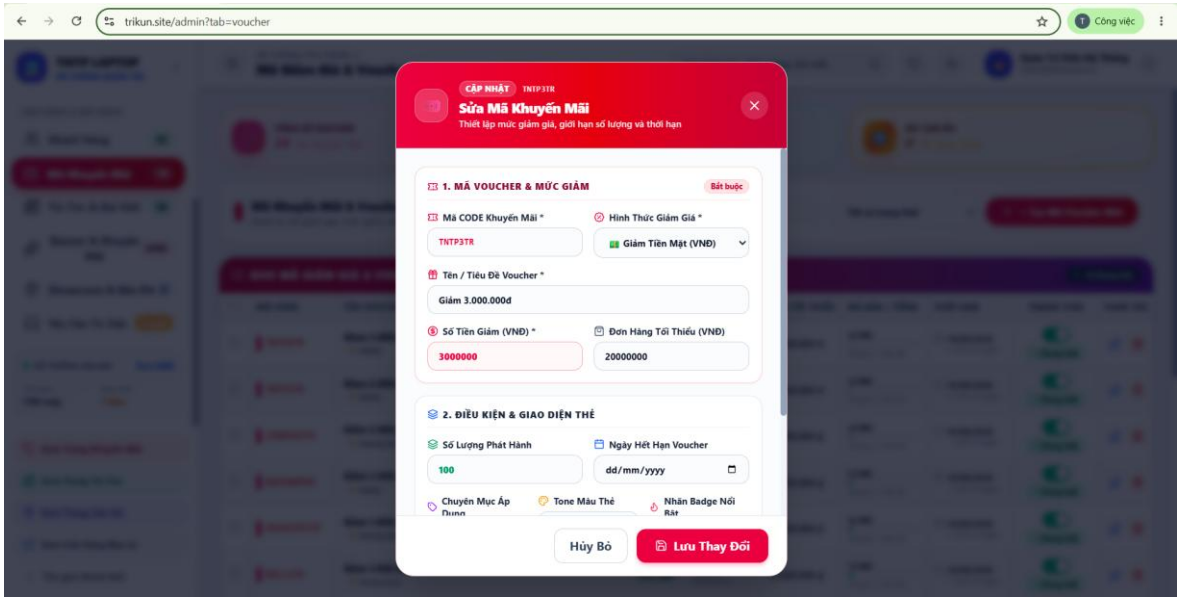
Hình 4.78. *Trang quản trị: biểu mẫu cập nhật tài khoản khách hàng*

4.16.9. Quản lý mã giảm giá (voucher)

Tab **Mã Khuyến Mãi** (Hình 4.79) thống kê số voucher, số đang kích hoạt và số đang tạm ẩn; mỗi mã hiển thị loại giảm (tiền mặt hoặc phần trăm), giá trị, đơn tối thiểu, số đã dùng trên tổng phát hành, hạn dùng, công tắc bật / tắt nhanh và nút sửa, xóa. Nút **Tạo Mã Voucher Mới** và nút sửa dùng chung một biểu mẫu (Hình 4.80): mã, hình thức giảm, tên, số tiền giảm, đơn tối thiểu, số lượng phát hành và ngày hết hạn. Khi khách áp mã lúc thanh toán, server kiểm tra lại mã, hạn dùng, số lượng còn lại và đơn tối thiểu rồi mới tính giảm giá.



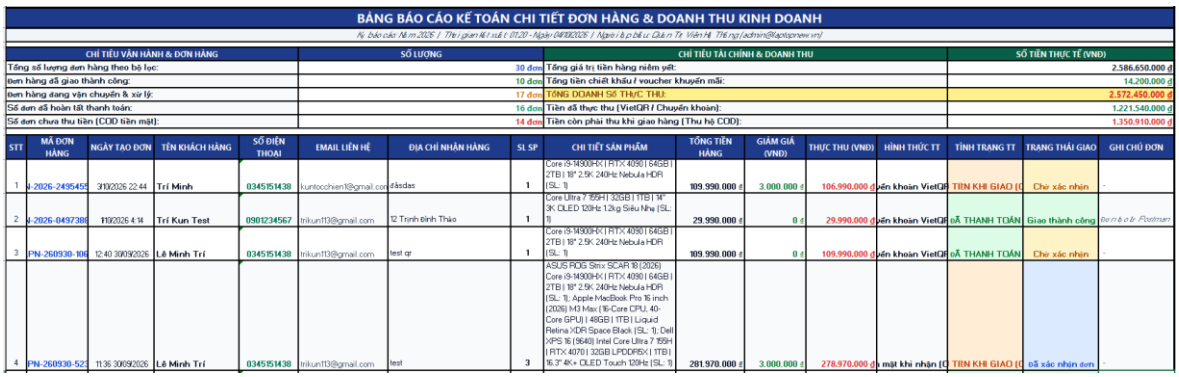
Hình 4.79. *Trang quản trị: danh sách mã giảm giá với số lượng đã dùng, hạn dùng và công tắc bật / tắt*



Hình 4.80. *Trang quản trị: biểu mẫu sửa mã khuyến mãi*

4.16.10. Xuất báo cáo kế toán ra Excel

Ở tab Đơn hàng, nút **Xuất Excel** tải về bảng báo cáo kế toán của các đơn đang lọc (Hình 4.81). Tệp gồm phần chỉ tiêu vận hành (tổng đơn, đơn đã giao, đang xử lý, đã thanh toán, chưa thu tiền COD), phần tài chính (tổng giá trị niêm yết, tổng chiết khấu và voucher, tổng doanh số thực thu, tiền đã thu, tiền còn thu hộ) và bảng chi tiết từng đơn: mã đơn, ngày tạo, khách hàng, địa chỉ, sản phẩm, tổng tiền, giảm giá, thực thu, hình thức và tình trạng thanh toán, trạng thái giao, ghi chú. Hình chụp phần đầu của tệp, dữ liệu là đơn hàng thử nghiệm.



BẢNG BÁO CÁO KẾ TOÁN CHI TIẾT ĐƠN HÀNG & DOANH THU KINH DOANH

Kỳ: Dữ liệu tất cả thời gian (từ khi bắt đầu đến 01:20 - Ngày 04/03/2026) | Người in: phải ưu tiên Trần Viết Hải | Nguồn: http://localhost:5000/admin

CHỈ TIÊU VẬN HÀNH & ĐƠN HÀNG	SỐ LƯỢNG	CHỈ TIÊU TÀI CHÍNH & DOANH THU	SỐ TIỀN THỰC TẾ (VNĐ)
Tổng số lượng đơn hàng theo bộ lọc:	30 đơn	Tổng giá trị tiền hàng niêm yết:	2,586,650,000 ₫
Đơn hàng đã giao thành công:	10 đơn	Tổng tiền chiết khấu / voucher khuyến mãi:	14,200,000 ₫
Đơn hàng đang vận chuyển & xử lý:	17 đơn	TỔNG DOANH SỐ THỰC THU:	2,572,450,000 ₫
Số đơn đã hoàn tất thanh toán:	16 đơn	Tiền đã thực thu (VietQR / Chuyển khoản):	1,221,540,000 ₫
Số đơn chưa thu tiền (COD tiền mặt):	14 đơn	Tiền còn phải thu khi giao hàng (Thu hộ COD):	1,350,910,000 ₫

STT	MÃ ĐƠN HÀNG	NGÀY TẠO ĐƠN	TÊN KHÁCH HÀNG	SỐ ĐIỆN THOẠI	EMAIL LIÊN HỆ	ĐỊA CHỈ NHẬN HÀNG	SL SP	CHI TIẾT SẢN PHẨM	TỔNG TIỀN HÀNG	GIẢM GIÁ (VNĐ)	THỰC THU (VNĐ)	HÌNH THỨC TT	TÌNH TRẠNG TT	TRẠNG THÁI GIAO	GHI CHÚ ĐƠN
1	-2026-2495455	3/02/2026 22:44	Trí Minh	0345151438	kuntocchien1@gmail.com	đâsdas	1	Core i9-14900HX | RTX 4090 | 64GB | 2TB | 18" 2.5K 240Hz Nebula HDR (SL: 1)	109.990.000 ₫	3.000.000 ₫	106.990.000 ₫	yển khoản VietQR	TIỀN KHI GIAO (C	Chờ xác nhận	-
2	-2026-0497388	1/02/2026 4:14	Trí Kun Test	0901234567	trikun113@gmail.com	12 Trịnh Đình Thảo	1	Core Ultra 7 155H | 32GB | 1TB | 14" 3K OLED 120Hz 1.2kg Siêu Nhẹ (SL: 1)	29.990.000 ₫	0 ₫	29.990.000 ₫	yển khoản VietQR	ĐÃ THANH TOÁN	Giao thành công	Bơm từ collection Postman
3	PN-260930-106	12:40 30/09/2026	Lê Minh Trí	0345151438	trikun113@gmail.com	test qr	1	Core i9-14900HX | RTX 4090 | 64GB | 2TB | 18" 2.5K 240Hz Nebula HDR (SL: 1)	109.990.000 ₫	0 ₫	109.990.000 ₫	yển khoản VietQR	ĐÃ THANH TOÁN	Chờ xác nhận	-
4	PN-260930-523	11:36 30/09/2026	Lê Minh Trí	0345151438	trikun113@gmail.com	test	3	ASUS ROG Strix SCAR 18 (2026) Core i9-14900HX | RTX 4090 | 64GB | 2TB | 18" 2.5K 240Hz Nebula HDR (SL: 1); Apple MacBook Pro 16 inch (2026) M3 Max (16-Core CPU, 40-Core GPU) | 48GB | 1TB | Liquid Retina XDR Space Black (SL: 1); Dell XPS 16 | 9640 | Intel Core Ultra 7 155H | RTX 4070 | 32GB LPDDR5X | 1TB | 16.3" 4K+ OLED Touch 120Hz (SL: 1)	281.970.000 ₫	3.000.000 ₫	278.970.000 ₫	ền mặt khi nhận (C	TIỀN KHI GIAO (C	ỗi xác nhận đơn	-

Hình 4.81. *Tệp Excel báo cáo kế toán do chức năng Xuất Excel tạo ra: chỉ tiêu tổng hợp và chi tiết từng đơn*

CHƯƠNG 5. XỬ LÝ LỖI VÀ BẢO MẬT

5.1. Chuẩn hóa mã trạng thái HTTP

Mọi controller đều bọc logic trong `try...catch` và trả về mã HTTP đúng ngữ nghĩa cùng thông điệp tiếng Việt dễ hiểu. Bảng sau tổng hợp các mã được dùng trong hệ thống.

Bảng 5.1. *Các mã trạng thái HTTP được sử dụng*

Mã	Tên	Tình huống thực tế trong hệ thống
200	OK	Đọc, cập nhật, xóa thành công; đăng nhập; kiểm tra voucher hợp lệ
201	Created	Tạo đơn hàng, sản phẩm, danh mục, voucher, đánh giá, phiếu liên hệ
400	Bad Request	Thiếu trường, sai định dạng email/SĐT, mật khẩu yếu, OTP sai/hết hạn, voucher chưa đủ điều kiện, chữ ký Webhook sai, JSON hỏng, dữ liệu bị lọc NoSQL Injection
401	Unauthorized	Thiếu/sai/hết hạn token; token đã bị thu hồi (sau đăng xuất, đổi mật khẩu, bị khóa); sai email hoặc mật khẩu
403	Forbidden	Không phải admin; tài khoản bị khóa / chưa kích hoạt; xem hoặc sửa đơn của người khác; đánh giá khi chưa mua hàng; origin không được phép (CORS)
404	Not Found	Không tìm thấy sản phẩm, đơn, voucher, người dùng; route không tồn tại
409	Conflict	Trùng mã sản phẩm / danh mục; voucher đã hết lượt lưu; tạo thanh toán cho đơn đã thanh toán; đánh giá trùng
410	Gone	API khôi phục showroom mặc định đã ngừng hỗ trợ
413	Payload Too Large	Body JSON vượt quá giới hạn 1 MB
429	Too Many Requests	Vượt giới hạn tần suất (300 request/phút/IP và hạn mức riêng của API nhạy cảm); nhập sai mật khẩu 5 lần → khóa đăng nhập 15 phút
500	Internal Server Error	Lỗi truy vấn DB hoặc lỗi chưa lường trước — bắt bởi try/catch hoặc middleware lỗi cuối
502	Bad Gateway	Không gửi được email OTP do dịch vụ email bên ngoài lỗi

5.2. Xử lý từng loại lỗi

5.2.1. Lỗi 400 – Bad Request

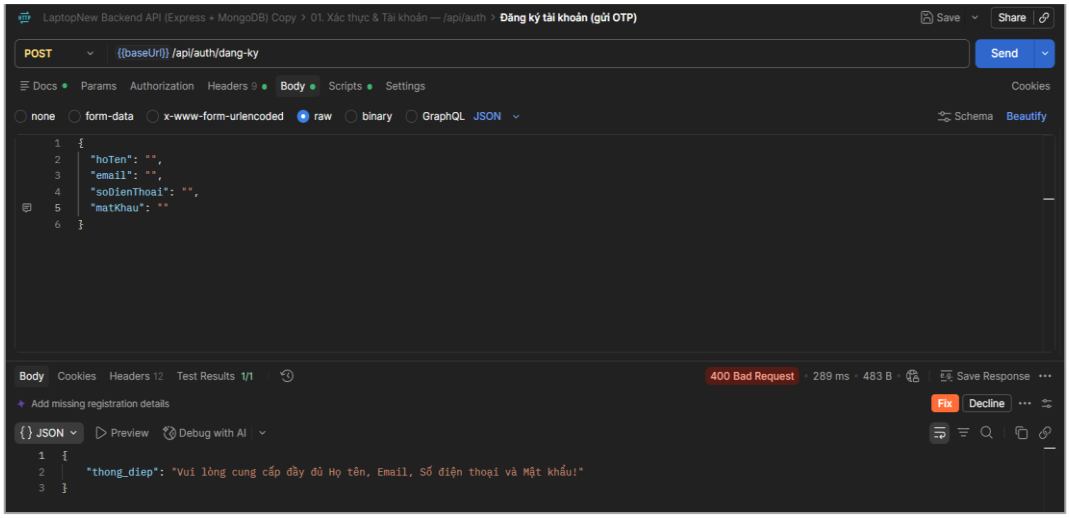
❖ **Nguyên nhân:**

- Dữ liệu client gửi lên thiếu hoặc sai định dạng (thiếu email, SĐT không đủ 10 số, mật khẩu mới dưới 6 ký tự).
- Mã OTP không đúng hoặc đã hết hạn; mã giảm giá chưa đủ giá trị đơn tối thiểu.

❖ **Cách xử lý:**

- Kiểm tra dữ liệu ngay đầu controller và `return res.status(400)` trước khi chạm vào cơ sở dữ liệu.
- Thông điệp nêu rõ cần sửa gì, ví dụ "Vui lòng cung cấp đầy đủ Họ tên, Email, Số điện thoại và Mật khẩu!".

```
POST /api/auth/dang-ky   { "hoTen": "A", "email": "x" }
HTTP/1.1 400 Bad Request
{ "thong_diep": "Vui lòng cung cấp đầy đủ Họ tên, Email, Số điện thoại và Mật khẩu!" }
```



Hình 5.1. *Kiểm thử lỗi 400 bằng Postman: đăng ký thiếu thông tin*

5.2.2. Lỗi 401 – Unauthorized

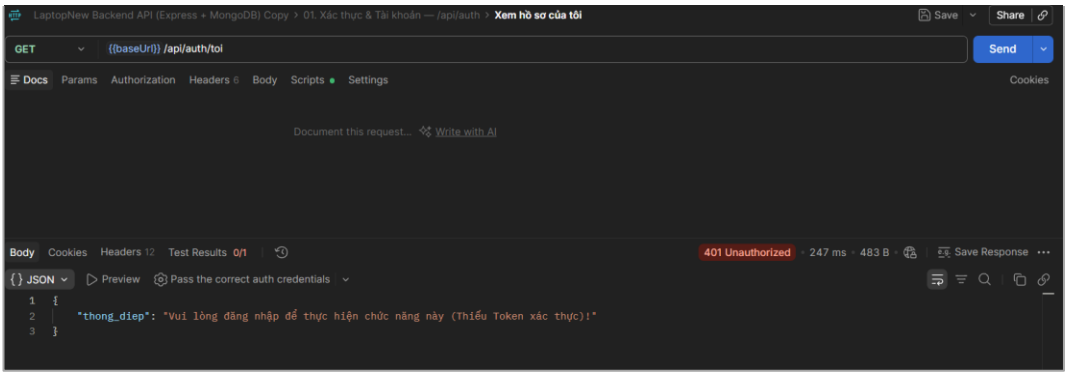
❖ **Nguyên nhân:**

- Gọi API cần đăng nhập nhưng không gửi token, token sai chữ ký hoặc đã hết hạn 7 ngày.
- Đăng nhập sai email hoặc mật khẩu.

❖ **Cách xử lý:**

- Middleware `xacThucToken` phân biệt `TokenExpiredError` (yêu cầu đăng nhập lại) với token bị sửa đổi.
- Khi đăng nhập sai, trả cùng một thông điệp cho cả hai trường hợp để kẻ tấn công không dò được email nào tồn tại.

```
GET /api/auth/toi        (không gửi header Authorization)
HTTP/1.1 401 Unauthorized
{ "thong_diep": "Vui lòng đăng nhập để thực hiện chức năng này (Thiếu Token xác thực)!" }

GET /api/auth/toi        Authorization: Bearer abc.def.ghi
HTTP/1.1 401 Unauthorized
{ "thong_diep": "Token xác thực không hợp lệ hoặc đã bị thay đổi!" }
```



Hình 5.2. *Kiểm thử lỗi 401 bằng Postman: gọi /api/auth/toi không gửi token*

5.2.3. Lỗi 403 – Forbidden

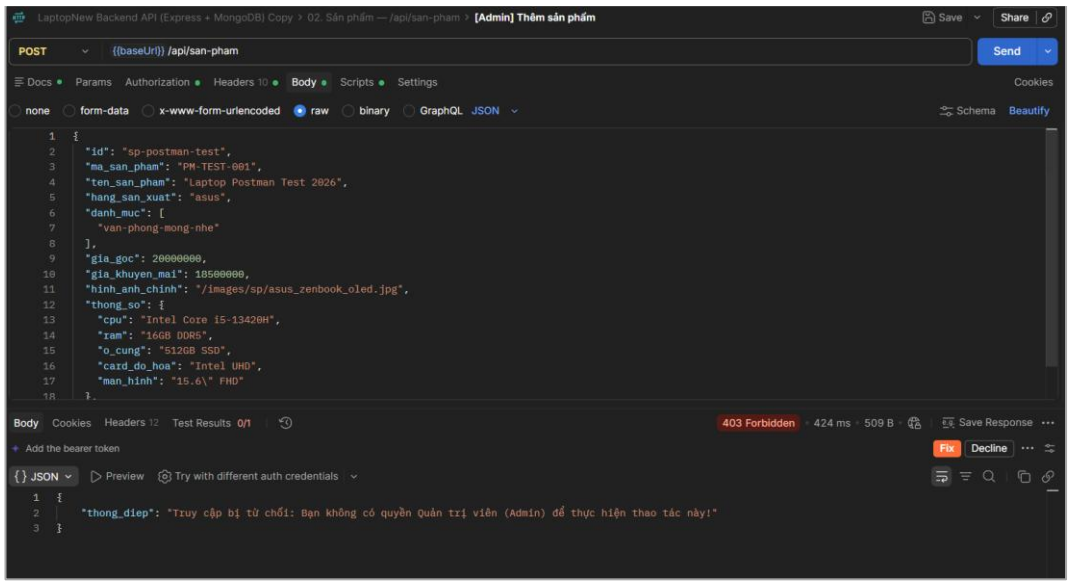
❖ **Nguyên nhân:**

- Tài khoản khách hàng gọi API dành cho admin (thêm/sửa/xóa sản phẩm, voucher, danh mục).
- Tài khoản bị khóa đăng nhập; khách cập nhật đơn không phải của mình.
- Khách tra cứu đơn hàng nhưng không nhập đúng số điện thoại nhận hàng, hoặc thành viên xem đơn của người khác.

❖ **Cách xử lý:**

- Middleware `chiAdmin` chặn trước khi vào controller; controller đơn hàng so sánh chủ đơn với `req.user`.

```
POST /api/san-pham       Authorization: Bearer <token của khach_hang>
HTTP/1.1 403 Forbidden
{ "thong_diep": "Truy cập bị từ chối: Bạn không có quyền Quản trị viên (Admin) để thực
hiện thao tác này!" }
```



Hình 5.3. *Kiểm thử lỗi 403 bằng Postman: tài khoản khách gọi API thêm sản phẩm*

5.2.4. Lỗi 404 – Not Found

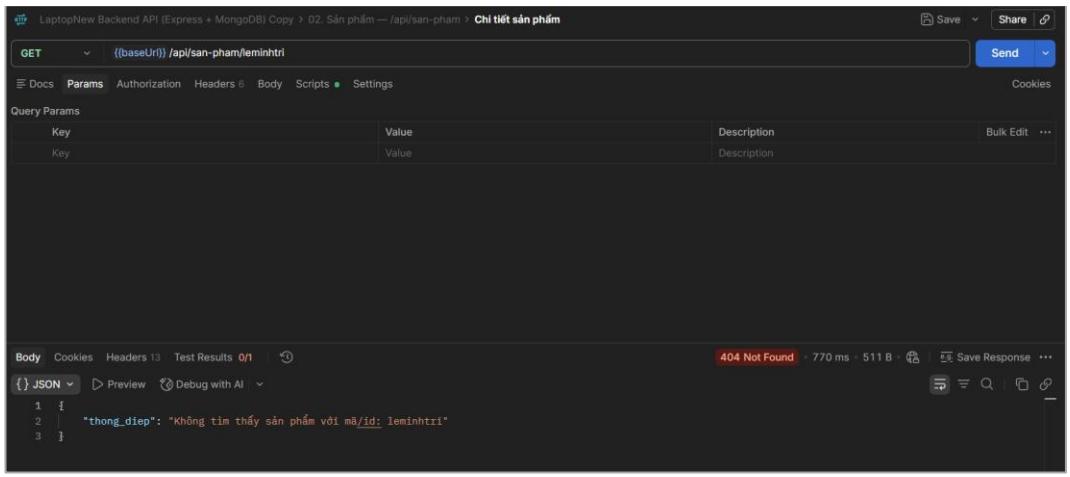
❖ **Nguyên nhân:**

- Tìm tài nguyên theo id/slug/mã không tồn tại.
- Gọi sai đường dẫn API.

❖ **Cách xử lý:**

- Controller kiểm tra kết quả truy vấn `null` → trả 404 kèm mã đã tìm.
- Middleware cuối `app.js` bắt mọi route không khớp.

```
GET /api/san-pham/khong-ton-tai-xyz
HTTP/1.1 404 Not Found
{ "thong_diep": "Không tìm thấy sản phẩm với mã/id: khong-ton-tai-xyz" }

GET /api/khong-co-route
HTTP/1.1 404 Not Found
{ "thong_diep": "Đường dẫn không tồn tại: /api/khong-co-route" }
```



Hình 5.4. *Kiểm thử lỗi 404 bằng Postman: sản phẩm không tồn tại*

5.2.5. Lỗi 500 – Internal Server Error

❖ **Nguyên nhân:**

- Mất kết nối MongoDB, lỗi ép kiểu dữ liệu, lỗi logic chưa lường trước.

❖ **Cách xử lý:**

- Mỗi controller có `catch` trả 500 kèm `chi_tiet: loi.message` để dễ gỡ lỗi.
- Express 5 tự chuyển lỗi của hàm `async` tới middleware lỗi cuối cùng, nên server không bị sập.
- Ghi log `console.error(err.stack)` để lập trình viên truy vết.

```
// app.js – middleware xử lý lỗi tập trung
app.use((req, res) => {                                    // 404 cho route không tồn tại
    res.status(404).json({ thong_diep: `Đường dẫn không tồn tại: ${req.originalUrl}` });
});
app.use((err, req, res, next) => {                         // 500 cho lỗi chưa được bắt
    console.error('❌ Lỗi máy chủ chưa được bắt:', err.stack);
    res.status(500).json({ thong_diep: 'Đã xảy ra lỗi nội bộ trên máy chủ!', chi_tiet:
err.message });
});
```

5.2.6. Lỗi 429 – Too Many Requests

❖ **Nguyên nhân:**

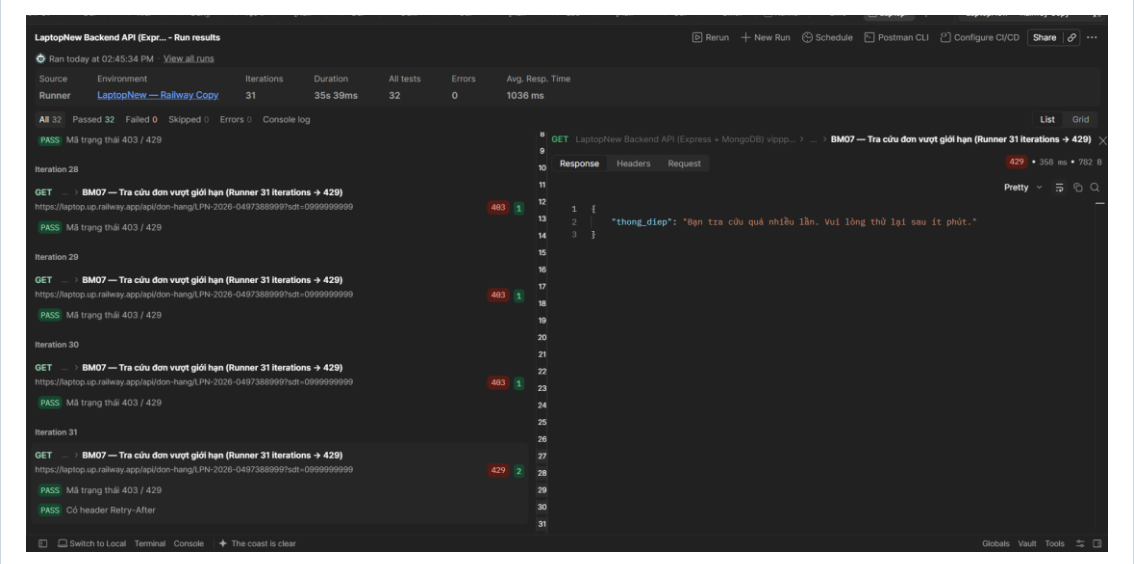
- Một IP gọi `GET /api/don-hang/:id` quá 30 lần trong 10 phút — dấu hiệu dò mã đơn hàng tự động.
- Một IP gửi quá nhiều request tới API bất kỳ (> 300/phút) hoặc tới API nhạy cảm (đăng nhập, OTP, thanh toán, chat AI...) vượt hạn mức riêng (Bảng 5.5).

- Một tài khoản bị nhập sai mật khẩu 5 lần liên tiếp — dấu hiệu dò mật khẩu (brute-force).

❖ **Cách xử lý:**

- Middleware dùng chung `gioiHanRequest({ windowMs, max })` đếm request theo IP cho từng bộ giới hạn, trả kèm header `RateLimit-Limit`, `RateLimit-Remaining` và `Retry-After` (số giây phải chờ).
- Vượt giới hạn thì dừng ngay trước controller, không truy vấn cơ sở dữ liệu.
- Khóa theo **tài khoản** (không chỉ theo IP): bộ đếm `soLanDangNhapSai` và mốc `khoaDangNhapDen` lưu trong MongoDB, nên kẻ tấn công đổi IP liên tục cũng không lách được.

```
GET /api/don-hang/LPN-260930-5239?sdt=…   (lần thứ 31 trong 10 phút)
HTTP/1.1 429 Too Many Requests
{ "thong_diep": "Bạn tra cứu quá nhiều lần. Vui lòng thử lại sau ít phút." }
```



Hình 5.5. *Kiểm thử lỗi 429 bằng Postman: tra cứu đơn vượt giới hạn*

5.3. Mô hình bảo mật nhiều lớp (Defense in Depth)

Nguyên tắc thiết kế của nhóm là **không tin bất kỳ dữ liệu nào từ client** và **không dựa vào một lớp bảo vệ duy nhất**. Mỗi request đi qua nhiều lớp kiểm tra; nếu một lớp bị vượt qua, lớp tiếp theo vẫn chặn được. Bảng 5.2 tóm tắt các lớp theo đúng thứ tự một request đi vào server.

***Bảng 5.2.** Các lớp bảo vệ một request theo thứ tự xử lý*

Lớp	Thành phần	Cơ chế bảo vệ	Chống lại
0. Biên mạng	Cloudflare	HTTPS bắt buộc (SSL Full, HTTP → 301), Bot Fight Mode, rate limit /api/auth/* tại biên	DDoS, bot tự động, dò mật khẩu quy mô lớn
1. Kết nối	cors, datHeaderBaoMat	CORS whitelist theo `CLIENT_URL`; security headers `nosniff`, `X-Frame-Options: DENY`, CSP, HSTS (production); ẩn `X-Powered-By`	Website lạ gọi API, Clickjacking, lộ công nghệ
2. Cửa vào	express.json, chongNoSqlInjection	Giới hạn body 1 MB; JSON hỏng → 400; xóa mọi khóa bắt đầu bằng `$` hoặc chứa `.`	DoS bằng payload lớn, NoSQL Injection
3. Tần suất	gioiHanRequest	300 request/phút/IP cho mọi API + hạn mức riêng cho API nhạy cảm (Bảng 5.5)	Spam, dò mã, DoS mức ứng dụng
4. Danh tính	xacThucToken	JWT HS256 (ghim thuật toán), hạn 7 ngày, kiểm tra `tokenVersion` + trạng thái khóa	Giả mạo token, dùng lại token đã thu hồi
5. Quyền	chiAdmin, laChuSoHuuDon	RBAC cho route quản trị; kiểm tra chủ sở hữu từng đơn hàng	Leo thang đặc quyền, IDOR
6. Dữ liệu vào	Controller	Ép kiểu chuỗi, whitelist trường được sửa, escape regex, kiểm tra định dạng	Mass Assignment, ReDoS, dữ liệu rác
7. Nghiệp vụ	Controller	Giá tính lại ở server; thao tác nguyên tử cho kho, voucher; Webhook kiểm chữ ký + số tiền + idempotent	Sửa giá, race condition, giả mạo thanh toán
8. Lưu trữ	Model	bcrypt cho mật khẩu, HMAC-SHA256 cho OTP, `toJSON()` ẩn trường nhạy cảm, bí mật trong `.env`	Lộ dữ liệu khi rò rỉ DB / API
9. Phản hồi	app.js	Ẩn chi tiết lỗi ở production; thông điệp chung chống dò email / mật khẩu	Thu thập thông tin (reconnaissance)

5.4. Đối chiếu với OWASP API Security Top 10 (2023)

OWASP API Security Top 10 là danh sách 10 rủi ro bảo mật API phổ biến nhất do tổ chức OWASP công bố, được dùng rộng rãi làm tiêu chuẩn đánh giá. Nhóm đối chiếu từng rủi ro với hệ thống LaptopNew như sau:

***Bảng 5.3.** Đối chiếu hệ thống LaptopNew với OWASP API Security Top 10*

Mã	Rủi ro	Biện pháp trong hệ thống	Kiểm chứng
API1	Broken Object Level Authorization (xem/sửa dữ liệu của người khác)	Chủ đơn xác định theo `id_nguoi_dung`; tra cứu công khai cần mã đơn + SĐT và dữ liệu bị che; khách chỉ được hủy đơn của mình	BM01, BM02, TC27–TC30
API2	Broken Authentication	bcrypt, JWT HS256 + thu hồi phiên, khóa đăng nhập sau 5 lần sai, OTP băm + giới hạn 5 lần, chống dò email, Google ID Token được xác minh	BM05, BM06, BM09, BM15, TC07–TC11
API3	Broken Object Property Level Authorization	Whitelist trường được sửa (hồ sơ, người dùng, đơn); `toJSON()` ẩn `matKhau`, `maOtp`, `tokenVersion`	BM03, BM04

Mã	Rủi ro	Biện pháp trong hệ thống	Kiểm chứng
API4	Unrestricted Resource Consumption	Rate limit toàn cục + riêng; body ≤ 1 MB; limit ≤ 200; escape regex chống ReDoS; timeout khi gọi PayOS / AI; rate limit lớp biên Cloudflare cho /api/auth/*	TC32, BM07
API5	Broken Function Level Authorization	`chiAdmin` trên mọi route quản trị (sản phẩm, danh mục, voucher, người dùng, tin tức, liên hệ, showroom, xóa đơn)	TC09, TC38, TC39
API6	Unrestricted Access to Sensitive Business Flows	Săn voucher nguyên tử; tạo thanh toán 20 lần/15 phút; đánh giá chỉ khi đã mua, 8 lần/giờ; liên hệ 5 phiếu/giờ	Hình 6.27
API7	Server Side Request Forgery	Server chỉ gọi các địa chỉ cố định (PayOS, Brevo, Telegram, Gemini); không bao giờ tải URL do người dùng nhập (URL ảnh chỉ được lưu)	Rà soát mã nguồn
API8	Security Misconfiguration	Security headers, ẩn `X-Powered-By`, CORS whitelist, bắt buộc `JWT_SECRET`, ẩn lỗi chi tiết ở production, `trust proxy` đúng 1 tầng	TC33, TC34, TC35
API9	Improper Inventory Management	74 route được thống kê đầy đủ (Bảng 3.3); collection Postman 141 request; API ngừng hỗ trợ trả 410 Gone	Bảng 3.3, Mục 6.5
API10	Unsafe Consumption of APIs	Webhook PayOS: chữ ký HMAC + số tiền + idempotent; Google: xác minh chữ ký, audience, `email_verified`; timeout và dự phòng khi dịch vụ ngoài lỗi	TC23, Hình 6.13

5.5. Rà soát và khắc phục lỗ hổng bảo mật

Sau khi hoàn thành chức năng, nhóm tự **đóng vai kẻ tấn công** để rà soát toàn bộ mã nguồn Back-End theo OWASP. Bảng 5.4 liệt kê **21 lỗ hổng** đã phát hiện: cách khai thác nếu chưa sửa, cách nhóm đã khắc phục và ca kiểm thử chứng minh. **Toàn bộ 21 lỗ hổng đã được khắc phục** trong mã nguồn hiện tại.

Bảng 5.4. *Các lỗ hổng phát hiện khi rà soát và cách đã khắc phục*

#	Lỗ hổng (mức độ)	Khai thác nếu chưa sửa	Đã khắc phục bằng	Kiểm chứng
Mức NGHIÊM TRỌNG				
1	Đăng nhập Google giả mạo	Gửi email bất kỳ không kèm ID Token hợp lệ → đăng nhập vào tài khoản người khác	Chỉ tin dữ liệu do Google cấp: `verifyIdToken` (ID Token) hoặc `getTokenInfo` + `userinfo` (Access Token, kiểm tra `aud`); bắt buộc `email_verified`	Email giả không token → 400; token rác → 401
2	API quản trị người dùng công khai	`POST /api/nguoi-dung` tự tạo tài khoản `vaiTro: admin`, mật khẩu không băm	`xacThucToken` + `chiAdmin`, whitelist trường, băm mật khẩu	TC38
3	Mật khẩu tài khoản Google đoán được	Mật khẩu mặc định dạng `GoogleOAuth_<sub>` → đăng nhập bằng mật khẩu	Mật khẩu ngẫu nhiên 32 byte; từ chối dạng mật khẩu cũ	Rà soát mã

#	Lỗ hổng (mức độ)	Khai thác nếu chưa sửa	Đã khắc phục bằng	Kiểm chứng
Mức CAO				
4	Lộ mã OTP qua API	`GET /api/auth/toi` trả về `maOtp` → chiếm tài khoản khi đang đặt lại mật khẩu	`toJSON()` loại `maOtp`, `hanOtp`, `loaiOtp`, `tokenVersion`, các bộ đếm; OTP lưu dạng băm	BM04
5	NoSQL Injection	Gửi `{"$ne": null}` thay cho giá trị chuỗi để vượt qua điều kiện truy vấn	Middleware `chongNoSqlInjection` + ép kiểu chuỗi ở controller	TC31
6	IDOR xem trộm đơn hàng	Sửa SĐT hồ sơ thành SĐT nạn nhân → thấy toàn bộ đơn kèm địa chỉ của họ	Chủ đơn chỉ xác định theo `id_nguoi_dung`	BM01
7	Khách tự xác nhận đã thanh toán	`PUT /don-hang/:id` với `da_thanh_toan: true` → nhận hàng không trả tiền	Chủ đơn chỉ được hủy đơn chờ xác nhận	BM02
8	Sửa giá đơn hàng / số tiền thanh toán	Gửi giá 1.000đ cho laptop 30 triệu; tạo link thanh toán với số tiền tùy ý	Server tính lại giá, voucher, tổng tiền từ DB; Webhook kiểm số tiền	TC37, BM02
9	CRUD tin tức, liên hệ, showroom không cần đăng nhập	Ai cũng sửa/xóa bài viết, đọc toàn bộ phiếu liên hệ (dữ liệu cá nhân)	Gắn `xacThucToken` + `chiAdmin`	TC39
Mức TRUNG BÌNH				
10	Dò OTP 6 số	Thử lần lượt 1.000.000 mã cho tới khi đúng	Sai 5 lần → hủy mã; tối đa 6 lần gửi OTP / 15 phút / IP; sinh bằng `crypto.randomInt`	BM06
11	OTP dùng sai mục đích	Dùng OTP đổi mật khẩu để gọi API đặt lại mật khẩu	Kiểm tra `loaiOtp` đúng luồng	Rà soát mã
12	Dò mật khẩu (brute-force)	Thử hàng nghìn mật khẩu cho một email	Khóa đăng nhập 15 phút sau 5 lần sai + 30 request/15 phút/IP	BM05
13	Token vẫn dùng được sau đăng xuất / bị khóa	Token bị lộ dùng tiếp được tới 7 ngày	`tokenVersion` thu hồi phiên phía server	BM09, BM15
14	Regex Injection / ReDoS	Tìm kiếm với `(a+)+$` làm CPU treo	`escapeRegex()` trước mọi `$regex` (sản phẩm, tin tức, liên hệ)	TC32
15	Mass Assignment hồ sơ	Gửi `vaiTro: "admin"` khi cập nhật hồ sơ	Whitelist trường được phép sửa	BM03
16	Race condition kho & voucher	Gửi 2 request "đã giao" cùng lúc → trừ kho 2 lần; săn voucher vượt số lượng	`findOneAndUpdate` có điều kiện (nguyên tử)	Hình 6.25, 6.27
17	HTML Injection trong email	Họ tên chứa thẻ `<a href=...>` → email lừa đảo gửi từ tên miền cửa hàng	`escapeHtml()` dữ liệu người dùng trong mẫu email	Rà soát mã
Mức THẤP				
18	Thiếu security headers, lộ lỗi	Lộ `X-Powered-By: Express`; JSON hỏng trả 500 kèm stack	Security headers; lỗi 400/413/403 rõ ràng; ẩn chi tiết ở production	TC33–TC35
19	Bản nháp tin tức xem được công khai	`GET /api/tin-tuc?tat_ca=true` trả cả bài chưa xuất bản	Chỉ Admin xem bản nháp	TC40
20	Cấu hình JWT chưa chặt	`JWT_SECRET` dự phòng viết cứng; chưa ghim thuật toán	Bắt buộc biến môi trường; `algorithms: ['HS256']`	Rà soát mã

#	Lỗ hổng (mức độ)	Khai thác nếu chưa sửa	Đã khắc phục bằng	Kiểm chứng
21	Dò email tồn tại	Quên mật khẩu trả 404 nếu email không có	Luôn trả 200 cùng thông điệp; thời gian phản hồi đồng đều	Rà soát mã

5.6. Hiện thực các cơ chế bảo mật tiêu biểu

5.6.1. Chống NoSQL Injection bằng middleware

MongoDB dùng các toán tử bắt đầu bằng `$` (như `$ne`, `$gt`). Nếu server đưa nguyên object client gửi vào truy vấn, kẻ tấn công có thể gửi `{ "matKhau": { "$ne": null } }` để biến điều kiện "mật khẩu bằng X" thành "mật khẩu khác rỗng". Middleware dưới đây chạy cho **mọi request**, xóa các khóa nguy hiểm trước khi tới controller; controller còn ép kiểu chuỗi thêm một lần nữa.

Middleware chongNoSqlInjection — middleware/bao-mat.middleware.js

```
const locKhoaNguyHiem = (giaTri, doSau = 0) => {
    if (doSau > 10 || giaTri === null || typeof giaTri !== 'object') return 0;
    let soKhoaDaXoa = 0;
    for (const khoa of Object.keys(giaTri)) {
        if (khoa.startsWith('$') || khoa.includes('.') || khoa === '__proto__') {
            delete giaTri[khoa]; soKhoaDaXoa += 1;            // xóa toán tử MongoDB /
prototype
        } else soKhoaDaXoa += locKhoaNguyHiem(giaTri[khoa], doSau + 1);  // duyệt đệ quy
    }
    return soKhoaDaXoa;
};
app.use((req, res, next) => { locKhoaNguyHiem(req.body); locKhoaNguyHiem(req.params);
next(); });
```

5.6.2. Khóa đăng nhập khi bị dò mật khẩu

Giới hạn theo IP chưa đủ vì kẻ tấn công có thể đổi IP liên tục. Hệ thống lưu thêm bộ đếm **theo tài khoản** trong MongoDB: sai 5 lần liên tiếp thì tài khoản bị tạm khóa đăng nhập 15 phút, đăng nhập đúng thì bộ đếm về 0.

Khóa đăng nhập 15 phút sau 5 lần sai — controllers/xac-thuc.controller.js

```
if (nguoiDung.khoaDangNhapDen && nguoiDung.khoaDangNhapDen > new Date())
    return res.status(429).json({ thong_diep: 'Tài khoản tạm khóa đăng nhập...' });
const matKhauDung = await nguoiDung.soSanhMatKhau(matKhau);          // bcrypt.compare
if (!matKhauDung) {
    nguoiDung.soLanDangNhapSai += 1;
    if (nguoiDung.soLanDangNhapSai >= 5) {
        nguoiDung.khoaDangNhapDen = new Date(Date.now() + 15 * 60 * 1000);
        nguoiDung.soLanDangNhapSai = 0;
    }
    await nguoiDung.save();
    return res.status(401).json({ thong_diep: 'Tài khoản hoặc Mật khẩu không chính
xác!' });
}
```

5.6.3. OTP an toàn: sinh ngẫu nhiên mật mã, lưu dạng băm, giới hạn số lần nhập

OTP là "chìa khóa tạm" của tài khoản nên được bảo vệ như mật khẩu: sinh bằng bộ sinh số ngẫu nhiên an toàn mật mã, **chỉ lưu bản băm** (lộ DB cũng không đọc được mã), so sánh bằng hàm thời gian hằng và tự hủy sau 5 lần nhập sai.

Tiện ích OTP — utils/bao-mat.js

```
const taoMaOtp = () => crypto.randomInt(100000, 1000000).toString();   // thay
Math.random
const bamOtp = (otp) => crypto.createHmac('sha256',
process.env.JWT_SECRET).update(otp).digest('hex');
const soSanhOtp = (giaTriDaLuu, otpNhap) => {
    const a = Buffer.from(giaTriDaLuu), b = Buffer.from(bamOtp(otpNhap));
    return a.length === b.length && crypto.timingSafeEqual(a, b);     // chống timing
attack
};
```

5.6.4. Chống IDOR khi xem và cập nhật đơn hàng

IDOR (Insecure Direct Object Reference) xảy ra khi server chỉ kiểm tra "đã đăng nhập" mà không kiểm tra "dữ liệu này có phải của bạn không". Hàm dưới đây là điểm kiểm tra duy nhất, được dùng chung cho xem đơn, cập nhật đơn và tạo thanh toán.

Xác định chủ đơn — controllers/don-hang.controller.js

```
const laChuSoHuuDon = (user, donHang) => {
    const idDon = String(donHang.id_nguoi_dung || '');
    if (idDon) return idDon === String(user.id) || idDon === String(user._id);
    // Đơn cũ chưa gắn id: so theo email (đã xác thực OTP). KHÔNG so theo SĐT tự khai.
    return user.email.toLowerCase() === donHang.thong_tin_giao_hang.email.toLowerCase();
};
```

5.6.5. Thu hồi phiên đăng nhập phía server (tokenVersion)

JWT là stateless nên bình thường không thể "hủy" trước hạn. Nhóm thêm trường `tokenVersion` vào người dùng và nhúng vào token khi đăng nhập. Mỗi khi **đăng xuất, đổi/đặt lại mật khẩu, bị Admin khóa hoặc đổi vai trò**, `tokenVersion` tăng 1 → mọi token cũ không còn khớp và bị từ chối ngay ở `xacThucToken`.

Đăng xuất và kiểm tra tokenVersion

```
// controllers/xac-thuc.controller.js — dangXuat
await NguoiDung.updateOne({ _id: req.user._id }, { $inc: { tokenVersion: 1 } });
// middleware/xac-thuc.middleware.js — xacThucToken
if (Number(decoded.tokenVersion || 0) !== Number(nguoiDung.tokenVersion || 0))
    return res.status(401).json({ thong_diep: 'Phiên đăng nhập đã bị thu hồi. Vui lòng
đăng nhập lại.' });
```

5.6.6. Giới hạn tần suất (Rate Limiting)

Mỗi bộ giới hạn có bộ đếm riêng **theo IP** (không theo đường dẫn), nên kẻ tấn công không lách được bằng cách đổi tham số trên URL. Bảng 5.5 là hạn mức đang áp dụng.

***Bảng 5.5.** Hạn mức giới hạn tần suất đang áp dụng*

API	Hạn mức / IP	Mục đích
Lớp biên Cloudflare: /api/auth/*	10 request / 10 giây (Block)	Chặn dò mật khẩu ngay tại biên, không phụ thuộc server
Mọi API `/api/*`	300 request / 1 phút	Chống spam, DoS mức ứng dụng
Đăng ký, kích hoạt, đăng nhập, Google, xác nhận OTP, đặt lại mật khẩu	30 request / 15 phút	Chống dò mật khẩu, dò OTP
Quên mật khẩu, gửi lại OTP, gửi OTP đổi mật khẩu	6 request / 15 phút	Chống spam email, dò OTP
Tra cứu đơn công khai `GET /don-hang/:id`	30 request / 10 phút	Chống dò mã đơn hàng
Tra cứu bảo hành	30 request / 15 phút	Chống dò dữ liệu khách hàng
Tạo link thanh toán / kiểm tra thanh toán	20 / 15 phút · 60 / 1 phút	Chống lạm dụng cổng PayOS
Kiểm tra voucher	40 request / 10 phút	Chống dò mã giảm giá
Gửi đánh giá / thích đánh giá	8 / 1 giờ · 30 / 1 phút	Chống spam đánh giá
Gửi phiếu liên hệ	5 request / 1 giờ	Chống spam phiếu tư vấn
Chat Trợ lý AI / trạng thái AI	12 / 1 phút · 30 / 1 phút	Bảo vệ hạn mức API key AI
Đăng nhập theo **tài khoản**	5 lần sai liên tiếp → khóa 15 phút	Chống brute-force kể cả khi đổi IP

Ngoài các bộ đếm trong ứng dụng, nhóm bổ sung một lớp giới hạn ở biên mạng bằng luật Rate limiting của Cloudflare cho mọi đường dẫn `/api/auth/*` (10 request / 10 giây / IP, hành động Block). Lớp này chặn request ngay tại Cloudflare trước khi tới server và không mất bộ đếm khi server khởi động lại (cấu hình và kết quả kiểm chứng ở mục 6.7.3, 6.7.5).

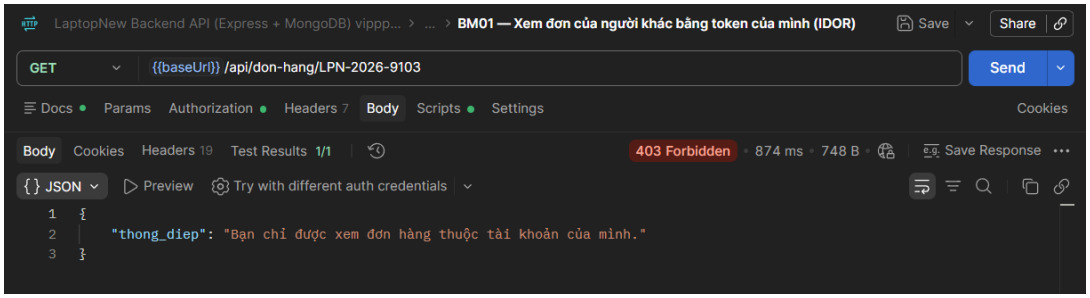
5.7. Kiểm thử bảo mật bằng Postman

Mỗi biện pháp ở trên đều có kịch bản tấn công thử trong thư mục **"16. Kịch bản bảo mật nâng cao"** của collection Postman (chạy theo đúng thứ tự). Ngoài ra 10 ca TC31–TC40 nằm trong bộ Runner tự động (Bảng 6.2).

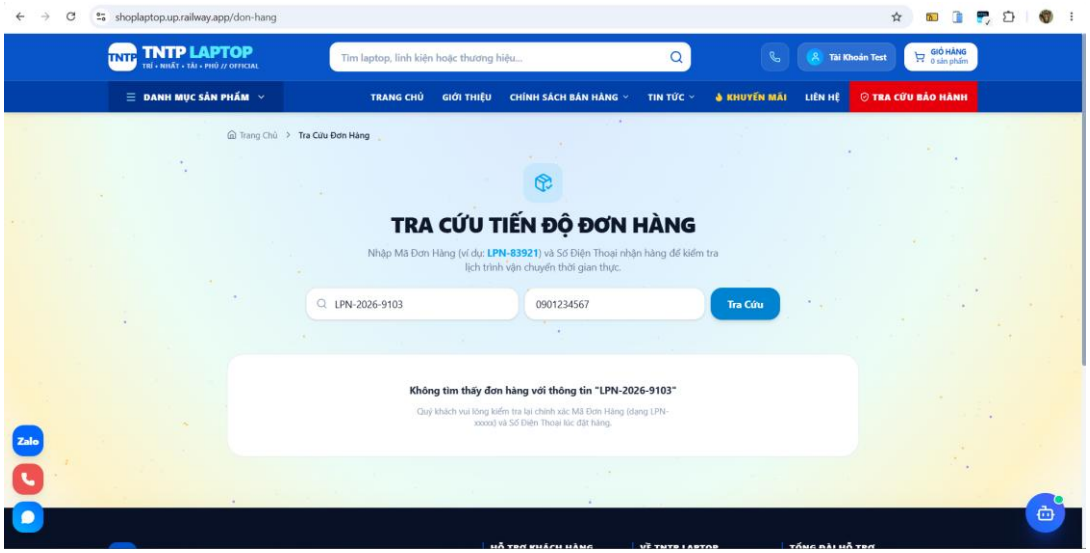
***Bảng 5.6.** Kịch bản kiểm thử bảo mật (thư mục 16 của collection Postman)*

Mã	Kịch bản tấn công thử	Kỳ vọng	Hình
BM01	Dùng token của mình xem đơn của tài khoản khác	403 Forbidden	5.6
BM02	Khách tự gửi `da_thanh_toan: true` cho đơn của mình	403 Forbidden	5.7

Mã	Kịch bản tấn công thử	Kỳ vọng	Hình
BM03	Gửi `vaiTro: "admin"` khi cập nhật hồ sơ	200, `vaiTro` vẫn là `khach_hang`	5.8
BM04	Xem hồ sơ `GET /api/auth/toi`	Không có `matKhau`, `maOtp`, `hanOtp`	5.9
BM05	Nhập sai mật khẩu 6 lần liên tiếp	5 lần 401, lần 6 trả 429 (khóa 15 phút)	5.10
BM06	Nhập sai OTP quên mật khẩu 5 lần	Đếm ngược số lần còn lại, lần 5 mã bị hủy	5.11
BM07	Tra cứu đơn công khai 31 lần / 10 phút	Lần 31 trả 429 + `Retry-After`	5.5
BM08–BM10	Admin khóa tài khoản → dùng token cũ → đăng nhập lại	Token cũ 401/403; đăng nhập 403	5.12
BM11, BM12	Admin mở khóa; Admin thử tự khóa chính mình	200; 400 (không cho tự khóa)	—
BM13–BM15	Đăng nhập → đăng xuất → dùng lại token cũ	Token cũ trả 401 "đã bị thu hồi"	5.13
TC31	NoSQL Injection `{"$ne": null}` vào đăng nhập	400, không cấp token	5.14
TC35	Kiểm tra HTTP security headers	Có `nosniff`, `DENY`, không có `X-Powered-By`	5.15



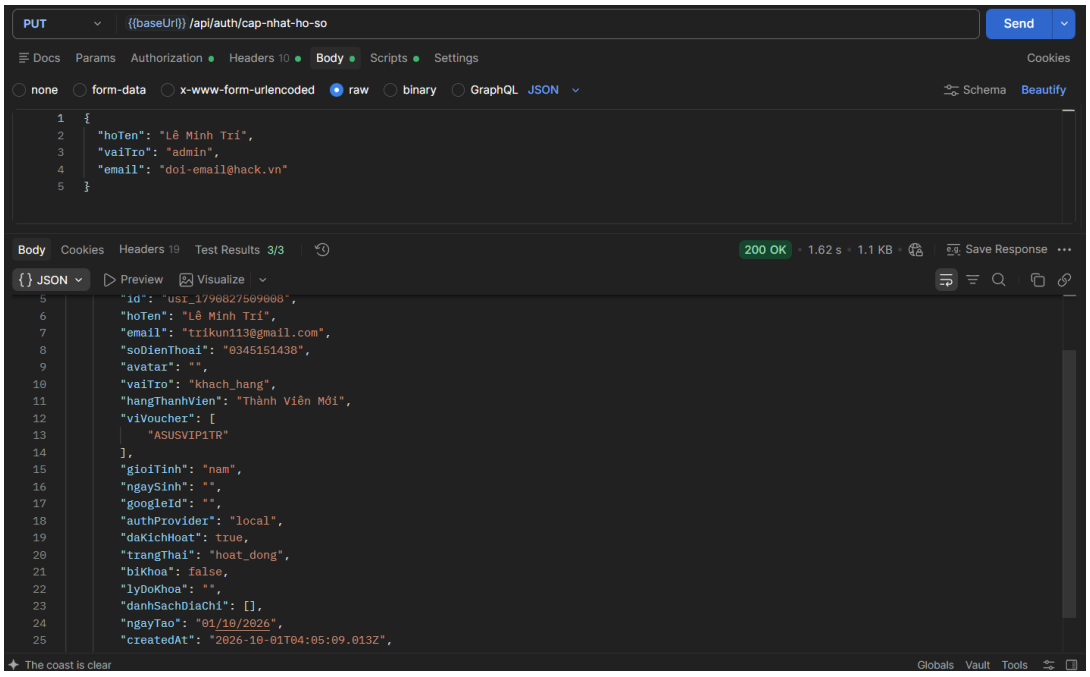
(a) Postman: token của Trí Kun xem đơn LPN-2026-9103 của người khác → 403 Forbidden



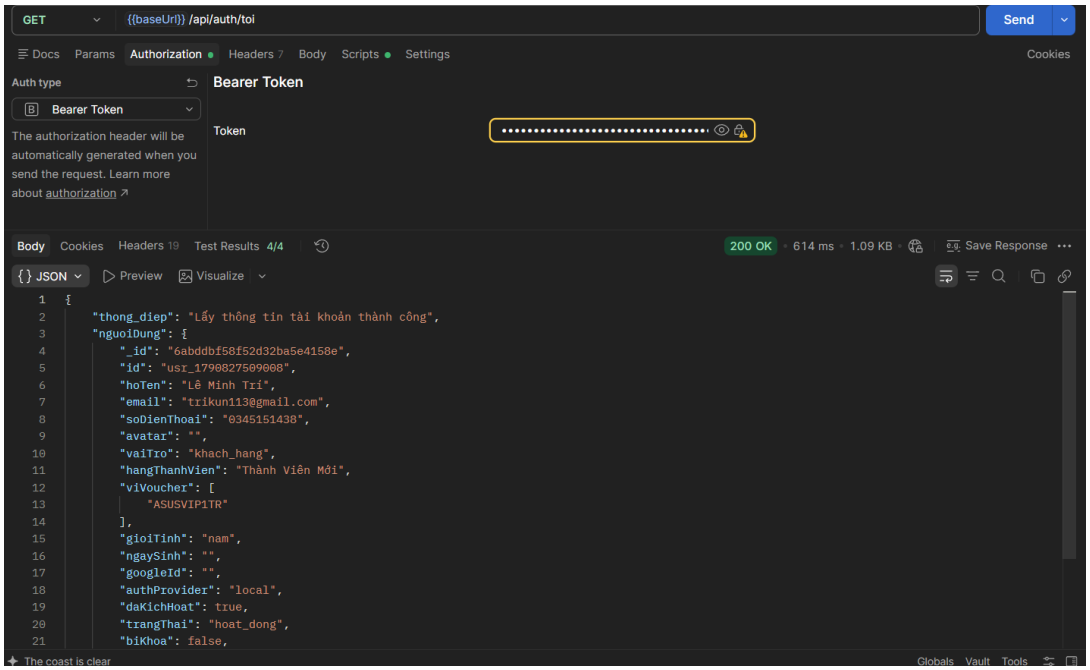
(b) Giao diện web: tra cứu đơn của người khác chỉ hiển thị "Không tìm thấy", không lộ đơn có tồn tại

Hình 5.6. *BM01 — Xem đơn của tài khoản khác bị chặn (403, chống IDOR)*

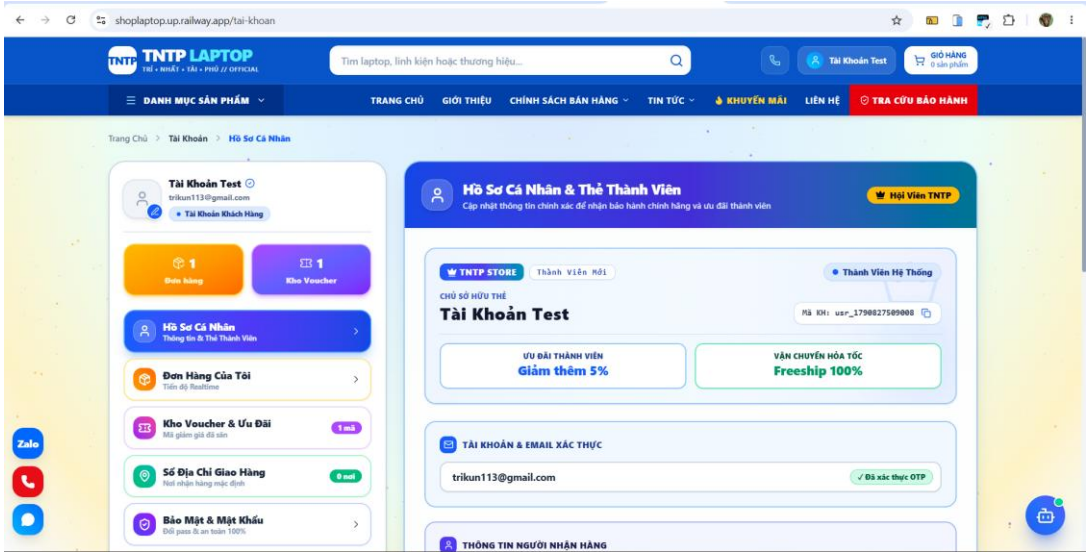
Hình 5.7. *BM02 — Khách tự xác nhận "đã thanh toán" bị từ chối (403)*



Hình 5.8. *BM03 — Gửi vaiTro: admin khi cập nhật hồ sơ nhưng không được nâng quyền*



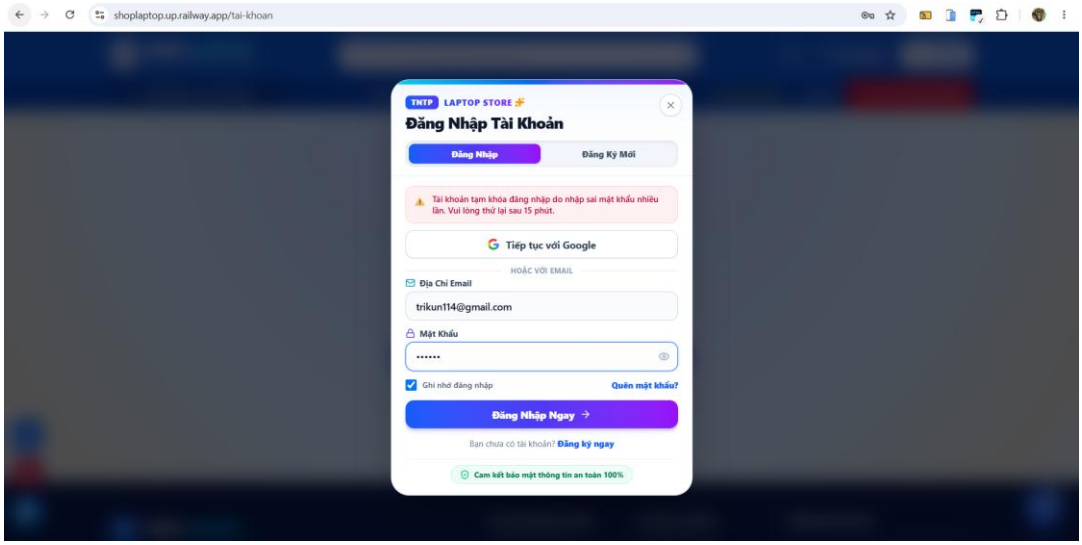
(a) *Postman: GET /api/auth/toi → 200, JSON không có matKhau, maOtp, hanOtp, tokenVersion*



(b) *Giao diện web: trang Hồ sơ cá nhân hiển thị đúng thông tin tài khoản*

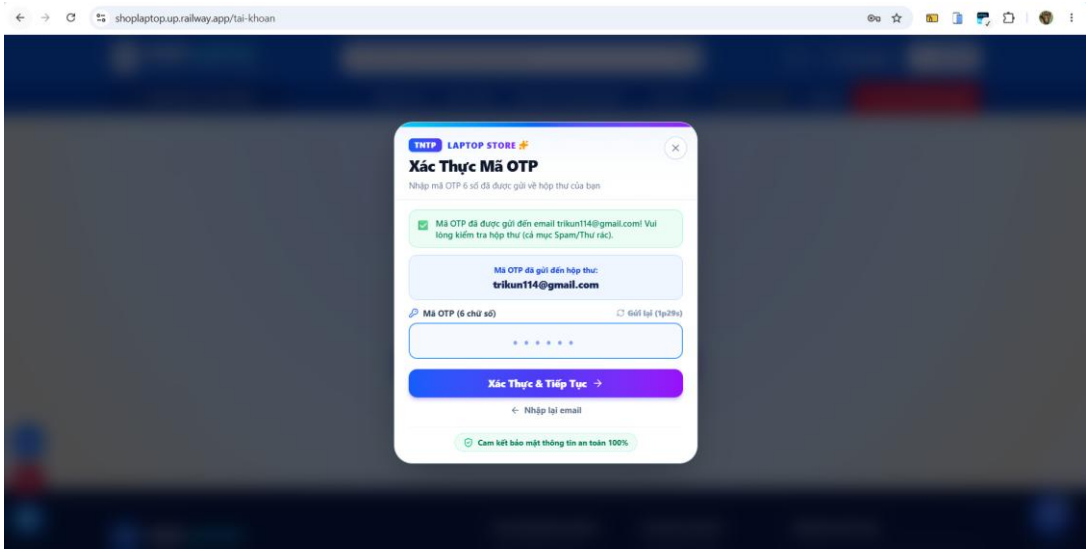
Hình 5.9. *BM04 — Hồ sơ người dùng không còn trả mật khẩu băm và OTP*

(a) *Postman: lần nhập sai mật khẩu thứ 5 → 429 "Tài khoản tạm khóa đăng nhập"*

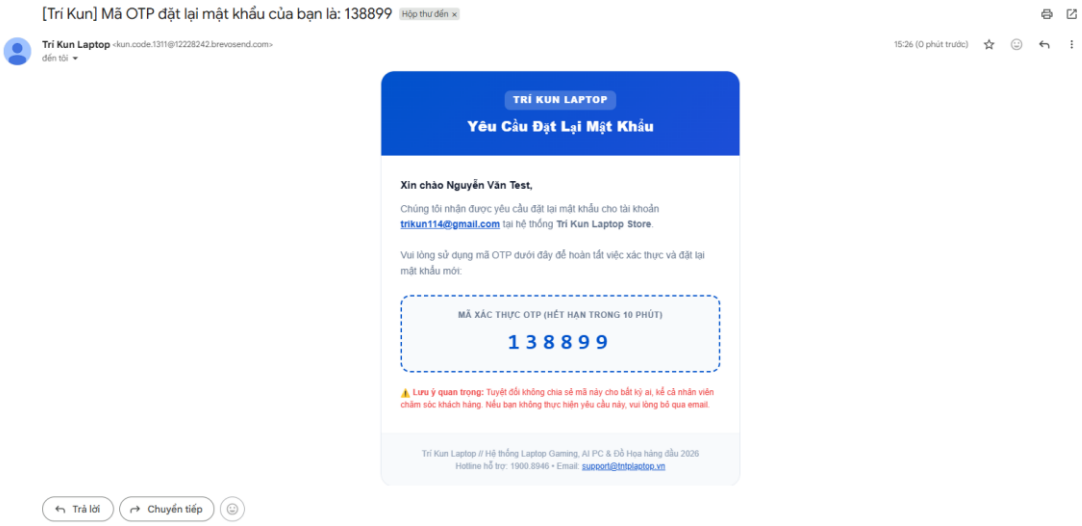


(b) *Giao diện web: form đăng nhập báo tài khoản đang tạm khóa 15 phút*

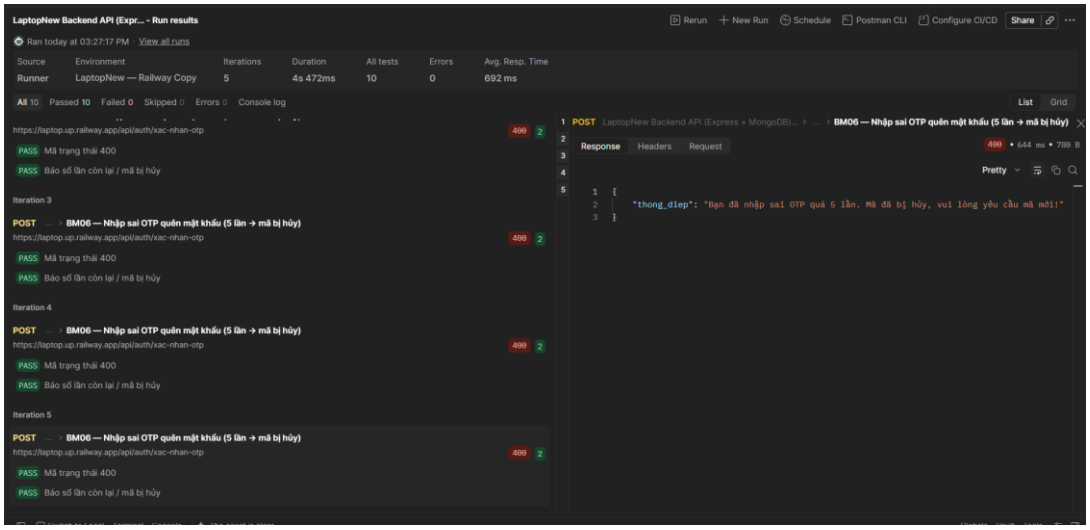
Hình 5.10. *BM05 — Nhập sai mật khẩu 5 lần, tài khoản bị tạm khóa đăng nhập (429)*



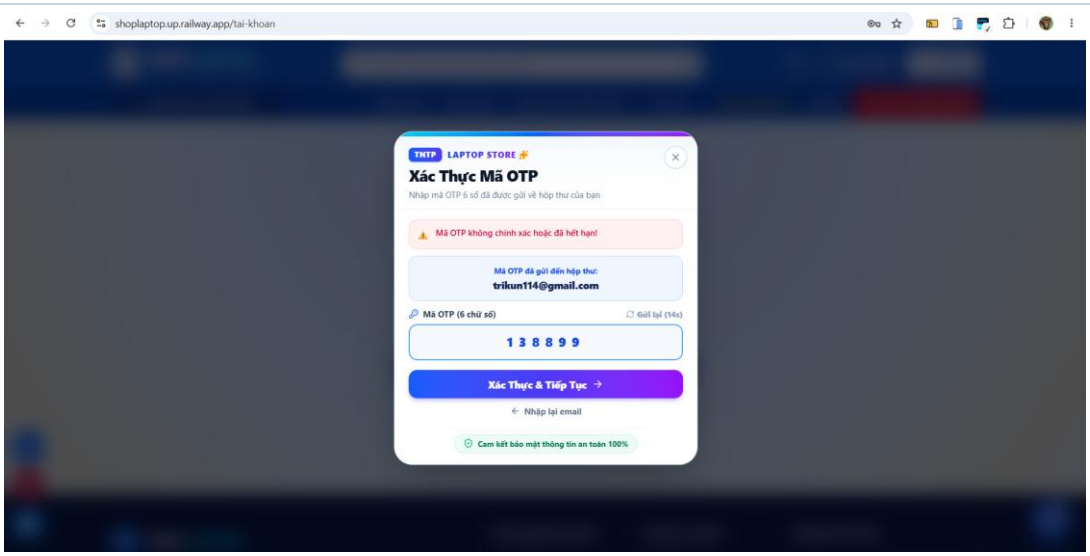
(a) *Giao diện web: bước nhập mã OTP xác thực*



(b) *Email OTP đặt lại mật khẩu gửi tới hộp thư người dùng*

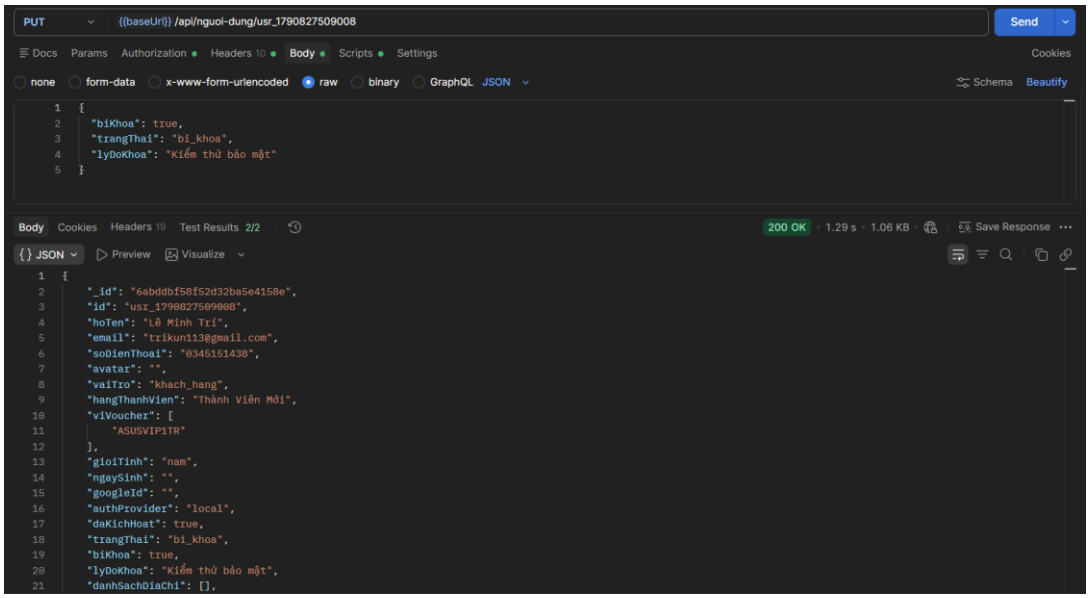


(c) *Postman Runner: 5 lần nhập sai OTP, thông điệp đếm ngược số lần còn lại*

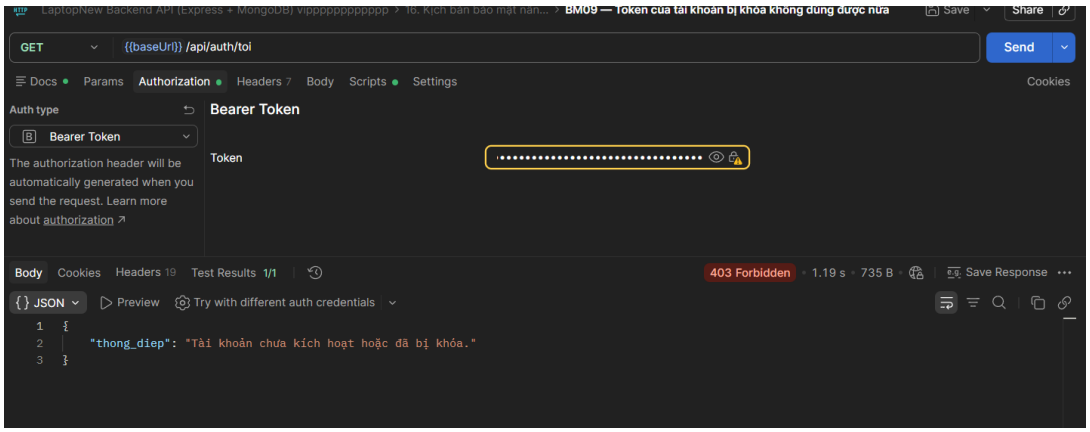


(d) *Giao diện web: báo OTP không chính xác / mã đã bị hủy*

Hình 5.11. *BM06 — Nhập sai OTP: đếm ngược số lần còn lại và hủy mã*

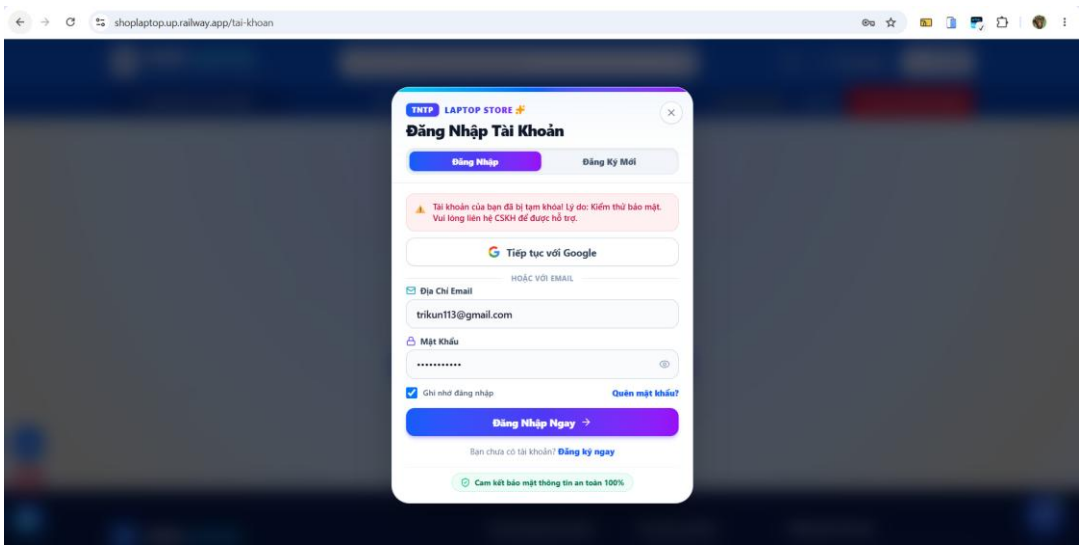


(a) *BM08 — Admin khóa tài khoản: PUT /api/nguoi-dung/:id → 200, biKhoa = true*



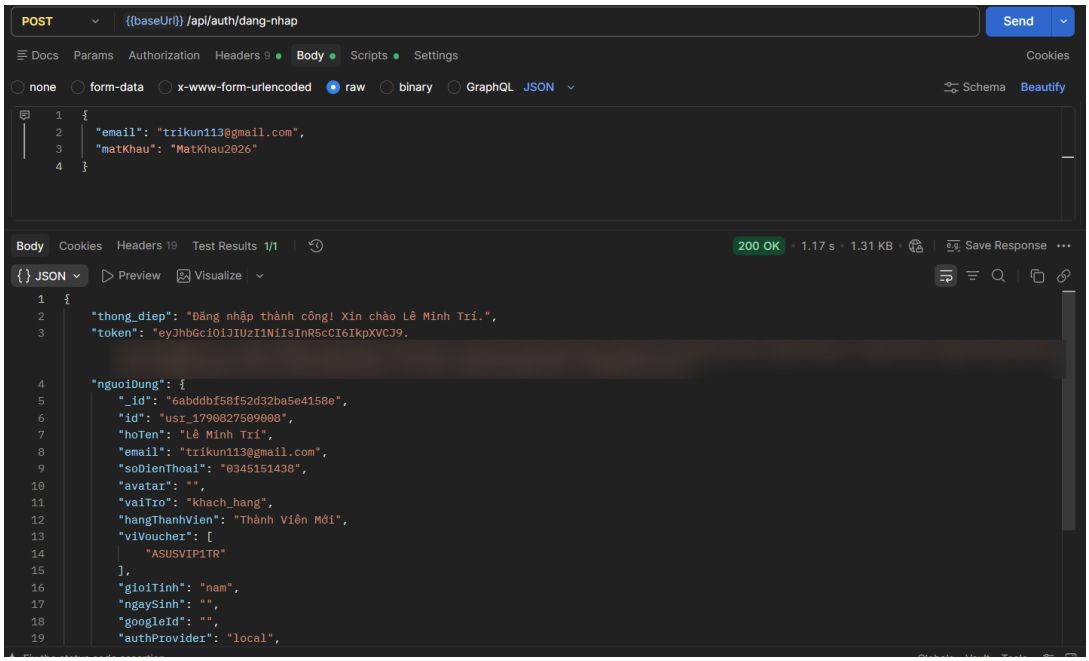
(b) *BM09 — Token cũ của tài khoản bị khóa: GET /api/auth/toi → 403*

(c) *BM10 — Đăng nhập lại tài khoản bị khóa → 403 kèm lý do khóa*

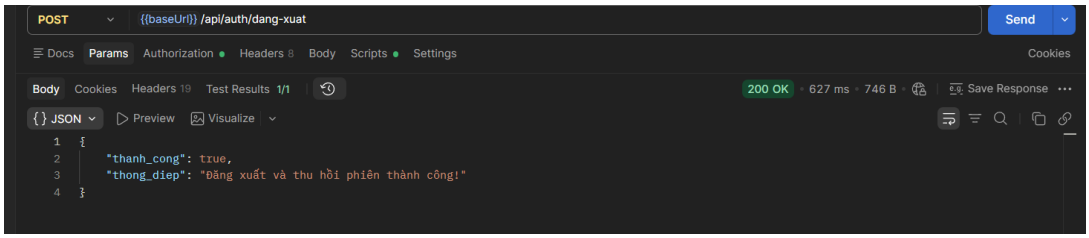


(d) *Giao diện web: form đăng nhập hiển thị thông báo tài khoản bị khóa*

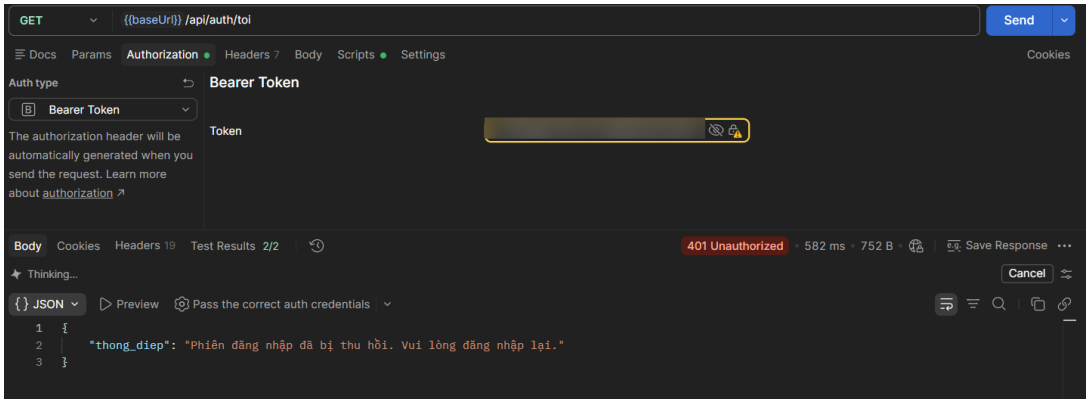
Hình 5.12. *BM08 – BM10 — Admin khóa tài khoản, token cũ và đăng nhập lại đều bị từ chối*



(a) *BM13 — Đăng nhập, nhận token mới (200)*



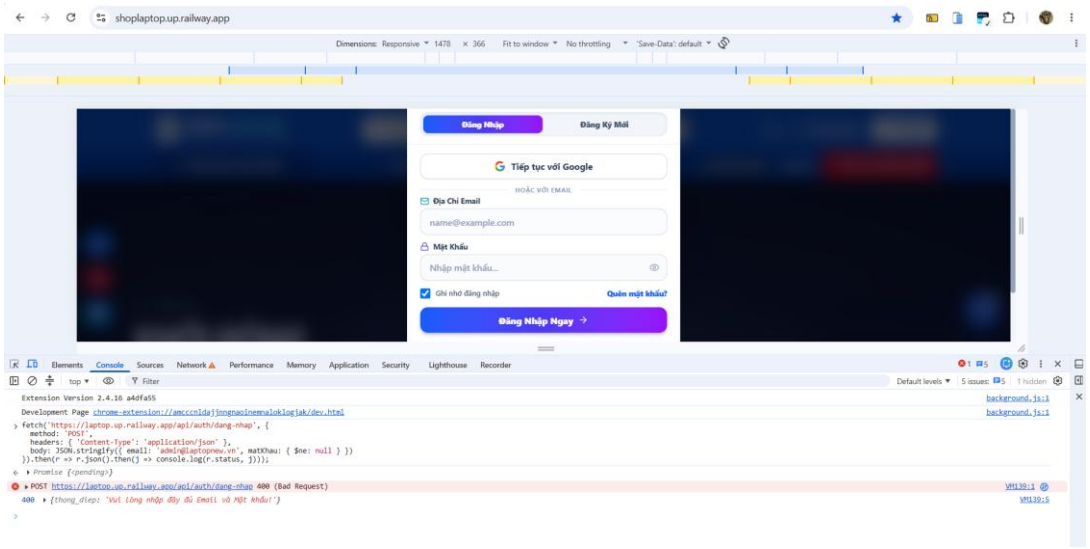
(b) *BM14 — Đăng xuất: server tăng tokenVersion, thu hồi phiên (200)*



(c) *BM15 — Dùng lại token cũ: 401 "Phiên đăng nhập đã bị thu hồi"*

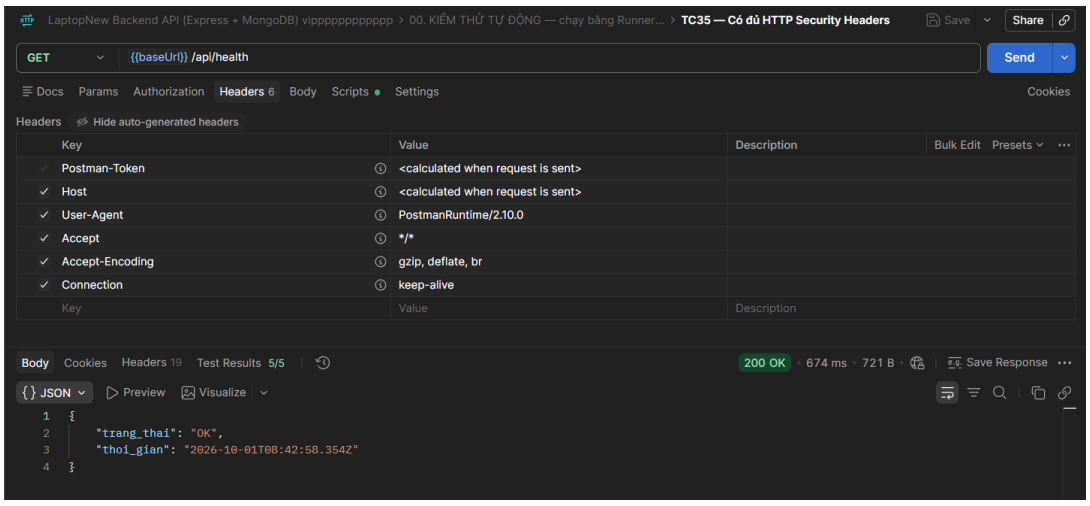
Hình 5.13. *BM15 — Token cũ bị thu hồi sau khi đăng xuất (401)*

(a) *Postman: matKhau = {"$ne": null} → 400, không cấp token*

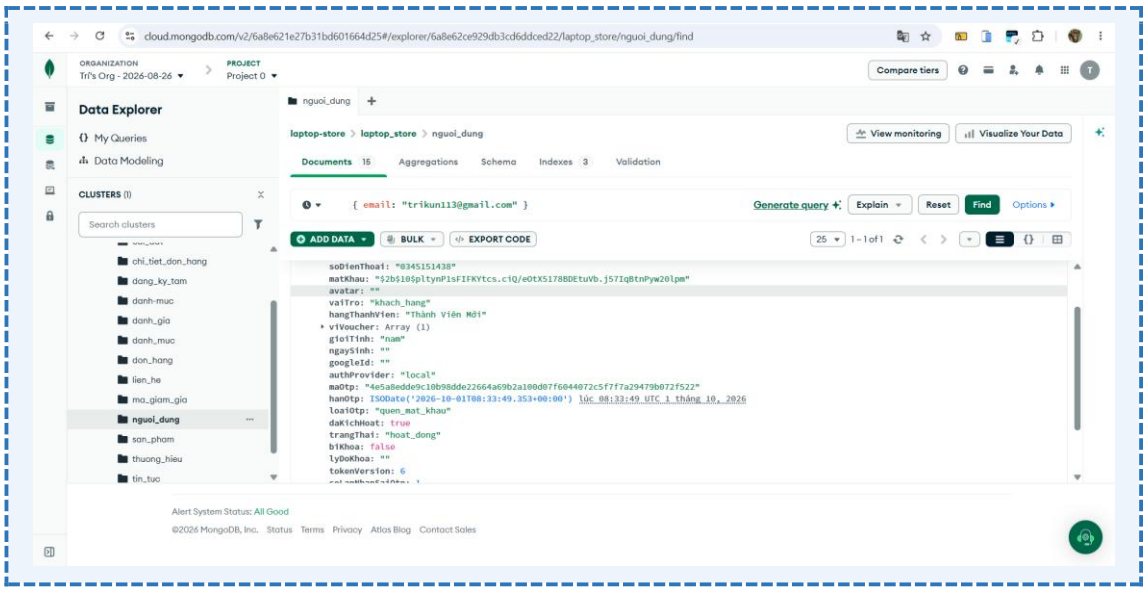


(b) *Trình duyệt (DevTools Console): gửi cùng payload tấn công → 400 Bad Request*

Hình 5.14. *TC31 — Tấn công NoSQL Injection vào API đăng nhập bị chặn*



Hình 5.15. *TC35 — HTTP Security Headers trong phản hồi của server*



Hình 5.16. *Mật khẩu (bcrypt) và OTP (HMAC-SHA256) chỉ được lưu dạng băm trong MongoDB*

5.8. Hạn chế còn lại và hướng hoàn thiện

Nhóm trình bày trung thực các điểm chưa làm được để định hướng giai đoạn sau:

- Bộ đếm giới hạn tần suất trong ứng dụng lưu trong bộ nhớ (Map) nên reset khi server khởi động lại; nhóm đã giảm thiểu bằng lớp rate limit ở biên Cloudflare (mục 6.7.3). Hướng tiếp theo: lưu bộ đếm bằng Redis để chia sẻ khi chạy nhiều server.
- Chưa có **refresh token** (access token ngắn hạn) và **xác thực hai lớp (2FA)** cho tài khoản Admin.
- Chưa ghi **nhật ký kiểm toán** (audit log) cho thao tác quản trị: ai khóa tài khoản, ai sửa giá, lúc nào.
- Chưa tự động quét thư viện có lỗ hổng (`npm audit`, Dependabot) trong quy trình CI.
- Thao tác nguyên tử mới áp dụng ở mức từng document; tạo đơn nhiều sản phẩm chưa dùng **transaction** nhiều document.

CHƯƠNG 6. TRIỂN KHAI VÀ KIỂM THỬ

6.1. Yêu cầu môi trường

- Node.js 18 trở lên (nhóm dùng Node.js 24) và npm.
- Tài khoản MongoDB Atlas, mở Network Access cho IP máy chạy server.
- Tùy chọn: tài khoản PayOS, Telegram Bot (BotFather), Brevo, Google Cloud (OAuth Client ID), Google AI Studio (Gemini API key).
- Môi trường production: Railway (service BE và FE), MongoDB Atlas, tên miền `trikun.site` quản lý DNS bằng Cloudflare (mục 6.7).

6.2. Cấu hình biến môi trường

Tạo tệp `BE/.env` với các biến sau (giá trị thật không đưa vào báo cáo vì là thông tin bí mật):

Bảng 6.1. *Danh sách biến môi trường của Back-End*

Biến	Bắt buộc	Ý nghĩa
`PORT`	Không	Cổng server, mặc định 5000
NODE_ENV	Khi deploy	`production`: ẩn chi tiết lỗi, bật HSTS, tin cậy 1 tầng proxy
`MONGODB_URI`	Có	Chuỗi kết nối MongoDB Atlas
`CLIENT_URL`	Có	Danh sách origin Front-End được phép gọi API (CORS), cách nhau dấu phẩy; origin đầu tiên là trang trả về sau thanh toán PayOS
`JWT_SECRET`, `JWT_EXPIRES_IN`	Có (server dừng nếu thiếu)	Khóa ký JWT (chuỗi ngẫu nhiên dài ≥ 32 ký tự) và thời hạn token (ví dụ 7d)
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`	Khi dùng Google	Xác minh đăng nhập Google
`EMAIL_USER`, `EMAIL_PASS`	Khi chạy local	Gmail và App Password cho Nodemailer
`BREVO_API_KEY`, `BREVO_SENDER_EMAIL`	Khi deploy cloud	Gửi email qua Brevo API
`PAYOS_CLIENT_ID`, `PAYOS_API_KEY`, `PAYOS_CHECKSUM_KEY`	Khi dùng PayOS	Khóa gọi API và ký/xác minh chữ ký

Biến	Bắt buộc	Ý nghĩa
`PAYMENT_BANK_BIN`, `PAYMENT_BANK_ACCOUNT_NO`, `PAYMENT_BANK_ACCOUNT_NAME`	Khi dùng VietQR	Tài khoản nhận tiền để dựng ảnh QR
`TELEGRAM_BOT_TOKEN`, `TELEGRAM_CHAT_ID`	Khi dùng Bot	Token bot và chat ID của quản trị viên
`GEMINI_API_KEYS`	Khi dùng AI	Nhiều key Gemini cách nhau dấu phẩy để xoay vòng
`NINE_ROUTER_API_KEY`, `NINE_ROUTER_BASE_URL`, `NINE_ROUTER_MODEL`	Tùy chọn	Cấu hình cổng AI 9Router

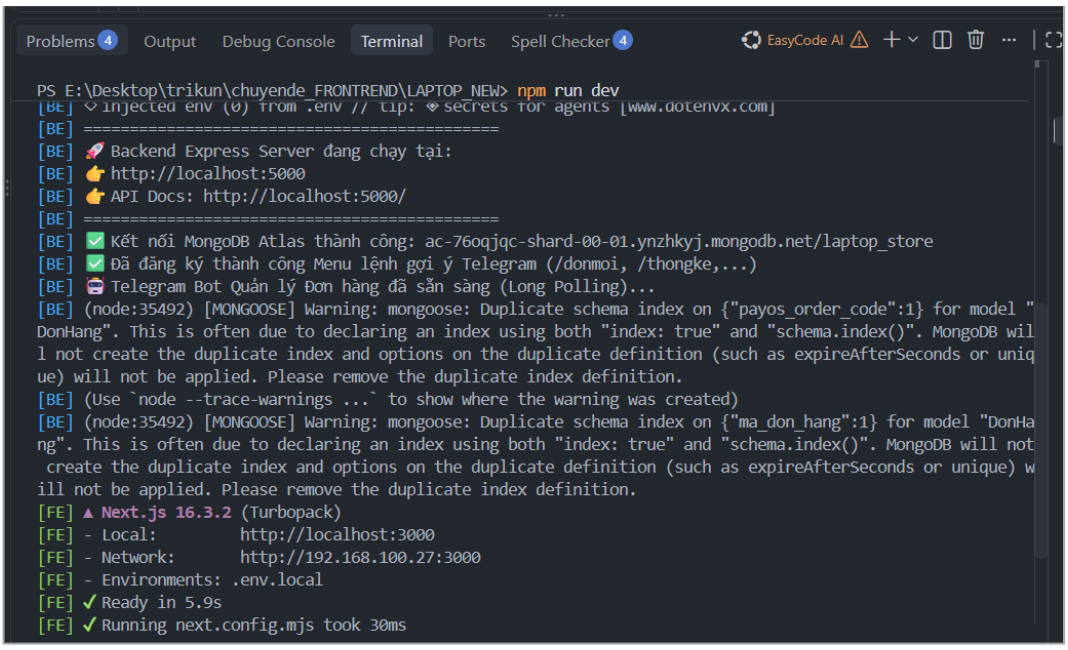
6.3. Cài đặt và khởi chạy

Các lệnh cài đặt và chạy dự án

```
# 1. Cài thư viện cho Back-End
cd LAPTOP_NEW/BE
npm install

# 2. Chạy ở chế độ phát triển (tự khởi động lại khi sửa code)
npm run dev          # nodemon server.js
# hoặc chạy thường
npm start            # node server.js
npm test             # 103 ca kiểm thử tự động bằng Jest (mục 6.8)
npm run lint         # ESLint: phát hiện lỗi tiềm ẩn trong mã nguồn
npm run format       # Prettier: tự định dạng mã nguồn

# 3. (Tùy chọn) Chạy Front-End để thử giao diện
cd ../FE
npm install
npm run dev          # http://localhost:3000

# Hoặc chạy đồng thời cả hai bằng tệp chay.bat ở thư mục gốc (Windows)
```

Khi khởi chạy thành công, cửa sổ terminal hiển thị địa chỉ server và thông báo kết nối MongoDB Atlas:

```
==============================================
🚀 Backend Express Server đang chạy tại:
👉 http://localhost:5000
👉 API Docs: http://localhost:5000/
==============================================
☑ Kết nối MongoDB Atlas thành công: ac-76oqjqc-shard-00-
00.ynzhkyj.mongodb.net/laptop_store
```



Hình 6.1. *Terminal khởi chạy Back-End ở máy phát triển (npm run dev)*

6.4. Kiểm thử API

Nhóm kiểm thử API theo phương pháp **hộp đen**: gửi request với dữ liệu hợp lệ và không hợp lệ, so sánh mã HTTP và nội dung phản hồi với kết quả mong đợi. Bộ kiểm thử tự động gồm **40 ca (TC01–TC40) với 58 bài kiểm tra (assertion)** trong thư mục 00 của collection, chạy bằng Postman Runner trên server Railway kết nối MongoDB Atlas. Ca TC01–TC26 xây dựng ngày 29/09/2026, TC27–TC30 ngày 30/09/2026, **TC31–TC40 (kiểm thử bảo mật bổ sung sau rà soát)** ngày 01/10/2026. Lần chạy cuối cùng ngày 01/10/2026: **58/58 bài kiểm tra đạt, 0 lỗi**. Bộ Runner chỉ dùng thao tác đọc và thao tác bị từ chối để không làm thay đổi dữ liệu thật.

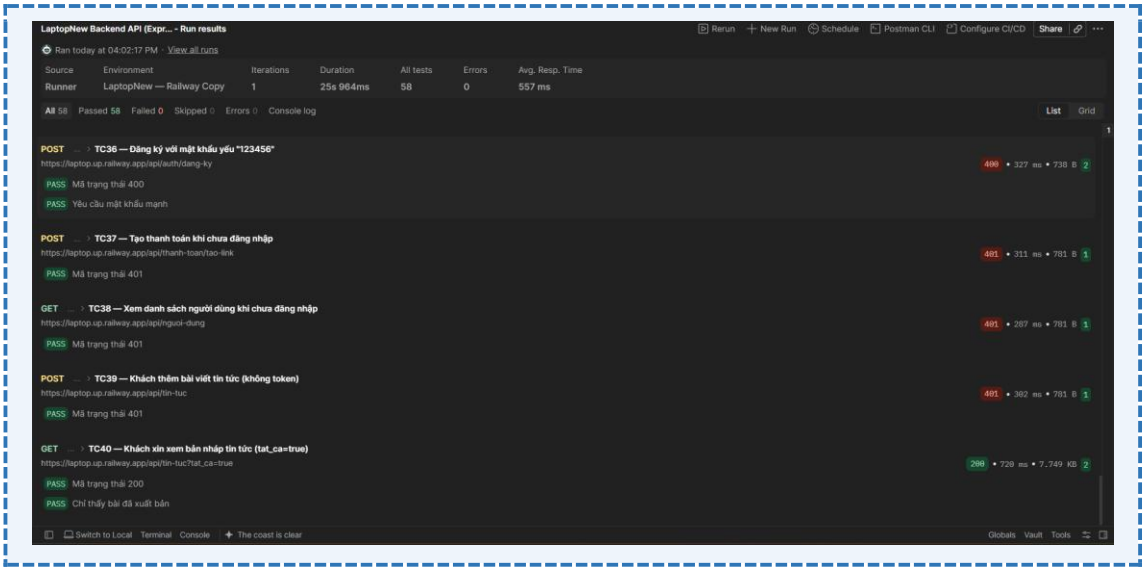
Bảng 6.2. *Kết quả kiểm thử API thực tế*

Mã	API	Dữ liệu vào	Mong đợi	Thực tế	KQ
Giám sát và sản phẩm					
TC01	GET /api/health	—	200, trang_thai OK	200 `{"trang_thai":"OK"}`	✓ Đạt
TC02	GET /	—	200, thông tin server	200, phien_ban 1.1.0	✓ Đạt
TC03	GET /api/san-pham	`limit=2&sap_xep=gia_tang_dan`	200, 2 sản phẩm giá tăng dần	200, 2 phần tử, rẻ nhất trước	✓ Đạt
TC04	GET /api/san-pham	`hang_san_xuat=asus&gia_max=30000000`	200, laptop ASUS ≤ 30 triệu	200, ASUS Zenbook 14 OLED 29.990.000đ	✓ Đạt
TC05	GET /api/san-pham/:id	`khong-ton-tai-xyz`	404	404 “Không tìm thấy sản phẩm...”	✓ Đạt
TC06	GET /api/khong-co-route	—	404 route	404 “Đường dẫn không tồn tại...”	✓ Đạt

Mã	API	Dữ liệu vào	Mong đợi	Thực tế	KQ
Xác thực và phân quyền					
TC07	GET /api/auth/toi	Không có token	401	401 “Thiếu Token xác thực”	✓ Đạt
TC08	GET /api/auth/toi	Token giả `abc.def.ghi`	401	401 “Token... không hợp lệ”	✓ Đạt
TC09	POST /api/san-pham	Không có token	401 (route admin)	401	✓ Đạt
TC10	POST /api/auth/dang-nhap	Email không tồn tại	401	401 “Tài khoản hoặc Mật khẩu không chính xác!”	✓ Đạt
TC11	POST /api/auth/dang-nhap	`email` rỗng	400	400 “Vui lòng nhập đầy đủ...”	✓ Đạt
TC12	POST /api/auth/dang-ky	Thiếu SĐT, mật khẩu	400	400 “Vui lòng cung cấp đầy đủ...”	✓ Đạt
Nghiệp vụ					
TC13	POST /api/ma-giam-gia/kiem-tra	Mã `KHONGCO999`	404, hop_le false	404, so_tien_giam 0	✓ Đạt
TC14	POST /api/ma-giam-gia/kiem-tra	Mã rỗng	400	400 “Vui lòng nhập mã giảm giá.”	✓ Đạt
TC15	GET /api/ma-giam-gia	`kich_hoat=true`	200, danh sách voucher	200, 14 voucher	✓ Đạt
TC16	GET /api/bao-hanh/tra-cuu	`tu_khoa=ab`	Từ chối từ khóa < 3 ký tự	200, hop_le false, thông báo quá ngắn	✓ Đạt
TC17	GET /api/bao-hanh/tra-cuu	Mã đơn không tồn tại	Không tìm thấy	200, hop_le false	✓ Đạt
TC18	GET /api/don-hang	Không gửi token	401, yêu cầu đăng nhập	401 “Vui lòng đăng nhập...”	✓ Đạt
TC19	POST /api/don-hang	Chưa đăng nhập	401, yêu cầu đăng nhập	401, yeu_cau_dang_nhap true	✓ Đạt
TC20	GET /api/danh-muc	—	200	200, 21 danh mục	✓ Đạt
TC21	GET /api/tin-tuc	—	200	200, thanhCong true	✓ Đạt
TC22	GET /api/cai-dat/showroom	—	200	200, 3 showroom	✓ Đạt
Thanh toán và AI					
TC23	POST /api/thanh-toan/webhook	Chữ ký sai	400	400 “Chữ ký không hợp lệ”	✓ Đạt
TC24	GET /api/thanh-toan/kiem-tra/123	orderCode chưa thanh toán	200, chưa thanh toán	200, da_thanh_toan false, PENDING	✓ Đạt
TC25	POST /api/tro-ly-ai/chat/stream	`tin_nhan` rỗng	400	400 “Vui lòng cung cấp nội dung...”	✓ Đạt
TC26	GET /api/tro-ly-ai/trang-thai	—	200, trạng thái AI	200, 9Router đã cấu hình, 5 key Gemini	✓ Đạt
Bảo mật tra cứu đơn hàng					

Mã	API	Dữ liệu vào	Mong đợi	Thực tế	KQ
TC27	GET /api/don-hang/:ma	Khách, không gửi `sdt`	403, yêu cầu xác minh SĐT	403, `yeu_cau_xac_minh_sdt` true	✓ Đạt
TC28	GET /api/don-hang/:ma	Khách, `sdt` sai	403	403 “Vui lòng nhập đúng mã đơn hàng và số điện thoại...”	✓ Đạt
TC29	GET /api/don-hang/:ma	Khách, `sdt` đúng	200, dữ liệu cá nhân đã che	200, họ tên `L* M*** T**`, SĐT `034****438`	✓ Đạt
TC30	GET /api/don-hang/:ma	Token của chủ đơn	200, đầy đủ thông tin	200, đầy đủ `thong_tin_giao_hang`	✓ Đạt
Kiểm thử bảo mật (bổ sung sau rà soát — 01/10/2026)					
TC31	POST /api/auth/dang-nhap	matKhau = {"$ne": null}	400, không cấp token	400 “Vui lòng nhập đầy đủ...”	✓ Đạt
TC32	GET /api/san-pham	tu_khoa=(a+)+$	200, không treo server	200 [] trong 0,07 giây	✓ Đạt
TC33	POST /api/auth/dang-nhap	JSON hỏng	400	400 “Dữ liệu JSON... không hợp lệ”	✓ Đạt
TC34	GET /api/health	Origin: website lạ	403 (CORS)	403 “Origin không được phép...”	✓ Đạt
TC35	GET /api/health	—	Có security headers	nosniff, DENY, RateLimit; không có X-Powered-By	✓ Đạt
TC36	POST /api/auth/dang-ky	matKhau = 123456	400, mật khẩu yếu	400 “...từ 8 đến 72 ký tự”	✓ Đạt
TC37	POST /api/thanh-toan/tao-link	Không có token	401	401 “Thiếu Token xác thực”	✓ Đạt
TC38	GET /api/nguoi-dung	Không có token	401	401	✓ Đạt
TC39	POST /api/tin-tuc	Không có token	401	401 “Thiếu Token xác thực”	✓ Đạt
TC40	GET /api/tin-tuc	tat_ca=true (khách)	Chỉ bài đã xuất bản	200, 10 bài, 0 bản nháp	✓ Đạt

Kết quả: 40/40 ca (58/58 bài kiểm tra) đạt. Các chức năng có ghi dữ liệu (đăng ký thật, đặt hàng, thanh toán, duyệt đơn Telegram) được kiểm thử thủ công qua giao diện và Postman (Chương 4, Mục 6.5); các kịch bản tấn công thử có ghi dữ liệu (khóa tài khoản, đăng xuất...) trình bày ở Mục 5.7; sự thay đổi dữ liệu trong MongoDB được minh chứng ở Mục 6.6.



Hình 6.2. *Postman Runner chạy 40 ca kiểm thử tự động (58 bài kiểm tra): 58/58 đạt, 0 lỗi*

6.5. Minh chứng kiểm thử API bằng Postman

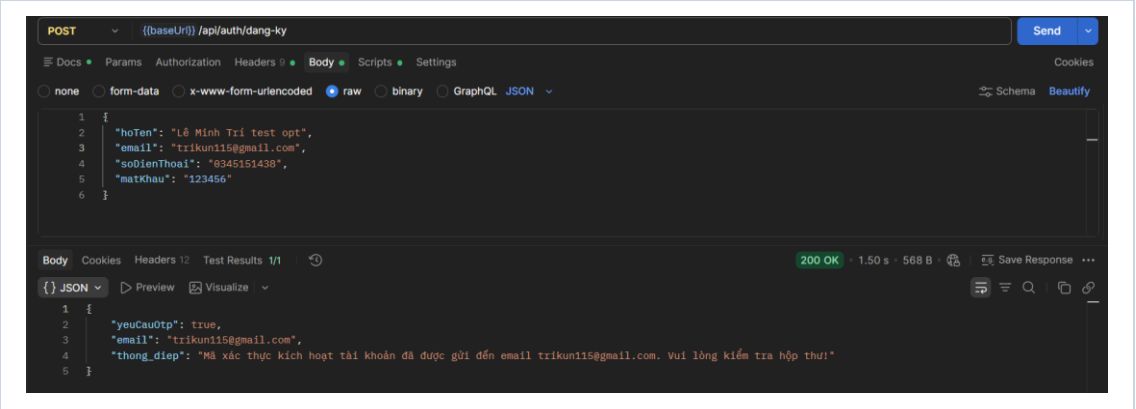
Toàn bộ API được gom trong collection **LaptopNew Backend API** (tệp `_tai_lieu_bao_cao/postman/`), gồm **17 thư mục, 141 request**: thư mục 00 là bộ Runner tự động 40 ca, thư mục 01–14 theo nhóm chức năng, 15 là kịch bản lỗi, 16 là kịch bản bảo mật nâng cao. Các request dùng chung biến môi trường `baseUrl`, `token`, `adminToken`, `productId`, `maDonHang`, `orderCode`...; request đăng nhập tự lưu token nên các request sau chỉ cần bấm Send. Bảng dưới đây tổng hợp minh chứng Postman cho từng nhóm API.

Bảng 6.3. *Tổng hợp minh chứng kiểm thử Postman theo nhóm API*

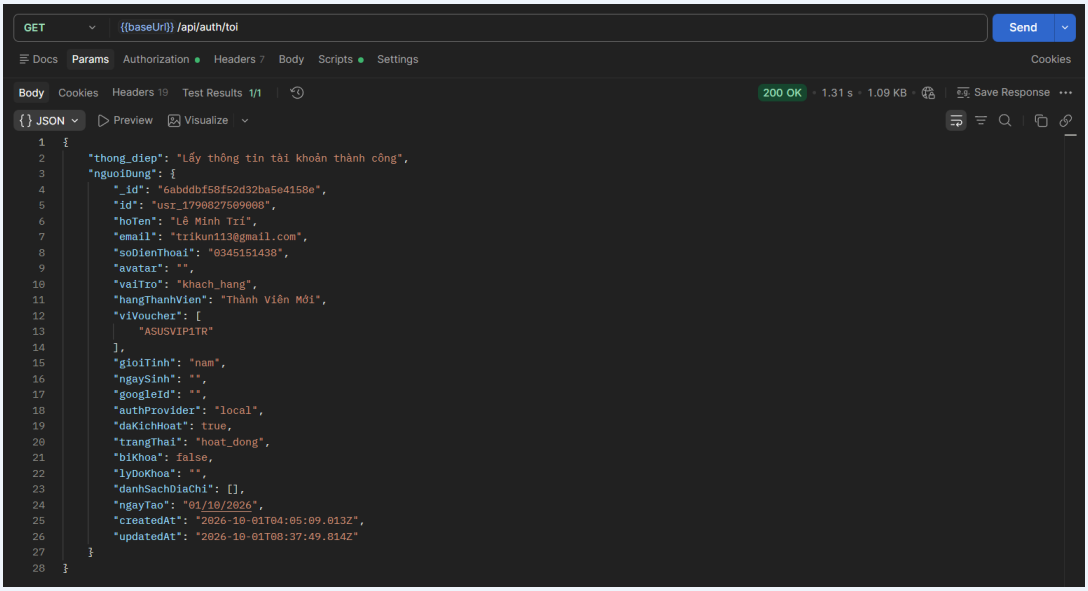
STT	Nhóm API	Request tiêu biểu	Hình minh chứng
1	Xác thực & tài khoản	Đăng ký, kích hoạt OTP, đăng nhập, xác nhận OTP, xem hồ sơ, đổi mật khẩu	Hình 4.7, 4.4, 4.15, 6.3, 6.4, 6.5
2	Sản phẩm & danh mục	Danh sách, lọc nhiều điều kiện, thêm sản phẩm (Admin), lọc thương hiệu	Hình 3.1, 6.6, 6.7, 6.8
3	Đơn hàng	Tạo đơn, tra cứu (chủ đơn / khách có SĐT / thiếu SĐT), cập nhật trạng thái	Hình 6.9, 4.25, 4.24, 4.26, 6.10
4	Thanh toán	Tạo QR, kiểm tra trạng thái, Webhook sai chữ ký	Hình 6.11, 6.12, 6.13
5	Mã giảm giá	Kiểm tra mã, lưu vào ví, xem ví	Hình 4.21, 6.14
6	Đánh giá, bảo hành, liên hệ	Gửi đánh giá, tra cứu bảo hành, gửi phiếu tư vấn	Hình 6.15, 6.16, 6.17
7	Tin tức, AI, showroom	Danh sách tin, chat AI, trạng thái AI, showroom	Hình 4.45, 4.40, 6.18, 6.19
8	Giám sát máy chủ	Thông tin server, health check	Hình 4.52, 6.20

STT	Nhóm API	Request tiêu biểu	Hình minh chứng
9	Kịch bản lỗi	400, 401, 403, 404, 429	Hình 5.1, 5.2, 5.3, 5.4, 5.5
10	Chạy tự động	Runner 40 ca (TC01–TC40), không ghi dữ liệu	Hình 6.2
11	Kịch bản bảo mật	IDOR, Mass Assignment, khóa đăng nhập, dò OTP, thu hồi phiên, NoSQL Injection, headers	Hình 5.5 – 5.16
12	Dữ liệu MongoDB	Trước / sau khi đăng ký, đặt hàng, giao hàng, hủy đơn, lưu voucher, đánh giá, khóa tài khoản	Hình 6.21 – 6.30

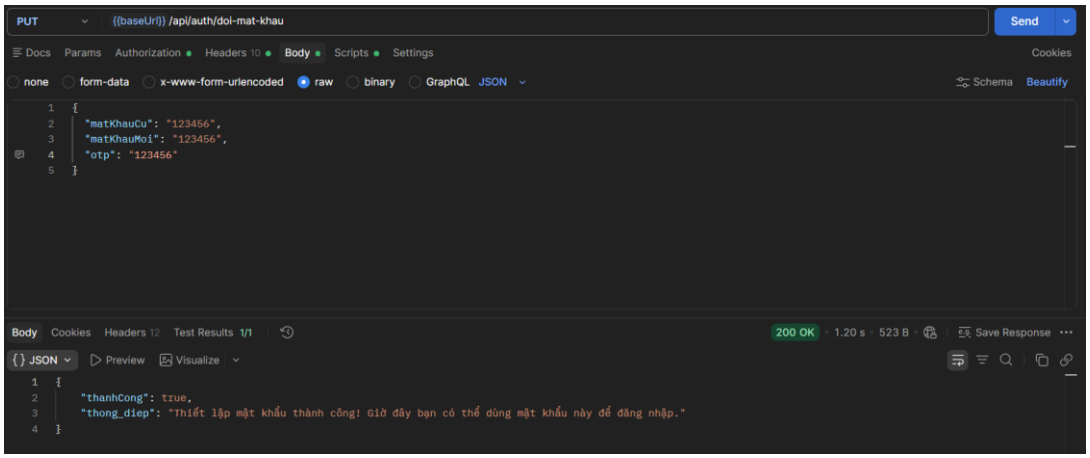
6.5.1. Nhóm Xác thực và Tài khoản



Hình 6.3. *Kiểm thử POST /api/auth/dang-ky bằng Postman: đăng ký hợp lệ, gửi OTP*

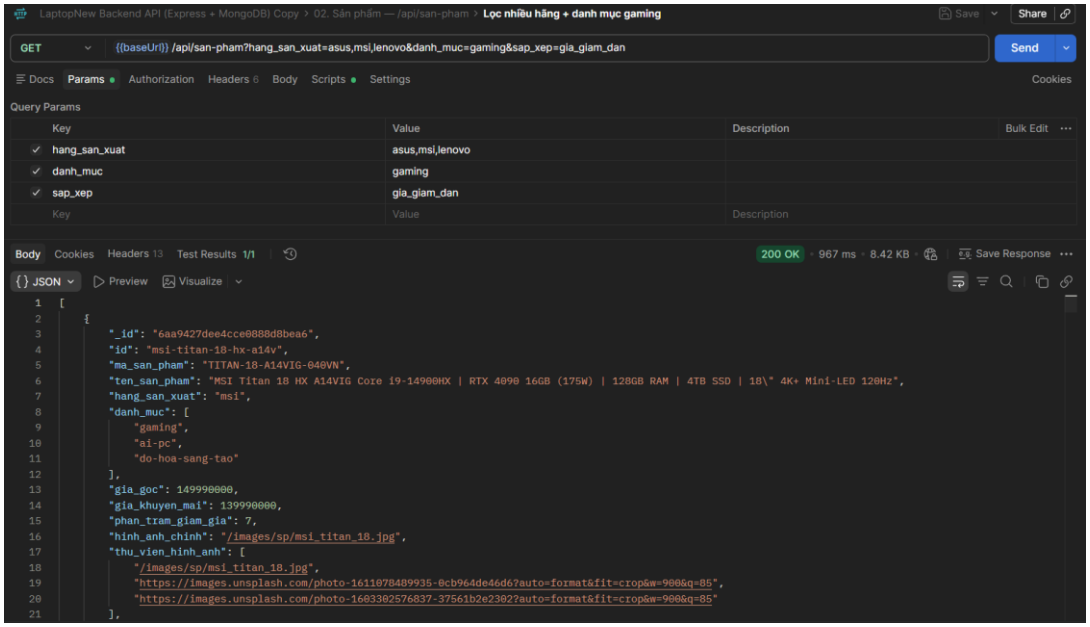


Hình 6.4. *Kiểm thử GET /api/auth/toi với token hợp lệ: hồ sơ không chứa mật khẩu băm và OTP*

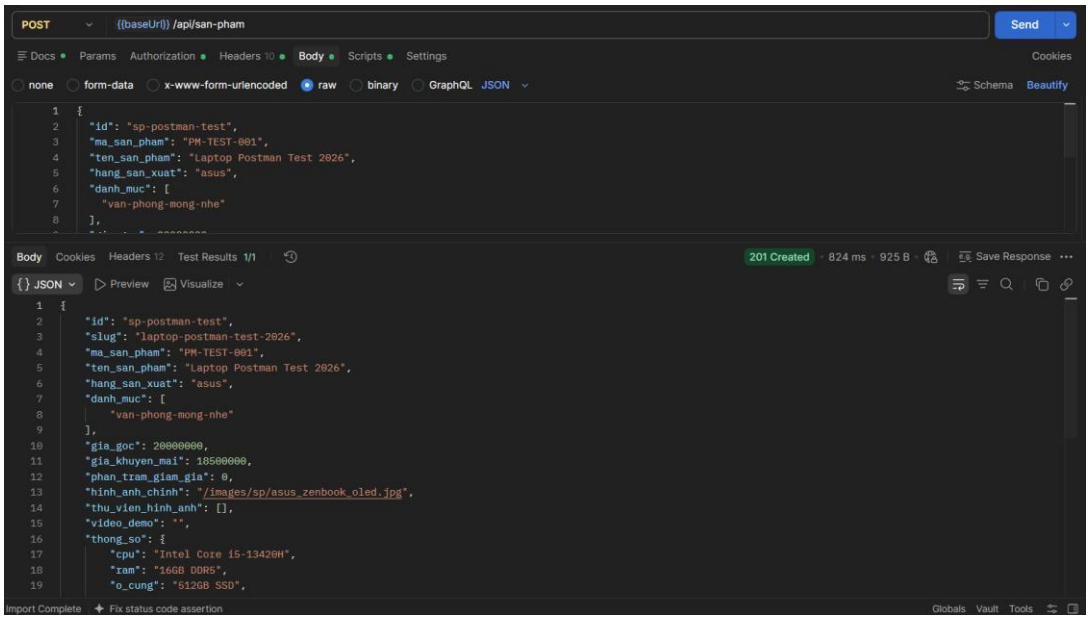


Hình 6.5. *Kiểm thử PUT /api/auth/doi-mat-khau bằng Postman*

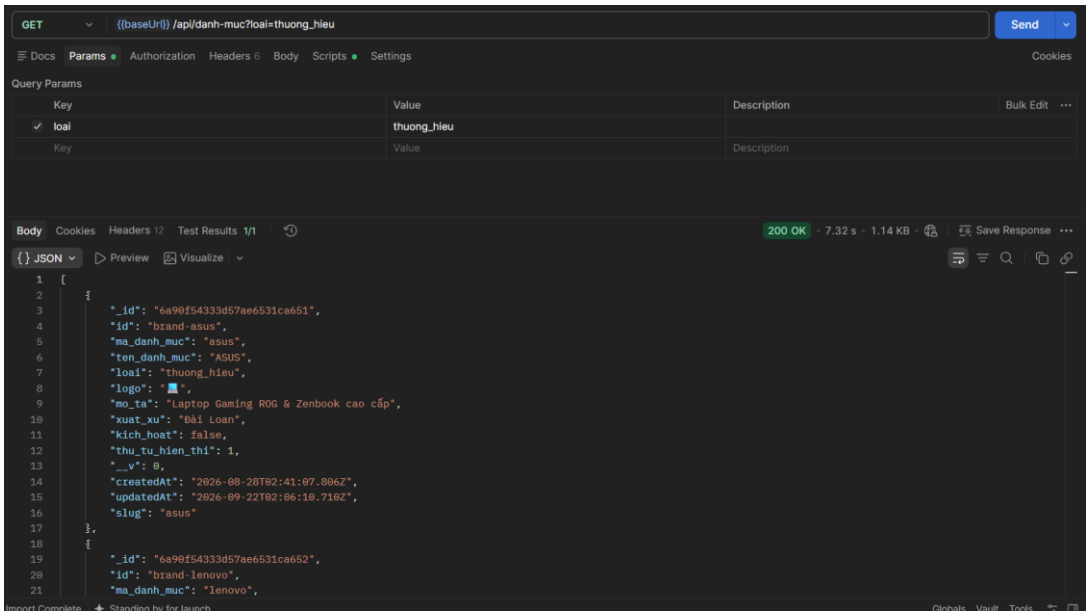
6.5.2. Nhóm Sản phẩm và Danh mục



Hình 6.6. *Kiểm thử lọc sản phẩm theo nhiều hãng và danh mục*

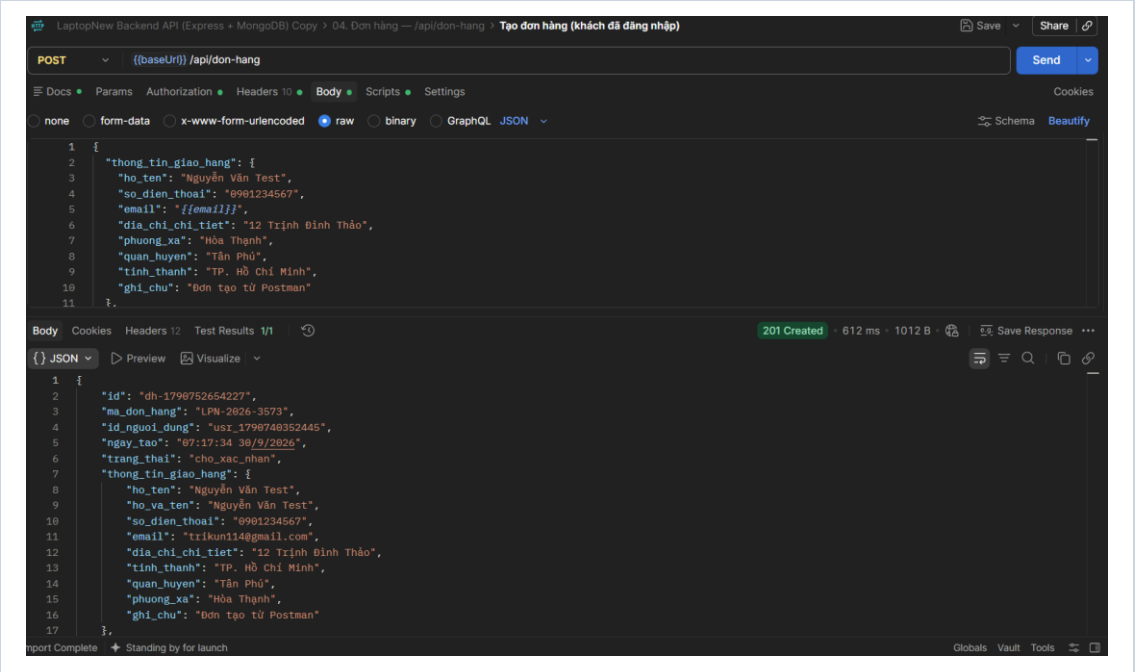


Hình 6.7. *Kiểm thử POST /api/san-pham bằng tài khoản Admin (201 Created)*

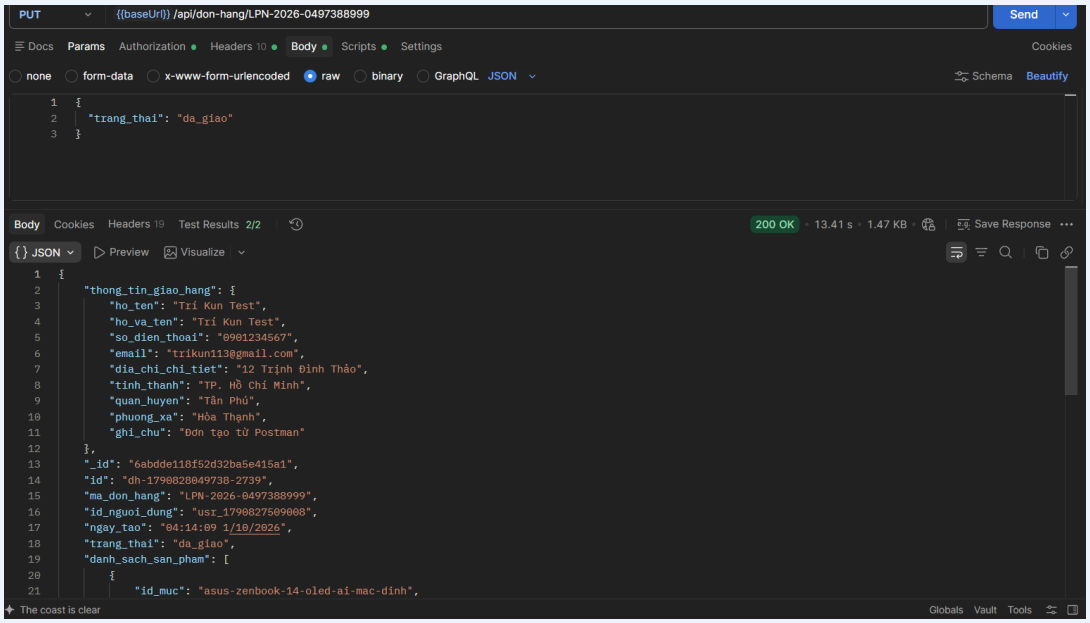


Hình 6.8. *Kiểm thử GET /api/danh-muc?loai=thuong_hieu*

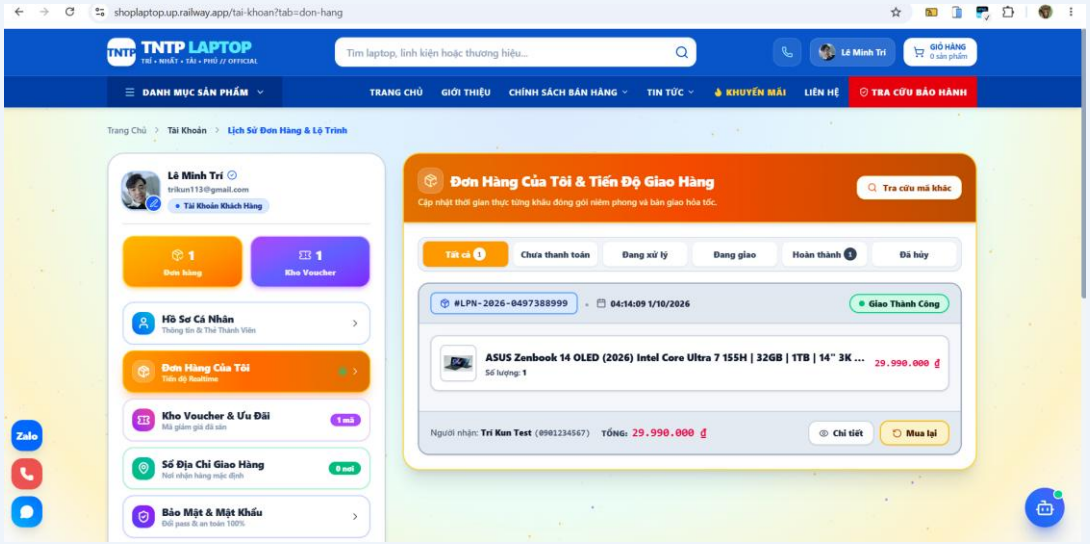
6.5.3. Nhóm Đơn hàng và Thanh toán



Hình 6.9. *Kiểm thử POST /api/don-hang: tạo đơn thành công (201 Created)*

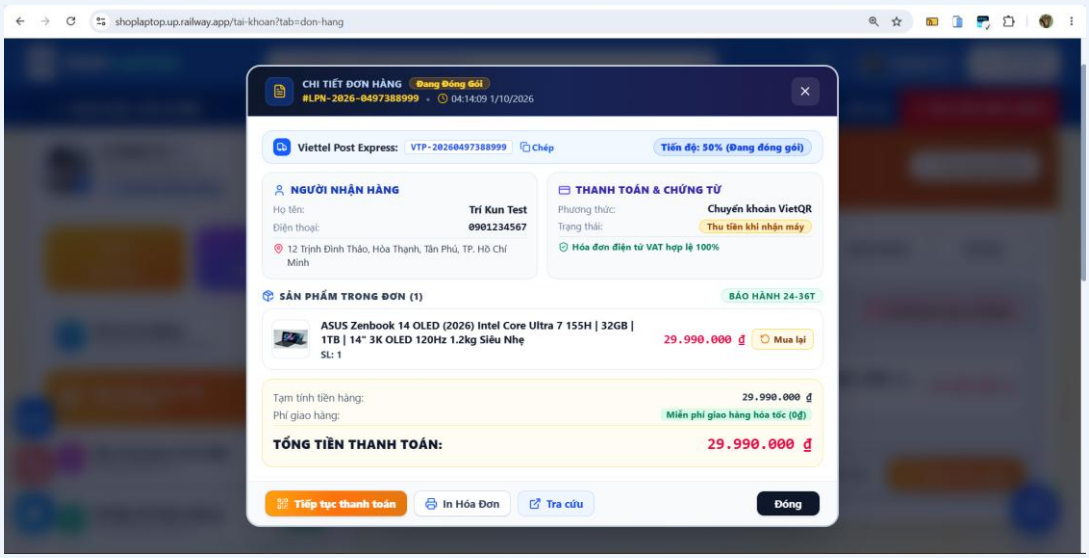


(a) *Postman: Admin chuyển đơn sang "da_giao" → 200, đồng bộ trạng thái thanh toán*

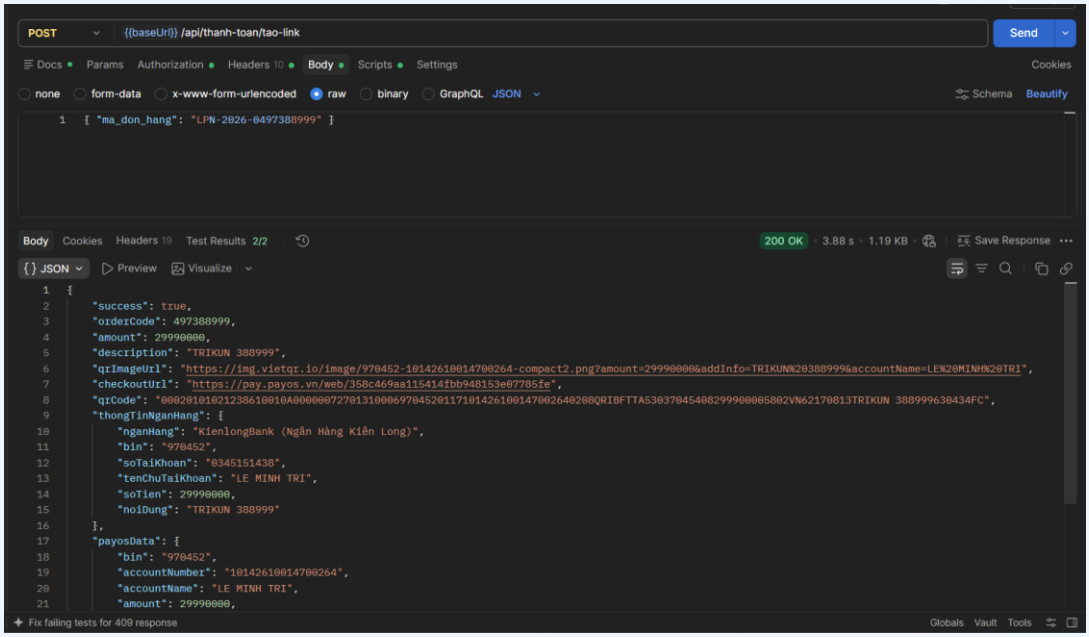


(b) *Giao diện web: trang Đơn hàng của tôi hiển thị "Giao thành công"*

Hình 6.10. *Kiểm thử PUT /api/don-hang/:id: Admin cập nhật trạng thái đơn*

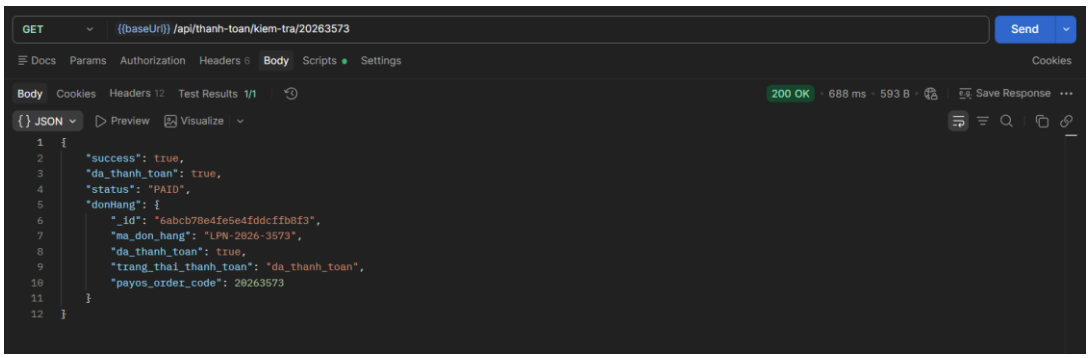


(a) *Giao diện web: chi tiết đơn hàng với phương thức chuyển khoản VietQR*

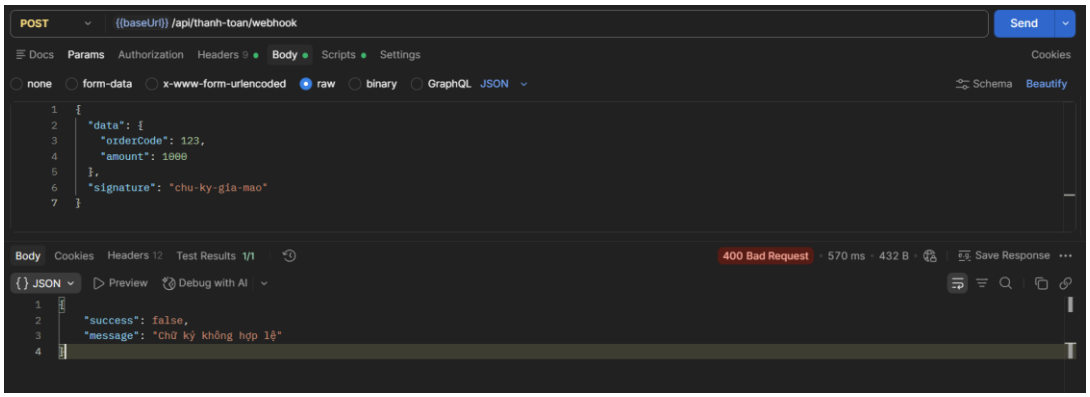


(b) *Postman: tạo link thanh toán bằng token chủ đơn → 200, amount lấy từ đơn trong DB*

Hình 6.11. *Kiểm thử POST /api/thanh-toan/tao-link bằng token chủ đơn: tạo mã QR thanh toán*

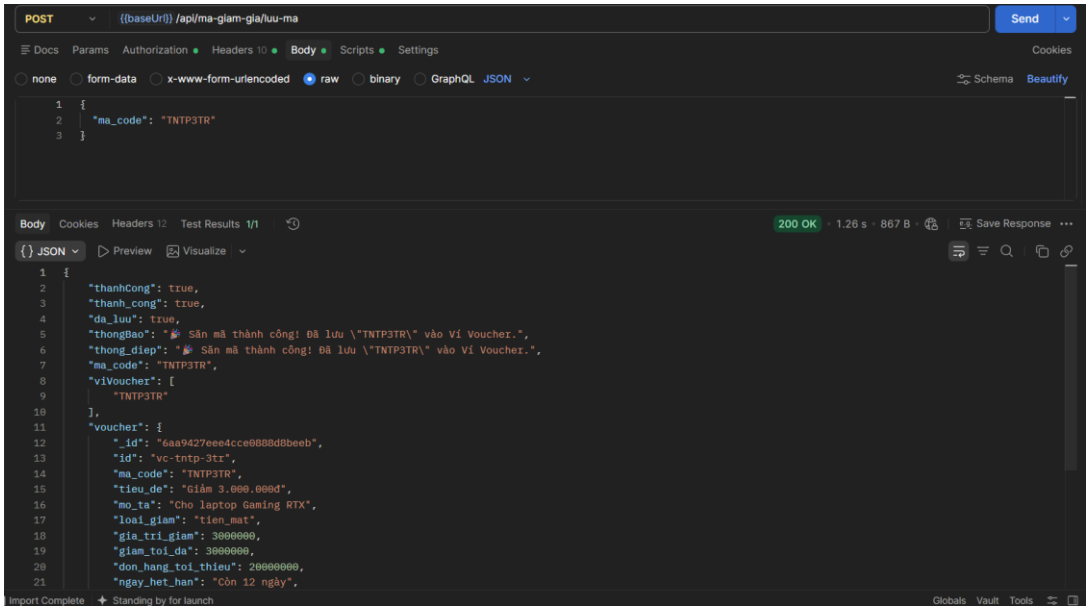


Hình 6.12. *Kiểm thử GET /api/thanh-toan/kiem-tra/:orderCode*

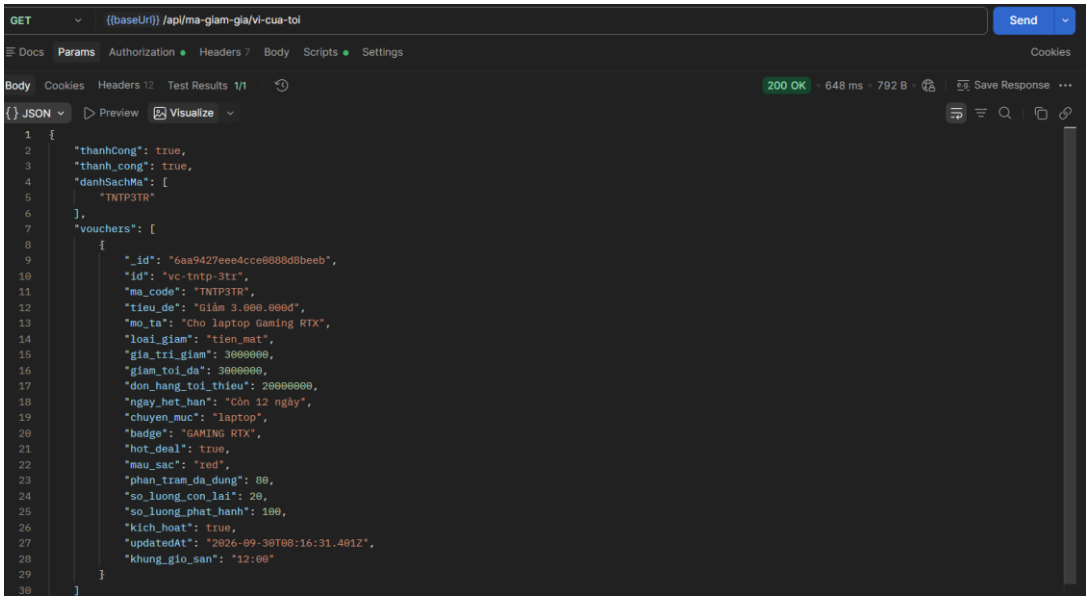


Hình 6.13. *Kiểm thử Webhook PayOS với chữ ký giả mạo bị từ chối (400)*

6.5.4. Nhóm Mã giảm giá, Đánh giá, Bảo hành và Liên hệ

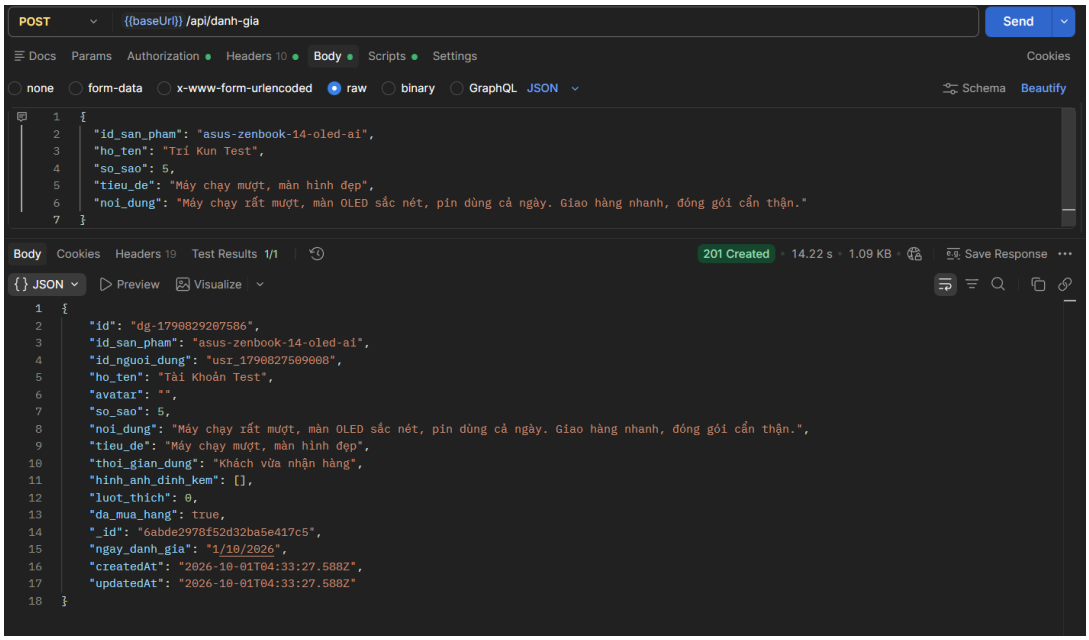


(a) *POST /api/ma-giam-gia/luu-ma: lưu voucher vào ví → 200*

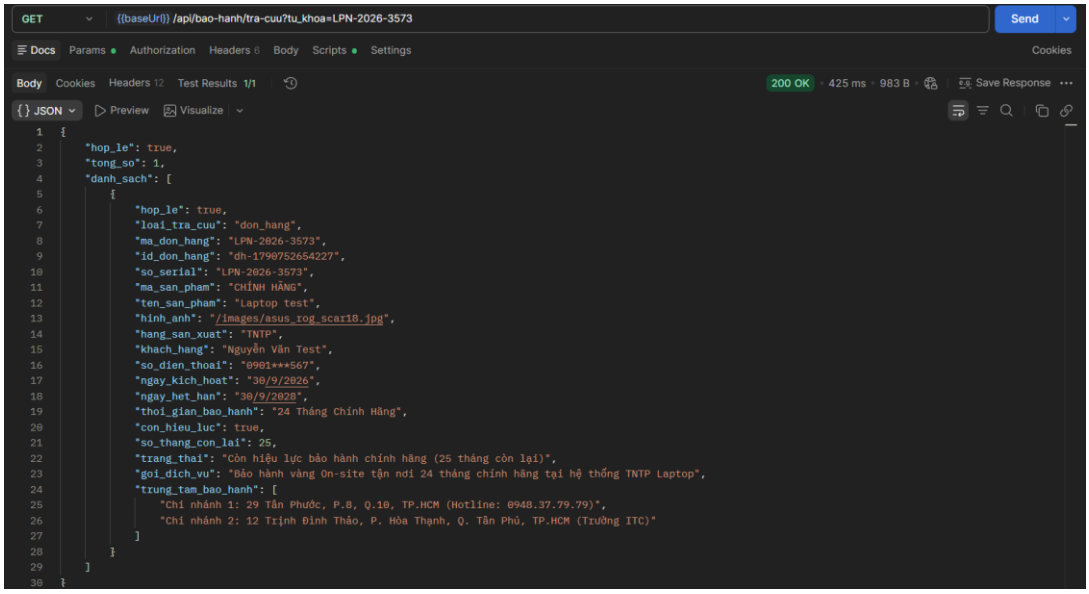


(b) *GET /api/ma-giam-gia/vi-cua-toi: danh sách voucher trong ví → 200*

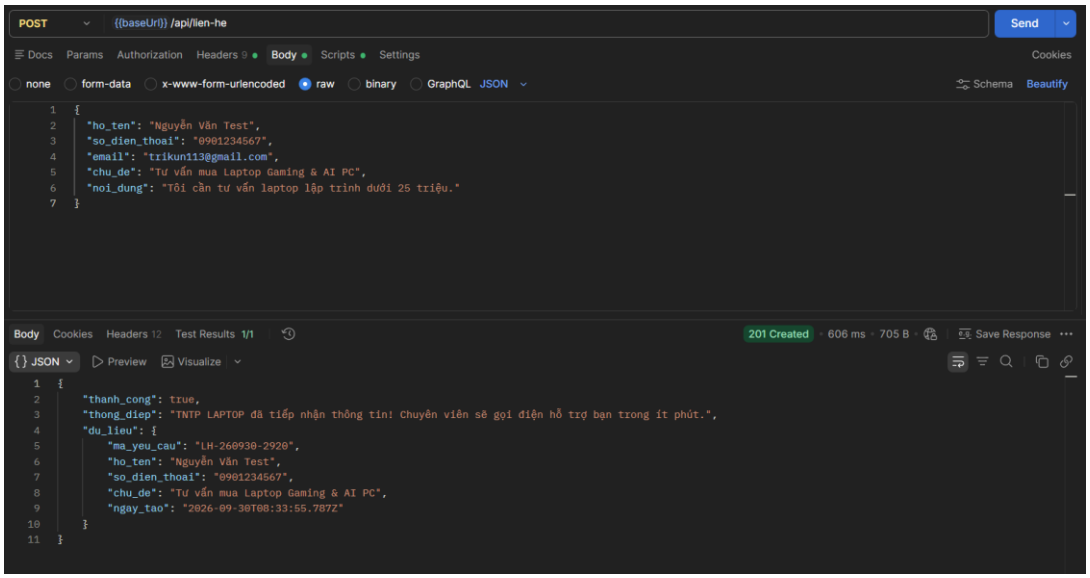
Hình 6.14. *Kiểm thử lưu voucher và xem ví voucher của tôi*



Hình 6.15. *Kiểm thử POST /api/danh-gia: gửi đánh giá sản phẩm*

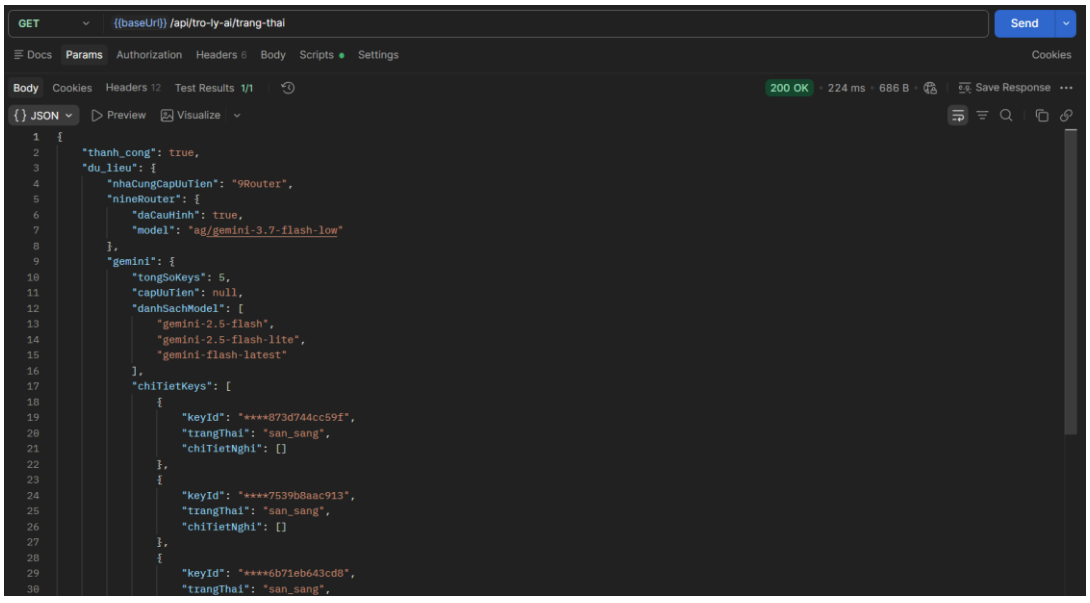


Hình 6.16. *Kiểm thử GET /api/bao-hanh/tra-cuu: tra cứu bảo hành điện tử*

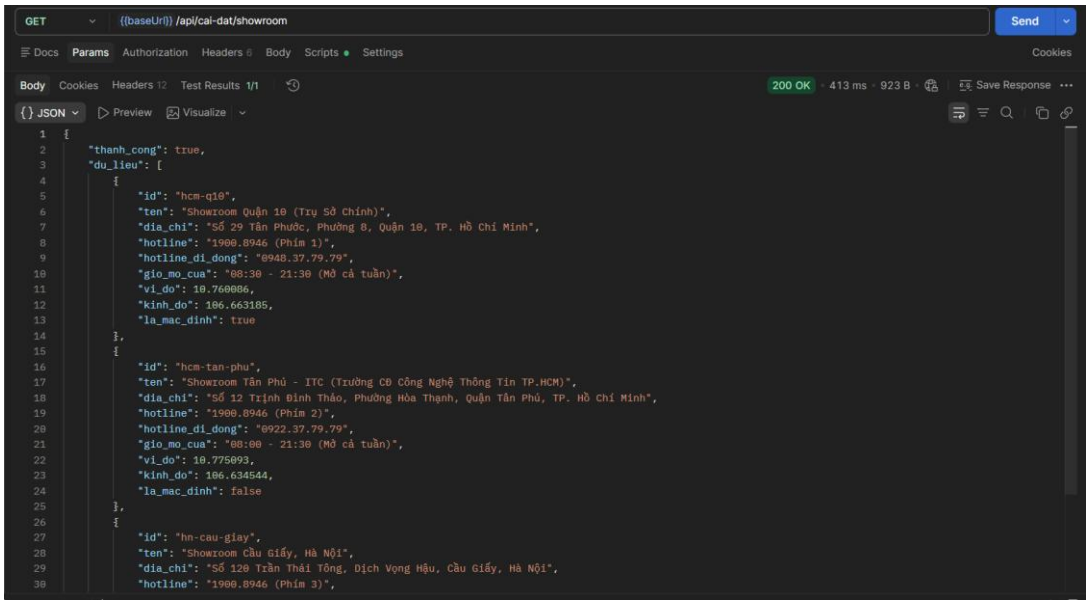


Hình 6.17. *Kiểm thử POST /api/lien-he: gửi phiếu yêu cầu tư vấn*

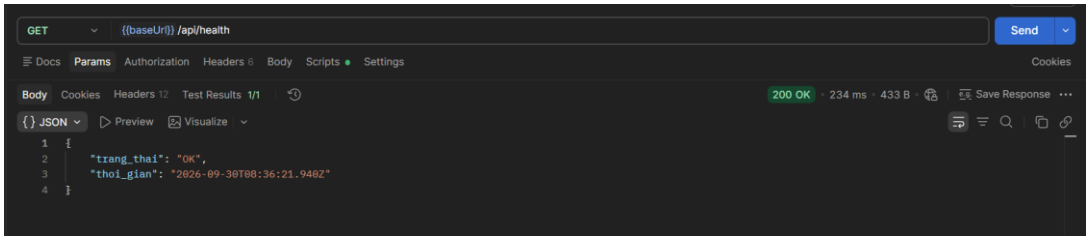
6.5.5. Nhóm Trợ lý AI, Cài đặt và Giám sát máy chủ



Hình 6.18. *Kiểm thử GET /api/tro-ly-ai/trang-thai*



Hình 6.19. *Kiểm thử GET /api/cai-dat/showroom*



Hình 6.20. *Kiểm thử GET /api/health: kiểm tra sức khỏe máy chủ*

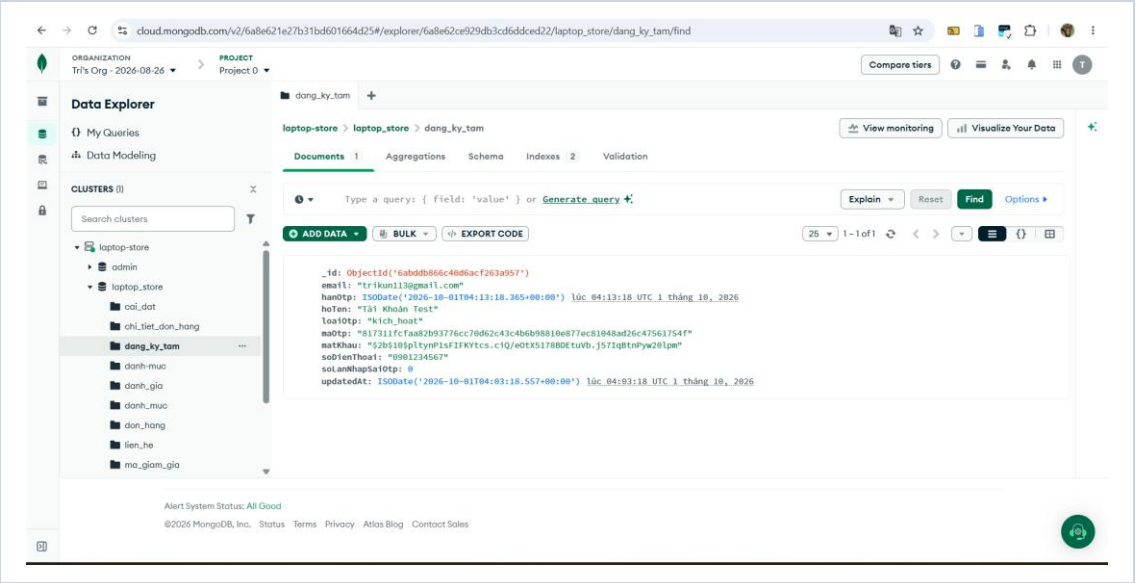
6.6. Minh chứng dữ liệu thay đổi đúng trong MongoDB

Response 200/201 của Postman mới chứng minh API **trả lời đúng**; để chứng minh API **ghi dữ liệu đúng**, nhóm chụp dữ liệu trong MongoDB Compass / Atlas **trước và sau** mỗi thao tác quan trọng. Bảng 6.4 tóm tắt các cặp minh chứng.

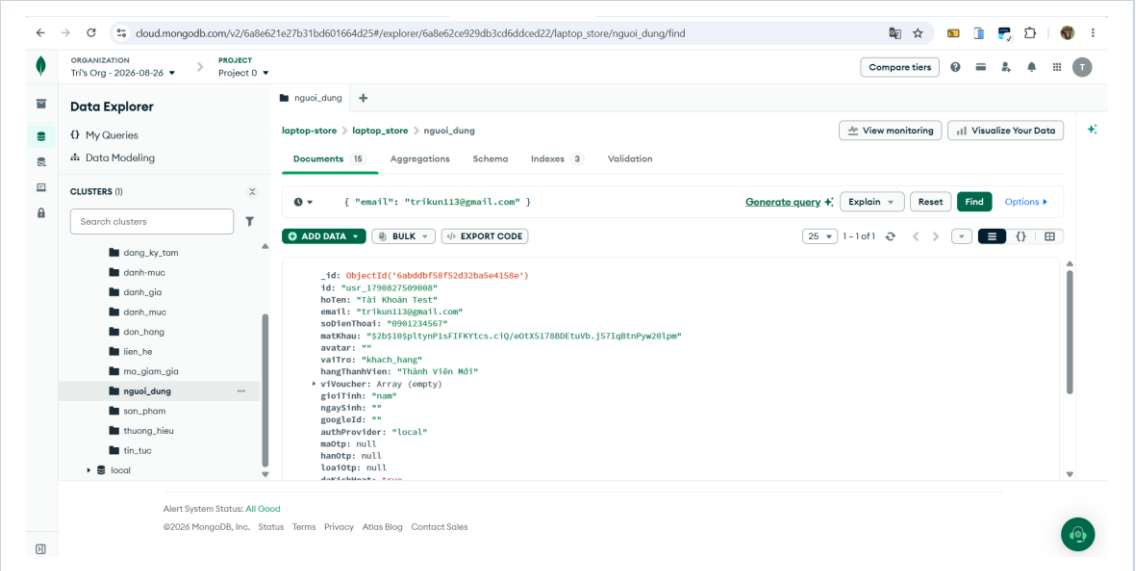
Bảng 6.4. *Các minh chứng dữ liệu MongoDB trước / sau thao tác*

Thao tác (Postman)	Collection	Dữ liệu cần thấy thay đổi	Hình
Đăng ký → Kích hoạt OTP	dang_ky_tam → nguoi_dung	Bản ghi tạm có `maOtp` băm; sau kích hoạt bản ghi tạm biến mất, xuất hiện người dùng mới	6.21, 6.22
Tạo đơn hàng	don_hang	Đơn mới `cho_xac_nhan`, `tong_tien_thanh_toan` do server tính	6.23
Admin chuyển “đã giao”	san_pham, don_hang	`so_luong_ton_kho` giảm, `so_luong_da_ban` tăng, đơn `da_tru_ton_kho: true`	6.24, 6.25
Admin xóa / hủy đơn đã giao	san_pham	Tồn kho được hoàn lại đúng số lượng	6.26
Lưu voucher vào ví	ma_giam_gia, nguoi_dung	`so_luong_con_lai` giảm 1, mã xuất hiện trong `viVoucher`	6.27

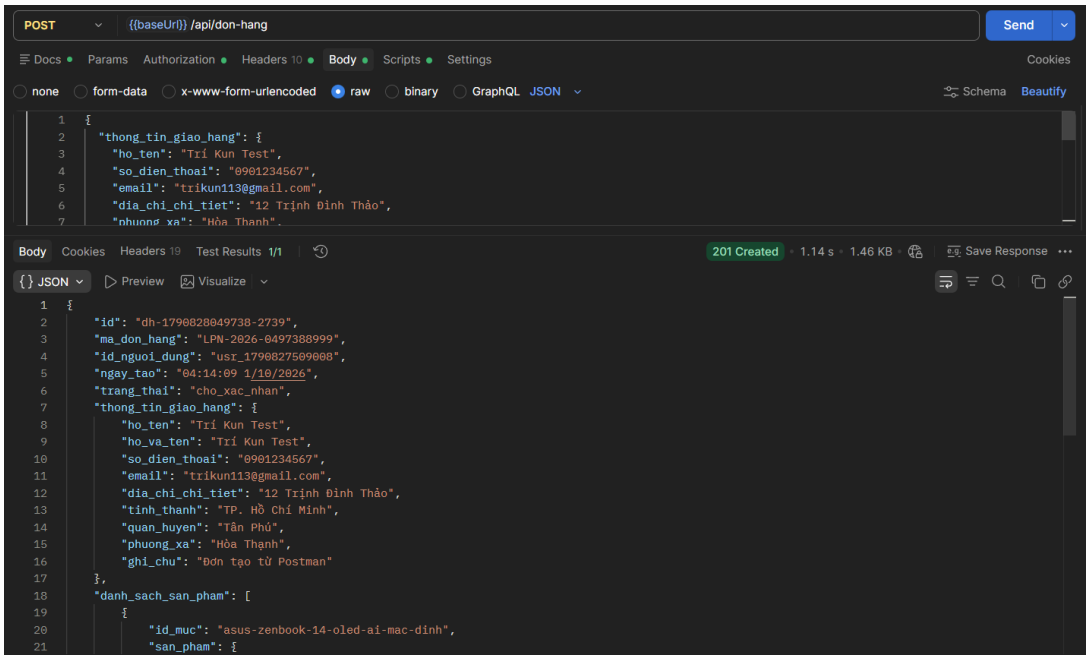
Thao tác (Postman)	Collection	Dữ liệu cần thấy thay đổi	Hình
Gửi đánh giá	danh_gia, san_pham	Đánh giá mới; `diem_danh_gia_tb`, `so_luong_danh_gia` cập nhật	6.28
Admin khóa tài khoản (BM08)	nguoi_dung	`biKhoa: true`, `tokenVersion` tăng	6.29
Gửi phiếu liên hệ	lien_he	Phiếu mới có `ma_yeu_cau` dạng LH-YYMMDD-XXXX	6.30



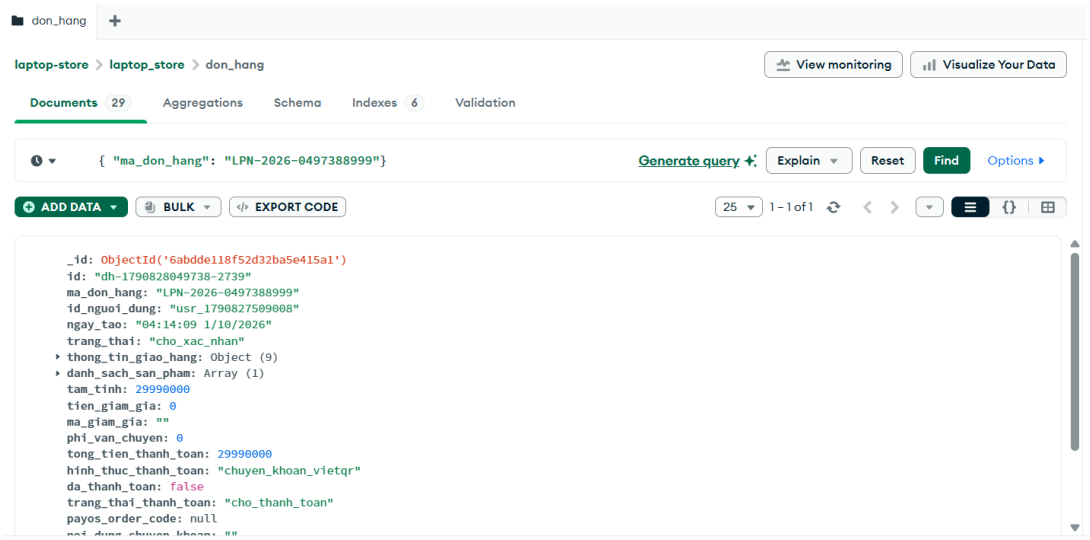
Hình 6.21. *Collection dang_ky_tam sau khi đăng ký: mật khẩu và OTP đều đã băm*



Hình 6.22. *Sau khi kích hoạt OTP: tài khoản xuất hiện trong nguoi_dung, bản ghi tạm đã bị xóa*

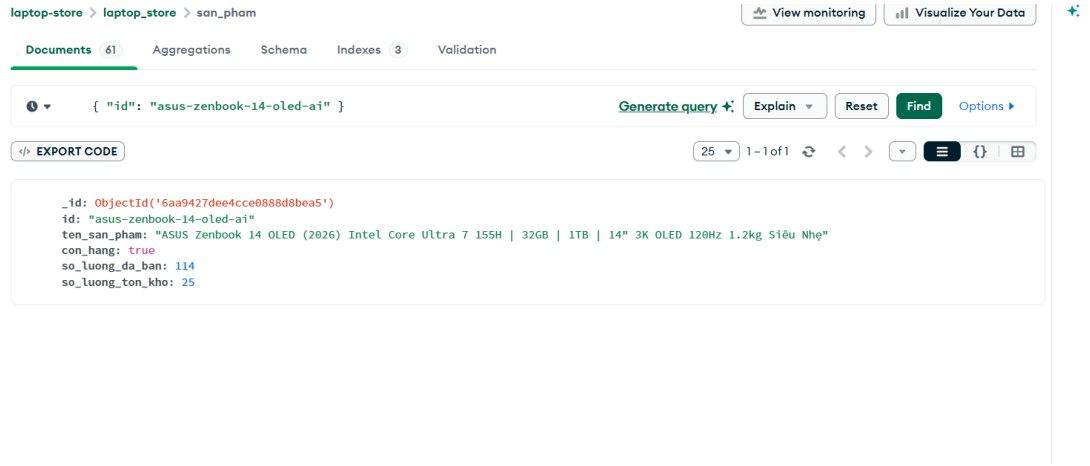


(a) *Postman: POST /api/don-hang → 201 Created, mã đơn LPN-2026-0497388999*

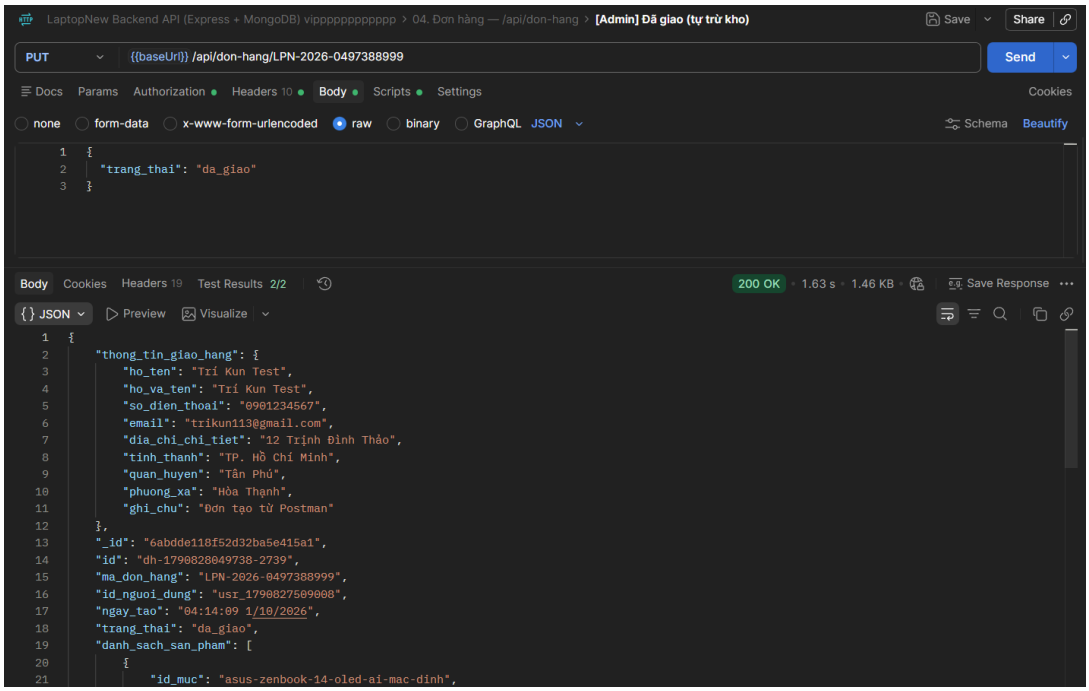


(b) *MongoDB Atlas: đơn cùng mã đã được ghi vào collection don_hang*

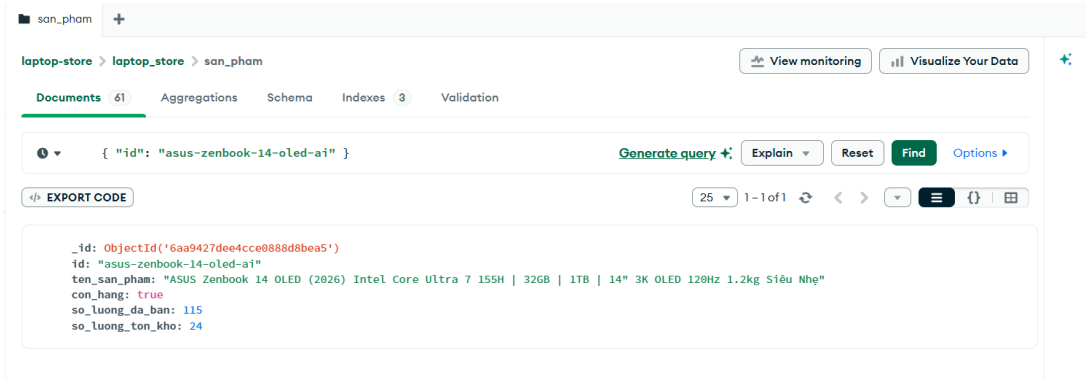
Hình 6.23. *Đơn hàng mới được ghi vào collection don_hang*



Hình 6.24. *Tồn kho sản phẩm TRƯỚC khi Admin chuyển đơn sang "đã giao"*

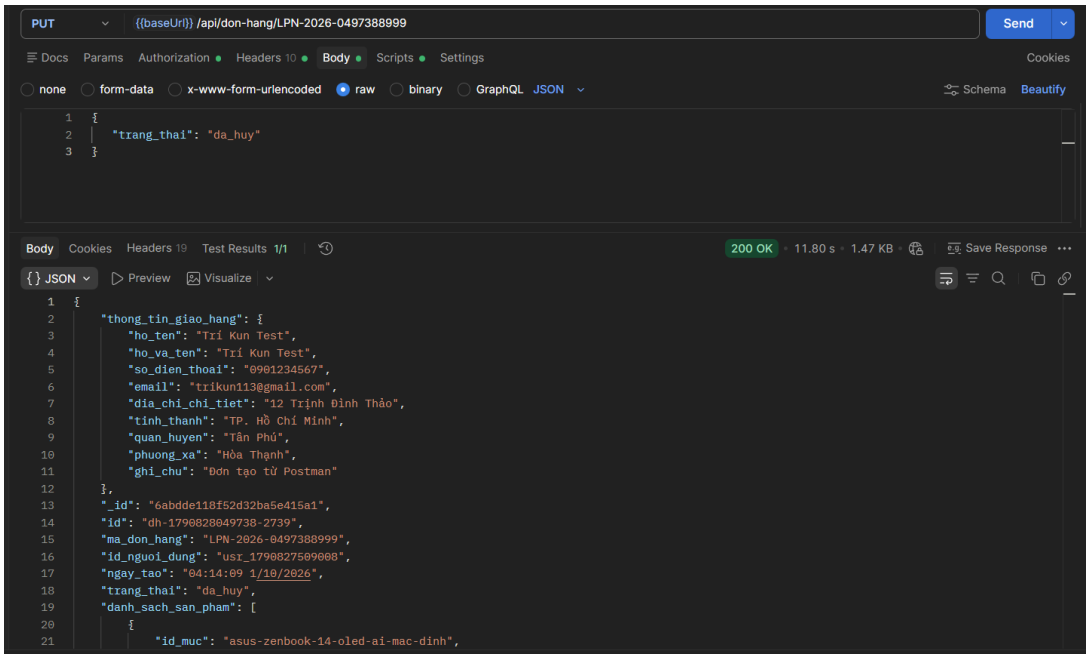


(a) *Postman: Admin chuyển đơn sang "da_giao" → 200*



(b) *MongoDB Atlas: so_luong_ton_kho 25 → 24, so_luong_da_ban 114 → 115*

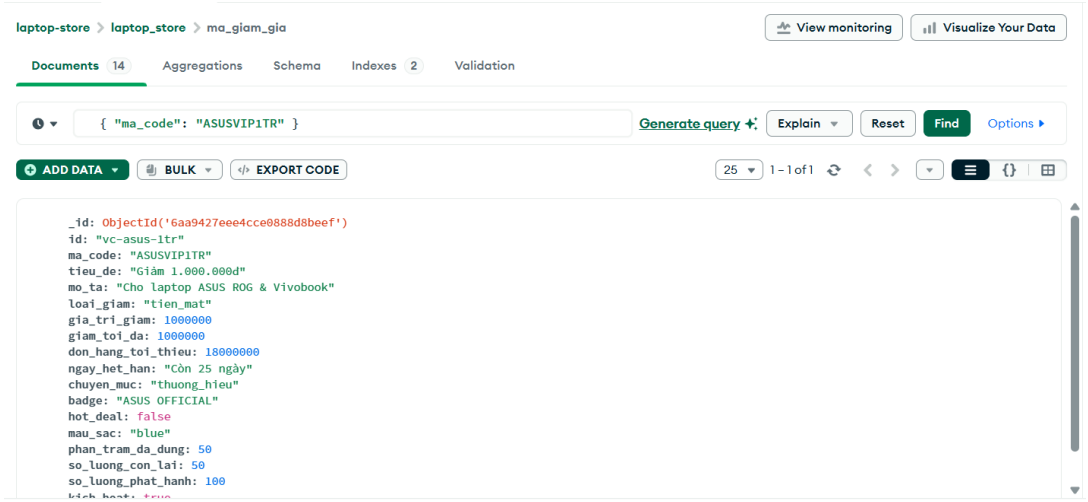
Hình 6.25. *Tồn kho SAU khi đơn "đã giao": giảm đúng số lượng, số đã bán tăng*



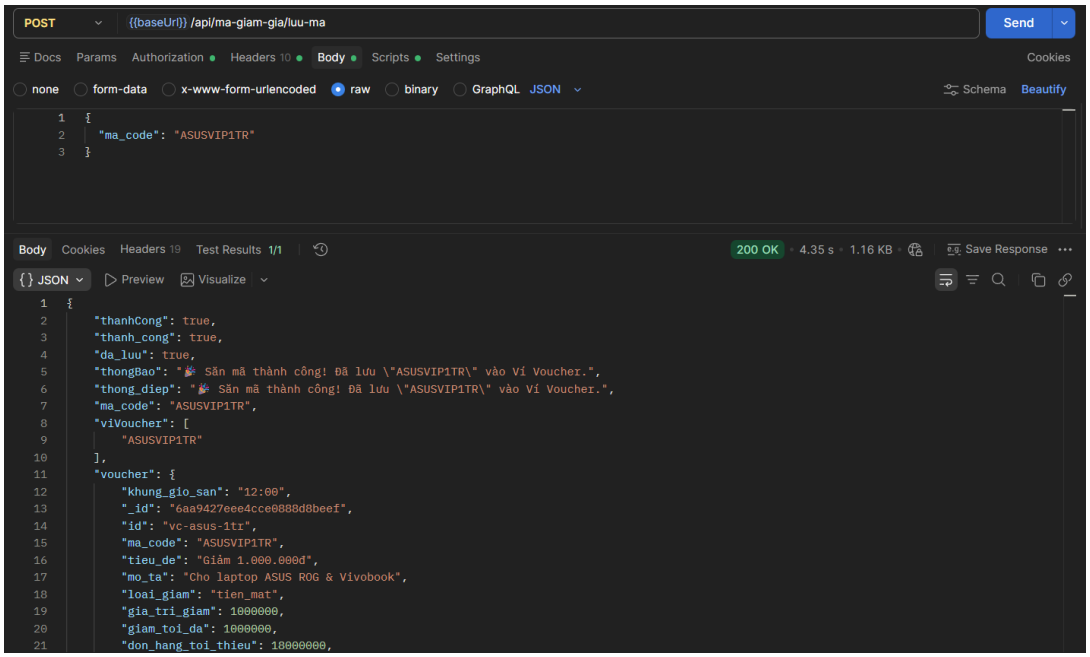
(a) *Postman: Admin chuyển đơn sang "da_huy" → 200*

(b) *MongoDB Atlas: tồn kho hoàn lại về 25, số đã bán về 114*

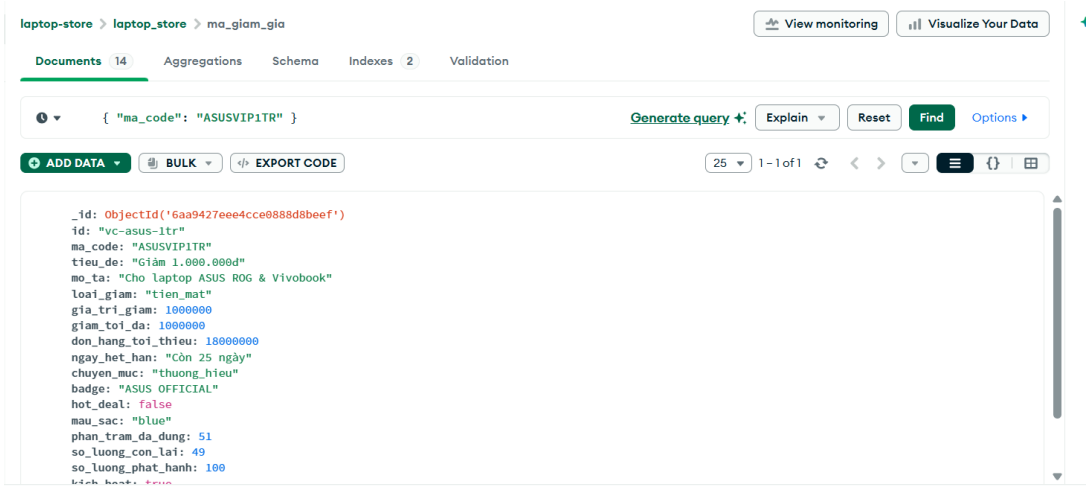
Hình 6.26. *Tồn kho được hoàn lại sau khi Admin hủy đơn*



(a) *MongoDB Atlas — TRƯỚC: voucher ASUSVIP1TR còn 50 lượt*

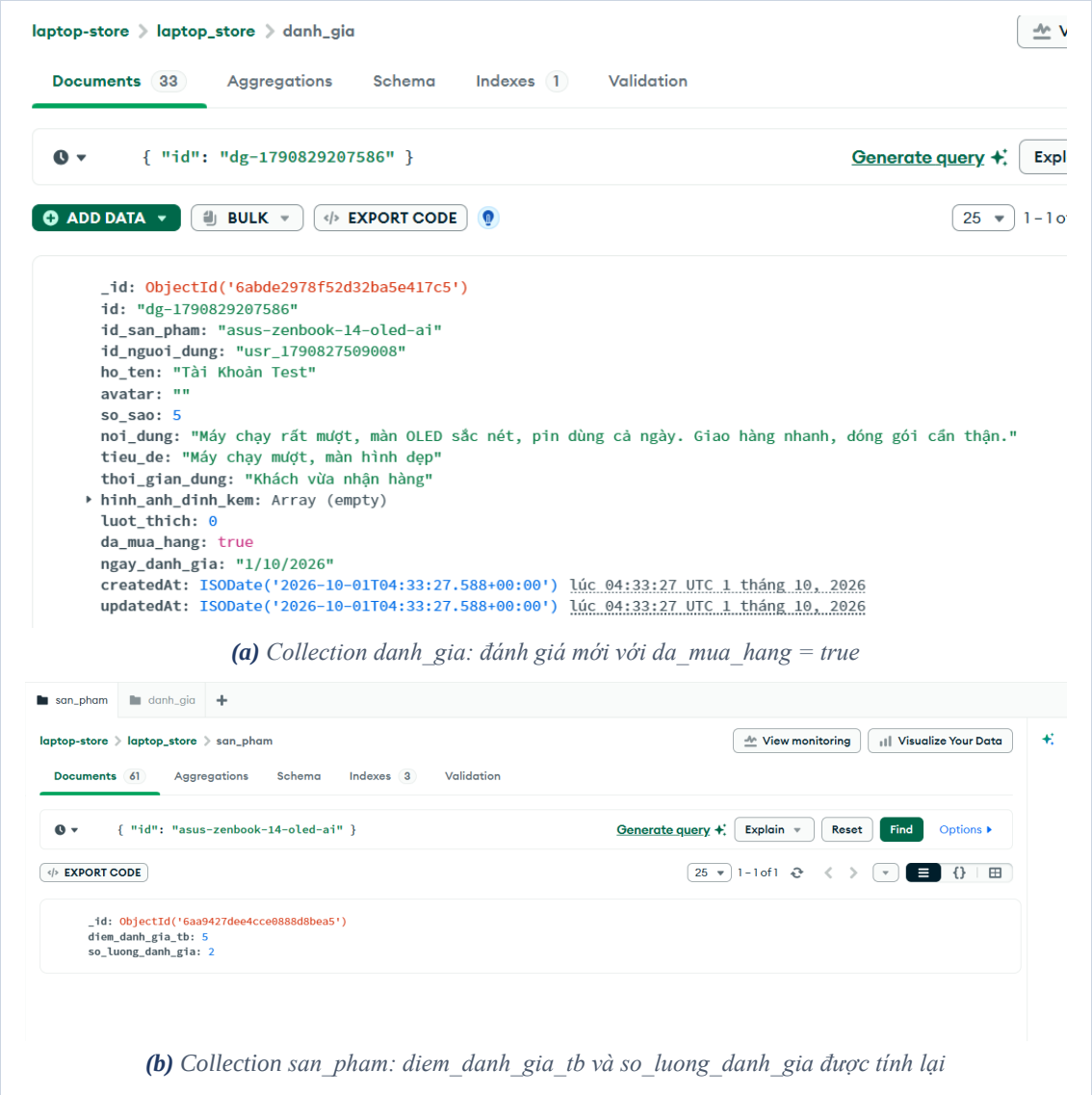


(b) *Postman: POST /api/ma-giam-gia/luu-ma → 200, mã được thêm vào viVoucher*



(c) *MongoDB Atlas — SAU: còn 49 lượt, phan_tram_da_dung tăng lên 51*

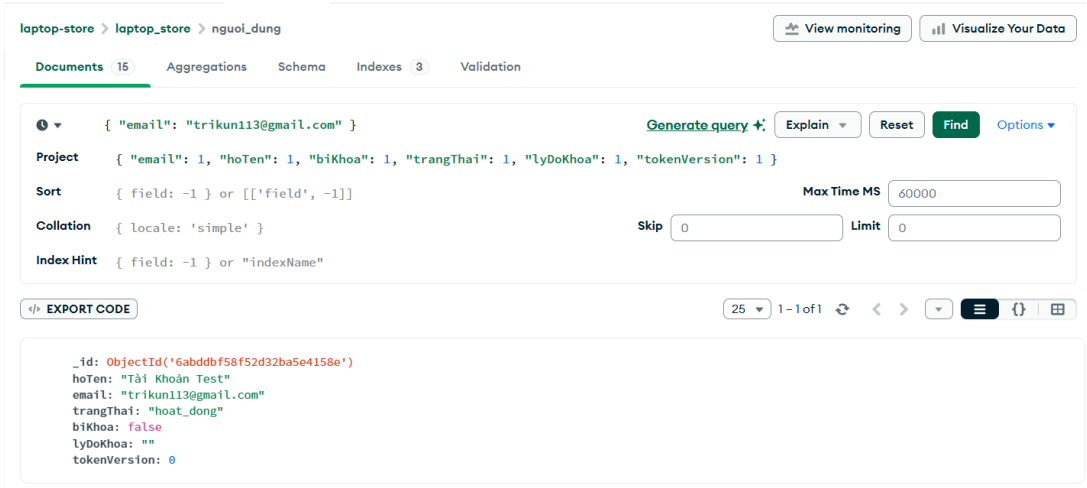
Hình 6.27. *Lưu voucher: số lượt còn lại giảm và mã được thêm vào ví người dùng*



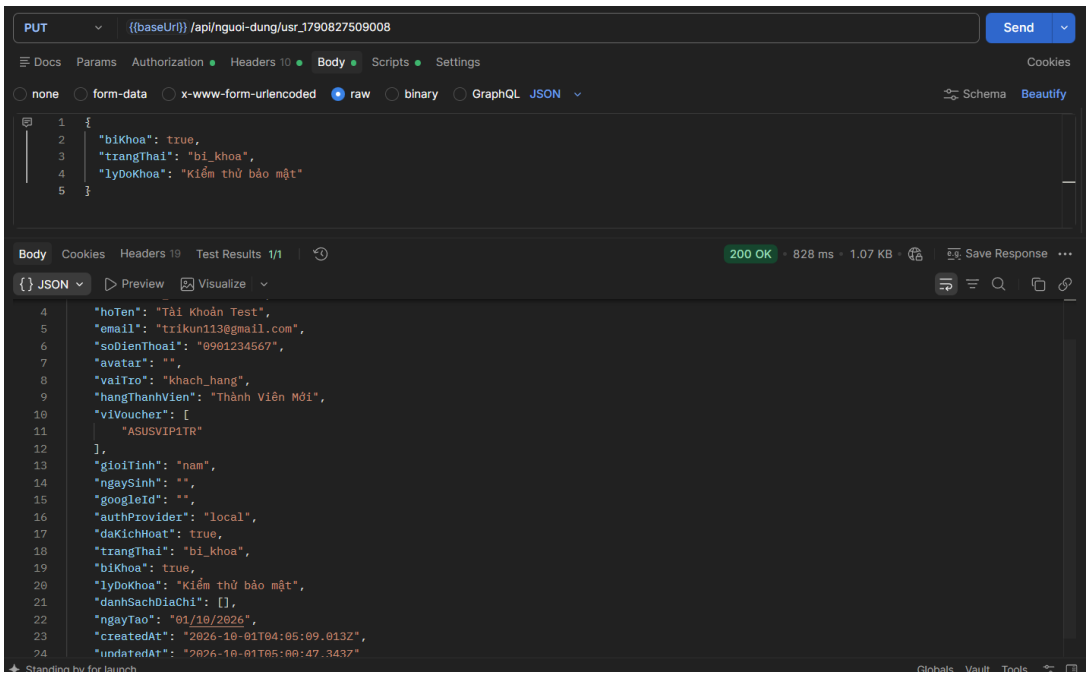
(a) *Collection danh_gia: đánh giá mới với da_mua_hang = true*

(b) *Collection san_pham: diem_danh_gia_tb và so_luong_danh_gia được tính lại*

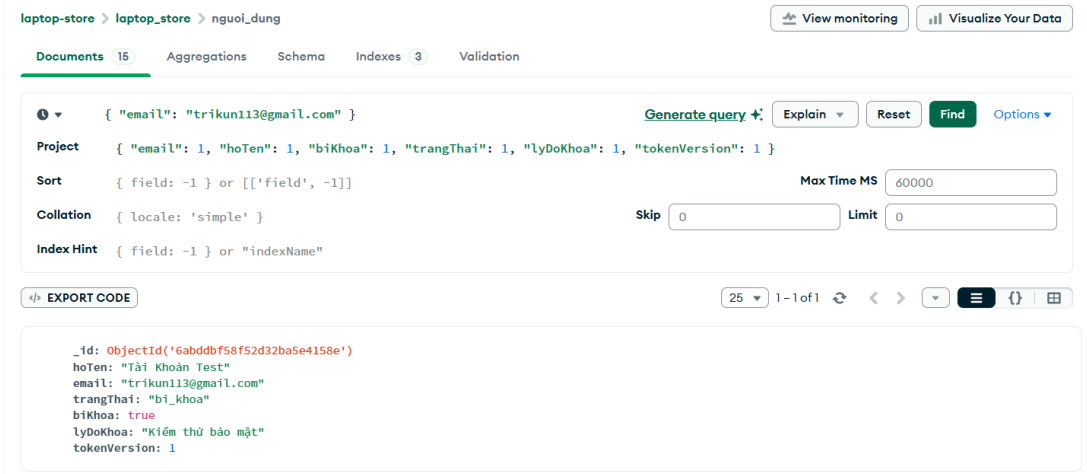
Hình 6.28. *Đánh giá mới và điểm trung bình sản phẩm được cập nhật*



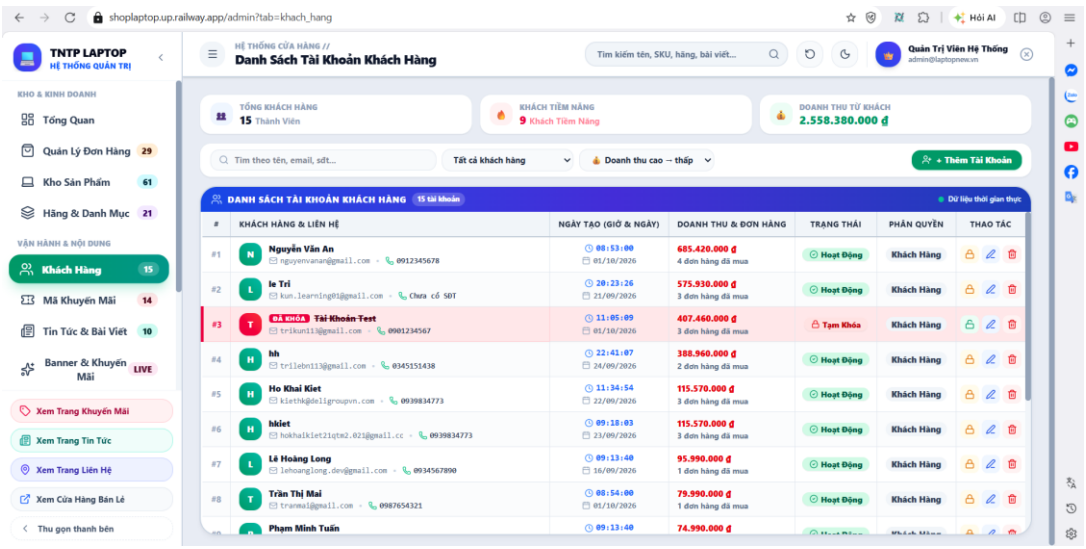
(a) *MongoDB Atlas — TRƯỚC: trangThai = hoat_dong, tokenVersion = 0*



(b) *Postman: Admin khóa tài khoản → 200*

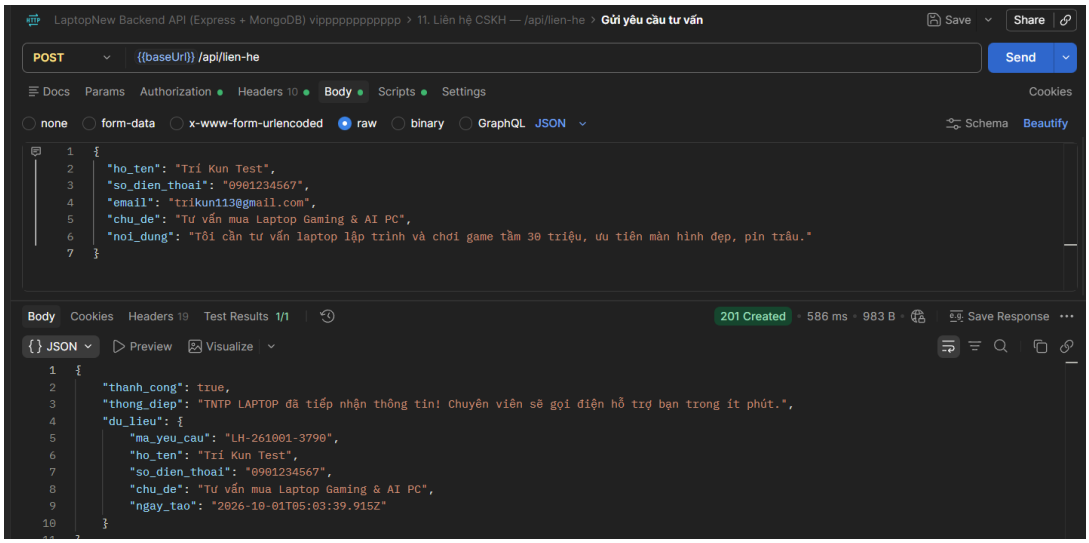


(c) *MongoDB Atlas — SAU: biKhoa = true, trangThai = bi_khoa, tokenVersion = 1*

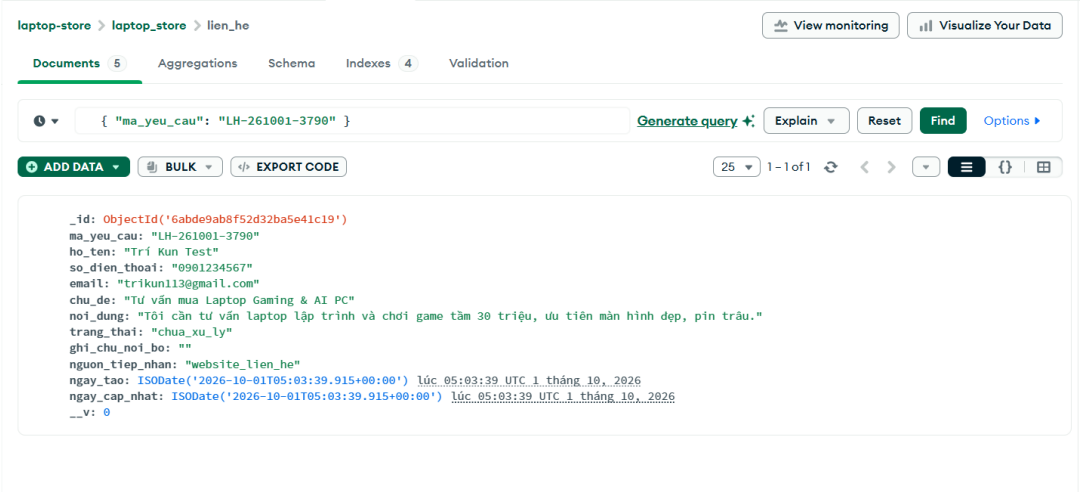


(d) *Giao diện quản trị: tài khoản được gắn nhãn "Đã khóa"*

Hình 6.29. *Tài khoản bị Admin khóa: biKhoa = true và tokenVersion tăng*



(a) *Postman: POST /api/lien-he → 201, mã phiếu LH-261001-3790*



(b) *MongoDB Atlas: phiếu cùng mã trong collection lien_he, trạng thái chua_xu_ly*

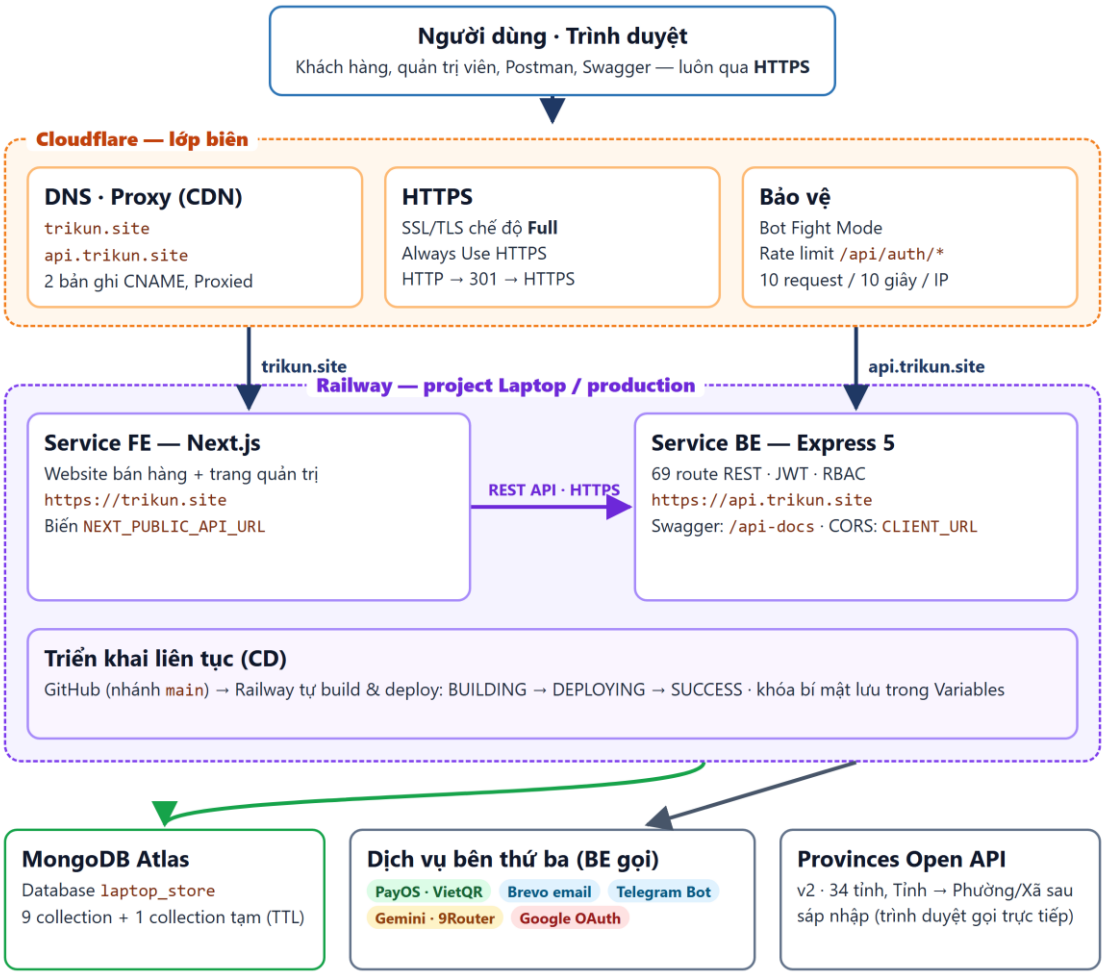
Hình 6.30. *Phiếu yêu cầu tư vấn mới trong collection lien_he*

6.7. Triển khai thực tế trên Railway, tên miền riêng và Cloudflare

Hệ thống không chỉ chạy ở máy cá nhân mà đã được đưa lên môi trường production và vận hành dưới tên miền riêng. Mọi truy cập từ Internet đi qua Cloudflare rồi mới tới hai service trên Railway; mã nguồn được tự động build và deploy mỗi lần push lên GitHub.

> **Website:** https://trikun.site · **API:** https://api.trikun.site · **Tài liệu API:** https://api.trikun.site/api-docs

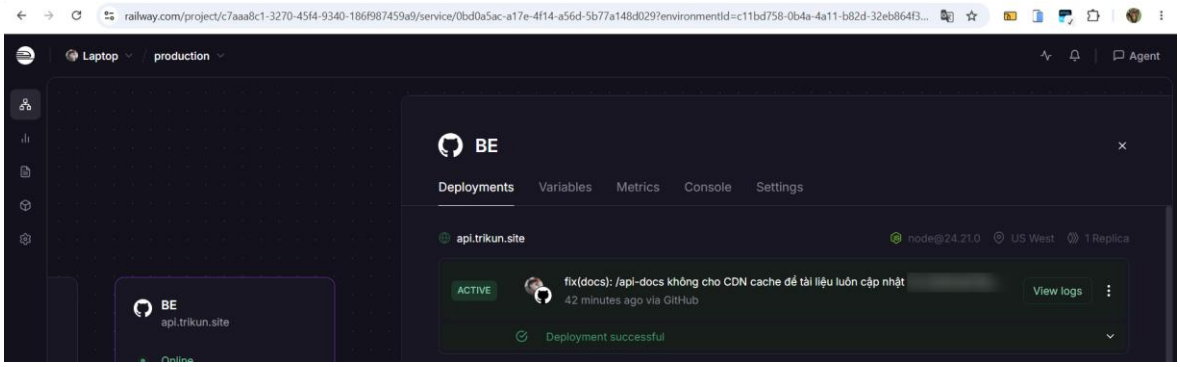
6.7.1. Mô hình triển khai



Hình 6.31. *Mô hình triển khai production: Cloudflare – Railway – MongoDB Atlas và các dịch vụ bên thứ ba*

- **Railway** chạy 2 service: FE (Next.js) và BE (Express 5, Node.js 24); khóa bí mật lưu trong Variables của Railway, không có trong mã nguồn.
- **Triển khai liên tục (CD):** mỗi lần push nhánh `main`, Railway tự build và deploy, trạng thái chuyển BUILDING → DEPLOYING → SUCCESS (Hình 6.32).

- **Cloudflare** đứng trước cả hai tên miền chính: xử lý HTTPS, chặn bot và giới hạn tần suất trước khi request tới Railway.
- **MongoDB Atlas** lưu dữ liệu; Back-End gọi các dịch vụ PayOS, Brevo, Telegram, Gemini/9Router và Google OAuth.



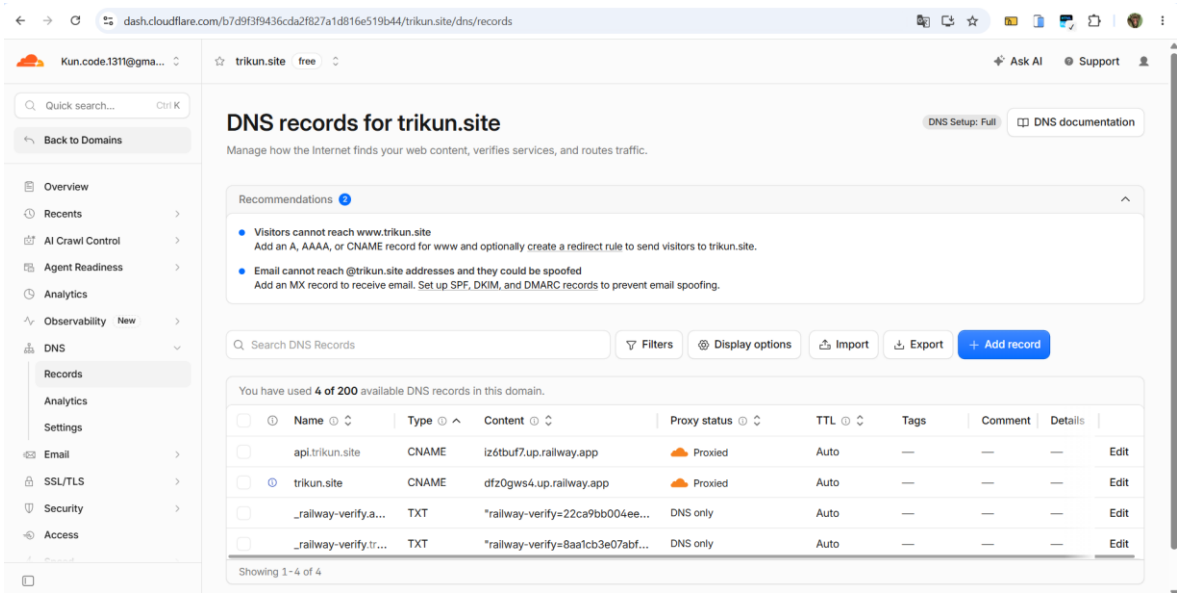
Hình 6.32. *Service BE trên Railway: bản deploy mới nhất từ GitHub ở trạng thái ACTIVE – Deployment successful*

6.7.2. Tên miền và cấu hình DNS

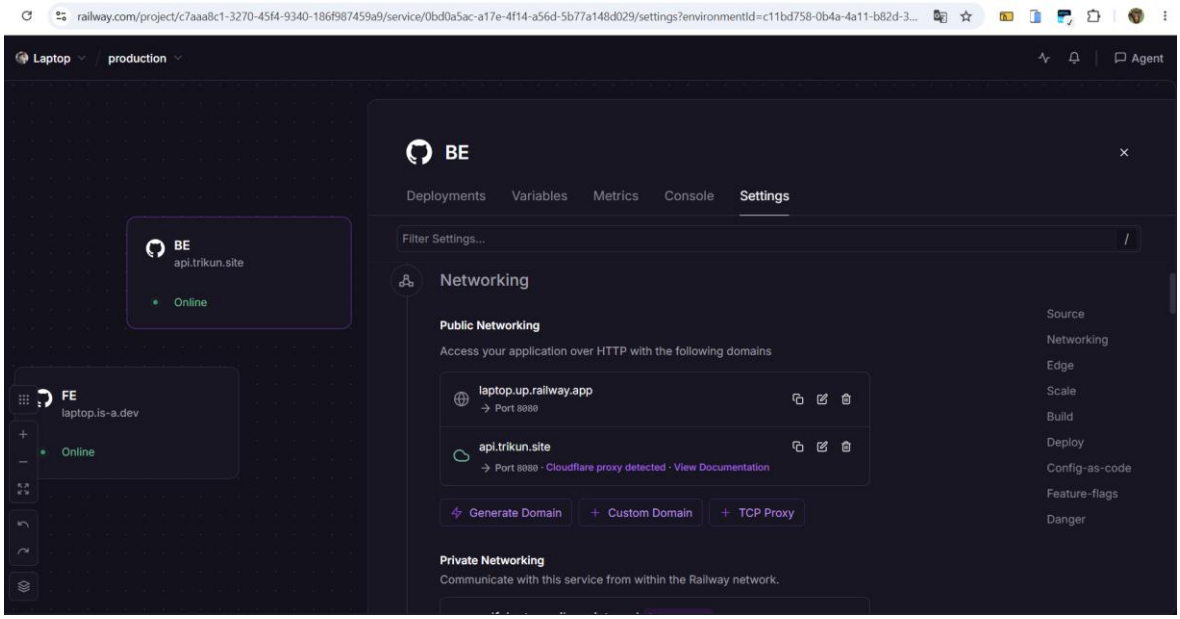
Tên miền `trikun.site` đăng ký tại TenTen, chuyển nameserver sang Cloudflare (`leo.ns.cloudflare.com`, `uma.ns.cloudflare.com`). Từ đó Cloudflare quản lý toàn bộ bản ghi DNS; mỗi service trên Railway nhận một tên miền riêng và được Railway xác minh quyền sở hữu bằng bản ghi TXT (Bảng 6.5).

Bảng 6.5. *Bản ghi DNS của tên miền trikun.site trên Cloudflare*

Tên	Loại	Trỏ tới	Proxy	Mục đích
`trikun.site`	CNAME	`dfz0gws4.up.railway.app`	Proxied	Website (service FE)
`api.trikun.site`	CNAME	`iz6tbuf7.up.railway.app`	Proxied	API + Swagger (service BE)
`_railway-verify`	TXT	railway-verify=…	DNS only	Railway xác minh trikun.site
`_railway-verify.api`	TXT	railway-verify=…	DNS only	Railway xác minh api.trikun.site



Hình 6.33. *Bản ghi DNS trên Cloudflare: hai CNAME ở chế độ Proxied và hai TXT xác minh*



Hình 6.34. *Railway gắn tên miền api.trikun.site cho service BE và nhận diện Cloudflare proxy*

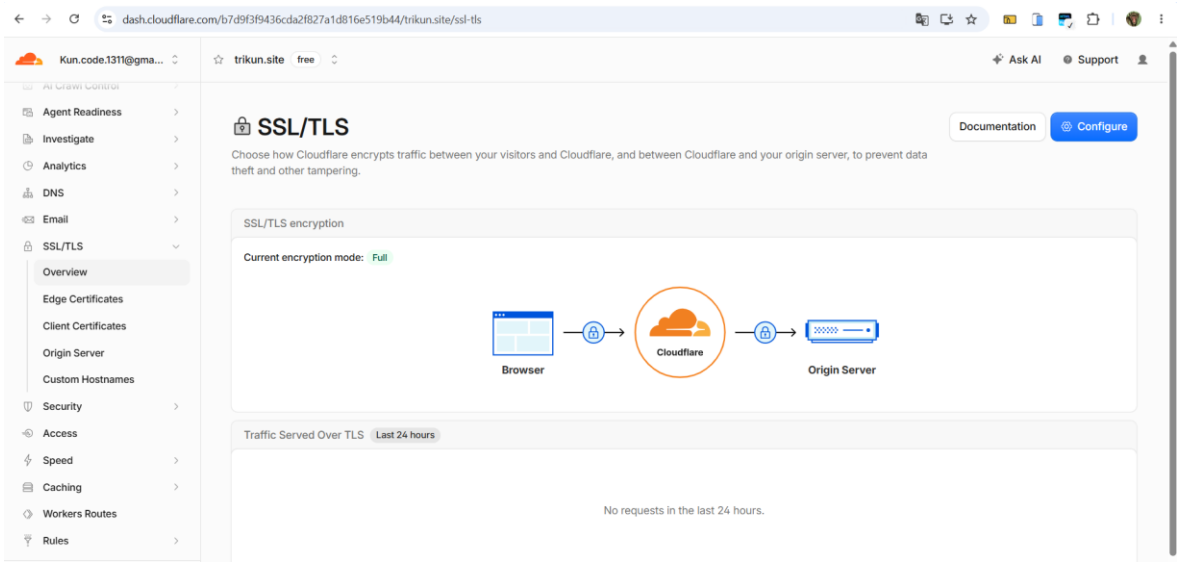
6.7.3. Bảo vệ lớp biên với Cloudflare

Cloudflare bổ sung một lớp bảo vệ đứng trước toàn bộ cơ chế bảo mật trong ứng dụng (mục 5.3): request bị xử lý ngay tại biên mạng, nên lưu lượng độc hại không tiêu tốn tài nguyên của server.

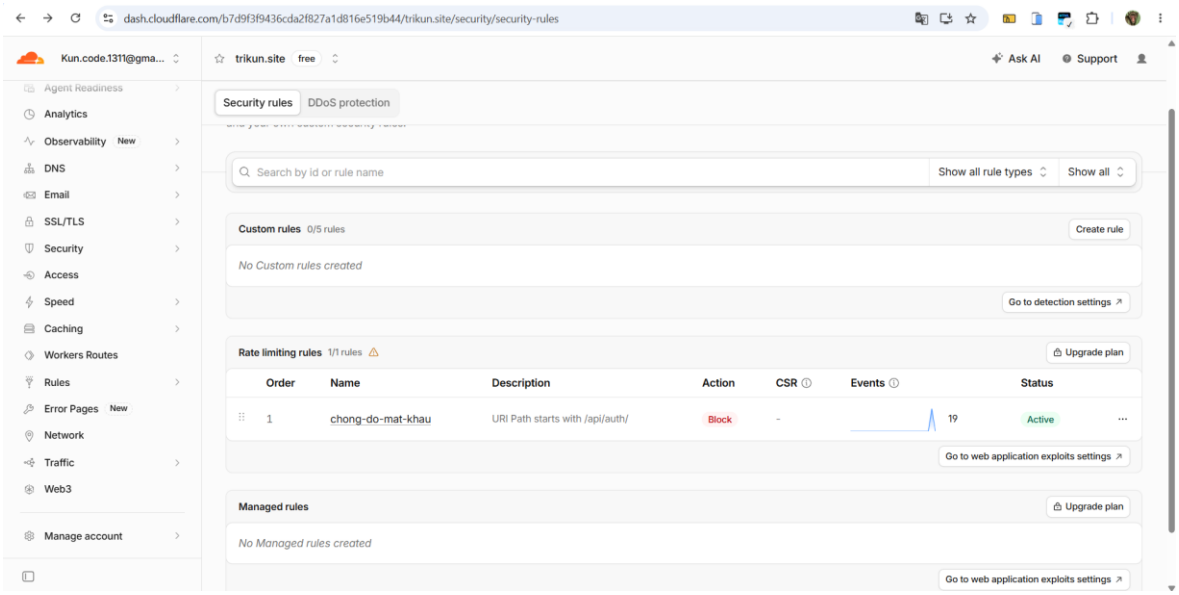
Bảng 6.6. *Cấu hình bảo vệ đã bật trên Cloudflare*

Tính năng	Cấu hình	Tác dụng
SSL/TLS	Chế độ **Full**	Mã hóa cả hai chặng: trình duyệt → Cloudflare và Cloudflare → Railway

Tính năng	Cấu hình	Tác dụng
Always Use HTTPS	Bật	Mọi truy cập `http://` được chuyển 301 sang `https://`
Bot Fight Mode	Bật	Thử thách / chặn bot tự động và công cụ quét lỗ hổng
Rate limiting rule	`chong-do-mat-khau`: URI Path bắt đầu `/api/auth/`, 10 request / 10 giây / IP, Block	Chặn dò mật khẩu, spam OTP ngay tại biên; không phụ thuộc server
Proxy (CDN)	Bật cho trikun.site và api.trikun.site	Request đi qua mạng biên Cloudflare; tài nguyên tĩnh được cache gần người dùng
HSTS (từ server)	`max-age=15552000; includeSubDomains`	Trình duyệt chỉ kết nối HTTPS trong 180 ngày



Hình 6.35. *SSL/TLS ở chế độ Full: mã hóa cả chặng Browser → Cloudflare và Cloudflare → Origin Server*



Hình 6.36. *Luật Rate limiting chong-do-mat-khau đang hoạt động cho /api/auth/*: ghi nhận 19 request bị chặn trong lần kiểm thử*

6.7.4. Cấu hình môi trường production

- **CORS:** biến `CLIENT_URL` gồm `https://trikun.site` và `https://shoplaptop.up.railway.app`; chỉ các origin này (và chính tên miền API, để nút Try it out của Swagger gọi được POST/PUT/DELETE) được gọi API, origin đầu tiên là trang trả về sau thanh toán.
- **Front-End:** `NEXT_PUBLIC_API_URL` trỏ tới `https://api.trikun.site/api`; biến được gắn lúc build nên FE được deploy lại sau khi đổi.
- **PayOS:** kênh thanh toán riêng cho LaptopNew, webhook trỏ tới `/api/thanh-toan/webhook` của `api.trikun.site`; webhook thử của PayOS nhận 200, chữ ký sai bị từ chối 400.
- **Google OAuth:** thêm `https://trikun.site` vào Authorized JavaScript origins của OAuth Client.

6.7.5. Kiểm chứng trên môi trường production

Sau khi chuyển sang tên miền riêng, nhóm kiểm chứng lại trên môi trường production bằng curl và Newman (trình chạy collection Postman bằng dòng lệnh). Kết quả ở Bảng 6.7.

Bảng 6.7. *Kết quả kiểm chứng trên môi trường production*

Kiểm chứng	Cách thực hiện	Kết quả
HTTP → HTTPS	Truy cập `http://api.trikun.site`	301 chuyển sang https://

Kiểm chứng	Cách thực hiện	Kết quả
Đi qua Cloudflare	Đọc header phản hồi	`Server: cloudflare`, có `CF-RAY`
Website	`/`, `/san-pham`, `/thanh-toan`, `/admin` trên trikun.site	200; mã JS gọi API tại `https://api.trikun.site/api`
API và tài liệu	`/api/health`, `/api/san-pham`, `/api/danh-muc`, `/api/tin-tuc`, `/api/cai-dat/showroom`, `/api-docs`	200
CORS	Preflight với Origin trikun.site và một website lạ	trikun.site: 204 cho phép · website lạ: 403
Rate limit lớp biên	60 request song song vào `/api/auth/toi`	42 request qua (401 vì không có token), **18 bị Cloudflare chặn 429**; request kế tiếp bị chặn mã 1015 → Cloudflare ghi nhận **19 sự kiện** (Hình 6.36)
Bảo mật ứng dụng	NoSQL Injection; gọi API Admin không token	400 · 401; đủ header HSTS, CSP, nosniff, X-Frame-Options
Webhook PayOS	Gửi webhook ký đúng và ký sai	200 · 400 (chữ ký không hợp lệ)
Kiểm thử tự động	Newman chạy thư mục 00 trên `https://api.trikun.site`	**36/36 ca** không cần dữ liệu đăng nhập đạt (50/50 bài kiểm tra)

Kết quả: website và API chạy ổn định trên tên miền riêng với HTTPS; mọi request đều đi qua Cloudflare và bị chặn ngay tại biên khi có dấu hiệu dò mật khẩu, trong khi các cơ chế bảo mật trong ứng dụng vẫn hoạt động đúng. Bốn ca TC27–TC30 cần token và mã đơn thật đã đạt trong lần chạy đầy đủ trên server Railway (Hình 6.2).

6.8. Kiểm thử tự động, chất lượng mã nguồn và CI

Bộ Postman ở các mục 6.4 – 6.7 kiểm thử hệ thống từ bên ngoài và cần server, database thật đang chạy. Để mỗi lần sửa mã đều được kiểm tra lại ngay, nhóm viết thêm bộ kiểm thử nằm trong mã nguồn bằng **Jest** và **Supertest**, đặt tại `BE/tests/`, và cho **GitHub Actions** tự chạy bộ kiểm thử này mỗi lần push.

6.8.1. Tổ chức bộ kiểm thử

- **Unit test** kiểm tra từng hàm nhỏ riêng lẻ (tiện ích bảo mật, chữ ký PayOS, middleware), không cần database.
- **Integration test** gửi request thật qua toàn bộ chuỗi Route → Middleware → Controller → Model bằng Supertest. Database là **mongodb-memory-server**:

một MongoDB chạy trong RAM, xóa sạch sau mỗi ca nên các ca độc lập với nhau và không đụng tới dữ liệu production.

- Dịch vụ bên ngoài được thay bằng **mock** (`jest.mock`): kiểm thử không gửi email Brevo hay tin nhắn Telegram thật. Biến môi trường dùng giá trị giả khai báo trong `tests/setup/bien-moi-truong.js`, không đọc tệp `.env`.
- Để Supertest nạp được ứng dụng, mã khởi động được tách làm hai: `app.js` chỉ cấu hình Express (middleware, route, xử lý lỗi) và export `app`; `server.js` nạp `.env`, kết nối MongoDB Atlas, mở cổng và khởi động Telegram Bot.

Bảng 6.8. *Bộ kiểm thử tự động Back-End (Jest + Supertest)*

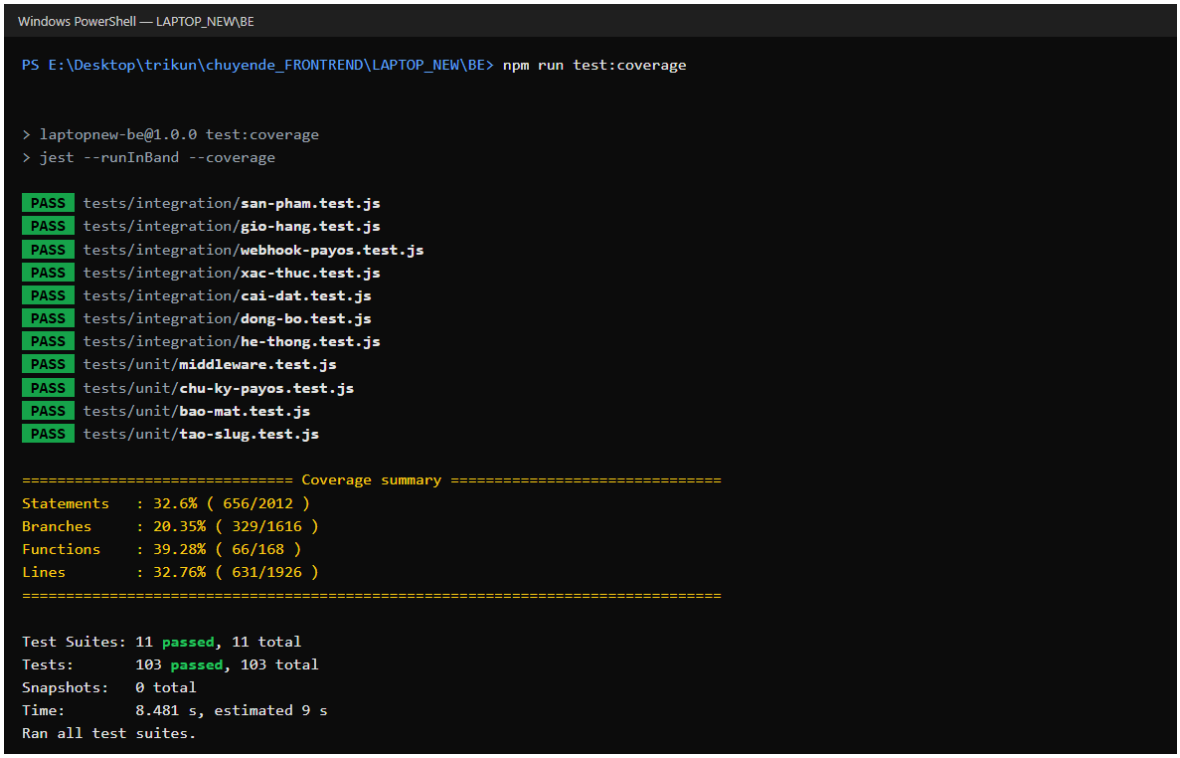
Tệp kiểm thử	Loại	Nội dung kiểm tra	Số ca
`unit/bao-mat.test.js`	Unit	Escape regex, ép kiểu chuỗi chống NoSQL Injection, sinh / băm / so sánh OTP, chính sách mật khẩu, lọc URL ảnh	15
`unit/middleware.test.js`	Unit	Lọc khóa `$` và `__proto__`, security headers, rate limit theo IP, phân quyền `chiAdmin`	7
`unit/chu-ky-payos.test.js`	Unit	Chữ ký HMAC-SHA256: sắp xếp khóa, sửa số tiền hoặc sai checksum key thì chữ ký đổi	4
`unit/tao-slug.test.js`	Unit	Tạo slug URL từ tên tiếng Việt có dấu	4
`integration/xac-thuc.test.js`	Integration	Đăng nhập 400 / 401 / 200, chặn `{"$ne": null}`, khóa sau 5 lần sai (429), tài khoản bị khóa (403), validate đăng ký, token giả, đăng xuất thu hồi token	14
`integration/san-pham.test.js`	Integration	Xem, tìm kiếm, lọc, từ khóa ReDoS, 404; sản phẩm ẩn / hết hàng tự ẩn với khách; phân quyền thêm / xóa (401 / 403 / 201 / 200); giá niêm yết gốc ≥ giá bán và % giảm do server tính	20
`integration/webhook-payos.test.js`	Integration	Thiếu chữ ký, chữ ký giả, sai số tiền → 400; webhook thử → 200; hợp lệ → đơn "đã thanh toán"; gửi lặp không xử lý lại	6
`integration/he-thong.test.js`	Integration	Health check, 404, JSON hỏng → 400, security headers, CORS (cho phép frontend và Swagger, chặn origin lạ), đặc tả OpenAPI	8
`integration/cai-dat.test.js`	Integration	Cấu hình khuyến mãi lưu trên server: khách đọc, chỉ Admin ghi, kiểm tra dữ liệu, đổi chữ ký đồng bộ	8

Tệp kiểm thử	Loại	Nội dung kiểm tra	Số ca
`integration/dong-bo.test.js`	Integration	API phiên bản dữ liệu: thêm / sửa / xóa làm đổi chữ ký, không đổi thì giữ nguyên, không cache	5
`integration/gio-hang.test.js`	Integration	Giỏ hàng theo tài khoản: cần đăng nhập, người này không thấy giỏ người khác, giá lấy mới nhất, bỏ qua giá giả, gỡ món ẩn / hết hàng, hạ số lượng về tồn kho	12
Tổng		**11 tệp**	**103**

Ví dụ một ca integration test: gửi webhook PayOS với chữ ký giả, kỳ vọng server từ chối và đơn hàng không đổi trạng thái.

```
test('chữ ký giả mạo → 400, đơn KHÔNG đổi trạng thái', async () => {
  await taoDonChoThanhToan();
  const data = { orderCode: MA_PAYOS, amount: SO_TIEN, code: '00' };
  const res = await request(app)
    .post('/api/thanh-toan/webhook')
    .send(taoWebhook(data, 'chu-ky-gia-mao'));

  expect(res.status).toBe(400);
  const don = await DonHang.findOne({ payos_order_code: MA_PAYOS });
  expect(don.da_thanh_toan).toBe(false);
});
```

Chạy bằng lệnh `npm test` (hoặc `npm run test:coverage` để kèm báo cáo độ phủ). Kết quả: **11/11 tệp, 103/103 ca đạt** trong khoảng 9 giây (Hình 6.37).



Hình 6.37. *Chạy bộ kiểm thử Jest: 11 tệp, 103/103 ca đạt, kèm tóm tắt độ phủ mã nguồn*

Độ phủ hiện đạt khoảng 33% số dòng lệnh. Nhóm ưu tiên các phần có rủi ro cao nhất là xác thực, phân quyền và thanh toán; các nghiệp vụ còn lại (đơn hàng, mã giảm giá, đánh giá) vẫn được kiểm thử bằng bộ Postman ở mục 6.4 – 6.6.

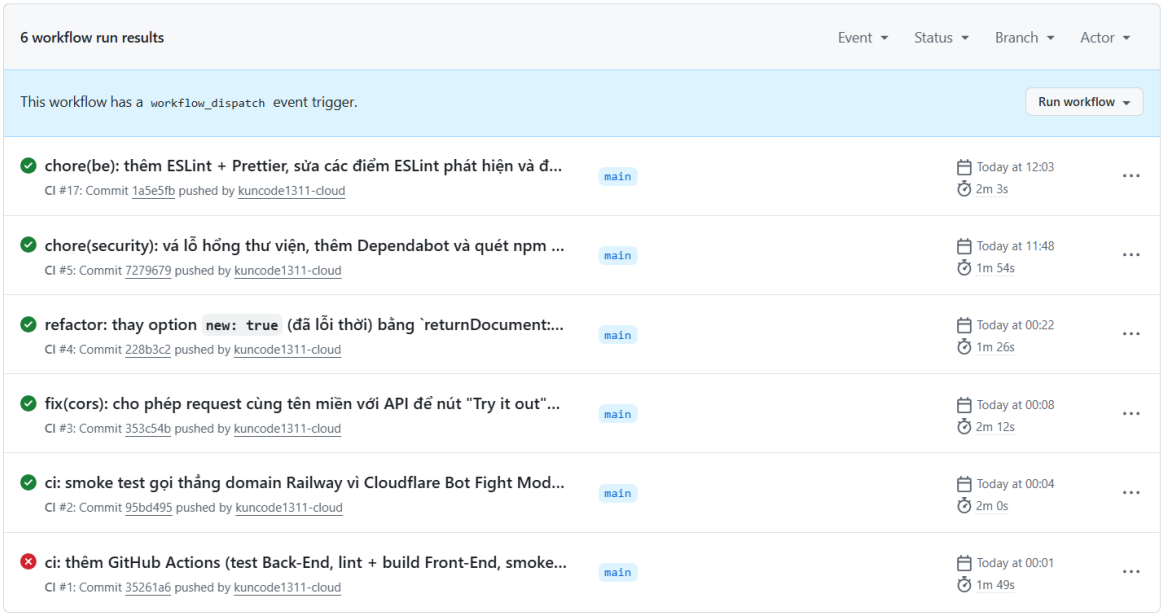
6.8.2. Tích hợp liên tục với GitHub Actions

Quy trình CI khai báo trong tệp `.github/workflows/ci.yml`, tự chạy mỗi khi push hoặc mở Pull Request vào nhánh `main`. Hai job đầu chạy song song; job cuối chỉ chạy khi cả hai đạt, nên mã làm hỏng kiểm thử bị phát hiện ngay trên commit (dấu ✗) trước khi được dùng để demo hay bàn giao.

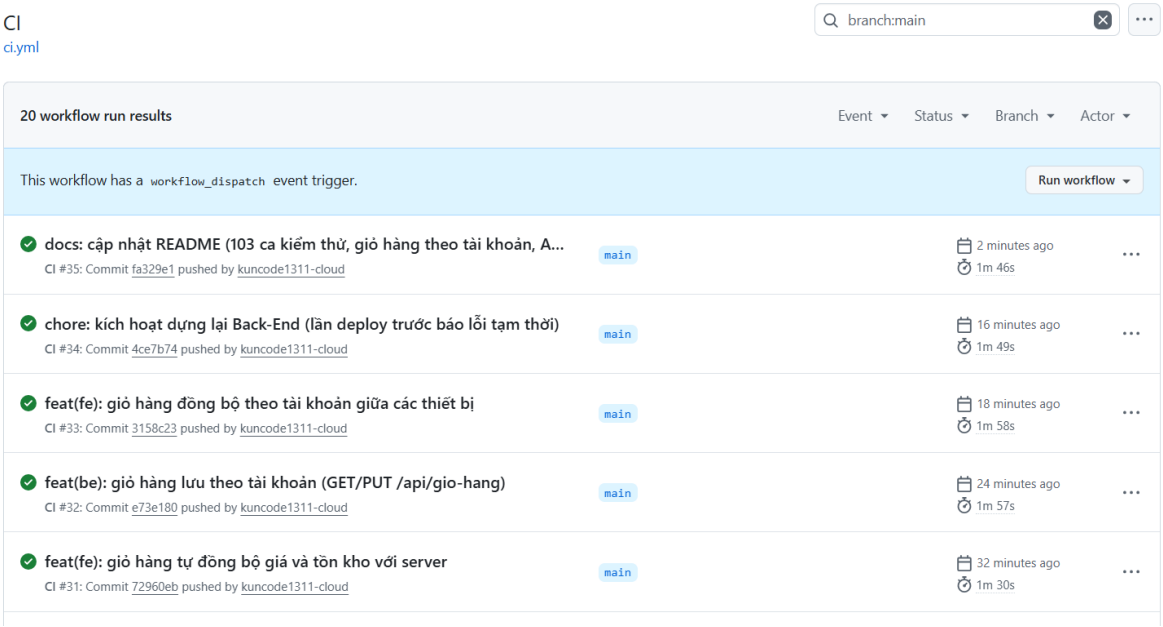
Bảng 6.9. *Các job trong quy trình CI*

Job	Các bước chính	Mục đích
`backend-test`	`npm ci` → `npm audit` → `npm run lint` → `npm run format:check` → `npm run test:coverage`; lưu báo cáo độ phủ (artifact)	Quét lỗ hổng thư viện, kiểm tra chất lượng và định dạng mã, chạy 103 ca Jest trên máy Ubuntu sạch
`frontend-build`	`npm ci` → `npm audit` → `npm run lint` → `npm run build`	Quét lỗ hổng, bảo đảm Front-End không lỗi cú pháp và build production được
`production-smoke`	Chỉ chạy khi 2 job trên đạt (`needs`); curl 6 điểm kiểm tra + Newman thư mục "14. Giám sát máy chủ"	Xác nhận hệ thống đang chạy thật vẫn đúng sau mỗi lần push

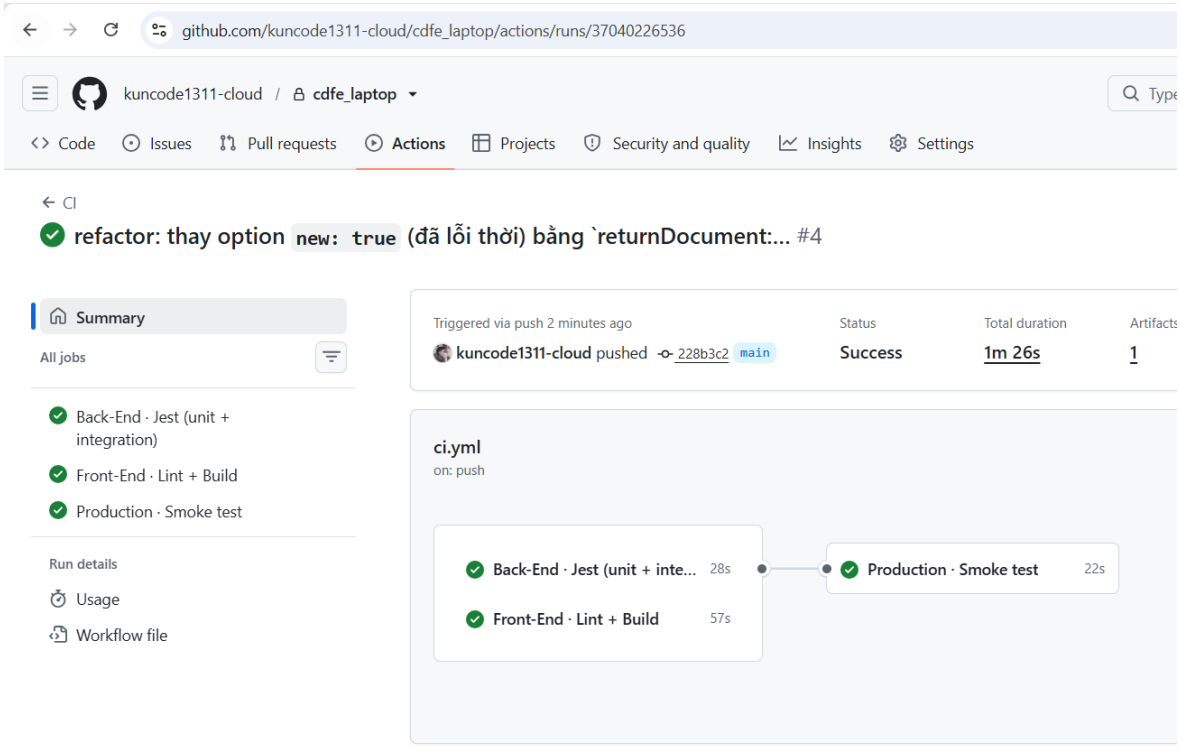
Lần chạy đầu, bước smoke test gọi `api.trikun.site` nhận mã **403**: Cloudflare Bot Fight Mode nhận diện máy chủ GitHub là bot và chặn ngay tại biên — đúng thiết kế ở mục 6.7.3. Nhóm cho smoke test kiểm tra thẳng hai service trên Railway (`laptop.up.railway.app`, `shoplaptop.up.railway.app`), còn tên miền chính vẫn được Cloudflare bảo vệ. Lịch sử các lần chạy ở Hình 6.38 cho thấy lần đầu không đạt (dấu đỏ) và các lần sau đều đạt, Hình 6.39 là các lần chạy gần nhất; Hình 6.40 là chi tiết một lần chạy với cả ba job.



Hình 6.38. *Lịch sử các lần chạy CI trên nhánh main: lần đầu bị Cloudflare chặn ở bước smoke test (đỏ), sau khi sửa các lần tiếp theo đều đạt, kể cả lần thêm ESLint, Prettier và npm audit*



Hình 6.39. *Các lần chạy CI gần nhất trên nhánh main (tổng 20 lần chạy) đều đạt, gồm các commit giỏ hàng theo tài khoản và đồng bộ giá, tồn kho*



Hình 6.40. *Chi tiết một lần chạy CI: cả 3 job Back-End, Front-End và Production smoke test đều đạt*

6.8.3. Chất lượng mã nguồn và an toàn thư viện

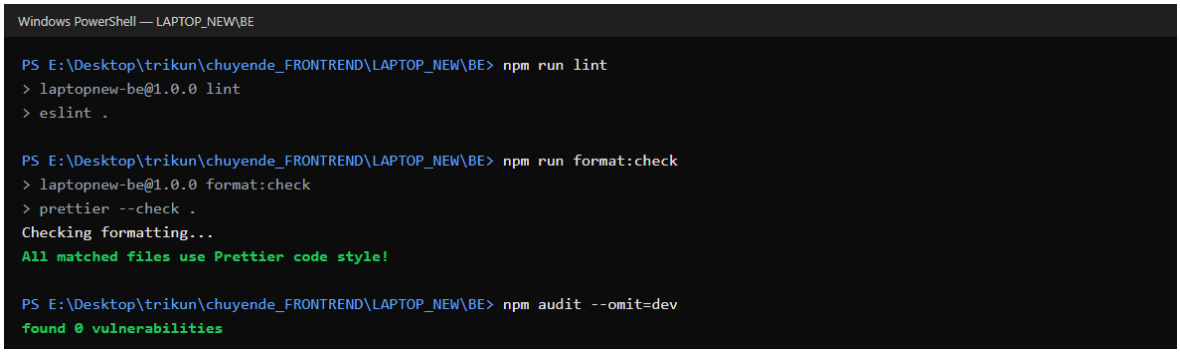
Ngoài kiểm thử chức năng, nhóm kiểm tra thêm chất lượng mã nguồn và các thư viện phụ thuộc bằng ba công cụ: **ESLint** phân tích tĩnh mã để phát hiện lỗi tiềm ẩn (bộ luật `eslint:recommended`), **Prettier** tự định dạng mã theo một chuẩn chung cho cả nhóm, và **npm audit** đối chiếu thư viện đang dùng với cơ sở dữ liệu lỗ hổng đã công bố. Mỗi công cụ có lệnh riêng (`npm run lint`, `npm run format`) và cũng được chạy trong CI. Lần quét đầu tiên cho kết quả ở Bảng 6.10.

Bảng 6.10. *Kết quả quét chất lượng mã nguồn và lỗ hổng thư viện*

Công cụ	Phát hiện ban đầu	Cách xử lý	Kết quả
ESLint	17 điểm: 8 lỗi, 9 cảnh báo	6 khối `catch {}` rỗng (cố ý bỏ qua lỗi parse JSON) → thêm chú thích nêu lý do; 2 ký tự escape thừa trong regex; 4 biến khai báo nhưng không dùng; 5 chỗ `let` nên là `const`	0 lỗi, 0 cảnh báo
Prettier	41 tệp chưa cùng một kiểu định dạng	Chạy `prettier --write`; cấu hình theo đúng phong cách sẵn có (thụt 4 ký tự, dấu nháy đơn, tối đa 120 ký tự mỗi dòng)	Toàn bộ tệp đạt

Công cụ	Phát hiện ban đầu	Cách xử lý	Kết quả
npm audit	FE: Next.js 16.3.2 có lỗ hổng mức **critical** (thực thi mã từ xa qua API tối ưu ảnh); `sharp` mức high; BE: `qs` mức moderate	Nâng Next.js lên 16.3.8 (bản vá cùng nhánh), cập nhật `sharp` và `qs`; build lại và deploy	FE và BE: 0 lỗ hổng

Việc sửa chỉ gồm dọn mã chết và đổi cách trình bày, không thay đổi logic; sau đó toàn bộ ca kiểm thử vẫn đạt và các chức năng trên production (đăng nhập, danh sách sản phẩm, trợ lý AI) vẫn hoạt động bình thường. Kết quả chạy lại các lệnh ở Hình 6.41. Ngoài ra, nhóm cấu hình **Dependabot** (tệp `.github/dependabot.yml`) để mỗi tuần GitHub tự mở Pull Request nâng cấp thư viện lỗi thời; các Pull Request này đi qua CI nên chỉ được gộp khi kiểm thử vẫn đạt.



Hình 6.41. *Chạy ESLint, kiểm tra định dạng Prettier và npm audit: không còn lỗi, không còn lỗ hổng*

> **Kết quả:** mỗi lần push, GitHub Actions quét lỗ hổng thư viện, kiểm tra chất lượng và định dạng mã, chạy 103 ca kiểm thử Back-End, lint và build Front-End, rồi kiểm tra hệ thống production; cả ba job đều đạt. Kết hợp với bộ Postman, hệ thống được kiểm thử ở cả mức hàm, mức API và môi trường thật.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết luận

❖ Kết quả đạt được

- Xây dựng hoàn chỉnh Back-End cho website bán laptop với **15 nhóm tài nguyên, 74 route**, tổ chức rõ ràng theo các tầng Route – Middleware – Controller – Service – Model.
- Thiết kế 10 collection trên MongoDB Atlas, khai thác tốt ưu điểm của mô hình document (nhúng thông số, phiên bản, tồn kho) và dùng TTL index cho dữ liệu tạm.
- Hoàn thiện luồng **đăng ký có OTP, đăng nhập JWT, Google, quên/đổi mật khẩu** và phân quyền khách hàng / quản trị viên.
- Hiện thực nghiệp vụ **đơn hàng có đồng bộ tồn kho hai chiều**, mã giảm giá, bảo hành điện tử.
- Tích hợp thành công các dịch vụ thực tế: **PayOS/VietQR** (Webhook có chữ ký), **Brevo/Nodemailer**, **Telegram Bot** tương tác hai chiều và **Trợ lý AI** có dự phòng nhiều tầng.
- Kiểm thử tự động **40 ca API** bằng Postman Runner đều đạt; bổ sung **15 kịch bản tấn công thử** và minh chứng dữ liệu MongoDB trước/sau thao tác; hệ thống chạy ổn định cùng Front-End Next.js.
- Viết 103 ca kiểm thử tự động bằng Jest + Supertest (unit và integration, MongoDB chạy trong bộ nhớ) và thiết lập CI trên GitHub Actions: mỗi lần push đều tự quét lỗ hổng thư viện, kiểm tra ESLint + Prettier, chạy kiểm thử Back-End, lint + build Front-End và kiểm tra hệ thống production; vá lỗ hổng Next.js mức critical do npm audit phát hiện.
- Xây dựng hệ thống **bảo mật nhiều lớp**, đối chiếu đủ 10 rủi ro của OWASP API Security Top 10; tự rà soát và **khắc phục 21 lỗ hổng** (đăng nhập giả mạo, IDOR, NoSQL Injection, sửa giá, race condition, dò OTP/mật khẩu...).
- **Triển khai thực tế:** website https://trikun.site và API https://api.trikun.site chạy trên Railway, tự động deploy từ GitHub, đặt sau Cloudflare (HTTPS bắt buộc,

Bot Fight Mode, rate limit lớp biên); công bố tài liệu API tương tác chuẩn OpenAPI 3.0 tại https://api.trikun.site/api-docs.

❖ **Kiến thức và kỹ năng thu được**

- Hiểu rõ quy trình xây dựng một hệ thống Back-End từ thiết kế kiến trúc, cơ sở dữ liệu đến đặc tả và kiểm thử API.
- Thành thạo Express (router, middleware, xử lý lỗi), Mongoose (schema, index, hook), JWT và bcrypt.
- Biết cách tích hợp API bên thứ ba an toàn: ký và xác minh HMAC, timeout, bộ nhớ đệm, cơ chế dự phòng khi dịch vụ lỗi.
- Hình thành tư duy bảo mật: luôn kiểm tra quyền sở hữu dữ liệu, không tin dữ liệu client, dùng thao tác nguyên tử và kiểm thử bằng chính các kịch bản tấn công.
- Rèn luyện kỹ năng đọc tài liệu kỹ thuật, làm việc nhóm với Git và tự rà soát bảo mật mã nguồn.

❖ **Hạn chế**

- Bộ đếm rate limit trong ứng dụng lưu trong bộ nhớ (đã có thêm lớp giới hạn ở biên Cloudflare); chưa có 2FA cho Admin.
- Kiểm thử bằng mã (Jest) mới phủ khoảng 33% mã nguồn, tập trung vào xác thực, phân quyền và thanh toán; chưa có kiểm thử giao diện đầu-cuối (E2E).
- Chưa dùng transaction khi trừ tồn kho nhiều sản phẩm; tìm kiếm dựa trên `$regex` chưa tối ưu cho dữ liệu lớn.
- Chưa có refresh token (access token ngắn hạn) và nhật ký kiểm toán (audit log) cho thao tác quản trị.

7.2. Hướng phát triển

- **Bảo mật:** chuyển bộ đếm rate limit sang Redis; thêm 2FA cho Admin, access token ngắn hạn + refresh token; ghi audit log; chạy `npm audit` / Dependabot trong CI; kiểm tra dữ liệu bằng thư viện schema (Joi / Zod).

- **Chất lượng:** nâng độ phủ kiểm thử Jest cho đơn hàng, mã giảm giá, đánh giá; thêm kiểm thử giao diện E2E bằng Playwright; thêm pre-commit hook (husky) tự chạy lint trước mỗi commit.
- **Hiệu năng:** thêm Redis cache cho danh sách sản phẩm; dùng Atlas Search / text index cho tìm kiếm; phân trang cho mọi API danh sách.
- **Dữ liệu:** dùng MongoDB transaction khi tạo đơn và trừ kho; chuẩn hóa định dạng ngày về kiểu `Date`.
- **Tính năng:** WebSocket để cập nhật trạng thái thanh toán tức thì thay cho polling; tích hợp đơn vị vận chuyển (GHN, GHTK) để tính phí và theo dõi đơn; AI gợi ý sản phẩm theo lịch sử xem.
- **Triển khai:** đóng gói Docker, thêm môi trường staging tách khỏi production; giám sát uptime và cảnh báo lỗi; nâng cấp gói Cloudflare để bật WAF Managed Rules và thêm luật rate limit.

TÀI LIỆU THAM KHẢO

[1] Node.js Documentation — https://nodejs.org/docs

[2] Express 5.x API Reference — https://expressjs.com/en/5x/api.html

[3] MongoDB Manual — https://www.mongodb.com/docs/manual/

[4] Mongoose Documentation — https://mongoosejs.com/docs/

[5] RFC 7519 — JSON Web Token (JWT) — https://datatracker.ietf.org/doc/html/rfc7519

[6] jsonwebtoken (npm) — https://github.com/auth0/node-jsonwebtoken

[7] bcrypt.js — https://github.com/dcodeIO/bcrypt.js

[8] Nodemailer Documentation — https://nodemailer.com/

[9] Brevo API v3 Reference — https://developers.brevo.com/reference

[10] PayOS Developer Documentation — https://payos.vn/docs/

[11] VietQR Quick Link — https://www.vietqr.io/danh-sach-api/link-tao-ma-nhanh/

[12] Telegram Bot API — https://core.telegram.org/bots/api

[13] Google Identity — Verify the Google ID token — https://developers.google.com/identity/gsi/web/guides/verify-google-id-token

[14] Google Gemini API — https://ai.google.dev/gemini-api/docs

[15] OWASP API Security Top 10 — https://owasp.org/API-Security/

[16] MDN Web Docs — HTTP response status codes — https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

[17] OWASP Cheat Sheet Series — Authentication, Password Storage, Mass Assignment — https://cheatsheetseries.owasp.org/

[18] MongoDB Manual — Atomicity and Transactions — https://www.mongodb.com/docs/manual/core/write-operations-atomicity/

[19] OpenAPI Specification 3.0.3 — https://spec.openapis.org/oas/v3.0.3

[20] Swagger UI — https://swagger.io/tools/swagger-ui/

[21] Railway Documentation — https://docs.railway.com/

[22] Cloudflare Docs — DNS, SSL/TLS, Rate limiting rules — https://developers.cloudflare.com/

[23] Provinces Open API — Danh mục đơn vị hành chính Việt Nam — https://provinces.open-api.vn/

[24] Tài liệu API LaptopNew (Swagger UI) — https://api.trikun.site/api-docs

[25] Jest Documentation — https://jestjs.io/docs/getting-started

[26] Supertest — https://github.com/ladjs/supertest

[27] GitHub Actions Documentation — https://docs.github.com/actions

[28] ESLint — https://eslint.org/docs/latest/

[29] Prettier — https://prettier.io/docs/